

Tutorials for pointcloud processing in Python

Jean-Loup GREGORIO

September 6, 2026

Introduction

The use of pointclouds has steadily increased over the years, driven by the growing availability of 3D acquisition systems and advanced 3D processing and visualization software. Pointclouds are nowadays used in many areas, including engineering and manufacturing, architecture and construction, cultural heritage documentation and preservation, robotics and autonomous systems, extended and immersive reality, geospatial and environmental analysis, to name just a few.

These tutorials are for those wishing to learn a little bit more about the basics of pointcloud processing. Having gone through this stage during my Ph.D., I hope here to share some of what I have learned so far.

The notebooks are designed to make pointcloud processing algorithms easier to understand, without compromising performance too much and trying to minimize the use of specialized third-party software. They require basic knowledge of Python and its main scientific libraries.

Dependencies

The code is in python and relies on `numpy`, `scipy`, `matplotlib`, `scikit-learn`, and `jupyterlab`.

These dependencies may be installed with pip (or conda) with

```
pip install numpy scipy matplotlib scikit-learn jupyterlab
```

JupyterLab may be started using the terminal or Anaconda prompt simply by typing

```
jupyter lab
```

Reusing and distributing

This notebook is licensed under the Creative Commons Attribution-NonCommercial 4.0 International License (CC-BY-NC-4.0). For the full license text, please visit <https://creativecommons.org/licenses/by-nc/4.0/>.

If you use these tutorials in your work, please cite them as:

```
@unpublished{gregorio2024tutorials,  
  author={Grégorio, Jean-Loup},  
  title={Tutorials for pointcloud processing in Python},  
  year={2024},  
}
```

Contents

1	Basics	5
1.1	What is a pointcloud?	5
1.2	Loading	5
1.3	Visualizing	6
1.4	Inspecting	8
1.4.1	Intensity	8
1.4.2	Colors	10
1.4.3	Normals	11
1.4.4	Scalar fields	14
1.5	Geometric properties	15
1.5.1	Centroid	15
1.5.2	Bounding box	17
1.5.3	Extent	18
1.6	Wrapping up	19
2	Transformations	20
2.1	Translations	20
2.2	Rotations	22
2.3	Reflections	29
2.4	Scaling	33
2.5	Going further	35
2.6	Wrapping up	36
3	Spatial indexing	37
3.1	About distances, neighborhoods and the “brute force” approach	37
3.1.1	Distances	37
3.1.2	Neighborhoods	39
3.1.3	The “brute-force” approach	40
3.2	Voxel grids	43
3.3	Octrees	50
3.4	kd-trees	60
3.5	Quick comparison	69
3.6	Going further	70
3.7	Wrapping up	71
4	Subsampling	73
4.1	Index-based	73
4.1.1	Random sampling	73
4.1.2	Stride sampling	75
4.2	Spatial partitioning	77
4.2.1	Voxel grid downsampling	77
4.2.2	Octree downsampling	81
4.3	Distance-based	85
4.3.1	Farthest point sampling	85
4.3.2	Radius sampling	88
4.3.3	Poisson-disk sample elimination	90

4.4	Quick comparison	92
4.5	Wrapping up	93
5	Cleaning	94
5.1	Outlier removal	94
5.1.1	Statistical Outlier Removal (SOR)	96
5.1.2	Radius Outlier Removal (ROR)	98
5.2	Denoising	99
5.2.1	Bilateral filter	101
5.2.2	Moving Least Squares	104
5.3	Going further	108
5.4	Wrapping up	109
6	Normals and curvatures	110
6.1	Normals	110
6.1.1	Tangent plane estimation	110
6.1.2	Orientation with Riemannian Graph	115
6.2	Curvatures	119
6.2.1	PCA-based surface variation	120
6.2.2	Local surface fitting	122
6.2.3	Discrete differential operators	133
6.2.4	Quick comparison	139
6.3	Going further	141
6.4	Wrapping up	143
7	Descriptors	144
7.1	Eigenvalue-based Features	145
7.2	Point Feature Histograms (PFH) and Fast Point Feature Histograms (FPFH)	152
7.3	Signature of Histograms of Orientations (SHOT)	158
7.4	Going further	163
7.5	Wrapping up	164
8	Segmentation	165
8.1	Surface Segmentation	166
8.1.1	Region Growing	166
8.1.2	The Hough Transform	171
8.1.3	RANSAC	180
8.2	Object Segmentation	185
8.3	Going further	188
8.4	Wrapping up	189
9	Primitive fitting	191
9.1	About fitting	191
9.2	Plane	192
9.3	Sphere	199
9.4	Cylinder	205
9.5	Cone	214
9.6	Torus	223
9.7	Validity of fitted primitives	231

9.8	Wrapping up	233
10	Registration	234
10.1	Registration between corresponding sets	234
10.1.1	Point-to-point	236
10.1.2	Point to plane	238
10.2	Registration between non-corresponding sets	240
10.2.1	The Iterative Closest Point (ICP) Algorithm	240
10.2.2	Global registration and Principal Axis Alignment	246
10.3	Wrapping up	249
11	Machine learning	250
11.1	Cleaning	250
11.2	Object Segmentation	255
11.2.1	K-means	256
11.2.2	DBSCAN	258
11.2.3	Agglomerative Clustering	259
11.3	Semantic Segmentation	261
11.3.1	Neighborhoods	263
11.3.2	Descriptors	264
11.3.3	Classifiers	271
11.3.4	Regularization	278
11.4	Wrapping up	281
12	Going further	283
12.1	Software	283
12.2	Resources	283
	References	285

1 Basics

This notebook introduces the basic structure of pointclouds and their most common attributes. It covers how pointcloud data is represented, loaded, visualized, and inspected, and introduces simple geometric properties such as centroids and bounding boxes. These fundamental concepts form the basis for more advanced processing techniques explored in the following notebooks.

```
[29]: # Necessary imports
import sys
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

sys.path.append("./utils")
from plot_utils import cuboid_to_poly3D
```

1.1 What is a pointcloud?

A pointcloud generally refers to a set of points defined in a geometric space. In the context of this notebook and the following ones, this space is three-dimensional, and each point is described by its coordinates (x, y, z) . A pointcloud may thus be seen as a **discrete representation of a shape or object, where geometry is captured by sampling points on its surface**. In contrast to continuous representations (e.g., parametric surfaces), this sampling provides only a finite approximation of the underlying geometry.

Formally, a pointcloud can be represented as a set of points:

$$P = \{p_0, p_1, \dots, p_{n-1}\}$$

where each point $p_i \in \mathbb{R}^3$ is defined by its coordinates $p_i = (x_i, y_i, z_i)$.

This mathematical description emphasizes that a pointcloud is fundamentally an unordered set of samples in 3D space. So, unlike other representations such as meshes or parametric surfaces, pointcloud are:

- **Unstructured** (no connectivity is defined between points)
- **Unordered** (the ordering of points itself carries no information)

Pointclouds are widely used in domains involving 3D data acquisition and analysis, such as engineering (reverse engineering and inspection), autonomous systems (perception for robots or self-driving vehicles), and geographic information systems (terrain mapping). Their properties are strongly influenced by the acquisition process. As a result, pointclouds are often:

- **Noisy** (measurement errors)
- **Irregularly sampled** (non-uniform density)
- **Incomplete** (occlusions or limited sensor coverage)

These characteristics make point clouds both flexible and challenging to process.

1.2 Loading

Pointclouds are stored using a variety of file formats, depending on acquisition systems, processing software, as well as data size and content.

The simplest formats are **ASCII formats**, with extensions such as *.txt*, *.pts*, and *.xyz*. These are plain text files in which values are separated by whitespace or commas, and each line corresponds to a point. ASCII formats are easy to read and edit, but not memory-efficient, making them best suited for small to medium-sized pointclouds (typically up to a few hundred thousand points).

More efficient formats rely on binary encoding to reduce storage size and improve input/output (I/O) performance, making them better suited for large-scale pointclouds. The most common open formats for storing 3D pointclouds include:

- **PLY** (developed at Stanford, also supporting ASCII), flexible and widely used in research
- **PCD** (native of PCL, the Point Cloud Library), flexible and widely used in robotics
- **LAS** and **LAZ** (ASPRS standard), standard formats for LiDAR data, commonly used in geospatial applications
- **E57** (ASTM standard), designed for efficient storage and interoperability

For the sake of simplicity, this notebook and the following ones use an ASCII-based format, with points coordinates separated by whitespaces.

Such files can be easily loaded using Python and its most commonly used numerical library **numpy**.

```
[30]: # Load the pointcloud data from the .xyz file with a simple line of code
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
print("Here is the content of our pointcloud:\n", points)
print("Its shape is:", points.shape)
```

Here is the content of our pointcloud:

```
[-25.5861 106.9803  42.684 ]
[-30.1628  93.312  -47.9823]
[ -8.5167 100.3486  73.1544]
...
[ 39.3746 100.979  -1.1485]
[-54.9932  84.2855 -32.3682]
[-77.221  104.7658 -26.0204]]
```

Its shape is: (4096, 3)

Here, data are stored in a Numpy array, which is better suited for numerical operations than than a simple list. Points are usually listed *row-wise* (one point per line), resulting in a vector of shape $(n, 3)$ (n being the total number of points), although the contrary (i.e., *column-wise*) is also found. Points are defined by their cartesian coordinates, meaning their position from the origin according to the x , y , and z axes.

The loaded pointcloud is derived from the *Stanford bunny*, which is a popular test model used in computer graphics. The original model was obtained by scanning a ceramic figurine of a rabbit. It is considered a simple model by today's standards, in terms of both size and geometric complexity. Our pointcloud is an even more simplified version to facilitate manipulation and visualization.

1.3 Visualizing

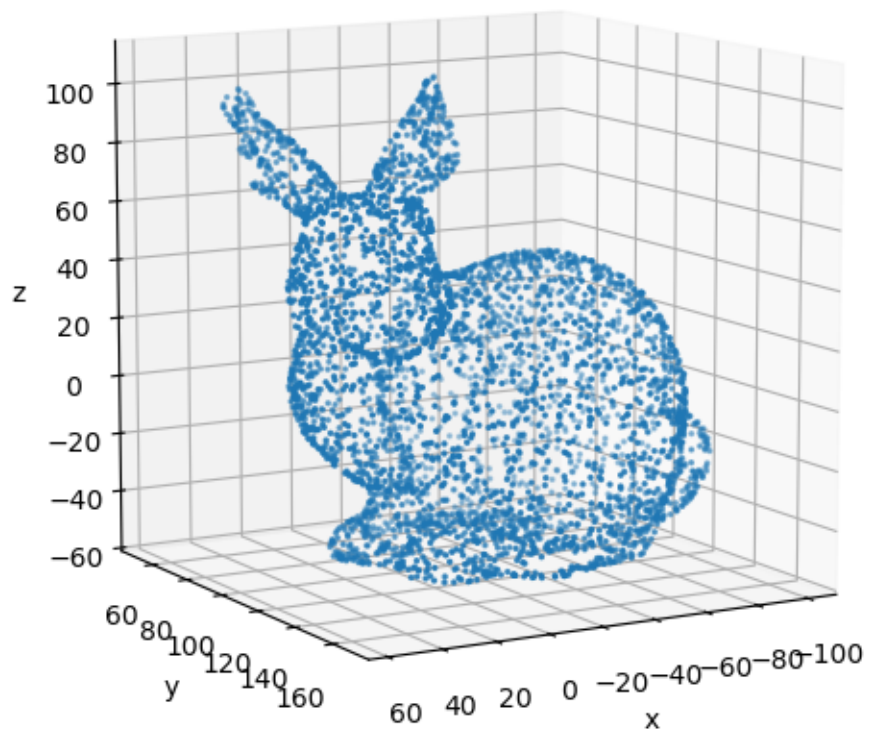
Visualizing a pointcloud is often the simplest step to gain an intuitive understanding of its geometry. Again, many options are available, depending on data format, size and content.

The simplest approach consists in representing each point with a scatter plot, using tools for data

visualization. This notebook and the following ones use `matplotlib`, which is among the most commonly used libraries in Python.

```
[31]: # Visualize the pointcloud with a simple 3D scatter plot using Matplotlib
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
# x, y, and z coordinates are stored in the first, second, and third columns
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           s=2.) # makes points smaller for better visualization
# Axis labels
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
# Ensure equal scaling along all axes (avoid distortion)
ax.set_box_aspect([1, 1, 1])
# Adjust the view angle for better visualization
ax.view_init(elev=10, azim=60)
# Set title and show the plot
ax.set_title("Stanford Bunny pointcloud")
plt.show()
```

Stanford Bunny pointcloud



It is important to keep in mind that pointclouds are often large, sparse, and irregularly sampled, which may affect both rendering performance and visual clarity.

For larger pointclouds (typically containing tens of thousands of points or more) and more interactive and advanced visualization (e.g., with shading), dedicated libraries, such as `open3D`, or software, such as CloudCompare, are generally preferred.

1.4 Inspecting

As seen before, each point p_i of the pointcloud P is defined by *a minima* three coordinates (x_i, y_i, z_i) , which describe its position in space. However, it is common for points to carry additional information, called *point attributes* (distinct from metadata). These attributes may be given directly by the acquisition system, or computed afterwards.

Among the most popular attributes are:

- **Intensity**, which measures the strength of the returned signal (typically obtained from LiDAR systems)
- **Colors** (typically obtained from imaging devices)
- **Normals**, which describe the local orientation of the surface (usually estimated)
- **Scalar fields**, which represent any per-point quantity

In practice, a pointcloud with attributes is represented by an array of shape (n, d) , with n the number of points and d the number of attributes per point (including its spatial coordinates). The (n, d) array is usually split into dedicated arrays (e.g., spatial coordinates, normals, colors) for clarity and ease of manipulation.

Inspection also provides an opportunity to perform basic checks, such as searching for invalid data.

```
[32]: # Load the pointcloud data with additional attributes (colors, normals, etc.)
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
print("Shape of our data:", data.shape)
# Basic check for inconsistencies or missing values (NaN)
print("Number of NaN values:", np.isnan(data).sum())
# Split the data into separate arrays for points, colors, scalars, and normals
points = data[:, :3]
normals = data[:, 3:6]
intensity = data[:, 6]
colors = data[:, 7:10]
mean_curvature = data[:, -2]
```

```
Shape of our data: (4096, 12)
```

```
Number of NaN values: 0
```

1.4.1 Intensity

Intensity measures the **strength of the signal returned by the sensor for each point**. It is commonly provided by active acquisition systems such as LiDAR. Intensity values depend on several

factors, including the acquisition conditions (i.e., distance to the object, the angle of incidence) and the material properties of the surface.

Intensity is typically represented as a single scalar value associated with each point. This gives a vector of shape $(n,)$, with n being the total number of points.

```
[42]: # Plot the pointcloud intensity
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
# Add intensity to the scatter plot
p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               s=2., c=intensity)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label("intensity")
ax.set_axis_off() # Hide axes for better visualization
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azim=60) # Adjust the view angle
ax.set_title("Pointcloud colored with intensity")
plt.show()
```

Pointcloud colored with intensity



1.4.2 Colors

Color attributes account for the visual appearance of each point. They may be acquired directly using imaging sensors or obtained by projecting images onto the pointcloud. In the first case, it is important to keep in mind that **colors may vary to the acquisition conditions** (the lighting in particular).

Colors are typically represented with **the RGB color model, that defines a color as an addition of the red, green and blue** primary colors. A RGB color vector associated to a point has three components, noted R, G, and B, with values generally ranging from 0 to 255 (integers) or from 0 to 1 (floats).

This gives a vector of shape $(n, 3)$, with n being the total number of points.

```
[33]: # Plot the pointcloud colored with RGB values
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
# Add colors to the scatter plot
ax.scatter(points[:, 0], points[:, 1], points[:, 2], s=2.,
           c=colors/255) # RGB values need to be normalized in Matplotlib
ax.set_axis_off() # Hide axes for better visualization
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azim=60) # Adjust the view angle
ax.set_title("Pointcloud colored with RGB values")
plt.show()
```

Pointcloud colored with RGB values



1.4.3 Normals

Normals describe the **local orientation of the surface at each point**. A normal associated with a point is typically assumed to be perpendicular to the underlying surface and oriented outward. Normals may be provided directly by the acquisition device, but are more commonly estimated from the pointcloud geometry (see the notebook on normals and curvatures).

A normal vector n_i associated to a point p_i has three components, noted $n_{i,x}$, $n_{i,y}$, and $n_{i,z}$. It is generally a unit vector, which means that its norm is equal to one: $\|n_i\| = 1$.

In practice, similar to colors, normals are stored in an array of shape $(n,3)$, where n is the total number of points.

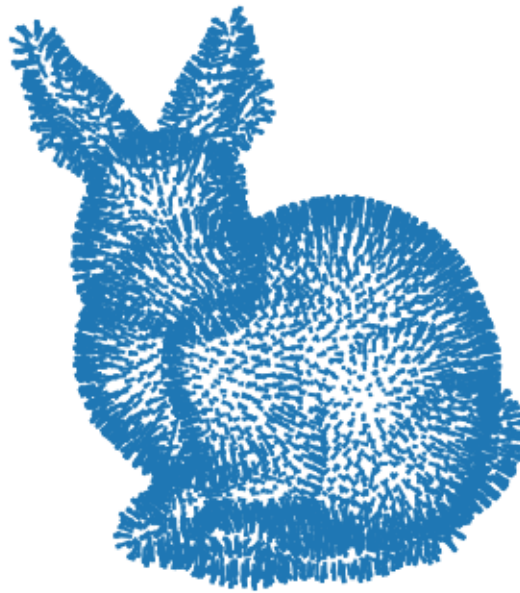
```
[34]: # Plot normals
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.quiver(points[:, 0], points[:, 1], points[:, 2], # points
          normals[:, 0], normals[:, 1], normals[:, 2], # normals
```

```

        length=5.) # make normals a little bigger for visualization purposes
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
ax.set_axis_off() # Hide axes for better visualization
ax.set_title("Pointcloud with normals")
plt.show()

```

Pointcloud with normals



Normals play a central role in many pointcloud processing algorithms, but are also used for visualization and rendering purposes. For instance, computing the angle between an incident light direction and normal vectors provides a simple model for simulating light reflection on a surface, as shown below.

```

[35]: # Incident ray (unit vector giving the direction of light)
v_dir = np.array([-1., 0., 0.])
# Magnitude and location of the light source (for visualization purposes)
v_mag = 25.
v_loc = np.array([120., 0., 100.])

```



```

# Angle between the normals and the incident ray
theta = np.arccos(normals @ v_dir)

# Plot the pointcloud colored with the lighting angle
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=theta, # color the points according to the angle
               cmap="bone" # use a black-and-white colormap
               )
ax.quiver(v_loc[0], v_loc[1], v_loc[2], # location of the light source
          v_dir[0], v_dir[1], v_dir[2], # direction of the incident ray
          length=v_mag, # length of the arrow representing the incident ray
          color='orange')
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('angle (rad)')
cbar.set_ticks([0., np.pi/2, np.pi]) # Tick positions at 0,  $\pi/2$ , and  $\pi$ 
cbar.set_ticklabels(['0', ' $\pi/2$ ', ' $\pi$ ']) # Tick labels to display in terms of  $\pi$ 
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
ax.set_axis_off() # Hide axes for better visualization
ax.set_title("Simulated lighting angle on pointcloud")
plt.show()

```

Simulated lighting angle on pointcloud



1.4.4 Scalar fields

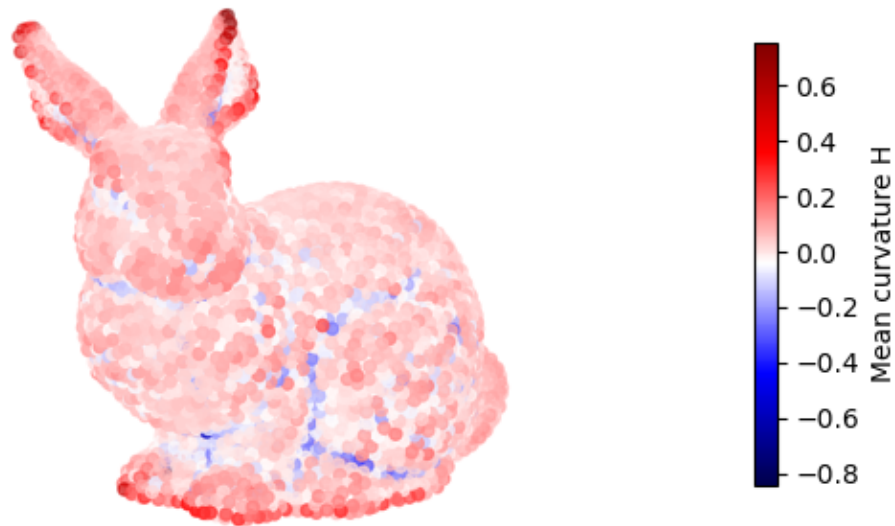
In general terms, a scalar field is a function associating a single number to every point in a space. Here, scalar fields simply designate **values associated with a point**. These values may represent a wide range of quantities, such as density or curvature.

Scalar fields are sometimes given by the acquisition device or may result of a calculation. In practice, a scalar field is used to associate any quantity of interest with the pointcloud. There may be several scalar fields associated with a given pointcloud.

```
[44]: # Plot the pointcloud colored with scalar values
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=mean_curvature, cmap="seismic")
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label("Mean curvature H")
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
```

```
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
ax.set_axis_off() # Hide axes for better visualization
ax.set_title("Pointcloud colored with scalar values")
plt.show()
```

Pointcloud colored with scalar values



1.5 Geometric properties

The following paragraphs detail simple geometric properties of the pointcloud. These properties are computed directly from the point coordinates and form the basis for more advanced processing.

1.5.1 Centroid

The centroid of a pointcloud is the average position of all its points. It provides a simple estimate of the center of mass of the object. Its formula is simply:

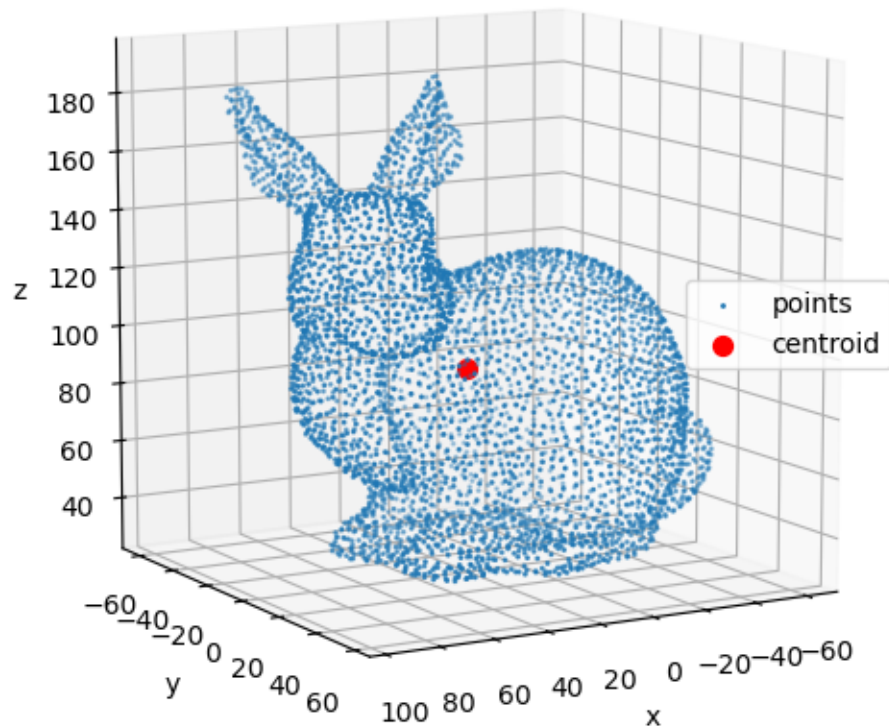
$$\bar{p} = \frac{1}{n} \sum_{i=0}^{n-1} p_i$$

The centroid is often used for centering or normalizing a pointcloud prior to further processing.

```
[37]: # Compute the centroid of the pointcloud
centroid = points.mean(axis=0)

# Show the centroid on the plot
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2], label="points", s=1.)
ax.scatter(centroid[0], centroid[1], centroid[2], c='r', label="centroid", s=50.)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
ax.set_title("Centroid of the pointcloud")
plt.show()
```

Centroid of the pointcloud



1.5.2 Bounding box

The axis-aligned bounding box (AABB) defines the smallest box, aligned with the coordinate axes, that contains all points. It is usually described by two vectors, the minimum coordinates and the maximum coordinates, also called the lower and upper bounds:

$$upper = \max_i(p_i) \quad lower = \min_i(p_i)$$

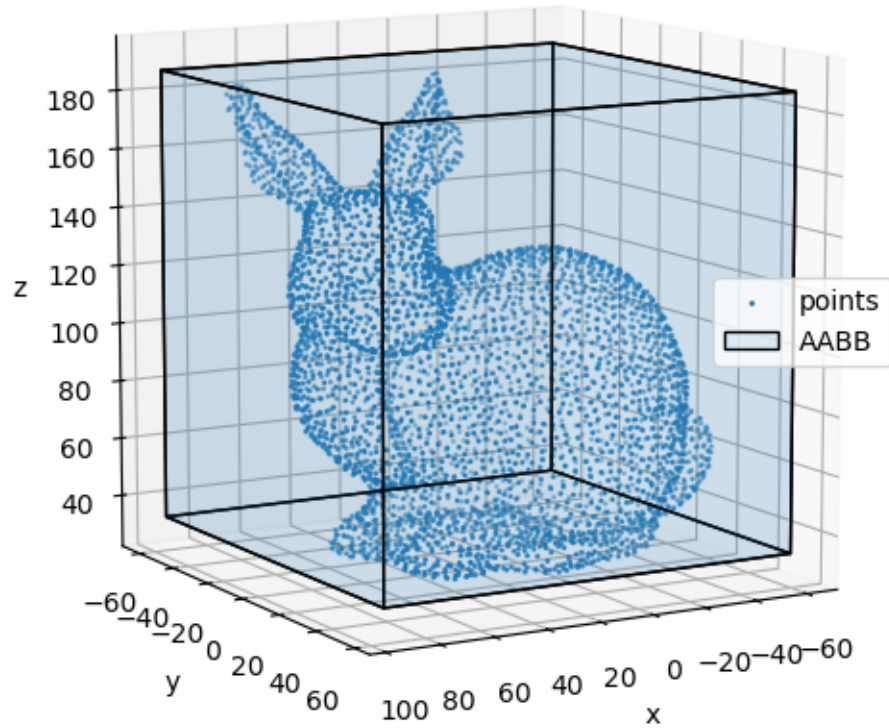
where the minimum and maximum are taken component-wise over all points.

The bounding box provides a quick approximation of the location and size of the pointcloud (and the sampled objects).

```
[ ]: # Compute the axis-aligned bounding box (AABB)
bbox_lower = points.min(axis=0)
bbox_upper = points.max(axis=0)

# Show the bounding box on the plot
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2], s=1, label="points")
# Plot the bounding box
bbox = cuboid_to_poly3D(bbox_lower, bbox_upper - bbox_lower) # custom function
pc = Poly3DCollection(bbox, edgecolor="k", alpha=0.1, label="AABB")
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
ax.set_title("Axis-aligned bounding box of the pointcloud")
plt.show()
```

Axis-aligned bounding box of the pointcloud



1.5.3 Extent

The extent of the pointcloud corresponds to its size along each coordinate axis. It can be computed as the difference between the maximum and minimum point coordinates:

$$ext = \max_i(p_i) - \min_i(p_i)$$

where the minimum and maximum are taken component-wise over all points.

This quantity is useful for understanding the scale of the data and for defining normalization or discretization strategies.

```
[39]: # Compute the extent
extent = np.ptp(points, axis=0)
with np.printoptions(precision=1, suppress=True): # for better readability
    print("Extent of the pointcloud (size along x, y, z):", extent)
```

Extent of the pointcloud (size along x, y, z): [155.4 120.5 154.]

1.6 Wrapping up

You should now have a better grasp of what a pointcloud is, how it is represented, and how to inspect and manipulate it in practice.

A key takeaway is that pointclouds represents a shape in a quite simple and flexible way, that is as a set of points in a 3D space, possibly enriched with additional attributes such as normals or colors. This representation is inherently **unordered** and **discrete**. It is also important to remember that real-world pointclouds are typically **noisy**, **irregularly sampled**, and often **incomplete**.

From a practical perspective, pointclouds can be handled efficiently using standard numerical tools such as Numpy. Relying on vectorized operations is essential for keeping the code both concise and fast. As a rule of thumb, explicit Python loops should be avoided whenever possible. On a modern machine (e.g., a laptop with a recent CPU and with several gigabytes of RAM), pointclouds containing several million points can be loaded and processed within reasonable time (in under a minute) for basic operations.

However, as pointclouds grow larger, dedicated visualization tools quickly become necessary. General-purpose plotting libraries like Matplotlib remain usable only for relatively small datasets (up to tens of thousands of points). Larger pointclouds require specialized software optimized for 3D visualization.

2 Transformations

This notebook explores spatial transformations or geometric operations applicable to 3D pointclouds. These transformations alter the position, orientation, and shape of objects in space. While a comprehensive mathematical treatment is beyond the scope of this notebook, it focuses on the most commonly used transformations in pointcloud processing: translations, rotations, reflections, and scaling. These transformations can be applied individually or in combination to achieve the desired manipulation of pointclouds.

Transformations can be classified based on the properties they preserve. Rigid motions, including translations, rotations, and reflections, maintain distances and angles between points, preserving the overall shape and size of the pointcloud. These are essential for tasks requiring geometric integrity, such as aligning pointclouds from different viewpoints. Affine transformations, which include rigid motions plus scaling and shearing, do not preserve distances and angles but maintain parallelism and ratios of distances along parallel lines. These are useful for resizing or reshaping the pointcloud.

```
[1]: # Necessary imports
import numpy as np
import matplotlib.pyplot as plt

# Load our bunny pointcloud
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
```

2.1 Translations

A translation moves every point of the pointcloud in the same direction.

In practice, a constant vector t is simply added to each point of the pointcloud:

$$p' = p + t$$

Thanks to broadcasting this operation may be vectorized in Numpy, resulting in more readable code and faster execution.

A translation may be used to “center” a pointcloud, for example to make it roughly coincide with the origin of the coordinate system or with another pointcloud. This allows to introduce the notion of centroid (or center of mass if all points are weighted equally), which is the mean position of all the points of the pointcloud. This notion is widely used in pointcloud processing algorithms.

```
[2]: # Translated points
translation = np.array([50., 40., -10.])
translated_points = points + translation
# Centered points
centroid = points.mean(axis=0)
centered_points = points - centroid

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
```

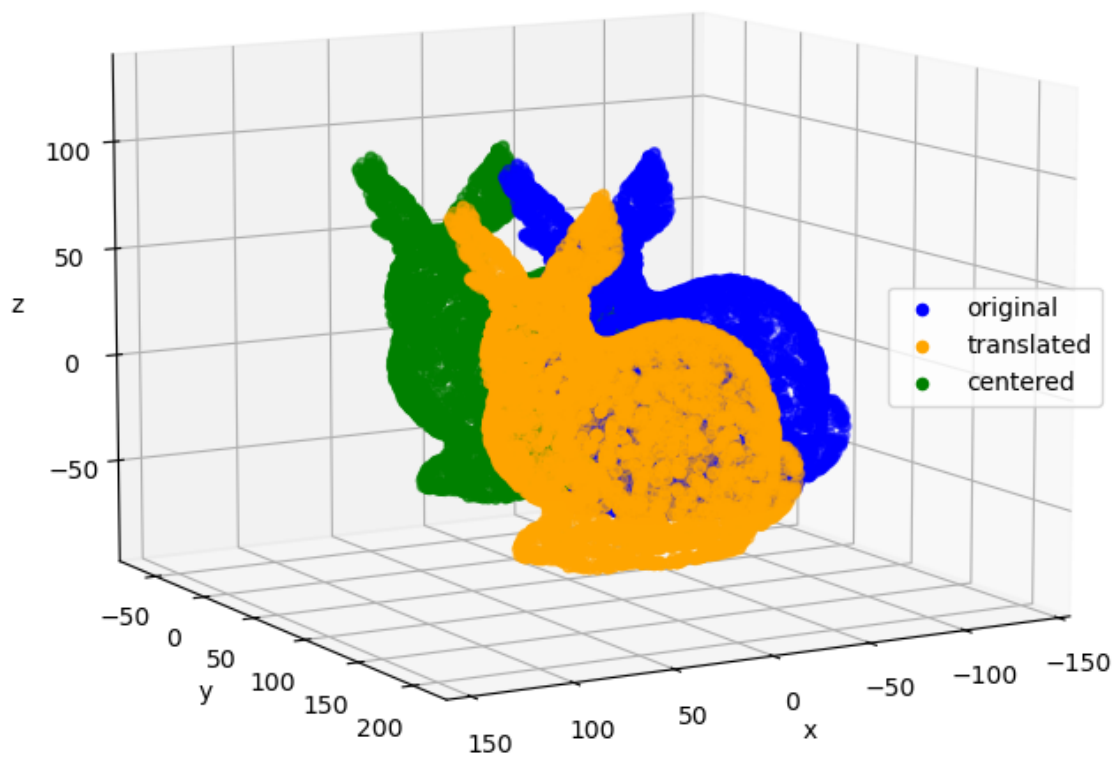


```

ax.scatter(translated_points[:, 0], translated_points[:, 1], translated_points[:, 2],
           c='orange', label="translated")
ax.scatter(centered_points[:, 0], centered_points[:, 1], centered_points[:, 2],
           c='green', label="centered")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Translated pointclouds")
plt.show()

```

Translated pointclouds



2.2 Rotations

A rotation moves every point of the pointcloud around a certain axis and by a certain angle (called θ here).

A rotation is said to be *elemental* (or *basic*) when considering one of the axes of the main coordinate system (i.e., x , y , and z). Rotation matrices around each axis are then given below. A rotation around the x axis leave the first coordinate of each point unchanged, and so on.

$$R_x(\theta_x) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_x & -\sin \theta_x \\ 0 & \sin \theta_x & \cos \theta_x \end{pmatrix} \quad R_y(\theta_y) = \begin{pmatrix} \cos \theta_y & 0 & \sin \theta_y \\ 0 & 1 & 0 \\ -\sin \theta_y & 0 & \cos \theta_y \end{pmatrix} \quad R_z(\theta_z) = \begin{pmatrix} \cos \theta_z & -\sin \theta_z & 0 \\ \sin \theta_z & \cos \theta_z & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

These rotations are counter-clockwise when the rotation axis is directed towards the observer (if the three-dimensional space is oriented according to the usual conventions, see the right-hand rule). Note that these rotations occur around the origin of the main coordinate system.

In practice, matrix multiplication is used to rotate each point of the pointcloud (do not forget that some attributes such as normals must also be rotated as well!). Due to the initial *row-wise* listing of points, some transpose operations are however necessary to keep dimensions consistent:

$$p' = (R(\theta).p^T)^T$$

Again, thanks to broadcasting this operation may be vectorized in Numpy, resulting in more readable code and faster execution.

Some extra steps are necessary if a *local* coordinate system were to be considered. For example, for an “in-place” rotation, around the pointcloud center of mass (without any change in the axes direction), a simple solution consists in centering the pointcloud first, applying the rotation, and then moving the pointcloud around its original centroid.

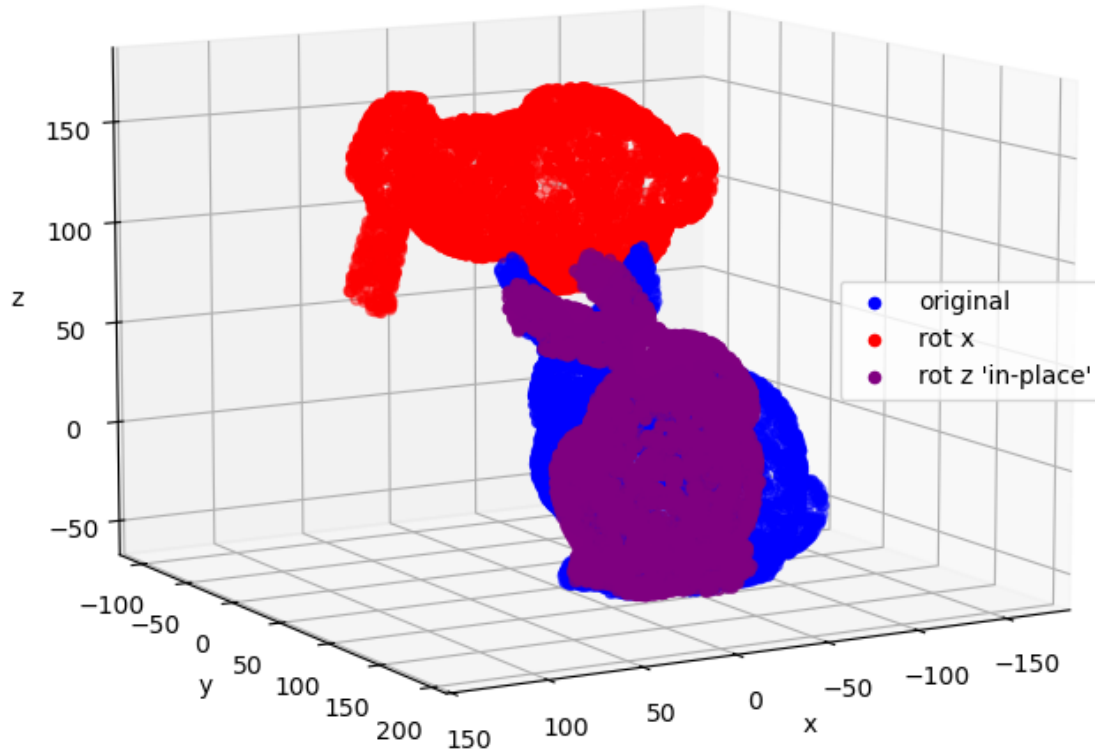
```
[3]: # Rotation around x
theta_x = np.pi/2 # or 90 degrees
R_x = np.array([
    [1., 0., 0.],
    [0., np.cos(theta_x), -np.sin(theta_x)],
    [0., np.sin(theta_x), np.cos(theta_x)]
])
points_rot_x = (R_x @ points.T).T
# Rotation around z "in-place"
theta_z = np.pi/3 # or 120 degrees
R_z = np.array([
    [np.cos(theta_z), -np.sin(theta_z), 0.],
    [np.sin(theta_z), np.cos(theta_z), 0.],
    [0., 0., 1.]
])
points_rot_z = (R_z @ centered_points.T).T + centroid
```

```

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_rot_x[:, 0], points_rot_x[:, 1], points_rot_x[:, 2],
           c='red', label="rot x")
ax.scatter(points_rot_z[:, 0], points_rot_z[:, 1], points_rot_z[:, 2],
           c='purple', label="rot z 'in-place'")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Rotated pointclouds around x, y and z axes")
plt.show()

```

Rotated pointclouds around x, y and z axes



A rotation matrix may also be computed using an arbitrary axis (i.e., different from one of the axes of the main coordinate system). Given a unit vector u (on the axis) and angle θ , the rotation matrix is written:

$$R_u(\theta_u) = \begin{pmatrix} u_x^2(1 - \cos \theta_u) + \cos \theta_u & u_x u_y(1 - \cos \theta_u) - u_z \sin \theta_u & u_x u_z(1 - \cos \theta_u) + u_y \sin \theta_u \\ u_x u_y(1 - \cos \theta_u) + u_z \sin \theta_u & u_y^2(1 - \cos \theta_u) + \cos \theta_u & u_y u_z(1 - \cos \theta_u) - u_x \sin \theta_u \\ u_x u_z(1 - \cos \theta_u) - u_y \sin \theta_u & u_y u_z(1 - \cos \theta_u) + u_x \sin \theta_u & u_z^2(1 - \cos \theta_u) + \cos \theta_u \end{pmatrix}$$

```
[4]: # Rotation around
u = np.array([np.sqrt(3)/3, np.sqrt(3)/3, np.sqrt(3)/3])
u_x, u_y, u_z = u
```

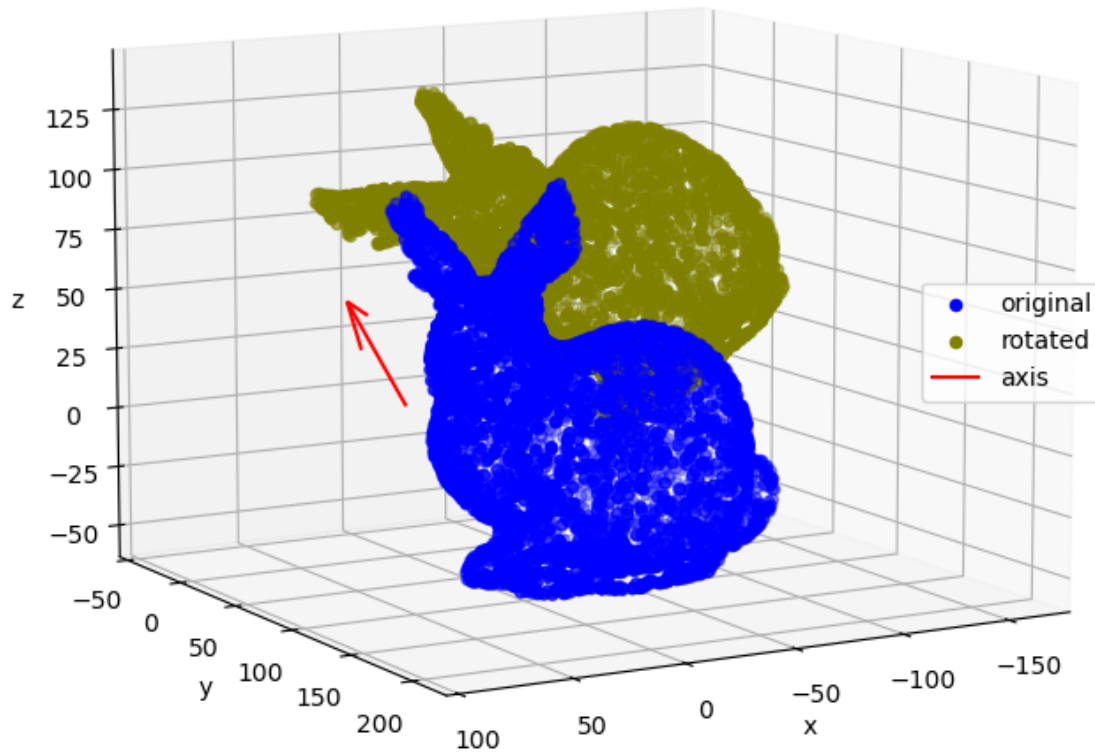
```

theta = np.pi/4 # or 45 degrees
c, s = np.cos(theta), np.sin(theta)
R_u = np.array([
    [(u_x**2)*(1-c)+c, u_x*u_y*(1-c)-u_z*s, u_x*u_z*(1-c)+u_y*s],
    [u_x*u_y*(1-c)+u_z*s, (u_y**2)*(1-c)+c, u_y*u_z*(1-c)-u_x*s],
    [u_x*u_z*(1-c)-u_y*s, u_y*u_z*(1-c)+u_x*s, (u_z**2)*(1-c)+c]
])
points_rot_u = (R_u @ points.T).T

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_rot_u[:, 0], points_rot_u[:, 1], points_rot_u[:, 2],
           c='olive', label="rotated")
ax.quiver(0., 0., 0., 100.*u_x, 100.*u_y, 100.*u_z,
          color='red', label="axis") # axis of rotation
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Rotated pointcloud around arbitrary axis")
plt.show()

```

Rotated pointcloud around arbitrary axis



It is then possible to obtain any rotation matrix from the three elemental rotation matrices seen above.

Although commonly used, this approach should be treated with caution. Remember, for example, that matrix multiplication is not commutative ($R_x.R_y \neq R_y.R_x$), so order does matter. Consequently, there are many possible rotation sequences (x-y-z, y-z-x, z-x-y, etc.). What's more, each additional rotation may also refer to either the global coordinate system (extrinsic rotation) or to the last rotated coordinate system (intrinsic rotation).

Without going into too much detail, note **there is not a single “good” way of defining a rotation matrix in terms of elemental rotations**. That's why quaternions are often preferred when compactness, efficiency, and numerical stability are important.

```

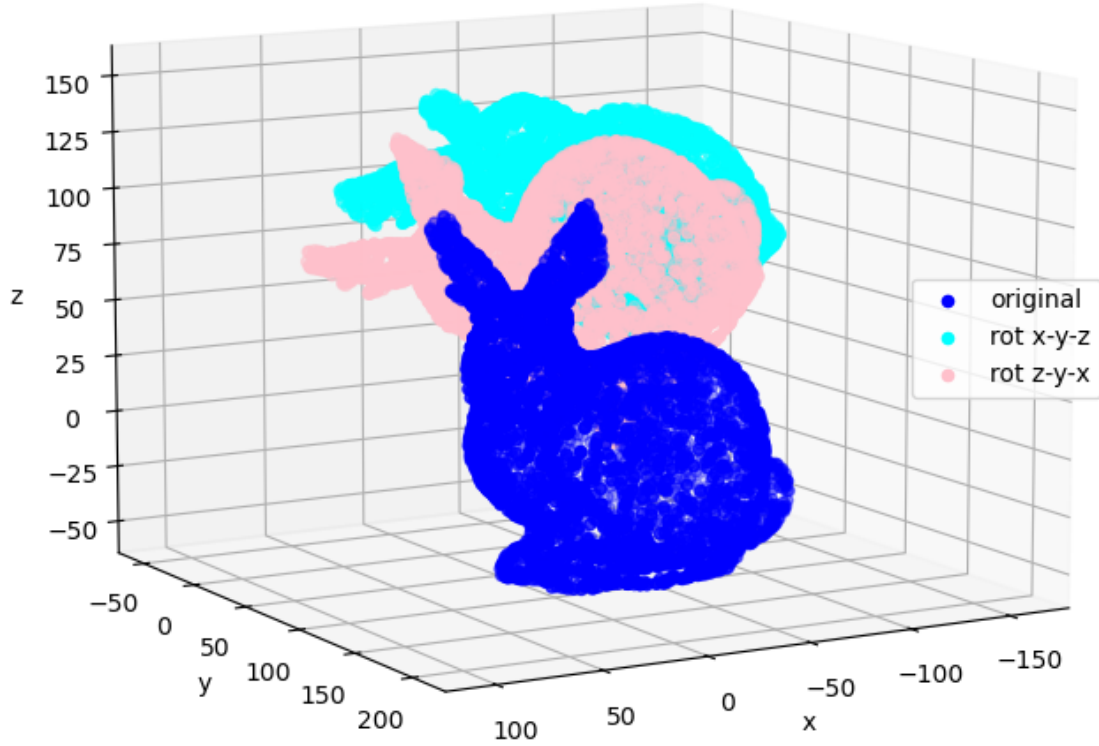
[5]: # Rotation around x
theta_x, theta_y, theta_z = np.pi/5, np.pi/6, np.pi/7
R_x = np.array([
    [1., 0., 0.],
    [0., np.cos(theta_x), -np.sin(theta_x)],
    [0., np.sin(theta_x), np.cos(theta_x)]
])
R_y = np.array([
    [np.cos(theta_y), 0., np.sin(theta_y)],
    [0., 1., 0.],
    [-np.sin(theta_y), 0., np.cos(theta_y)]
])
R_z = np.array([
    [np.cos(theta_z), -np.sin(theta_z), 0.],
    [np.sin(theta_z), np.cos(theta_z), 0.],
    [0., 0., 1.]
])
# First sequence of rotations (x-y-z)
R_xyz = R_x @ R_y @ R_z
points_rot_xyz = (R_xyz @ points.T).T
# Second sequence of rotations (z-y-x)
R_zyx = R_z @ R_y @ R_x
points_rot_zyx = (R_zyx @ points.T).T
# Comparison between the two sequences (not the same result)
print("Check if rotations R_t1 and R_t2 are the same:", np.
      ↪allclose(points_rot_xyz, points_rot_zyx))

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_rot_xyz[:, 0], points_rot_xyz[:, 1], points_rot_xyz[:, 2],
           c='cyan', label="rot x-y-z")
ax.scatter(points_rot_zyx[:, 0], points_rot_zyx[:, 1], points_rot_zyx[:, 2],
           c='pink', label="rot z-y-x")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Rotated pointclouds with different rotation sequences")
plt.show()

```

Check if rotations R_t1 and R_t2 are the same: False

Rotated pointclouds with different rotation sequences



We have seen how to compose R_t given the angles $(\theta_x, \theta_y, \theta_z)$, but the inverse operation is also possible.

Given a rotation matrix and considering (for example) the extrinsic rotation z-y-x (the corresponding angles are often called *yaw*, *pitch*, and *roll* in aeronautics), the equations is written:

$$R_t = \begin{pmatrix} r_{00} & r_{01} & r_{02} \\ r_{10} & r_{11} & r_{12} \\ r_{20} & r_{21} & r_{22} \end{pmatrix}$$

$$= \begin{pmatrix} \cos \theta_y \cos \theta_z & \sin \theta_x \sin \theta_y \cos \theta_z - \cos \theta_x \sin \theta_z & \cos \theta_x \sin \theta_y \cos \theta_z + \sin \theta_x \sin \theta_z \\ \cos \theta_y \sin \theta_z & \sin \theta_x \sin \theta_y \sin \theta_z + \cos \theta_x \cos \theta_z & \cos \theta_x \sin \theta_y \sin \theta_z - \sin \theta_x \cos \theta_z \\ -\sin \theta_y & \sin \theta_x \cos \theta_y & \cos \theta_x \cos \theta_y \end{pmatrix}$$

Using the bottom and left (simpler) terms for identification and remembering that $\tan \theta = \sin \theta / \cos \theta$, it is possible to deduce that:

$$\theta_x = \arctan2(r_{21}, r_{22}) \quad \theta_y = \arctan2(-r_{20}, \sqrt{r_{21}^2 + r_{22}^2}) \quad \theta_z = \arctan2(r_{10}, r_{00})$$

with *arctan2* being the element-wise arc tangent of the ratio of the two numbers choosing the quadrant correctly.

Note that these angles satisfy the conditions $\theta_x \in [-\pi, \pi], \theta_y \in [-\frac{\pi}{2}, \frac{\pi}{2}], \theta_z \in [-\pi, \pi]$.

```
[6]: r_00, _, _, r_10, _, _, r_20, r_21, r_22 = R_zyx.flatten()
# Compute angles
theta_x_calc = np.arctan2(r_21, r_22)
theta_y_calc = np.arctan2(-r_20, np.sqrt(r_21**2 + r_22**2))
theta_z_calc = np.arctan2(r_10, r_00)
# Compare to initial angles
print("Check if theta_x_calc and theta_x are the same:", np.allclose(theta_x,
    ↪theta_x_calc))
print("Check if theta_y_calc and theta_y are the same:", np.allclose(theta_y,
    ↪theta_y_calc))
print("Check if theta_z_calc and theta_z are the same:", np.allclose(theta_z,
    ↪theta_z_calc))
```

```
Check if theta_x_calc and theta_x are the same: True
Check if theta_y_calc and theta_y are the same: True
Check if theta_z_calc and theta_z are the same: True
```

2.3 Reflections

A reflection or “mirror” flips the pointcloud in a mirror plane so that each point is the same distance from the mirror plane as its reflected point.

As for rotations, it is possible to consider reflections in the three planes built from the main coordinate system (i.e., *Oxy*, *Oyz*, and *Ozx*). A reflection in these planes leave the coordinates of each point unchanged except in the direction of their normals (e.g., the normal of the *Oxy* plane is *y*) where the coordinate is “flipped”.

$$Rf_x = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad Rf_y = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad Rf_z = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{pmatrix}$$

In practice, matrix multiplication is once again used to flip each point of the pointcloud (do not forget that some attributes such as normals must also be flipped as well!):

$$p' = (Rf \cdot p^T)^T$$

Again, centering the pointcloud first is necessary for an “in-place” mirror operation.

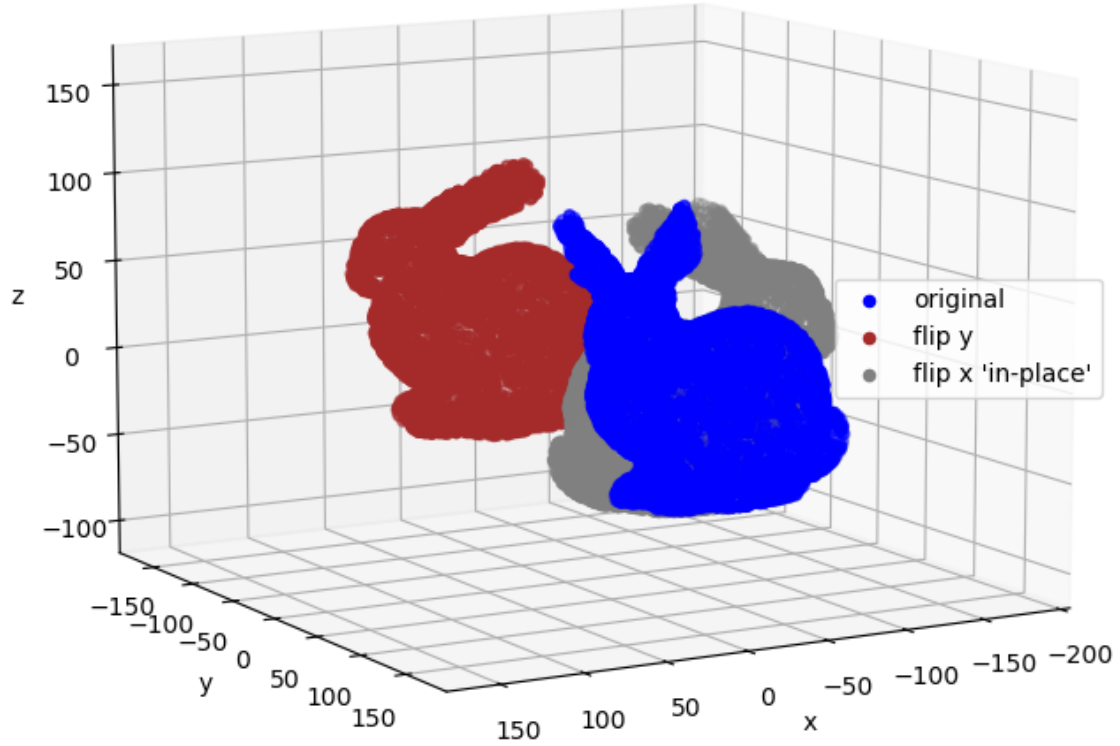
```

[7]: # Flip around y
Rf_y = np.array([
    [1., 0., 0.],
    [0., -1., 0.],
    [0., 0., 1.]
])
points_flip_y = (Rf_y @ points.T).T
# Flip around x "in-place"
Rf_x = np.array([
    [-1., 0., 0.],
    [0., 1., 0.],
    [0., 0., 1.]
])
points_flip_x = (Rf_x @ centered_points.T).T + centroid

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_flip_y[:, 0], points_flip_y[:, 1], points_flip_y[:, 2],
           c='brown', label="flip y")
ax.scatter(points_flip_x[:, 0], points_flip_x[:, 1], points_flip_x[:, 2],
           c='gray', label="flip x 'in-place'")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Reflected pointclouds around Oxy, Oyz and Ozx planes")
plt.show()

```

Reflected pointclouds around Oxy, Oyz and Ozx planes



A reflection matrix may also be computed using an arbitrary plane.

A 3D plane P is typically described by its cartesian equation $a.x + b.y + c.z + d = 0$, with a , b , and c describing its orientation and d its distance to origin. In this case, the plane normal is defined by $n = [a, b, c]$ (generally a unit vector) and any given point $p_i \in P$ verifies $p_i \cdot n = -d$.

Remembering that a projection matrix on a plane going through the origin ($d = 0$) is given by:

$$Pr_O = \frac{n^T \cdot n}{||n^2||} \iff Pr_O = n^T \cdot n \quad \text{for} \quad ||n|| = 1$$

It is possible to deduce that the reflection matrix is:

$$Rf_O = I - 2.n^T \cdot n = \begin{pmatrix} 1 - 2a^2 & -2ab & -2ac \\ -2ab & 1 - 2b^2 & -2bc \\ -2ac & -2bc & 1 - 2c^2 \end{pmatrix}$$

(which may be seen as the original points' position subtracted by two times the distance to the plane).

Hence the coordinates of the mirrored points is given by:

$$p'_O = (Rf_O \cdot p^T)^T$$

If the plane do not goes through the origin, a simple translation is added:

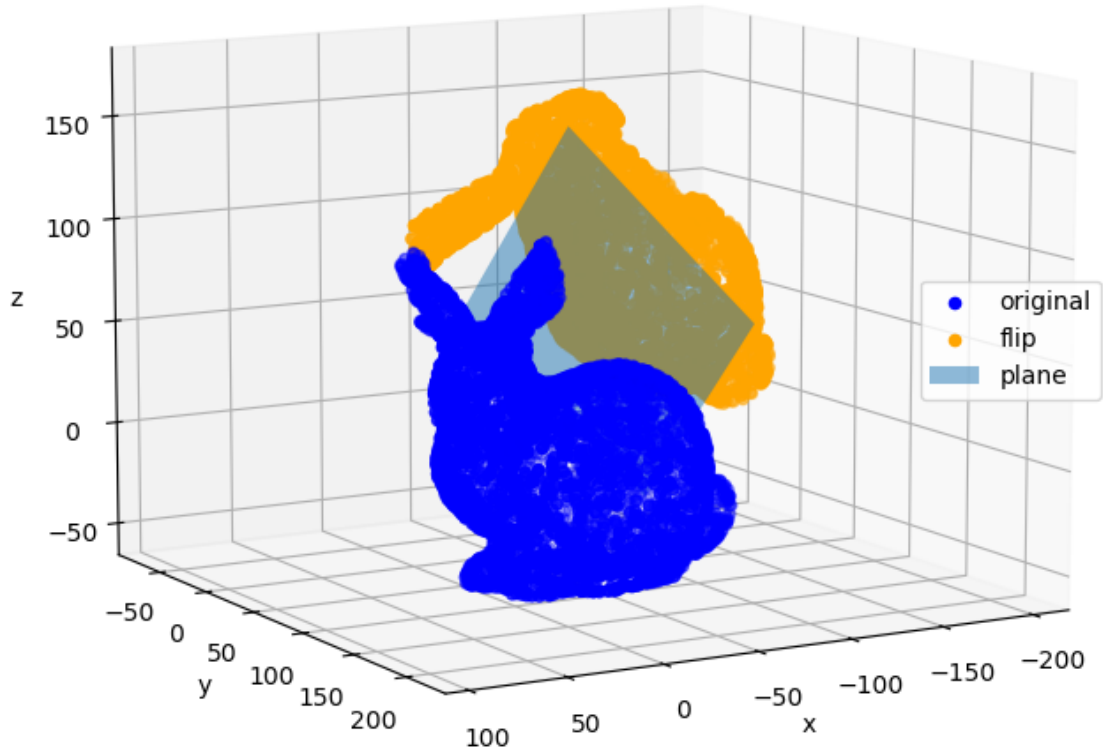
$$p' = p'_O - 2.d.n = (I - 2.n^T \cdot n) \cdot p - 2.d.n$$

Again, thanks to broadcasting this operation may be vectorized in Numpy, resulting in more readable code and faster execution.

```
[8]: # Plane coordinates
n = np.array([-np.sqrt(3)/3, -np.sqrt(3)/3, np.sqrt(3)/3])
a, b, c = n
d = -10.
# Projection
Rf_O = np.eye(3) - 2 * n.reshape(-1, 1) @ n.reshape(1, -1) # n initial shape is (3,) and the dot product needs to be (3, 1)*(1, 3)
points_flip = (Rf_O @ points.T).T - 2 * d * n

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_flip[:, 0], points_flip[:, 1], points_flip[:, 2],
           c='orange', label="flip")
# plot plane
xx, yy = np.meshgrid(np.linspace(-100, 0, 2), np.linspace(50, 150, 2))
z = -(a*xx + b*yy + d)/c
ax.plot_surface(xx, yy, z, alpha=0.5, label="plane")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Reflected pointcloud around arbitrary plane")
plt.show()
```

Reflected pointcloud around arbitrary plane



Note that, as for rotations, it is perfectly possible to sequence the reflection operations seen above.

2.4 Scaling

Scaling or resizing increases or diminishes the dimensions of the pointcloud by a scale factor.

When the scale factor is the same in all direction, scaling is said to be uniform or isotropic. In practice, the scale factor s is simply used to multiply each point of the pointcloud:

$$p' = s.p$$

When the scale factor is different in some directions, scaling is said to be non-uniform or anisotropic. Considering the three axes of the main coordinate system, we may deduce the following scaling

matrix:

$$Sc = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{pmatrix}$$

with s_x , s_y , and s_z being the scale factors in the x , y , and z direction respectively.

In practice, matrix multiplication is once again used to scale each point of the pointcloud (do not forget that some attributes such as normals must also be scaled as well!):

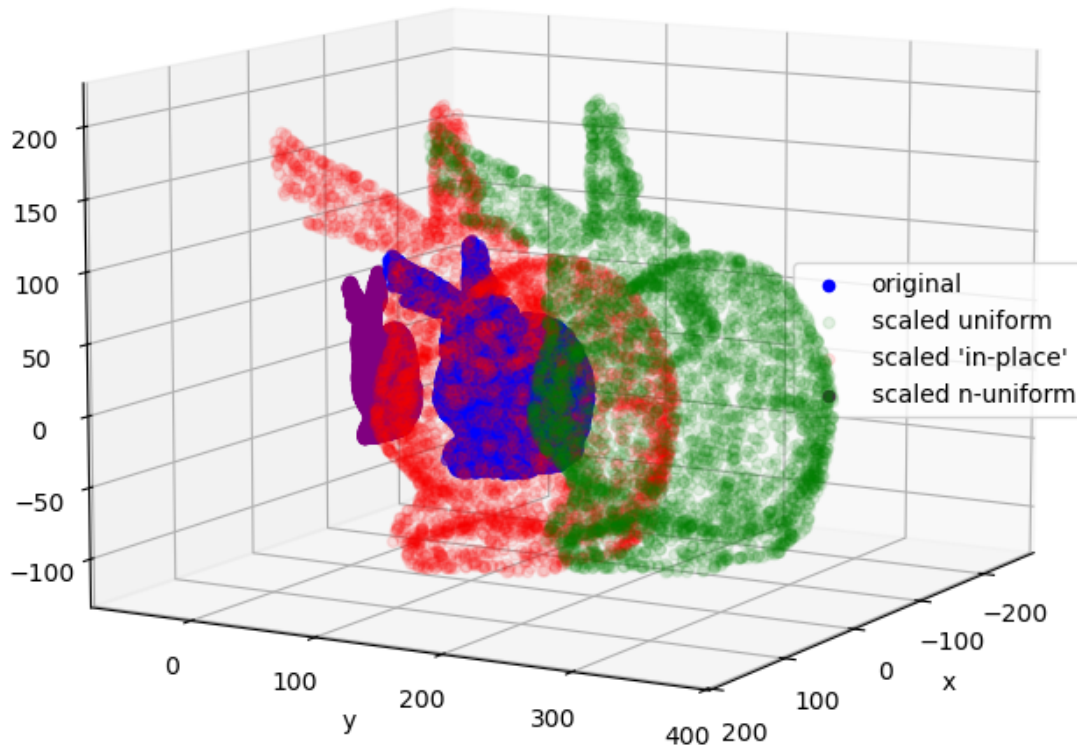
$$p' = (Sc.p^T)^T$$

Again, centering the pointcloud first is necessary for an “in-place” scaling operation. You can indeed note that the scaled pointcloud “moves away” from the original one if the latter is not centered.

```
[9]: # Uniform scaling
s = 2.
points_u_scaled_1 = s * points
points_u_scaled_2 = s * centered_points + centroid
# Non uniform scaling
Sc = np.array([
    [.5, 0., 0.],
    [0., .2, 0.],
    [0., 0., .75]
])
points_nu_scaled = (Sc @ points.T).T

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_u_scaled_1[:, 0], points_u_scaled_1[:, 1], points_u_scaled_1[:, 2],
           c='green', alpha=0.1, label="scaled uniform")
ax.scatter(points_u_scaled_2[:, 0], points_u_scaled_2[:, 1], points_u_scaled_2[:, 2],
           c='red', alpha=0.1, label="scaled 'in-place'")
ax.scatter(points_nu_scaled[:, 0], points_nu_scaled[:, 1], points_nu_scaled[:, 2],
           c='purple', label="scaled n-uniform")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 30)
plt.axis("equal")
plt.title("Scaled pointclouds")
plt.show()
```

Scaled pointclouds



2.5 Going further

The latest subsections gave a glimpse of the most commonly found pointcloud transformations (i.e., affine transformations that include translations, rotations, reflections, scaling, and shearing) and how to implement them in a quite painless manner. You may however be starting to realize that this topic is not so simple at all. What's more, some aspects have been left out, such as shearing. A way longer notebook would have been necessary to go through them all in detail.

You may have noticed that **all the discussed transformations may be expressed as a combination of matrix multiplications and vector additions**. That's because we used **Cartesian coordinates**. In computer graphics and computer vision (both fields make extensive use of pointclouds or 3D representations that share some of their properties such as meshes), **these operations are often represented using a matrix multiplication only**, allowing simpler and more efficient

processing, **thanks to Homogeneous coordinates.**

In Homogeneous coordinates, transformations are all expressed in the form of a $(4,4)$ matrix. A first consequence is that the shape of pointcloud is slightly altered, with the addition of a fourth coordinate equal to 1, such as:

$$p_i = [x_i, y_i, z_i, 1]$$

hence a basic pointcloud shape becomes $(n, 4)$ (n being the total number of points).

With this notation, a translation matrix is written:

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotations around the main axes are written:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta_x & -\sin \theta_x & 0 \\ 0 & \sin \theta_x & \cos \theta_x & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \begin{pmatrix} \cos \theta_y & 0 & \sin \theta_y & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta_y & 0 & \cos \theta_y & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \begin{pmatrix} \cos \theta_z & -\sin \theta_z & 0 & 0 \\ \sin \theta_z & \cos \theta_z & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

and so on.

No need to be surprised if you encounter Homogeneous coordinates in more advanced pointcloud processing libraries or software then.

2.6 Wrapping up

You should now have a better grasp of the fundamental spatial transformations applicable to 3D pointclouds, including translations, rotations, reflections, and scaling. These transformations are essential tools to deal with the more advanced processing algorithms that are discussed in the next notebooks.

One key takeaway is that these transformations are implemented quite simply using Numpy (again, vectorization is key to keep the code both readable and efficient). Basic manipulations of pointclouds of up to a few million points in a “reasonable” time should be possible, even with a standard laptop.

3 Spatial indexing

This notebook is about spatial indexing, which is the process of organizing points to make queries and data processing more efficient. It uses spatial data structures to quickly find subsets of points based on their location. This speeds up distance and neighborhood computations, which are key steps in many pointcloud algorithms.

After introducing the concepts of distance, neighborhoods, and the limits of brute-force search, this notebook presents three common spatial data structures: voxel grids, octrees, and kd-trees.

```
[24]: # Necessary imports
import sys
import heapq
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

sys.path.append("./utils")
from plot_utils import cuboid_to_poly3D

# Load our bunny pointcloud
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
```

3.1 About distances, neighborhoods and the “brute force” approach

3.1.1 Distances

A distance d measures how far apart two points p and q are.

The most common distance for pointclouds is the **Euclidean distance**, defined as:

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2 + (p_z - q_z)^2}$$

It corresponds to the length of the straight line between the two points.

Although Euclidean distance is used most of the time, other distance definitions are sometimes useful. For example, the **Manhattan distance** (or **1-distance**) is defined as:

$$d_1(p, q) = |p_x - q_x| + |p_y - q_y| + |p_z - q_z|$$

and the **Chebyshev distance** (or **∞ -distance**) as:

$$d_\infty(p, q) = \max(|p_x - q_x|, |p_y - q_y|, |p_z - q_z|)$$

Unless stated otherwise, distances in this notebook refer to the Euclidean distance.

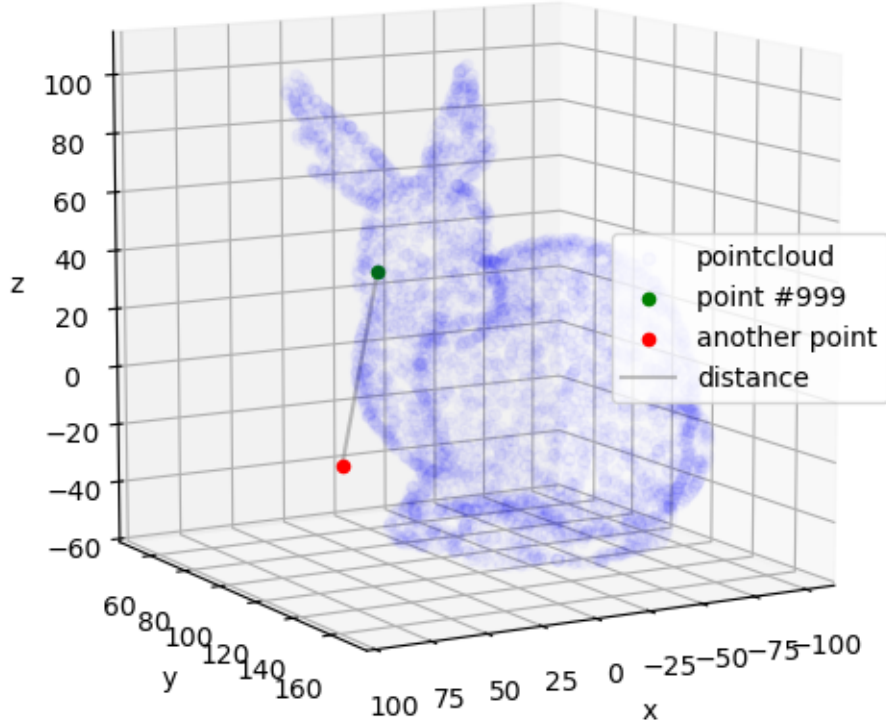
In practice, the Euclidean distance between two points is obtained by subtracting their coordinates and computing the norm of the resulting vector. NumPy provides this operation through `np.linalg.norm`.

```
[ ]: # Create another point
another_point = np.array([90., 150., -10.])
# Compute the Euclidean distance to point #999
point_999 = points[999]
dist = np.linalg.norm(point_999 - another_point)
print(f"Euclidean distance between {another_point} and {point_999} is: {dist}")

# Plot pointcloud and the two points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.02, label="pointcloud")
ax.scatter(point_999[0], point_999[1], point_999[2],
           color="green", label="point #999")
ax.scatter(another_point[0], another_point[1], another_point[2],
           color="red", label="another point")
ax.plot([another_point[0], point_999[0]], [another_point[1], point_999[1]],
        [another_point[2], point_999[2]], color='gray', alpha=0.5, label="distance")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_box_aspect([1, 1, 1])
ax.set_title("Distance between two points")
plt.show()
```

Euclidean distance between [90. 150. -10.] and [54.3485 127.7255 45.5547] is:
69.66711917820342

Distance between two points



3.1.2 Neighborhoods

A neighborhood is the set of points located close to a **query point** q .

In pointcloud processing, two types of neighborhoods are commonly used:

1. **Radius neighborhood** (or spherical neighborhood): all points located within a distance r of the query point are selected. This search is often called a **ball query**.

$$N_r(q) = \{p \in P : d(p, q) < r\}$$

The number of neighbors is not known in advance and depends on the local point density.

2. **k-nearest-neighbor neighborhood** (or k-nn): the k points closest to the query point are selected. In this case, the neighborhood always contains exactly k points, while the distance to the farthest neighbor may vary from one query point to another.

The following query point and search parameters will be used throughout the notebook: k number of neighbors and r search radius.

```
[26]: # Query point used through the notebook
query_point = another_point
# Parameters for neighborhood queries
k = 5
r = 50
```

3.1.3 The “brute-force” approach

The simplest way to compute a neighborhood is the *brute-force approach*.

Given a query point, it consists of:

1. Computing the distance to every point in the pointcloud.
2. Selecting the points that belong to the neighborhood:
 - the k closest points for a k-nn search;
 - all points within a radius r for a radius search.

This approach is easy to understand and implement, but it becomes inefficient for large pointclouds because **every query requires checking all points**, including points that are very far from the query point. For a pointcloud containing n points, computing a neighborhood for every point requires on the order of n^2 distance computations.

The main idea behind spatial structures is to **partition the 3D space into regions containing subsets of the pointcloud**. During a neighborhood search, these regions help identify where relevant points may be located, making it unnecessary to compare the query point with every point in the pointcloud.

```
[27]: def brute_force_search(points, query_point, k):
      """Find the k nearest neighbors using brute force search."""

      # Compute the distance from the query point to every point
      dists = np.linalg.norm(points - query_point, axis=1)
      # Sort the points by distance
      sorted_inds = np.argsort(dists)
      # Return the indices of the k closest points
      return sorted_inds[:k]

def brute_force_ball_search(points, query_point, r):
      """Find all points within a radius r using brute force search."""

      # Compute the distance from the query point to every point
      dists = np.linalg.norm(points - query_point, axis=1)
      # Return the indices of points within radius r
      return np.flatnonzero(dists < r)

[ ]: # Try brute force search
knn_brute_force = brute_force_search(points, query_point, k)
with np.printoptions(precision=2, suppress=True):
```

```

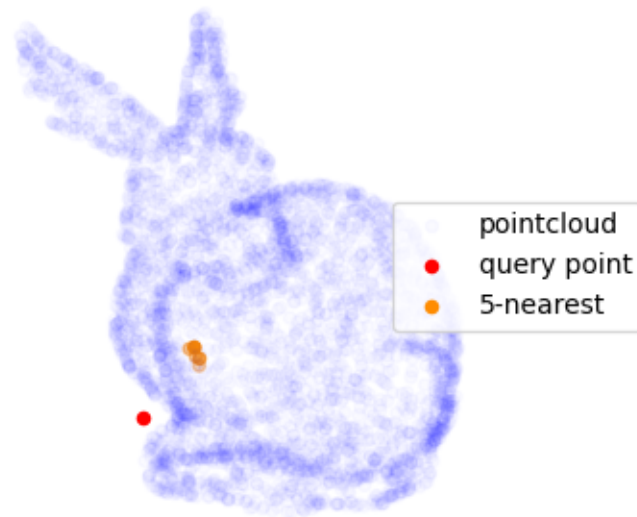
print(f"The {k}-nearest neighbors are points #", knn_brute_force)

# Plot results
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.02, label="pointcloud")
ax.scatter(query_point[0], query_point[1], query_point[2],
           color="red", label="query point")
ax.scatter(points[knn_brute_force, 0], points[knn_brute_force, 1],
           points[knn_brute_force, 2], color="darkorange", label=f"{k}-nearest")
ax.legend(loc="right")
ax.view_init(10, 50)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.set_title("k-nn search using brute force")
plt.show()

```

The 5-nearest neighbors are points # [183 893 3550 53 221]

k-nn search using brute force

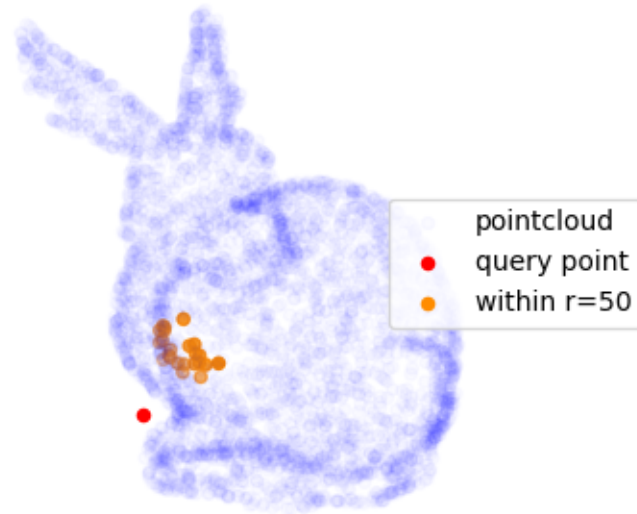


```
[ ]: # Try brute force ball search
ball_brute_force = brute_force_ball_search(points, query_point, r)
print(f"Neighbors within radius {r} are points #", ball_brute_force)

# Plot results
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.02, label="pointcloud")
ax.scatter(query_point[0], query_point[1], query_point[2],
           color="red", label="query point")
ax.scatter(points[ball_brute_force, 0], points[ball_brute_force, 1],
           points[ball_brute_force, 2], color="darkorange", label=f"within r={r}")
ax.legend(loc="right")
ax.view_init(10, 50)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.set_title("Radius search using brute force")
plt.show()
```

```
Neighbors within radius 50 are points # [ 53  183  221  268  287  436  449  893
1135 1342 1402 1900 1952 2002
2014 2216 2369 2399 3106 3550 3663 4021 4057]
```

Radius search using brute force



3.2 Voxel grids

A simple way to partition the space around a pointcloud is to use a **regular grid**. The grid is made of cells of equal size called **voxels**. A voxel is the 3D equivalent of a pixel, its name is derived from *volume* and *pixel*.

A voxel grid partitions the space into voxels and associates the points with them. In practice, it can be viewed as placing points into numbered cubic boxes.

A voxel grid is defined by its **voxel size**, that is, the edge length of each voxel. Smaller voxels result in a finer partition of the space, while larger voxels group more points together. The voxel size therefore controls the level of detail at which the pointcloud is organized and has a direct impact on the efficiency of neighborhood searches.

In the implementation below, each voxel is identified by its coordinates in the grid. For example, with a voxel size of 20, the voxel identified by $(1, 2, 3)$ starts at position $[20., 40., 60.]$ in the pointcloud coordinate system.

Building a voxel grid consists of two steps:

1. Determine the voxel containing each point.
2. Group points belonging to the same voxel.

In practice, voxel grids are often implemented using a **dictionary** (or **hash table**). The keys are voxel coordinates, such as (1,2,3), and the values are the points contained in the corresponding voxels. In this notebook, point indices are stored instead of point coordinates and only occupied voxels are stored, making the structure memory-efficient.

The example below illustrates the construction of a voxel grid in 2D. For visualization purposes, only the (x,z) coordinates are used, so voxels appear as squares rather than cubes.

```
[30]: def points_to_voxels(points, voxel_size):
        """Associate each point with a voxel."""

        # Works for both a single point and an array of points
        if points.ndim==1:
            points = points.reshape((1, -1))
        # Integer voxel coordinates
        voxel_coords = np.floor(points/voxel_size).astype(int)
        # Tuples can be used as dictionary keys
        return [tuple(c.tolist()) for c in voxel_coords]

    def create_voxel_grid(points, voxel_size):
        """Create a voxel grid containing point indices."""

        # Associate each point with a voxel
        voxel_coords = points_to_voxels(points, voxel_size)
        # Create a dictionary {voxel: [point indices]}
        voxel_grid = {}
        for i, coord in enumerate(voxel_coords):
            cell = voxel_grid.setdefault(coord, [])
            cell.append(i) # store point index in the cell

        return voxel_grid
```

```
[31]: # Voxel size used consistently below
voxel_size = 20.

# Create a 2D pointcloud by keeping one point out of eight and only the x and z
points_sample_2d = points[:, :8, [0,2]]
# Build 2D voxel grid
voxel_grid_2d = create_voxel_grid(points_sample_2d, voxel_size)

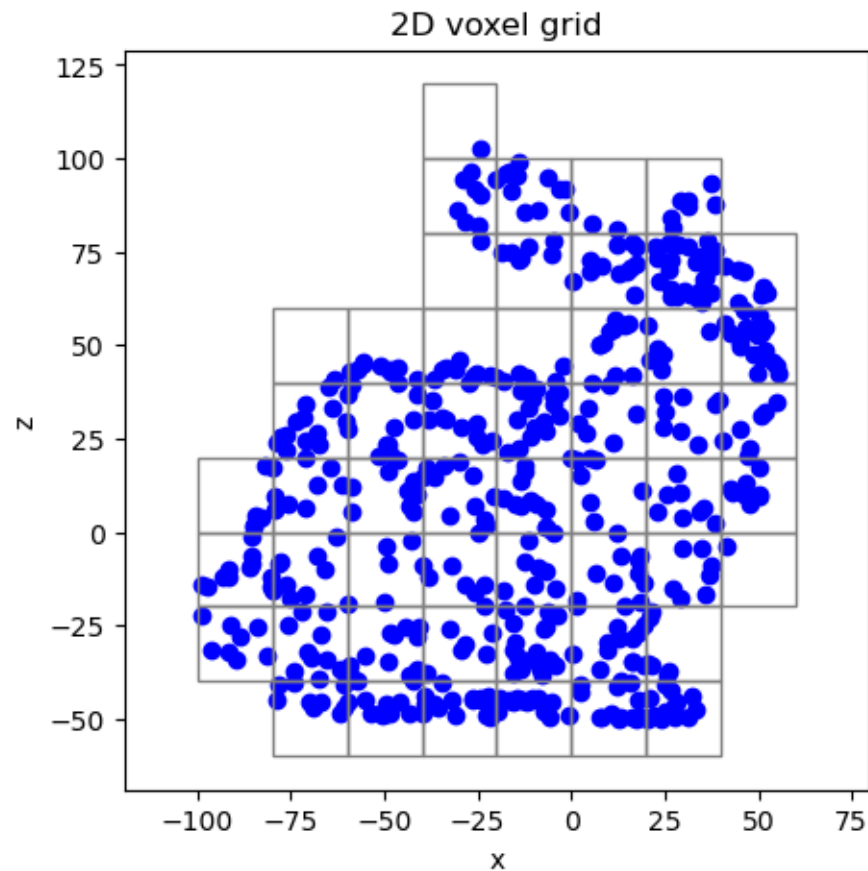
# Plot the 2D occupied cells of the voxel grid
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(points_sample_2d[:, 0], points_sample_2d[:, 1], color="blue")
# Plot the 2D cells
for key in voxel_grid_2d.keys():
```



```

    anchor_point = [voxel_size * d for d in key]
    rect = Rectangle(anchor_point, voxel_size, voxel_size,
                     color="gray", fill=False, lw=1)
    ax.add_patch(rect)
ax.set_xlabel('x')
ax.set_ylabel('z')
ax.axis("equal")
ax.set_title("2D voxel grid")
plt.show()

```



The process is exactly the same in 3D. The only difference is that squares become cubes, so each voxel is identified by three coordinates instead of two. Only occupied voxels are displayed, since empty voxels are not stored in the voxel grid.

```

[32]: # Build 3D voxel grid
voxel_grid_3d = create_voxel_grid(points, voxel_size)

# Plot the 3D occupied cells of the voxel grid
fig = plt.figure(figsize=(8, 6))

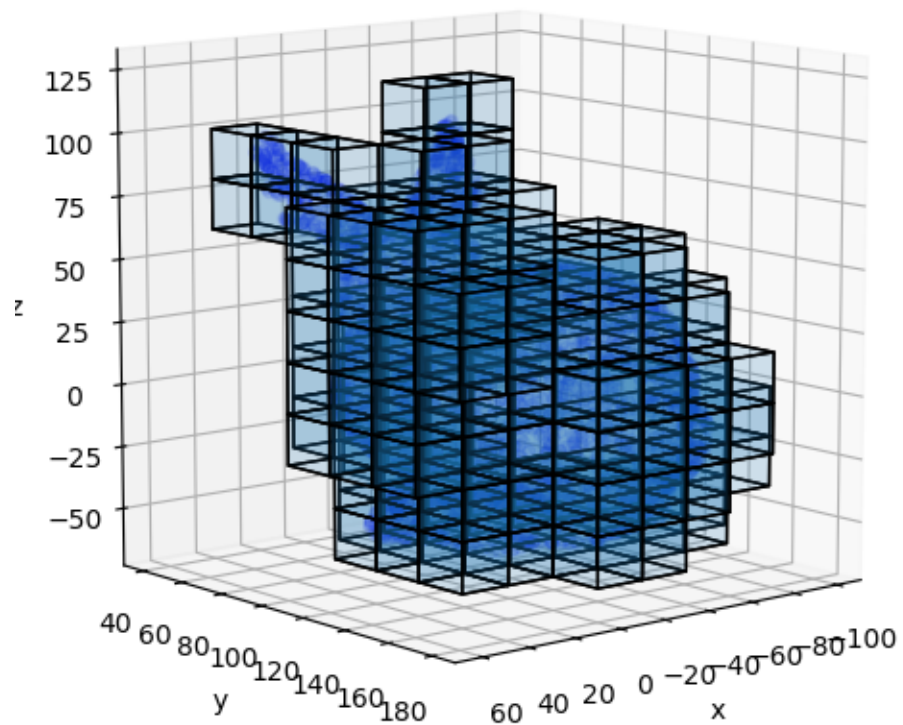
```

```

ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2], color="blue", alpha=0.1)
polys = []
for key in voxel_grid_3d.keys():
    anchor_point = [voxel_size * d for d in key]
    length = width = height = voxel_size
    polys.append(cuboid_to_poly3D(anchor_point, (length, width, height)))
polys = np.concatenate(polys)
pc = Poly3DCollection(polys, edgecolor="k", alpha=0.1)
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.view_init(10, 50)
ax.set_box_aspect([1, 1, 1])
ax.set_title("3D voxel grid")
plt.show()

```

3D voxel grid



The main advantage of a voxel grid is that **neighborhood searches only need to inspect voxels close to the query point**.

The voxel containing the query point is identified first, then neighboring voxels are explored. Using voxel coordinates, voxel neighborhoods can be defined with the **Chebyshev distance**. A 1-neighborhood contains the 26 voxels surrounding a query voxel, while neighborhoods of degree 2 and 3 contain 124 and 342 voxels, respectively.

The `VoxelGrid` class below implements this idea by storing point indices in occupied voxels and providing neighborhood search methods. Neighboring voxels are generated directly from voxel coordinates rather than by scanning the entire grid.

```
[33]: class VoxelGrid:
        """Partition 3D space into voxels."""

        class Voxel:
            """Cubic voxel."""

            def __init__(self):
                self.indices = []

        def __init__(self, points, voxel_size):

            self.points = points
            self.voxel_size = voxel_size
            self.voxels = {}
            self._build()

        def _build(self):
            """Associate each point with a voxel."""

            for i, point in enumerate(self.points):
                coords = self.voxel_coordinates(point)
                voxel = self.voxels.setdefault(coords, self.Voxel())
                voxel.indices.append(i)

        def voxel_coordinates(self, point):
            """Coordinates of voxel containing a given point."""
            return tuple(np.floor(point / self.voxel_size).astype(int))

        def voxel_neighbors(self, query_coords, degree):
            """Voxels at a given Chebyshev distance from a query voxel."""

            neighbors = []
            # Generate voxel coordinates at the requested distance
            q_x, q_y, q_z = query_coords
            for d_x in range(-degree, degree + 1):
```

```

        for d_y in range(-degree, degree + 1):
            for d_z in range(-degree, degree + 1):
                # Keep only coordinates exactly at desired distance
                if max(abs(d_x), abs(d_y), abs(d_z)) != degree:
                    continue
                coords = (q_x + d_x, q_y + d_y, q_z + d_z)
                voxel = self.voxels.get(coords)
                # Only occupied voxels are stored
                if voxel is not None:
                    neighbors.append(voxel)

    return neighbors

def knn_search(self, query_point, k):
    """Nearest neighbors search."""

    k = min(k, len(self.points)) # avoid infinite loop

    candidate_inds = []
    # Loop through voxel neighborhoods until k points are found
    query_coords = self.voxel_coordinates(query_point)
    degree = 0
    while len(candidate_inds) < k:
        for voxel in self.voxel_neighbors(query_coords, degree):
            candidate_inds.extend(voxel.indices)
        degree += 1
    # Include one extra neighborhood
    for voxel in self.voxel_neighbors(query_coords, degree):
        candidate_inds.extend(voxel.indices)
    # Brute-force search
    candidate_inds = np.asarray(candidate_inds)
    squared_dists = np.sum(
        (self.points[candidate_inds] - query_point)**2,
        axis=1,
    )
    return candidate_inds[np.argsort(squared_dists)[:k]]

def ball_search(self, query_point, radius):
    """Search neighbors within a radius."""

    candidate_inds = []
    # Loop through all voxels within radius
    query_coords = self.voxel_coordinates(query_point)
    max_degree = int(np.ceil(radius / self.voxel_size))
    for degree in range(max_degree + 1):
        for voxel in self.voxel_neighbors(query_coords, degree):

```

```

        candidate_inds.extend(voxel.indices)
    # Brute-force search
    candidate_inds = np.asarray(candidate_inds)
    squared_dists = np.sum(
        (self.points[candidate_inds] - query_point)**2,
        axis=1,
    )
    return candidate_inds[squared_dists < radius**2]

```

Given a query point, a ***k*-nearest-neighbor search** proceeds as follows:

1. Find the voxel containing the query point.
2. Explore neighboring voxels while increasing the neighborhood distance.
3. Stop once at least k candidate points have been collected.
4. Include the next neighborhood to avoid missing closer points located across voxel boundaries.
5. Perform a brute-force k -nn search on the collected points.

The first four steps identify a reduced set of candidate points. The final k -nn search is then performed only on these candidates rather than on the entire pointcloud.

To improve efficiency, the implementation above uses squared distances instead of Euclidean distances. Computing a square root is not necessary when comparing distances, as it does not affect the ordering of neighbors. This simple optimization is commonly used in practice. More advanced implementations also use smarter stopping criteria to further reduce the number of candidate points that need to be inspected.

```

[34]: # Build the voxel grid and perform a k-nn search
voxel_grid = VoxelGrid(points, voxel_size)
knn_voxel_grid = voxel_grid.knn_search(query_point, k)

print(f"The {k} nearest neighbors are points #", knn_voxel_grid)
print("Same neighbors as brute force:",
      np.array_equal(np.sort(knn_voxel_grid), np.sort(knn_brute_force)))

```

```

The 5 nearest neighbors are points # [ 183  893 3550   53  221]
Same neighbors as brute force: True

```

Given a query point, a **radius search** proceeds as follows:

1. Find the voxel containing the query point.
2. Collect the voxels that may contain points within the search radius r .
3. Perform a brute-force search on the points contained in these voxels.

As for the k -nn search, the first two steps identify a reduced set of candidate points. The final radius search is then performed only on these candidates rather than on the entire pointcloud.

To improve efficiency, the implementation above also uses squared distances and a squared search radius. This avoids computing square roots while producing exactly the same result.

```
[35]: ball_voxel_grid = voxel_grid.ball_search(query_point, r)

print(f"Points within radius {r} are points #", ball_voxel_grid)
print("Same neighbors as brute force:",
      np.array_equal(np.sort(ball_voxel_grid), np.sort(ball_brute_force)))
```

```
Points within radius 50 are points # [ 268  287 1900 2002 2014 2369 3106 3550
4057   53 183  221  436  449
      893 1135 1342 1402 1952 2216 2399 3663 4021]
Same neighbors as brute force: True
```

3.3 Octrees

An octree is a **hierarchical spatial structure** that represents 3D space through recursive subdivision into equal axis-aligned cuboids called **octants** [1].

Starting from a cube enclosing the entire pointcloud, the space is repeatedly divided into eight equal sub-cuboids. Each of these sub-cuboids can in turn be subdivided into eight smaller cuboids, and so on. The resulting hierarchy adapts naturally to the local distribution of points as dense regions are subdivided more deeply than sparse regions.

Each element of the hierarchy is called a **node**. Depending on the implementation, a node may store the points it contains, its geometric extent, and links to its children.

Before introducing octrees, it is helpful to consider their 2D counterpart, the **quadtree**. A quadtree recursively subdivides the plane into four equal regions called quadrants. The same ideas apply in 3D, except that each subdivision produces eight children instead of four.

In the implementation below, each quadtree node stores:

- The coordinates of the axis-aligned-rectangle (here the position of its bottom-left and top-right corners).
- The index of the 2D points it contains (empty in case of inner node).
- Its children (empty in case of leaf nodes).

The tree is built recursively. Starting from a root node containing all points of the pointcloud (typically its bounding box), each node is subdivided into four quadrants whenever it contains more than a given number of points. Subdivision stops when this condition is no longer met, producing the leaf nodes of the tree. As only leaf nodes store point indices, once a node is subdivided, its points are distributed among its children and the indices are removed from the parent node.

In the implementation below, node bounds are inclusive on their lower side and exclusive on their upper side. The root bounding box is therefore slightly enlarged to ensure that points lying on the maximum coordinates are correctly assigned to a leaf node.

```
[36]: class QuadTree:
      """Partition 2D space by recursively subdividing it into four quadrants."""

      class QuadNode:
          """A node of our QuadTree or quadrant."""
```

```

def __init__(self, bound_min, bound_max, indices):

    self.bound_min = bound_min # Bottom-left corner
    self.bound_max = bound_max # Top-right corner
    self.indices = indices # Point indices
    self.children = [] # Leaf

@property
def geometry(self):
    """Geometry of the node as an axis-aligned rectangle."""

    anchor_point = self.bound_min
    length, width = self.bound_max - self.bound_min

    return anchor_point, length, width

@property
def is_leaf(self):
    """A terminal/leaf node does not have children."""

    return not len(self.children)

def __init__(self, points, max_leaf_size=10):

    self.points = points
    self.max_leaf_size = max_leaf_size
    # Create root node
    bound_min = points.min(axis=0)
    bound_max = points.max(axis=0) + 1e-8 # slightly enlarge
    indices = np.arange(len(points), dtype=int)
    self.root = QuadTree.QuadNode(bound_min, bound_max, indices)
    # Recursively build the tree
    self._split_node(self.root)

def _split_node(self, node):
    """Recursively split a node into four subnodes/children."""

    # Do not split if node is already too small
    if len(node.indices) <= self.max_leaf_size:
        return
    # Compute the bounds of the four candidate subnodes
    x_center, y_center = (node.bound_min + node.bound_max)/2
    x_min, y_min = node.bound_min
    x_max, y_max = node.bound_max
    new_bounds = [
        # South-West
        (np.array([x_min, y_min]), np.array([x_center, y_center])),

```

```

        # North-West
        (np.array([x_min, y_center]), np.array([x_center, y_max])),
        # North-East
        (np.array([x_center, y_center]), np.array([x_max, y_max])),
        # South-East
        (np.array([x_center, y_min]), np.array([x_max, y_center]))
    ]

    # Compute the node points falling into each subnode
    for (bound_min_new, bound_max_new) in new_bounds:
        in_subnode = np.logical_and(
            (self.points[node.indices] >= bound_min_new).all(axis=1),
            (self.points[node.indices] < bound_max_new).all(axis=1)
        )
        # A subnode is created only if it contains some points
        if np.any(in_subnode):
            subnode = QuadTree.QuadNode(
                bound_min_new,
                bound_max_new,
                node.indices[in_subnode]
            )
            node.children.append(subnode)
            # Recursively subdivide until leaves are small enough
            self._split_node(subnode)
        # Clear node indices after the node is split (only store for leaves)
        node.indices = []

    @property
    def nodes(self):
        """List of QuadNodes composing our QuadTree."""

        quadtree_nodes = [self.root]
        # Breadth-first traversal
        queue = [self.root]
        while queue:
            node = queue.pop(0)
            for subnode in node.children:
                quadtree_nodes.append(subnode)
                queue.append(subnode)

        return quadtree_nodes

```

As observed below, the size of the quadrants adapts to the local point density. Dense regions are subdivided more deeply, resulting in smaller quadrants, while sparse regions are represented by larger quadrants.

```

[37]: # Build the quadtree
quadtree = QuadTree(points_sample_2d)

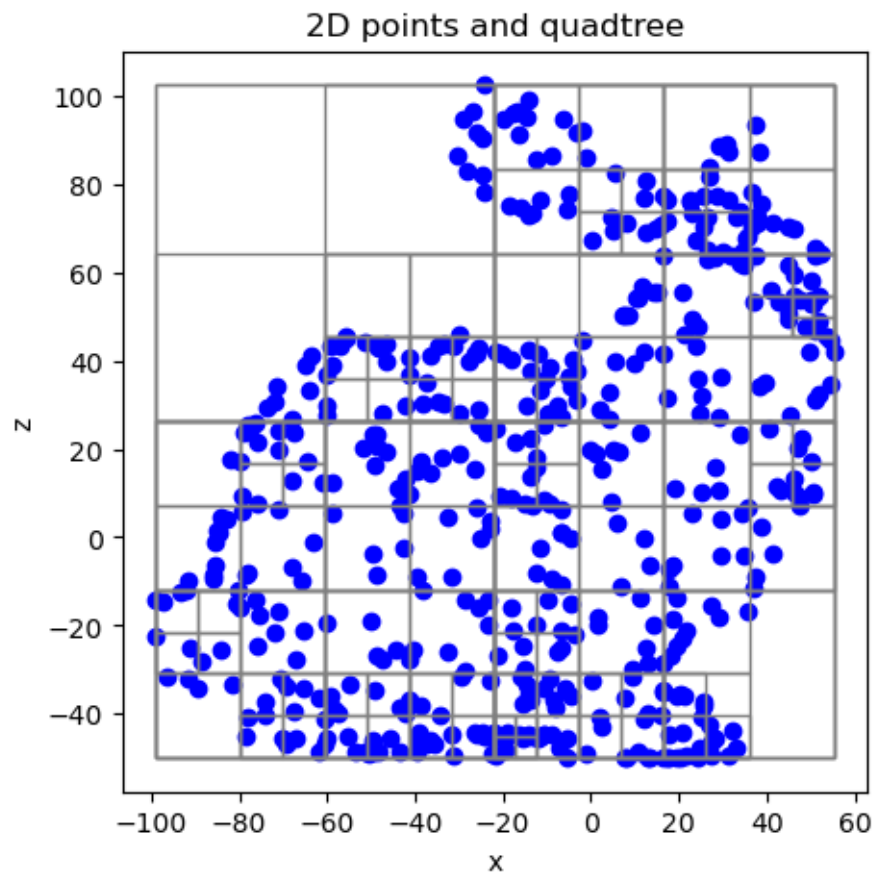
```



```

# Plot the 2D points with the quadtree
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot()
ax.scatter(points_sample_2d[:, 0], points_sample_2d[:, 1], color="blue")
for node in quadtree.nodes:
    anchor_point, length, width = node.geometry
    rect = Rectangle(anchor_point, length, width,
                    color="gray", fill=False, lw=1)
    ax.add_patch(rect)
ax.set_xlabel('x')
ax.set_ylabel('z')
ax.axis("equal")
ax.set_title("2D points and quadtree")
plt.show()

```



The same principles can be used to construct an octree. Instead of recursively subdividing the space into four quadrants, each node is subdivided into eight octants.

The implementation below also includes a maximum subdivision depth to prevent the tree from

growing indefinitely.

```
[ ]: class OctTree:
    """Partition 3D space by recursively subdividing it into eight octants."""

    class OctNode:
        """A node of our Octree or octant."""

        def __init__(self, depth, bound_min, bound_max, indices):

            self.depth = depth # Distance from the root
            self.bound_min = bound_min # Bottom-left corner
            self.bound_max = bound_max # Top-right corner
            self.indices = indices # Point indices
            self.children = [] # Leaf by default

        @property
        def geometry(self):
            """Geometry of the node as a rectangular cuboid."""

            anchor_point = self.bound_min
            length, width, height = self.bound_max - self.bound_min

            return anchor_point, length, width, height

        @property
        def is_leaf(self):
            """A terminal/leaf node does not have children."""

            return not len(self.children)

        def distance(self, point):
            """Distance between a point and the cuboid."""

            offset = np.max(
                [[0., 0., 0.],
                 self.bound_min - point,
                 point - self.bound_max],
                axis=0
            )
            # Squared distances for efficiency
            return np.sum(offset**2)

    def __init__(self, points, max_leaf_size=10, max_depth=10):

        self.points = points
        self.max_leaf_size = max_leaf_size
```

```

self.max_depth = max_depth
# Create root node
bound_min = points.min(axis=0)
bound_max = points.max(axis=0) + 1e-8 # slightly enlarge
indices = np.arange(len(points), dtype=int)
self.root = OcTree.OcTreeNode(0, bound_min, bound_max, indices)
# Recursively build the tree
self._split_node(self.root)

def _split_node(self, node):
    """Recursively split a node into eight subnodes/children."""

    # Do not split if already too small or if reached the maximum depth
    if (len(node.indices) <= self.max_leaf_size
        or node.depth >= self.max_depth):
        return
    # Compute the bounds of the eight candidate subnodes
    x_center, y_center, z_center = (node.bound_min + node.bound_max)/2
    x_min, y_min, z_min = node.bound_min
    x_max, y_max, z_max = node.bound_max
    new_bounds = [
        # South-West-Back
        (np.array([x_min, y_min, z_min]),
         np.array([x_center, y_center, z_center])),
        # North-West-Back
        (np.array([x_min, y_min, z_center]),
         np.array([x_center, y_center, z_max])),
        # North-East-Back
        (np.array([x_min, y_center, z_center]),
         np.array([x_center, y_max, z_max])),
        # South-East-Back
        (np.array([x_min, y_center, z_min]),
         np.array([x_center, y_max, z_center])),
        # South-West-Front
        (np.array([x_center, y_min, z_min]),
         np.array([x_max, y_center, z_center])),
        # North-West-Front
        (np.array([x_center, y_min, z_center]),
         np.array([x_max, y_center, z_max])),
        # North-East-Front
        (np.array([x_center, y_center, z_center]),
         np.array([x_max, y_max, z_max])),
        # South-East-Front
        (np.array([x_center, y_center, z_min]),
         np.array([x_max, y_max, z_center]))
    ]
    # Compute the node points falling into each subnode

```

```

for (bound_min_new, bound_max_new) in new_bounds:
    in_subnode = np.logical_and(
        (self.points[node.indices] >= bound_min_new).all(axis=1),
        (self.points[node.indices] < bound_max_new).all(axis=1)
    )
    # A subnode is created only if it contains some points
    if np.any(in_subnode):
        subnode = OctTree.OctNode(
            node.depth + 1, # Increment depth
            bound_min_new,
            bound_max_new,
            node.indices[in_subnode]
        )
        node.children.append(subnode)
        # Recursively subdivide until leaves are small enough
        self._split_node(subnode)
    # Clear node indices after the node is split (only store for leaves)
    node.indices = []

@property
def nodes(self):
    """List of OctNodes composing our OctTree."""

    octree_nodes = [self.root]
    # Breadth-first search
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        for subnode in node.children:
            octree_nodes.append(subnode)
            queue.append(subnode)

    return octree_nodes

def knn_search(self, query_point, k):
    """Nearest neighbors search"""

    # Store the list of nearest neighbors as (distance, index) using a
    # max-heap so that the farthest neighbor is always the first element
    # (distance need to be negated to use the min-heap of heapq)
    nearest_neighbors = []
    # Store the list of nodes to explore as (distance, node) using a
    # min-heap so that the closest node is always the first element
    priority_queue = []
    heapq.heappush(priority_queue, (0., self.root))
    while priority_queue:
        # Select the node with the smallest distance

```

```

dist, node = heapq.heappop(priority_queue)
# Skip nodes that are too far during backtracking
if len(nearest_neighbors) == k and dist > -nearest_neighbors[0][0]:
    continue
# If it's a leaf node, check all points in it
if node.is_leaf:
    squared_dists = np.sum(
        (self.points[node.indices] - query_point)**2,
        axis=1
    )
    for ind, dist in zip(node.indices, squared_dists):
        # Add the point to the list of nearest neighbors
        if len(nearest_neighbors) < k:
            heapq.heappush(nearest_neighbors, (-dist, ind))
        # if already neighbors, check if worth adding
        else:
            # replaces the current farthest neighbor
            if dist < -nearest_neighbors[0][0]:
                heapq.heapreplace(nearest_neighbors, (-dist, ind))
# Else add its children to the priority queue
    else:
        for child in node.children:
            # If not k neighbors yet, add all children to the queue
            child_dist = child.distance(query_point)
            heapq.heappush(priority_queue, (child_dist, child))

# Extract the indices of the k nearest neighbors
d, i = zip(*sorted(nearest_neighbors, reverse=True))

return np.sqrt(-np.asarray(d)), np.array(i)

def ball_search(self, query_point, r):
    """Search neighbors within a radius"""

    neighbors_within_radius = []
    # Dive into the tree
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        # Only consider nodes that are within a radius r of the query point
        if node.distance(query_point) <= r**2: # distances are squared
            # If it's a leaf node, check all points in it
            if node.is_leaf:
                # Compute distances to the query point
                squared_dists = np.sum(
                    (self.points[node.indices] - query_point)**2,
                    axis=1

```

```

    )
    # Select points within a radius r
    inds_within_r = node.indices[
        np.argwhere(squared_dists < r**2).flatten()
    ]
    # Add to list of neighbors
    neighbors_within_radius.extend(inds_within_r.tolist())
    # Else, add child nodes to the queue
    else:
        queue.extend(node.children)

    return neighbors_within_radius

```

As observed below, octant size adapts to the local point density. Dense regions are subdivided more deeply, resulting in smaller octants, while sparse regions are represented by larger octants.

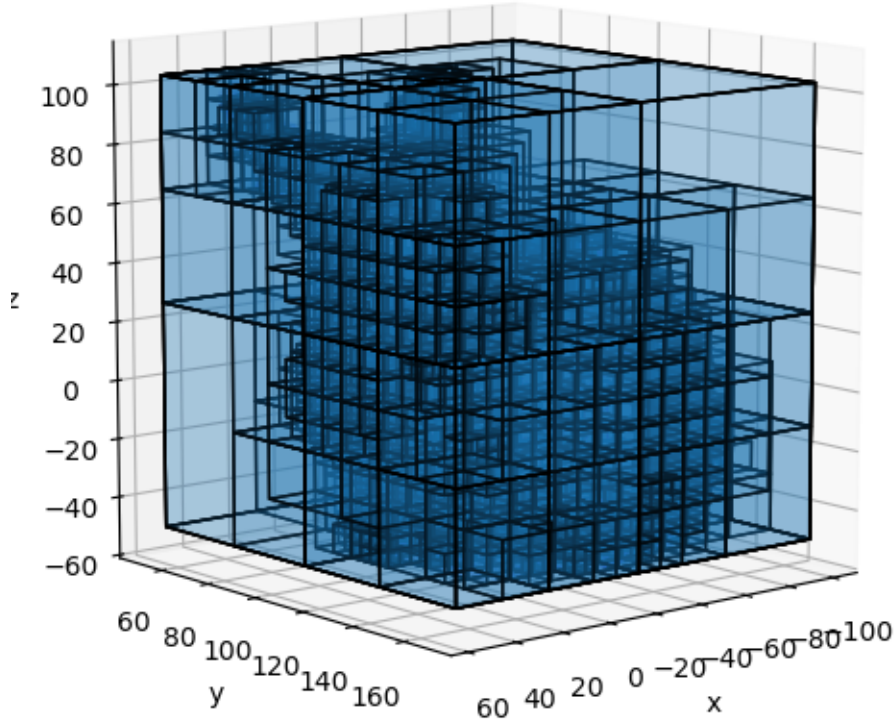
```

[39]: # Build octree with our 3D pointcloud
octree = OcTree(points)

# Plot the octree nodes as cuboids
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.1)
polys = []
for octnode in octree.nodes:
    anchor_point, length, width, height = octnode.geometry
    polys.append(cuboid_to_poly3D(anchor_point, (length, width, height)))
polys = np.concatenate(polys)
pc = Poly3DCollection(polys, edgecolor="k", alpha=0.1)
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.view_init(10, 50)
ax.set_box_aspect([1, 1, 1])
plt.title("3D points and octree")
plt.show()

```

3D points and octree



The main advantage of an octree is that it adapts to the local point density. Dense regions are subdivided into many small octants, while sparse regions are represented by fewer and larger octants.

A ***k*-nearest-neighbor search** proceeds as follows:

1. Start from the root node.
2. Visit the octants that are closest to the query point first.
3. When a leaf node is reached, compare the query point with the points it contains.
4. Keep track of the current *k* nearest neighbors found so far.
5. Skip octants that cannot contain closer neighbors.

This allows large parts of the pointcloud to be ignored during the search.

To decide whether an octant should be explored, the distance between the query point and the octant is computed. This distance is zero when the point lies inside the octant and increases as the point moves away from it.

For an octant with bounds (b_{min}, b_{max}) , the distance is given by:

$$d(p, C) = \sqrt{d_x^2 + d_y^2 + d_z^2}$$

where

$$d_x = \max\{0, b_{\min,x} - p_x, p_x - b_{\max,x}\}$$

and similarly for d_y and d_z .

In practice, the implementation below uses squared distances to avoid unnecessary square-root computations.

```
[40]: _, knn_octree = octree.knn_search(query_point, k)

print(f"The {k} nearest neighbors are points #", knn_octree)
print("Same neighbors as brute force:",
      np.array_equal(np.sort(knn_octree), np.sort(knn_brute_force)))
```

The 5 nearest neighbors are points # [183 893 3550 53 221]

Same neighbors as brute force: True

A **radius search** proceeds as follows:

1. Start from the root node.
2. Ignore octants whose distance to the query point is greater than the search radius r .
3. When a leaf node is reached, compare the query point with the points it contains.
4. Keep only the points whose distance to the query point is smaller than r .

As for the k-nn search, large parts of the tree can be skipped during the search. Only octants that may contain points within the search radius need to be explored.

The distance between the query point and an octant is computed as described above.

```
[41]: ball_octree = octree.ball_search(query_point, r)

print(f"Points within radius {r} are points #", ball_octree)
print("Same neighbors as brute force:",
      np.array_equal(np.sort(ball_octree), np.sort(ball_brute_force)))
```

Points within radius 50 are points # [4021, 449, 1952, 2216, 3663, 1402, 4057, 221, 2014, 2369, 3550, 268, 3106, 287, 436, 1135, 1342, 1900, 2002, 2399, 53, 183, 893]

Same neighbors as brute force: True

3.4 kd-trees

A kd-tree is a **binary tree data structure** designed to organize points in a k -dimensional space through recursive space partitioning [2].

Unlike an octree, which divides the space into eight regions at each subdivision, a kd-tree splits the space into only two regions at a time. Each subdivision is performed along a single dimension, with the splitting dimension changing as the tree grows. The partition is usually chosen so that each child node contains approximately the same number of points. This results in a relatively balanced tree, in which most leaves are located at a similar depth.

In the implementation below, each node of the kd-tree stores:

- The indices of the points it contains (empty in case of inner node);
- Its child nodes (usually named “left” and “right”, empty in case of leaf nodes);
- The splitting dimension (empty in case of leaf nodes);
- The splitting value (empty in case of leaf nodes).

As for the octree implementation, only leaf nodes store point indices. Once a node is subdivided, its points are distributed among its children and the indices are removed from the parent node. Unlike the octree, nodes do not explicitly store their geometric extent. The corresponding regions can be reconstructed from the sequence of splits applied from the root node. The tree is built as follows:

1. Create a root node containing all points of the pointcloud.
2. Select a splitting dimension.
3. Compute the median along this dimension.
4. Split the points into two groups around the median.
5. Repeat the last steps recursively until each leaf contains no more than a given number of points.

Other stopping criteria may also be used, such as limiting the maximum depth of the tree.

```
[ ]: class KdTree:

    class KdNode:

        def __init__(self, indices):

            self.indices = indices # indices of points it contains
            # Assume terminal by default
            self.left = None # Left child
            self.right = None # Right child
            self.split_dim = None # Splitting dimension
            self.split_val = None # Splitting value

        @property
        def is_leaf(self):
            """A terminal/leaf node does not have children."""
            return self.left is None and self.right is None

    def __init__(self, points, max_leaf_size=10):

        self.points = points
        self.max_leaf_size = max_leaf_size
        self.n_dims = points.shape[-1] # the dimension or k of the kd-tree
        self.bounding_box = (points.min(axis=0), points.max(axis=0))
        # Create root node
        indices = np.arange(len(points), dtype=int)
        self.root = KdTree.KdNode(indices)
        # Recursively build the tree
```

```

self._split_node(self.root, split_dim=0)

def _split_node(self, node, split_dim):
    """Recursively split a node into two subnodes."""

    # Do not split if node is already too small
    if len(node.indices) <= self.max_leaf_size:
        return
    # Store the split dimension
    node.split_dim = split_dim
    # Compute the split value
    split_val = np.median(self.points[node.indices, split_dim])
    node.split_val = split_val
    # Compute the node points falling into the left and right subnodes
    in_left_subnode = self.points[node.indices, split_dim] <= split_val
    in_right_subnode = self.points[node.indices, split_dim] > split_val
    # Avoid degenerate split and (infinite recursion)
    if np.all(in_left_subnode) or np.all(in_right_subnode):
        return
    left_subnode = KdTree.KdNode(node.indices[in_left_subnode])
    right_subnode = KdTree.KdNode(node.indices[in_right_subnode])
    node.left = left_subnode
    node.right = right_subnode
    # Split subnodes further (until children are too small)
    new_split_dim = (split_dim + 1) % self.n_dims
    self._split_node(left_subnode, new_split_dim)
    self._split_node(right_subnode, new_split_dim)
    # Clear node indices after the node is split (only store for leaves)
    node.indices = []

@property
def nodes(self):
    """List of KdNodes composing our KdTree."""

    kdtree_nodes = [self.root]
    # Explore the tree using breadth-first search
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        if not node.is_leaf:
            kdtree_nodes.append(node.left)
            queue.append(node.left)
            kdtree_nodes.append(node.right)
            queue.append(node.right)

    return kdtree_nodes

```

```

@property
def nodes_geometry(self):
    """Hyper-rectangle geometry of the KdTree nodes."""

    # Store the hyper-rectangle of the root node to start
    hyper_rectangles = []
    # Explore the tree using breadth-first search
    queue = [(self.root, self.bounding_box)]
    while queue:
        node, bounds = queue.pop(0)
        # Store the hyper-rectangle of the current node
        b_min, b_max = bounds
        hyper_rectangles.append((b_min, b_max - b_min))
        # Compute the bound of the two subnodes
        if not node.is_leaf:
            b_mid_min = np.copy(b_min)
            b_mid_min[node.split_dim] = node.split_val
            b_mid_max = np.copy(b_max)
            b_mid_max[node.split_dim] = node.split_val
            left_bounds = (b_min, b_mid_max)
            right_bounds = (b_mid_min, b_max)
            # Add the two subnodes to the queue
            queue.append((node.left, left_bounds))
            queue.append((node.right, right_bounds))

    return hyper_rectangles

def knn_search(self, query_point, k):
    """Nearest neighbors search."""

    # Store the list of nearest neighbors as (distance, index) using a
    # max-heap so that the farthest neighbor is always the first element
    # (distance need to be negated to use the min-heap of heapq)
    nearest_neighbors = []
    # Store nodes to explore in order of increasing distance
    priority_queue = []
    heapq.heappush(priority_queue, (0., self.root))
    while priority_queue:
        # Select the node with the smallest distance
        dist, node = heapq.heappop(priority_queue)
        # Skip nodes that are too far during backtracking
        if len(nearest_neighbors) == k and dist > -nearest_neighbors[0][0]:
            continue
        # If it's a leaf node, check all points in it
        if node.is_leaf:
            squared_dists = np.sum(
                (self.points[node.indices] - query_point)**2,

```

```

        axis=1
    )
    for ind, dist in zip(node.indices, squared_dists):
        # Add the point to the list of nearest neighbors
        if len(nearest_neighbors) < k:
            heapq.heappush(nearest_neighbors, (-dist, ind))
        # if already k neighbors, check if is worth adding
        else:
            # replaces the current farthest neighbor
            if dist < -nearest_neighbors[0][0]:
                heapq.heapreplace(nearest_neighbors, (-dist, ind))
    # Else add its children to the priority queue
    else:
        diff = query_point[node.split_dim] - node.split_val
        # Add child node on the same side as the query point first
        if diff <= 0:
            close, far = node.left, node.right
        else:
            close, far = node.right, node.left
        # Add the closest child node to the queue
        heapq.heappush(priority_queue, (0., close))
        # Check if we need to explore the other child node
        heapq.heappush(priority_queue, (diff**2, far))

    # Extract the indices of the k nearest neighbors
    d, i = zip(*sorted(nearest_neighbors, reverse=True))

    return np.sqrt(-np.asarray(d)), np.array(i)

def ball_search(self, query_point, r):
    """Search neighbors within a radius."""

    # If the query point is too far from the bounding box, return empty
    bound_min, bound_max = self.bounding_box
    deltas = np.max([
        [0., 0., 0.],
        bound_min - query_point,
        query_point - bound_max
    ], axis=0)
    dist_to_bbox = np.sum(deltas**2)
    if dist_to_bbox > r**2: # squared distances
        return np.array([])

    neighbors_within_radius = []
    # Explore the tree breadth-first
    queue = [self.root]
    while queue:

```

```

        # Select the node with the smallest distance
        node = queue.pop(0)
        # If it's a leaf node, check all points in it
        if node.is_leaf:
            # Compute distances to the query point
            squared_dists = np.sum(
                (self.points[node.indices] - query_point)**2,
                axis=1
            )
            # Select points within a radius r
            inds_within_r = node.indices[squared_dists < r**2]
            # Add to list of neighbors
            neighbors_within_radius.extend(inds_within_r.tolist())

        # Else add its children to the queue
        else:
            diff = query_point[node.split_dim] - node.split_val
            if diff <= 0:
                close, far = node.left, node.right
            else:
                close, far = node.right, node.left

            if abs(diff) <= r:
                queue.append(close)
                queue.append(far)
            else:
                queue.append(close)

    return np.array(neighbors_within_radius)

```

As observed below, the size and shape of the cells are not uniform. Because kd-trees split one dimension at a time, some cells may become long and thin.

```

[ ]: # Build the kd-tree
kdtree = KdTree(points_sample_2d)

# Plot the points and kd-tree
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot()
ax.scatter(points_sample_2d[:, 0], points_sample_2d[:, 1],
           color="blue")

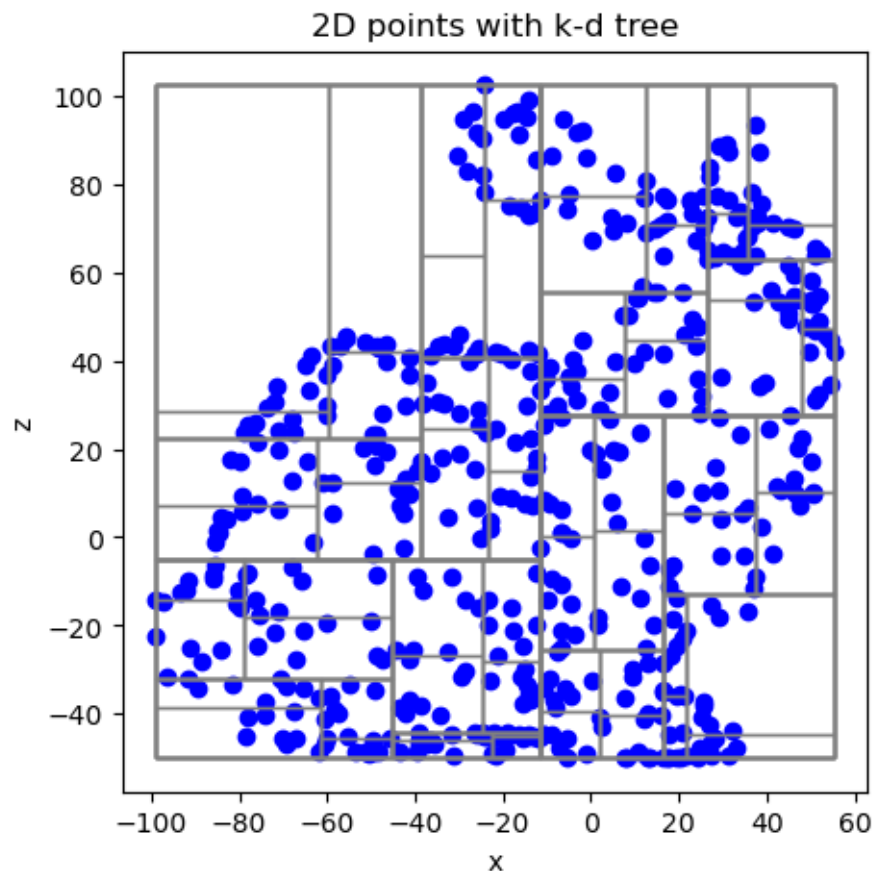
# Plot the kdtree nodes as rectangles
for hyper_rectangle in kdtree.nodes_geometry:
    anchor_point, dimensions = hyper_rectangle
    length, width = dimensions
    # kd-tree nodes as rectangles
    rect = Rectangle(anchor_point, length, width,

```

```

        color="gray", fill=False, lw=1)
    ax.add_patch(rect)
    ax.set_xlabel('x')
    ax.set_ylabel('z')
    plt.axis("equal")
    plt.title("2D points with kd-tree")
    plt.show()

```



The same applies in 3D. Unlike octrees, which produce regular subdivisions, kd-trees often generate cuboids of very different sizes and shapes.

```

[ ]: # Build the kd-tree
kdtree = KdTree(points)

# Plot the points and kdtree
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.1)

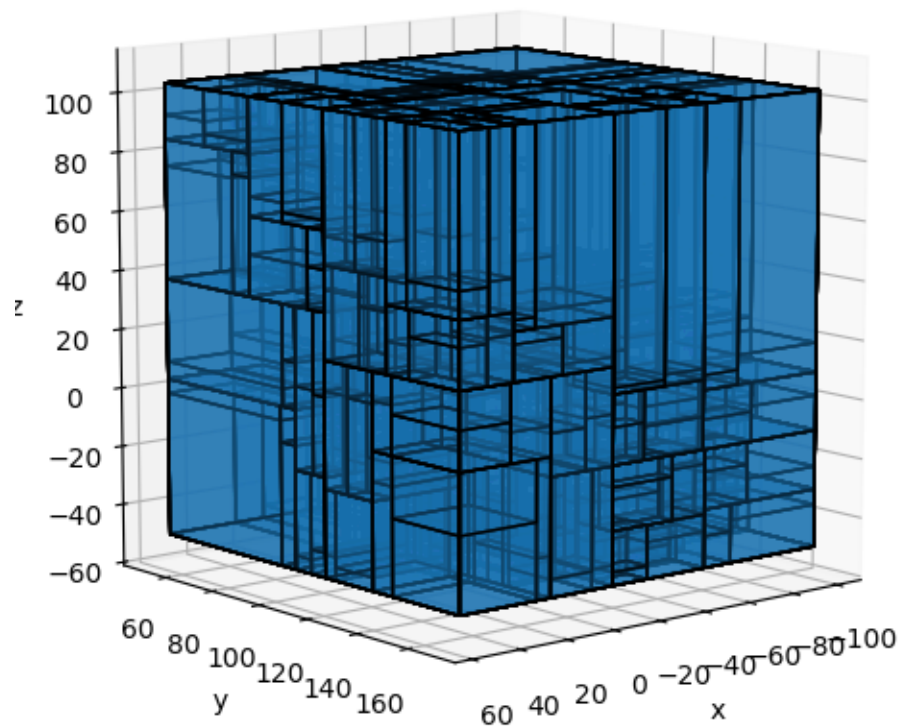
```

```

# Plot the kdtree nodes as cuboids
polys = []
for hyper_rectangle in kdtree.nodes_geometry:
    anchor_point, dimensions = hyper_rectangle
    length, width, height = dimensions
    polys.append(cuboid_to_poly3D(anchor_point, (length, width, height)))
polys = np.concatenate(polys)
pc = Poly3DCollection(polys, edgecolor="k", alpha=0.1)
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.view_init(10, 50)
ax.set_box_aspect([1, 1, 1])
ax.set_title("3D points with kd-tree")
plt.show()

```

3D points with k-d tree



The ***k*-nearest-neighbor** search is similar to the one used with the octree and takes advantage of

the hierarchical nature of the kd-tree.

It proceeds as follows:

1. Start from the root node.
2. Explore the side of each split that contains the query point first.
3. When a leaf node is reached, compare the query point with the points it contains.
4. Keep track of the current k nearest neighbors found so far.
5. Explore the opposite side of a split only if it may contain closer neighbors.

This allows large parts of the tree to be ignored during the search.

To decide whether the opposite side of a split must be explored, the distance between the query point p and the splitting plane P is computed. For a splitting dimension n and a splitting value v , this distance is given by:

$$d(p, P) = p_n - v$$

The sign indicates on which side of the split the query point lies. A negative value corresponds to the left side of the split, while a positive value corresponds to the right side.

The distance to the opposite side of the split is therefore simply $|p_n - v|$. If this distance is larger than the distance to the current farthest neighbor, the opposite side can safely be ignored.

```
[ ]: _, knn_kdtree = kdtree.knn_search(query_point, k)

print(f"The {k} nearest neighbors are points #", knn_kdtree)
print("Same neighbors as brute force:",
      np.array_equal(np.sort(knn_kdtree), np.sort(knn_brute_force)))
```

The 5 nearest neighbors are points # [183 893 3550 53 221]

Same neighbors as brute force: True

A **radius search** proceeds as follows:

1. Start from the root node.
2. Explore the side of each split that contains the query point.
3. Explore the opposite side only if it may contain points within the search radius r .
4. When a leaf node is reached, compare the query point with the points it contains.
5. Keep only the points whose distance to the query point is smaller than r .

As for the k -nn search, large parts of the tree can be ignored during the search.

To determine whether both sides of a split must be explored, the distance between the query point p and the splitting plane P is computed. For a splitting dimension n and splitting value v , this distance is given by:

$$d(p, P) = |p_n - v|$$

If $d(p, P) \leq r$, the search sphere intersects the splitting plane and both child nodes must be explored. Otherwise, only the child node located on the same side of the split as the query point needs to be considered.


```
[ ]: ball_kdtree = kdtree.ball_search(query_point, r)

print(f"Points within radius {r} are points #", ball_kdtree)
print("Same neighbors as brute force:",
      np.array_equal(np.sort(ball_kdtree), np.sort(ball_brute_force)))
```

```
Points within radius 50 are points # [3106 4021   53  183  221  893 2014 2369
3550  436  449 1135 1952 2216
 2399 3663 1402  287 1342  268 1900 2002 4057]
Same neighbors as brute force: True
```

3.5 Quick comparison

The previous sections introduced three common data structures used for pointcloud processing. But how do they compare in practice?

The comparison below focuses on three common operations: construction from points, insertion/deletion of points and nearest neighbor searches. It uses **time-complexity**, which describes how the running time of an algorithm grows with the size of its input.

Using the “big-O notation”, with n the number of points, $O(1)$ denotes constant time (same amount of time regardless of the input size), $O(\log n)$ logarithmic time (time increases slowly as the input size grows), $O(n)$ linear time (time increases proportionally to the input size), and $O(n \log n)$ linearithmic time (time grows faster than linear time but slower than quadratic time).

The values shown below correspond to typical or average-case behavior. Actual performance depends on implementation details, point distributions, and the parameters used to build each structure.

Structure	Construction	Insertion/Deletion	Nearest Neighbor Search
Voxel grid	$O(n)$	$O(1)$	$O(1)$ (best), $O(n)$ (worst)
Octree	$O(n \log n)$	$O(\log n)$	$O(\log n)$
kd-tree	$O(n \log n)$	$O(\log n)$	$O(\log n)$

A voxel grid is the simplest structure to build and maintain. Constructing it only requires assigning each point to a voxel, which is done in linear time. Inserting or deleting a point only requires updating the corresponding voxel. The efficiency of nearest-neighbor searches depends strongly on the chosen voxel size. If the resolution is appropriate, only a small number of voxels need to be explored. If the resolution is poorly chosen, it may approach the cost of a brute-force search.

Octrees require recursively subdividing the space into octants. This additional hierarchy increases construction costs but allows large regions of space to be ignored during neighborhood searches. The performance of insertion, deletion, and nearest-neighbor queries is typically logarithmic when the tree remains reasonably balanced.

kd-trees recursively partition points using splitting planes, usually chosen from the data distribution itself. As a result, the tree is often more balanced than an octree and may require visiting fewer regions during nearest-neighbor searches. However, frequent insertions and deletions may require rebuilding or rebalancing the tree, so kd-trees are particularly well suited to static or slowly changing pointclouds.

In practice, there is no universally best structure. Voxel grids are simple, memory-efficient, and easy to update. Octrees adapt naturally to variations in point density. kd-trees often provide the best nearest-neighbor search performance on static datasets. The most appropriate choice depends on the characteristics of the pointcloud and the required operations.

3.6 Going further

The implementations presented in this notebook are kept simple to illustrate the underlying principles of spatial indexing. In practice, many variants and optimizations exist, often targeting specific objectives. This section briefly presents a few common alternatives.

Voxel grid

Although voxel grids are conceptually simple, several implementation choices are possible in practice:

1. **Dense voxel grids.** All voxels are stored explicitly in a multidimensional array. This allows direct indexing but becomes impractical when the occupied space is sparse.
2. **Sparse voxel grids.** Only occupied voxels are stored, typically using a hash table. This is the approach used in this notebook and in many pointcloud processing libraries.
3. **Hierarchical voxel grids.** Multiple voxel sizes are used simultaneously, allowing the pointcloud to be represented at different resolutions. Octrees can be viewed as a particular form of hierarchical voxel grid.

Instead of storing only point indices, voxels may also store aggregated information such as point counts, average coordinates, colors, or normals. This is commonly used for downsampling and machine-learning applications.

Octree

Several data structures may be used to implement an octree in practice, depending on the application and constraints:

1. **Tree-based octrees.** This is the most intuitive representation and the one used in this notebook. Each node explicitly stores links to its children, making the tree structure easy to navigate and modify.
2. **Array-based octrees.** This approach is well suited to *full octrees*, in which every internal node has exactly eight children. All nodes can then be stored in a flat array instead of using explicit parent-child links. The position of each node in the array implicitly defines its location in the hierarchy.
3. **Linear octrees.** In this approach, nodes are stored in a hash table and identified by a code describing their position in the tree. Typically, each node is assigned a 3-bit index (from 0 to 7) corresponding to its octant, which is concatenated with its parent's code to form an octonary identifier. This encoding leads to what is commonly referred to as a *linear octree*, by analogy with the *linear quadtree* [3].
4. **Point-code-based octrees.** In this variant, instead of assigning a code to each node, a code is assigned directly to each point, representing its location across all octree subdivision levels. As in the linear octree representation, the code consists of a concatenation of 3-bit octant

indices. Sorting these codes enables efficient neighborhood queries and traversal operations. This approach is notably used in CloudCompare [4].

The choice of representation depends on the operations to be performed and the available computational resources. Tree-based octrees are generally easier to modify, while linear representations often reduce memory consumption and improve traversal efficiency.

Also note that the octree cells may also have different shapes, the most common ones being axis-aligned cuboids and axis-aligned cubes. Axis-aligned cuboids better fit the extent of the pointcloud, while cubic cells produce more regular subdivisions and often simplify computations.

kd-tree

Several variants of kd-trees exist in practice. One of the most important design choices is the **splitting rule**, which determines how the space is partitioned at each node:

1. **The standard split.** The splitting dimension is chosen as the dimension in which the points have the greatest spread. The splitting value is the median of the points along this dimension. This usually produces well-balanced trees, but may still generate long and thin cells.
2. **The midpoint split.** The splitting dimension is again chosen as the dimension with the greatest spread, but the split is performed at the center of the cell rather than at the median of the points. This approach is similar to a binary version of quadtrees and octrees. It produces more regular cells, but may create empty regions that still need to be explored during queries.
3. **The sliding midpoint split.** This approach starts with a midpoint split. If all points lie on the same side of the midpoint, the splitting plane is moved toward the points until both child cells contain data. This tends to work well for highly non-uniform point distributions and is the splitting strategy used in the kd-tree implementation of `scipy` [5].

Many other variants and optimizations exist. Depending on the application, memory usage and construction time may be just as important as query performance. For example, approximate median calculations are often used to accelerate the construction of kd-trees for very large pointclouds.

3.7 Wrapping up

You should now have a better understanding of spatial indexing for 3D pointclouds, whose goal is to accelerate common operations such as distance calculations and neighborhood searches. This notebook introduced three widely used data structures: voxel grids, octrees, and kd-trees, each offering different trade-offs for pointcloud processing.

Voxel grids are simple to build and manipulate. They can also serve as a structured representation of a pointcloud, simplifying or even enabling downstream processing tasks (e.g., the use of techniques originally developed for regular image data, such as convolutional neural networks). Their main limitation is that performance depends strongly on the chosen voxel size, which determines the level of detail at which the pointcloud is organized.

Octrees and kd-trees rely on hierarchical subdivisions of space and are generally better suited to neighborhood searches. Octrees are relatively simple to implement and naturally support dynamic pointclouds through point insertion and deletion. kd-trees are typically more efficient for nearest-neighbor searches, especially when point distributions are highly non-uniform, although they may require rebalancing after many updates.

Ultimately, no single data structure is best in all situations. The most appropriate choice depends on the characteristics of the pointcloud, the type of queries being performed, and whether the dataset is static or frequently updated.

4 Subsampling

This notebook introduces subsampling techniques for **reducing the size of pointclouds to enable more efficient processing**. Real-world pointclouds often contain millions of points, many of which provide similar geometric information. The number of points can often be reduced significantly **while preserving the main structure and features of the scene or object**.

This notebook focuses on several widely used subsampling methods, which are classified into the following categories: **index-based**, **spatial partitioning** and **distance-based**. Representative techniques from each category are discussed and compared.

To improve memory efficiency and preserve access to all original point attributes (e.g., colors and normals), the implementations return the indices of sampled points rather than their coordinates.

```
[1]: # Necessary imports
from collections import deque

import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import KDTree

# Load our bunny pointcloud
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
kdtree = KDTree(points)
```

4.1 Index-based

Index-based subsampling selects points using only their indices or ordering, without considering their spatial distribution. These techniques are simple and fast, but they do not guarantee a uniform distribution of points. As a result, important geometric details may be lost, especially when the original pointcloud is unevenly sampled. Because of these limitations, they are often used as baselines in benchmarks or for visualization.

4.1.1 Random sampling

Random sampling is one of the simplest subsampling techniques. It works by **selecting a subset of points at random from the original pointcloud**, either by keeping a fixed number of points k or by using a fixed sampling ratio. In the standard version, every point has the same chance of being selected, and a point can only be selected once. A common implementation randomly shuffles the point indices and keeps the first k points. Another approach is *Bernoulli* sampling, where each point is kept independently with probability p . In this case, the number of selected points is random, with an average of pn , where n is the number of input points. For very large or streaming pointclouds, reservoir sampling can be used to select a fixed number of points in a single pass without storing the entire point cloud in memory [6].

The implementation below follows the first approach and selects k random indices.

```
[2]: def random_sampling(points, num_samples):
      """Returns indices of randomly selected points."""
```

```

N, _ = points.shape
if num_samples > N:
    raise ValueError("num_samples must not exceed the number of points.")
rng = np.random.default_rng()

return rng.choice(N, num_samples, replace=False)

```

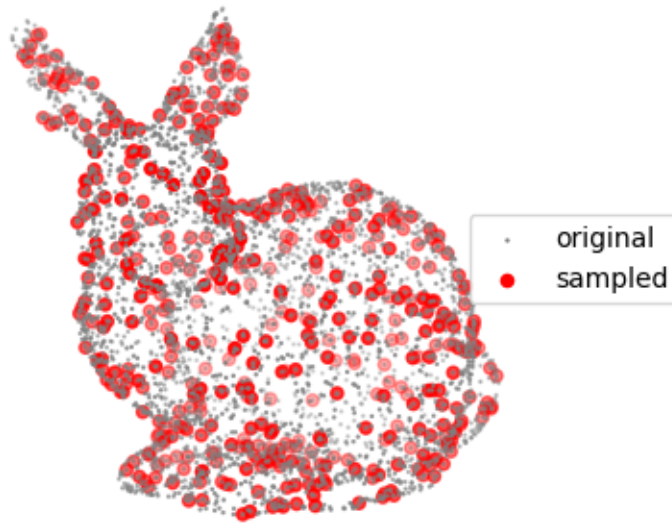
```

[3]: # Randomly sample 500 points from the bunny pointcloud
random_sampled_indices = random_sampling(points, 500)
random_sampled_points = points[random_sampled_indices]

# Visualize the original and randomly sampled points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(random_sampled_points[:, 0], random_sampled_points[:, 1],
           random_sampled_points[:, 2],
           color='red', label="sampled")
ax.set_title(f"Random sampling ({len(random_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
plt.show()

```

Random sampling (500 points)



4.1.2 Stride sampling

Stride Sampling, sometimes called uniform index sampling, is a simple and fast technique that works by **selecting points at regular index intervals**. Unlike random sampling, it is deterministic and therefore reproducible. Although it does not explicitly consider the spatial position of points, it may provide an approximately uniform spatial distribution when the point ordering reflects spatial locality, as is often the case for sequential acquisition systems such as RGB-D cameras or linear laser scanners. However, when this assumption does not hold, stride sampling may produce uneven spatial coverage.

The implementation is straightforward and amounts to keeping every n -th point according to the initial ordering.

The implementation below uses NumPy's *slicing*, with an option to modify the initial ordering to add some randomness.

```
[4]: def stride_sampling(points, stride, shuffle=False):  
      """Selects point indices at regular intervals."""
```

```

N, _ = points.shape
if not (isinstance(stride, int) and 1 <= stride < N):
    raise ValueError(
        "stride must be an int >= 1 and less than the number of points."
    )
indices = np.arange(N)
if shuffle: # optionally shuffle the indices to reduce ordering bias
    np.random.shuffle(indices)

return indices[::stride]

```

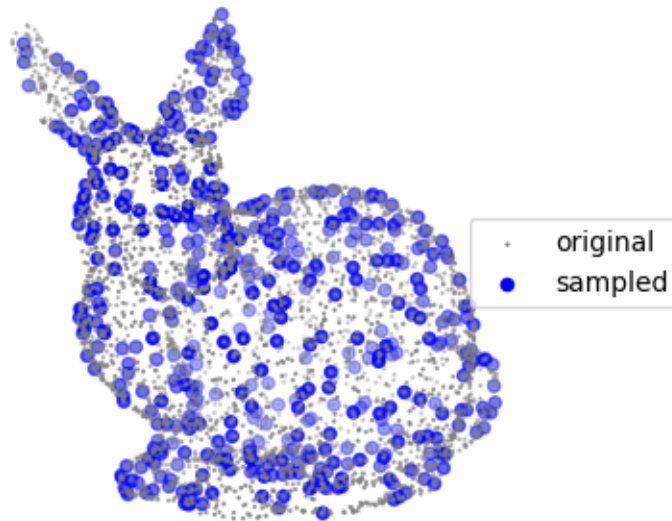
```

[5]: # Apply stride sampling to the bunny pointcloud
# using a stride of 8 to obtain a similar number of points as before
stride_sampled_indices = stride_sampling(points, stride=8)
stride_sampled_points = points[stride_sampled_indices]

# Visualize the original and stride-sampled points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(stride_sampled_points[:, 0], stride_sampled_points[:, 1],
           stride_sampled_points[:, 2], color='blue', label="sampled")
ax.set_title(f"Stride sampling ({len(stride_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
plt.show()

```


Stride sampling (512 points)



4.2 Spatial partitioning

Spatial partitioning divides three-dimensional space into regularly spaced cells that contain subsets of points (see the notebook on spatial indexing). The core idea is to **select one or more points from each cell**. By enforcing a more uniform spatial distribution of points, these techniques generally provide a more uniform spatial distribution than index-based methods and better preserve the overall shape of the pointcloud.

Spatial partitioning techniques are categorized according to the underlying spatial structure: a uniform grid with *voxel grid downsampling* and a hierarchical grid with *octree downsampling*. kd-trees are not considered here because their partitioning is irregular and directly related to geometric scale, making them less suitable for the subsampling tasks.

4.2.1 Voxel grid downsampling

Voxel grid downsampling works by **dividing space into a regular grid of voxels and selecting one or more representative points per voxel**. The voxel size controls the resolution of the subsampled pointcloud: larger voxels produce fewer points, while smaller voxels preserve more

detail. This makes voxel grid downsampling **particularly suitable when the desired resolution is known in advance**.

Once the voxel grid has been built, downsampling is efficient because each occupied voxel can be processed independently.

Several strategies can be used to select the representative point. Common choices include keeping the first point in the voxel, selecting a random point, or choosing the point closest to the voxel center. Choosing the point closest to the voxel center generally provides a more regular spatial distribution.

A custom voxel grid implementation based on a dictionary (hash map) is presented below, with options to keep either the first point in each voxel or the point closest to the voxel center.

```
[6]: class VoxelGrid:
    """Partition 3D space into voxels."""

    class Voxel:
        """Cubic voxel."""

        def __init__(self, center):
            self.center = center # center of the cube
            self.indices = []

    def __init__(self, points, voxel_size):

        self.points = points
        self.voxel_size = voxel_size
        self.voxels = {}
        self._build()

    def _build(self):
        """Create the voxel grid by filling cells with points."""

        for i, point in enumerate(self.points):
            coords = self.voxel_coordinates(point)
            voxel = self.voxels.setdefault(
                coords,
                self.Voxel(center=(np.array(coords) + 0.5) * self.voxel_size)
            )
            voxel.indices.append(i)

    def voxel_coordinates(self, point):
        """Coordinates of a voxel for a given point"""
        return tuple(np.floor(point / self.voxel_size).astype(int))

    def sample_indices(self, closest_to_center=False):
        """Sample one index per voxel."""
```

```

sampled_indices = []
for voxel in self.voxels.values():
    if closest_to_center:
        # Find the point in the voxel closest to the voxel center
        points_in_voxel = self.points[voxel.indices]
        # Squared distances for efficiency
        distances = np.sum((points_in_voxel - voxel.center)**2, axis=1)
        closest_point_idx = np.argmin(distances)
        chosen_index = voxel.indices[closest_point_idx]
    else:
        chosen_index = voxel.indices[0]
    sampled_indices.append(chosen_index)

return sampled_indices

```

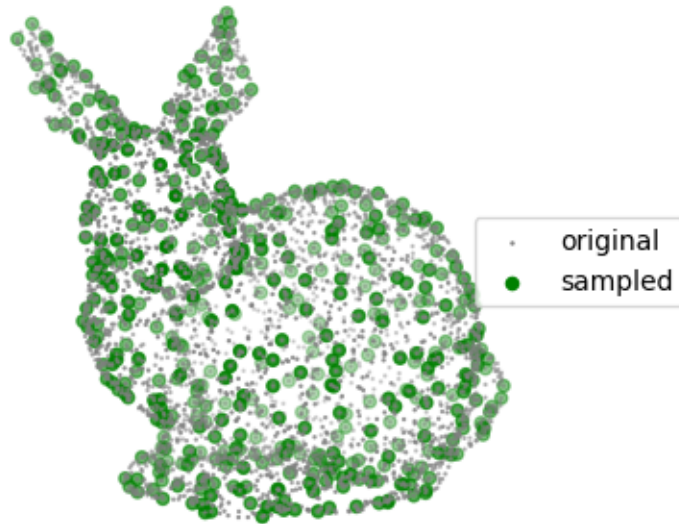
```

[8]: # Voxel grid downsampling
# using a voxel size of 12. to obtain a similar number of points as before
voxel_grid = VoxelGrid(points, voxel_size=12.)
# Compare two representative-point selection strategies
for closest_to_center in [False, True]:
    sampled_indices = voxel_grid.sample_indices(closest_to_center)
    voxel_sampled_points = points[sampled_indices]

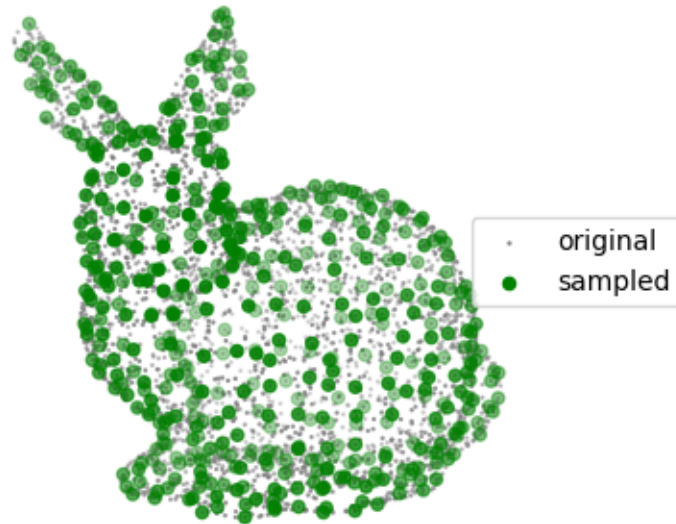
    # Visualize the original and voxel grid downsampled pointclouds
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               color='grey', s=0.5, label='original')
    ax.scatter(voxel_sampled_points[:, 0], voxel_sampled_points[:, 1],
               voxel_sampled_points[:, 2], color='green', label="sampled")
    ax.set_title(f"Voxel grid downsampling ({len(voxel_sampled_points)}" +
                f" points)\n(closest_to_center={closest_to_center})")
    ax.legend(loc="right")
    ax.view_init(10, 60)
    ax.set_axis_off()
    ax.set_box_aspect([1, 1, 1])
    plt.show()

```

Voxel grid downsampling (497 points)
(closest_to_center=False)



Voxel grid downsampling (497 points) (closest_to_center=True)



4.2.2 Octree downsampling

Octree downsampling works similarly to voxel grid downsampling but uses a hierarchical spatial subdivision instead of a regular grid. Regions containing many points are subdivided more finely, while sparse regions remain larger. This makes octree downsampling **well suited to pointclouds with highly non-uniform point densities or when the desired level of detail is not known in advance**.

Compared to voxel grids, octrees are more complex to build. However, once the octree has been constructed, different subsampling resolutions can be obtained by selecting different tree depths.

As with voxel grids, representative points may be chosen using simple strategies such as keeping the first point or selecting one at random or using geometry-aware criteria such as proximity to the node center or centroid. Because the number of occupied nodes varies with depth, the final number of sampled points is not known in advance.

The custom octree implementation presented below selects the first point contained in each node at a given subdivision level.

```
[9]: class OctTree:
    """Partition 3D space by recursively subdividing it into eight octants."""

    class OctNode:
        """A node of our Octree or octant."""

        def __init__(self, depth, bound_min, bound_max, indices):

            self.depth = depth # Distance from the root
            self.bound_min = bound_min # Bottom-left corner
            self.bound_max = bound_max # Top-right corner
            self.indices = indices # Point indices
            self.children = [] # Leaf by default

        @property
        def is_leaf(self):
            """A terminal/leaf node does not have children."""
            return not len(self.children)

    def __init__(self, points, max_leaf_size=10, max_depth=10):

        self.points = points
        self.max_leaf_size = max_leaf_size
        self.max_depth = max_depth
        # Create root node
        bound_min = points.min(axis=0)
        bound_max = points.max(axis=0) + 1e-8 # slightly enlarge
        indices = np.arange(len(points), dtype=int)
        self.root = OctTree.OctNode(0, bound_min, bound_max, indices)
        # Recursively build the tree
        self._split_node(self.root)

    def _split_node(self, node):
        """Recursively split a node into eight subnodes/children."""

        # Do not split if node is already too small or if reached maximum depth
        if (len(node.indices) <= self.max_leaf_size
            or node.depth >= self.max_depth):
            return

        # Compute the bounds of the eight candidate subnodes
        x_center, y_center, z_center = (node.bound_min + node.bound_max)/2
        x_min, y_min, z_min = node.bound_min
        x_max, y_max, z_max = node.bound_max
        new_bounds = [
            # South-West-Back
            (np.array([x_min, y_min, z_min]),
             np.array([x_center, y_center, z_center])),
```

```

        # North-West-Back
        (np.array([x_min, y_min, z_center]),
         np.array([x_center, y_center, z_max])),
        # North-East-Back
        (np.array([x_min, y_center, z_center]),
         np.array([x_center, y_max, z_max])),
        # South-East-Back
        (np.array([x_min, y_center, z_min]),
         np.array([x_center, y_max, z_center])),
        # South-West-Front
        (np.array([x_center, y_min, z_min]),
         np.array([x_max, y_center, z_center])),
        # North-West-Front
        (np.array([x_center, y_min, z_center]),
         np.array([x_max, y_center, z_max])),
        # North-East-Front
        (np.array([x_center, y_center, z_center]),
         np.array([x_max, y_max, z_max])),
        # South-East-Front
        (np.array([x_center, y_center, z_min]),
         np.array([x_max, y_max, z_center]))
    ]

    # Compute the node points falling into each subnode
    for (bound_min_new, bound_max_new) in new_bounds:
        in_subnode = np.logical_and(
            (self.points[node.indices] >= bound_min_new).all(axis=1),
            (self.points[node.indices] < bound_max_new).all(axis=1)
        )
        # A subnode is created only if it contains some points
        if np.any(in_subnode):
            subnode = OctTree.OctNode(
                node.depth + 1, # Increment depth
                bound_min_new,
                bound_max_new,
                node.indices[in_subnode]
            )
            node.children.append(subnode)
            # Recursively subdivide until leaves are small enough
            self._split_node(subnode)
        # Clear node indices after the node is split (only store for leaves)
        node.indices = []

    def sample_indices(self, subdivision_level):
        """Select the first point per node at a given subdivision level
        (if exceeding the octree depth, it just samples from leaves)."""

        sampled_indices = []

```

```

# Explore the tree using breadth-first search
queue = deque([self.root])
while queue:
    node = queue.popleft()
    # Check if we reached the desired subdivision level
    if node.depth < subdivision_level and not node.is_leaf:
        # If not, continue exploring the tree
        queue.extend(node.children)
    else:
        # Continue down to a leaf node via the first child
        while not node.is_leaf:
            node = node.children[0]
        # Sample the first points from this leaf node
        sampled_indices.append(node.indices[0])

return sampled_indices

```

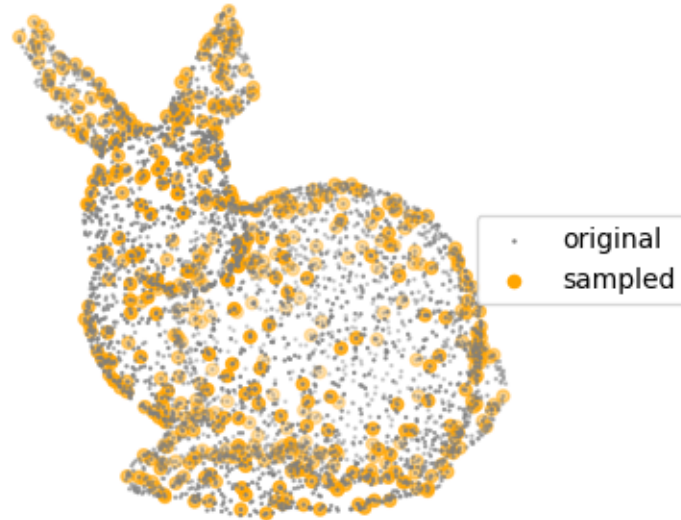
```

[10]: # Octree downsampling
# using a leaf size of 25 to obtain a similar number of points as before
octree = OcTree(points, max_leaf_size=25)
sampled_indices = octree.sample_indices(subdivision_level=4)
octree_sampled_points = points[sampled_indices]

# Visualize the original and octree downsampled points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(octree_sampled_points[:, 0], octree_sampled_points[:, 1],
           octree_sampled_points[:, 2], color='orange', label="sampled")
ax.set_title(f"Octree downsampling ({len(octree_sampled_points)} points)\n" +
            "(first point per node)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
plt.show()

```


Octree downsampling (496 points) (first point per node)



4.3 Distance-based

Distance-based techniques rely on pairwise distances between points to guide subsampling. Spatial coverage and uniformity are preserved by avoiding the selection of points that are too close to each other.

4.3.1 Farthest point sampling

Farthest point sampling (FPS) works by **iteratively selecting the point that is farthest from the points already sampled**. This produces a well-distributed subset that generally covers the original pointcloud more uniformly than random sampling. The main limitation of FPS is its computational cost, since distances must be updated after each new point is selected.

A common optimization avoids recomputing distances to all previously selected points. Instead, each point stores its current distance to the nearest sampled point. When a new point is selected, only this value needs to be updated. Implementations also typically use squared Euclidean distances to avoid unnecessary square-root computations. This results in the classical naïve implementation of FPS, which still requires updating distances for all remaining points at each iteration.

Several acceleration strategies have been proposed to reduce the number of distance evaluations. One simple approach uses spatial structures to restrict distance updates to points near the newly selected sample. Since only nearby points are likely to find a closer sampled point, many distance computations can be avoided. More advanced methods rely on techniques such as approximate nearest-neighbor search.

An implementation of FPS supporting both the naïve approach and a spatial-structure-based acceleration is proposed below. The implementation stores the distance between each point and its nearest sampled point. At each iteration, the point with the largest stored distance is selected, and the distances are updated accordingly. In the naïve version, all points are updated after each selection. The accelerated version updates distances only for points located within a radius equal to the current FPS distance.

```
[11]: def farthest_point_sampling(points, num_samples, neighbor_finder=None):
    """Return point indices selected with Farthest Point Sampling."""

    N, _ = points.shape
    if num_samples > N:
        raise ValueError("num_samples must be less than the number of points.")

    # In the naive implementation, all points are updated
    if neighbor_finder is None:
        neighbor_finder = lambda point, radius: np.arange(N)
    # Randomly select the first point
    first_index = np.random.randint(N)
    sampled_indices = np.empty(num_samples, dtype=int)
    sampled_indices[0] = first_index
    first_point = points[sampled_indices[0]] # first sampled point
    # Initialize distances
    squared_distances = np.sum((points - first_point) ** 2, axis=1)
    squared_distances[first_index] = 0.

    for i in range(1, num_samples):
        # Select the farthest point
        farthest_index = np.argmax(squared_distances)
        sampled_indices[i] = farthest_index
        farthest_point = points[farthest_index]
        # Compute radius to find points that need distance updates
        farthest_squared_distance = squared_distances[farthest_index]
        radius = np.sqrt(farthest_squared_distance)
        neighbor_indices = neighbor_finder(farthest_point, radius)
        # Update distances to the last sampled point
        dist_to_last = np.sum(
            (points[neighbor_indices] - farthest_point)**2,
            axis=1
        )
        squared_distances[neighbor_indices] = np.minimum(
            squared_distances[neighbor_indices], dist_to_last
```

```

    )
    squared_distances[farthest_index] = 0.0

    return sampled_indices

```

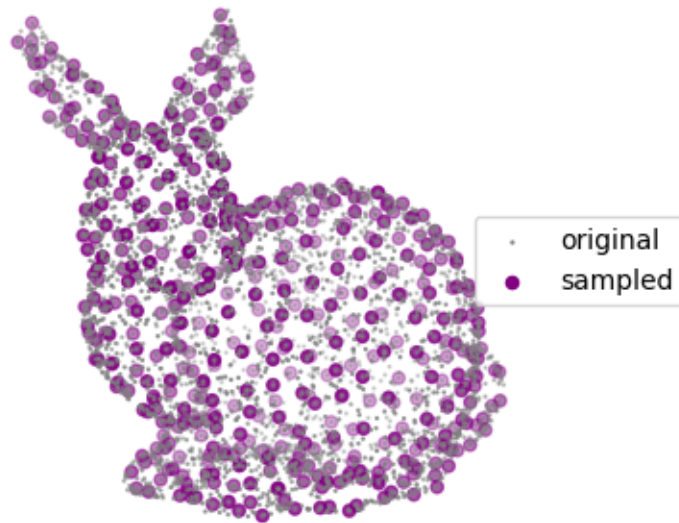
```

[12]: # Farthest Point Sampling with kd-tree acceleration
neighbor_finder = lambda point, radius: kdtree.query_ball_point(point, radius)
fps_sampled_indices = farthest_point_sampling(points, 500, neighbor_finder)
fps_sampled_points = points[fps_sampled_indices]

# Visualize the original and FPS sampled points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(fps_sampled_points[:, 0], fps_sampled_points[:, 1],
           fps_sampled_points[:, 2], color='purple', label="sampled")
ax.set_title(f"Farthest Point Sampling ({len(fps_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
plt.show()

```

Farthest Point Sampling (500 points)



4.3.2 Radius sampling

Radius sampling (also called minimum-distance sampling) **selects points such that no two retained points lie within a distance r of each other**. It works by processing points one by one. When a point is selected, all other points within radius r are excluded from further consideration. This guarantees a minimum spacing of at least r between sampled points. For a fixed point ordering and radius, the result is deterministic.

Efficient implementation relies on spatial search structures to quickly identify neighbors within the radius r .

The implementation below relies on a kd-tree for fast neighborhood searches and uses a Boolean mask to track selected points.

```
[13]: def radius_sampling(points, neighbor_finder):  
      """Return point indices selected by radius sampling."""  
  
      N, _ = points.shape
```

```

# Boolean mask keeping track of selected points
selected = np.ones(N, dtype=bool)

for ind in np.arange(N):
    # Skip this point if it has already been discarded
    if not selected[ind]:
        continue
    # Find neighbors of this point
    neighbors = neighbor_finder(points[ind])
    neighbors = np.array(neighbors)
    # Skip further steps if this point has no neighbors (and keep it)
    if not neighbors.size:
        continue
    # Check only neighbors that have not been processed yet
    mask = neighbors > ind
    if not np.any(mask):
        continue
    # Discard all relevant neighbors of this point
    selected[neighbors[mask]] = False

return np.flatnonzero(selected)

```

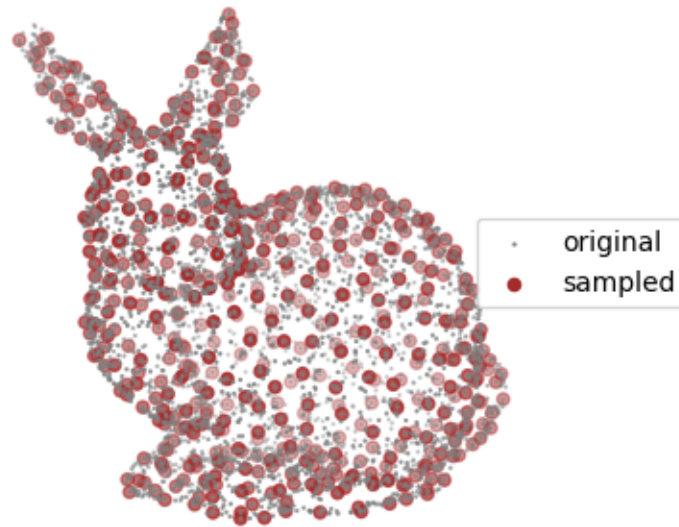
```

[14]: # Radius sampling using a greedy algorithm
# using a radius of 7.5 to obtain a similar number of points as before
neighbor_finder = lambda point: kdtree.query_ball_point(point, r=7.5)
radius_sampled_indices = radius_sampling(points, neighbor_finder)
radius_sampled_points = points[radius_sampled_indices]

# Visualize the original and radius sampled points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(radius_sampled_points[:, 0], radius_sampled_points[:, 1],
           radius_sampled_points[:, 2], color='brown', label="sampled")
ax.set_title(f"Radius sampling ({len(radius_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
plt.show()

```

Radius sampling (532 points)



4.3.3 Poisson-disk sample elimination

Poisson-disk sample elimination generates a well-distributed subset by **iteratively removing points from an initially dense pointcloud**. Each point is assigned a weight based on the density of its neighborhood, with nearby neighbors contributing more than distant ones. At each iteration, the point with the largest weight is removed and the weights of neighboring points are updated. The process continues until the desired number of points remains, producing a distribution that approximates Poisson-disk sampling without generating new points.

The implementation below follows this elimination strategy and uses a kd-tree to efficiently identify neighboring points. This approach is similar in spirit to the `sample_points_poisson_disk` function provided by `Open3D`, which is based on the sample elimination method proposed by Yuksel [7].

```
[ ]: def poisson_disk_elimination(points, num_samples, neighbor_finder, radius):  
    """Poisson-disk sample elimination."""  
  
    N, _ = points.shape  
    if num_samples >= N:
```

```

        raise ValueError("num_samples must be less than the number of points.")

    # Neighborhoods for all points
    neighbor_inds = neighbor_finder(points)
    # Pairwise contributions and initial weights
    contributions = []
    weights = np.zeros(N)
    for i, neighbors in enumerate(neighbor_inds):
        neighbors = np.asarray(neighbors)
        neighbors = neighbors[neighbors != i]
        if neighbors.size:
            distances = np.linalg.norm(points[neighbors] - points[i], axis=1)
            contrib = np.maximum(0.0, 1.0 - distances / radius)
        else:
            contrib = np.empty(0)
        contributions.append((neighbors, contrib))
        weights[i] = contrib.sum()

    active = np.ones(N, dtype=bool)
    while np.sum(active) > num_samples:
        # Remove the most crowded point
        active_weights = np.where(active, weights, -np.inf)
        remove_idx = np.argmax(active_weights)
        active[remove_idx] = False
        # Update neighboring weights
        neighbors, contrib = contributions[remove_idx]
        for j, c in zip(neighbors, contrib):
            if active[j]:
                weights[j] -= c

    return np.flatnonzero(active)

```

```

[21]: # Poisson-disk sample elimination
neighbor_finder = lambda point: kdtree.query_ball_point(point, r=10.)
poisson_sampled_indices = poisson_disk_elimination(
    points,
    500, # num_samples
    neighbor_finder,
    10. # radius
)
poisson_sampled_points = points[poisson_sampled_indices]

# Visualize the original and Poisson-disk sampled points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')

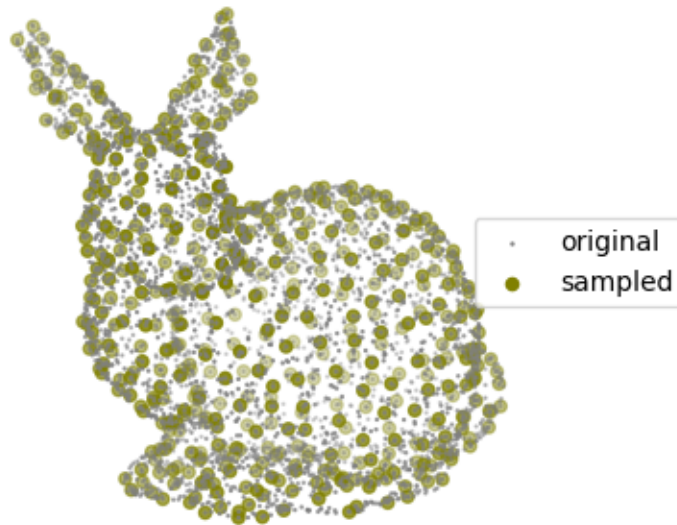
```

```

ax.scatter(poisson_sampled_points[:, 0], poisson_sampled_points[:, 1],
           poisson_sampled_points[:, 2], color='olive', label="sampled")
ax.set_title(f"Poisson-disk sample elimination "
             f"({len(poisson_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
plt.show()

```

Poisson-disk sample elimination (500 points)



4.4 Quick comparison

Below is a table summarizing all subsampling techniques discussed in this notebook. Time complexity uses the “big-O notation”, where n is the initial number of points and k the desired sample size. The cost of building spatial structures for acceleration is not included in the table but discussed below. m is the total number of voxel cells or octree cells at a given depth.

Technique	Category	Deterministic	Count control	Uniformity	Time complexity
Random	Index	No	Exact	Low	$O(k)$ to $O(n)$
Stride	Index	Yes	Exact	Low	$O(k)$ to $O(n)$
Voxel Grid	Spatial	Yes	Approximate	Medium	$O(m)$ to $O(n)$
Octree	Spatial	Yes	Approximate	Medium	$O(m)$ to $O(n)$
FPS (naïve)	Distance	Yes	Exact	High	$O(nk)$
Radius	Distance	Yes	Approximate	Medium	$O(n \log n)$
Poisson (naïve)	Distance	Yes	Exact	High	$O(n^2)$

Index-based techniques are extremely fast. *Random sampling* runs in linear time either in k when drawing k random indices, or in n when scanning the full array with Bernoulli probability or after generating a random permutation. *Stride sampling* behaves similarly, with a cost proportional to k when using direct indexing, or n when scanning and retaining every fixed step.

Spatial partitioning methods remain efficient once their structures are built. *Voxel grids* are constructed in linear time, while *octrees* require linear to linearithmic time depending on tree balance. Sampling then simply extracts one representative per voxel or octree leaf, linear time in m occupied cells, or up to linear time in n when selecting the point closest to a cell center.

Distance-based techniques are more expensive. Naïve *Farthest Point Sampling* updates distances to all n points at each of the k iterations, giving linear time in both n and k . *Radius sampling* uses spatial queries to enforce a minimum spacing, leading to linearithmic time with KD-trees or grids. The computational cost of *Poisson-disk sample elimination* depends on how point weights are updated and maintained during the elimination process. The simplified implementation presented in this notebook repeatedly searches for the point with the highest weight, resulting in quadratic complexity. More advanced implementations use priority queues and efficient neighborhood updates to reduce the cost considerably.

4.5 Wrapping up

You should now have a better grasp of 3D pointcloud subsampling and its role within the larger processing pipeline. Raw pointclouds often contain millions of points, far more than needed for tasks such as registration, surface reconstruction, semantic segmentation, or real-time inference. Subsampling is therefore a crucial early step, as it reduces redundancy, controls density, and improves computational tractability while attempting to preserve geometric structure.

Depending on the application, one may choose extremely fast index-based techniques, spatially aware voxel and octree reductions, or more sophisticated distance-based techniques that maximize uniformity. No single technique is universally optimal and each represents a specific trade-off between speed, regularity, determinism, and the ability to target an exact number of samples. Understanding these tradeoffs makes it easier to choose the right technique for downstream algorithms, ensuring that subsequent stages operate efficiently without sacrificing essential geometric information.

5 Cleaning

This notebook addresses cleaning, a fundamental preprocessing step in 3D pointcloud workflows. Real-world pointclouds typically contain various forms of noise, outliers, and artifacts introduced by sensor imperfections or reconstruction errors. This severely degrades the quality of downstream tasks such as registration, segmentation, or feature extraction. Cleaning aims to improve the reliability and interpretability of pointclouds by removing points that are unlikely to represent the actual geometry of the scene. By filtering out noise and outliers, we obtain a dataset that is easier to process for subsequent algorithms that often assume a certain level of smoothness or geometric coherence.

This notebook focuses on outlier removal and denoising, some of the most common cleaning operations applied in practice.

```
[33]: # Necessary imports
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from scipy.spatial import KDTree

sys.path.append("./utils")
from manip_utils import orthonormal_basis_from_w

# Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
normals = data[:, 3:6]
kdtree = KDTree(points)
```

5.1 Outlier removal

Outliers are points that are geometrically inconsistent with their local neighborhood. They may appear isolated, too far from surfaces, or part of spurious structures caused by reflections, occlusions, or reconstruction noise.

Outlier-removal algorithms aim to detect and eliminate such points by analyzing local spatial properties, such as distances or neighborhood statistics. This section presents two widely used techniques: *Statistical Outlier Removal* (SOR) and *Radius Outlier Removal* (ROR). Both rely on neighborhood queries but differ in how they classify a point as an outlier.

```
[34]: # Add outliers to the pointcloud
n_outliers = 100
outliers = np.random.uniform(
    points.min(axis=0),
    points.max(axis=0),
    (n_outliers, 3)
)
points_outliers = np.vstack((points, outliers))
```

```

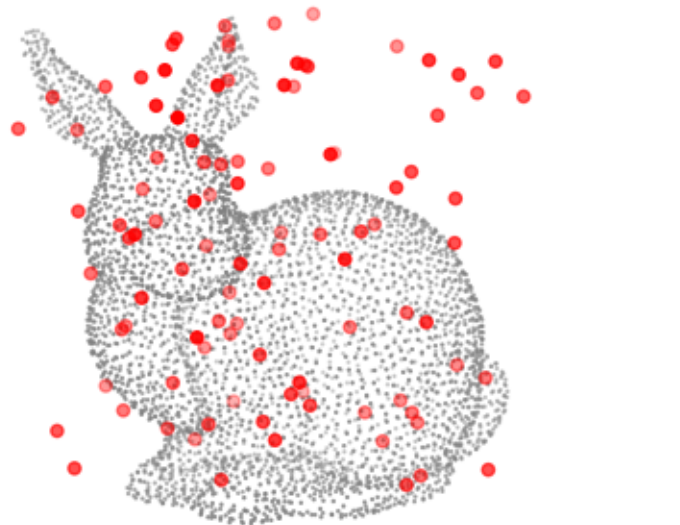
# Create a mask to identify the outliers in the combined pointcloud
outliers_mask_true = np.hstack((np.zeros(points.shape[0], dtype=bool),
                                np.ones(outliers.shape[0], dtype=bool)))

# Build a KDTree for the pointcloud with outliers
kdtree_outliers = KDTree(points_outliers)

# Visualize the pointcloud with outliers
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers[:, 0], outliers[:, 1], outliers[:, 2],
           c="red", label="outliers")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.legend()
ax.set_title("pointcloud with generated outliers")
plt.show()

```

pointcloud with generated outliers



5.1.1 Statistical Outlier Removal (SOR)

Statistical Outlier Removal (SOR) is a statistics-based filtering method that classifies points according to how unusual their local spacing is.

For each point, SOR computes the mean distance to its k nearest neighbors. It then models the distribution of these mean distances across the entire pointcloud. Points whose mean distance deviates from the global distribution are considered statistical anomalies and removed. This is typically done by removing points lying more than a user-defined number of standard deviations from the mean.

SOR is particularly suitable when noise is roughly evenly distributed and when a global statistical model makes sense.

```
[35]: def statistical_outlier_removal(points, neighbor_dist_function, std_ratio=2.0):  
    """Remove points that are statistical outliers based on their average  
    distance to neighbors."""  
  
    # Compute the average distance to the nearest neighbors for each point  
    dists = neighbor_dist_function(points)  
    dists = dists[:, 1:] # exclude the distance to itself  
    avg_distances = np.mean(dists, axis=1)  
  
    # Compute the mean and standard deviation of the average distances  
    mean_distance = np.mean(avg_distances)  
    std_distance = np.std(avg_distances)  
  
    # Identify outliers based on the distance threshold  
    threshold = mean_distance + std_ratio * std_distance  
    outliers_mask = avg_distances > threshold  
  
    return outliers_mask
```

```
[36]: # Apply statistical outlier removal  
neighbor_dist_function = lambda points: kdtree_outliers.query(points, k=10)[0]  
outliers_mask_sor = statistical_outlier_removal(  
    points_outliers,  
    neighbor_dist_function,  
    std_ratio=2.0  
)  
outliers_sor = points_outliers[outliers_mask_sor]  
print(f"Number of outliers: {len(outliers_sor)} (expected: {n_outliers})")  
missed_mask_sor = np.logical_xor(outliers_mask_sor, outliers_mask_true)  
missed_sor = points_outliers[missed_mask_sor]  
  
# Visualize the inliers and outliers
```

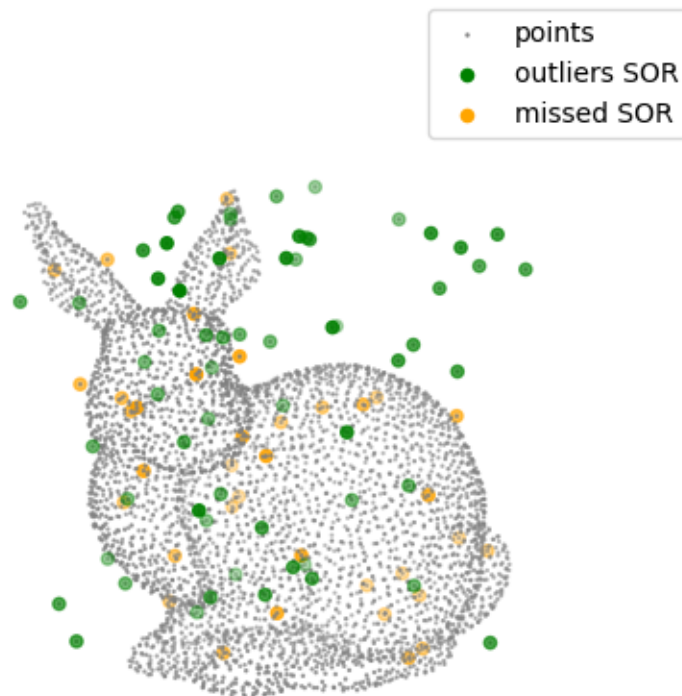
```

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_outliers[:, 0], points_outliers[:, 1], points_outliers[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers_sor[:, 0], outliers_sor[:, 1], outliers_sor[:, 2],
           c="green", label="outliers SOR")
ax.scatter(missed_sor[:, 0], missed_sor[:, 1], missed_sor[:, 2],
           c="orange", label="missed SOR")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.legend()
ax.set_title("Statistical outlier removal (SOR)")
plt.show()

```

Number of outliers: 63 (expected: 100)

Statistical outlier removal (SOR)



5.1.2 Radius Outlier Removal (ROR)

Radius Outlier Removal (ROR) is a density-based filtering method. Instead of analyzing global statistics, ROR examines local point density.

For each point, it counts how many neighbors lie within a given radius. If a point has fewer neighbors than a user-defined threshold, it is considered to be in a locally sparse region and is removed.

This makes ROR well suited for eliminating isolated points or small clusters. ROR is especially helpful when point spacing is relatively uniform or when one wants to enforce a minimal local density.

```
[37]: def radius_outlier_removal(points, neighbor_finder, nb_points=5):
        """Remove points that have fewer than a specified number of neighbors
        within a given radius."""

        # Find neighbors within the specified radius for each point
        neighbors = neighbor_finder(points)
        # Count the number of neighbors for each point
        neighbor_counts = np.array([len(n) for n in neighbors])
        # Identify outliers based on the neighbor count threshold
        outliers_mask = neighbor_counts < nb_points

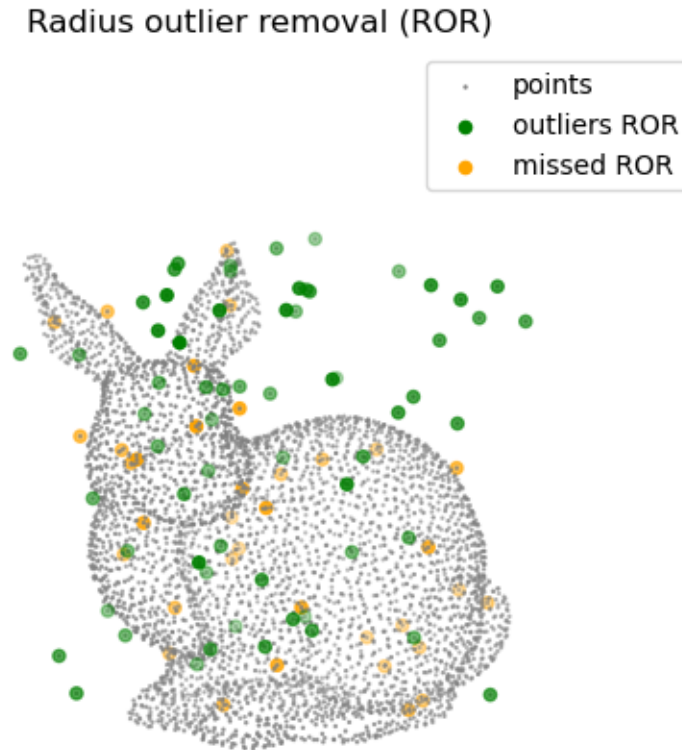
        return outliers_mask

[38]: # Apply radius outlier removal
neighbor_finder = lambda points: kdtree_outliers.query_ball_point(points, r=10.)
outliers_mask_ror = radius_outlier_removal(
    points_outliers,
    neighbor_finder,
    nb_points=5
)
outliers_ror = points_outliers[outliers_mask_ror]
print(f"Number of outliers: {len(outliers_ror)} (expected: {n_outliers})")
missed_mask_ror = np.logical_xor(outliers_mask_ror, outliers_mask_true)
missed_ror = points_outliers[missed_mask_ror]

# Visualize the inliers and outliers
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_outliers[:, 0], points_outliers[:, 1], points_outliers[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers_ror[:, 0], outliers_ror[:, 1], outliers_ror[:, 2],
           c="green", label="outliers ROR")
ax.scatter(missed_ror[:, 0], missed_ror[:, 1], missed_ror[:, 2],
           c="orange", label="missed ROR")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
```

```
ax.legend()
ax.set_title("Radius outlier removal (ROR)")
plt.show()
```

Number of outliers: 64 (expected: 100)



5.2 Denoising

Pointclouds are often affected by noise or small fluctuations in point positions caused by sensor inaccuracy or reconstruction artifacts. Unlike outliers, which are isolated and quite easy to identify, noise affects all points, subtly perturbing the surface geometry.

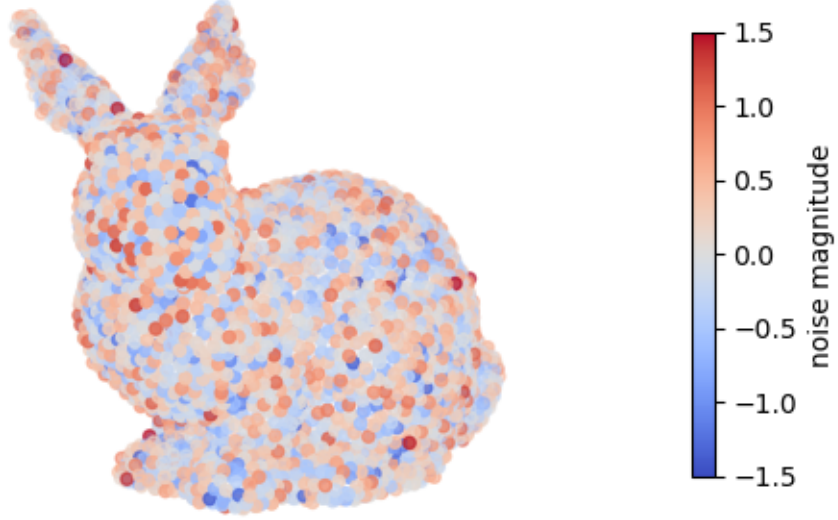
Denoising aims to smooth the pointcloud while preserving important geometric features, such as edges, corners, and fine details. The challenge is to reduce small-scale noise without oversmoothing the underlying shape or blurring sharp transitions.

The following sections focus on some of the most widely used denoising techniques for pointclouds: the bilateral filter and the Moving Least Squares filter.

```
[39]: # Add noise to the pointcloud
sigma_noise = 0.5
noise = np.random.normal(scale=sigma_noise, size=len(points))
points_noise = points + noise[:, None] * normals

# Visualize the noisy pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
norm = matplotlib.colors.TwoSlopeNorm(
    vcenter=0,
    vmin=-3*sigma_noise,
    vmax=3*sigma_noise
)
im = ax.scatter(points_noise[:, 0], points_noise[:, 1], points_noise[:, 2],
                c=noise, cmap="coolwarm", norm=norm)
cbar = fig.colorbar(im, ax=ax, shrink=0.5, pad=0)
cbar.set_label("noise magnitude")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.set_title(f"pointcloud with random noise ( $\sigma$ = $\{sigma\_noise\}$ )")
plt.show()
```


pointcloud with random noise ($\sigma=0.5$)



5.2.1 Bilateral filter

The bilateral filter is a feature-preserving smoothing technique that reduces noise while avoiding the blurring typical of classical averaging filters. Originally proposed for image denoising [8], the bilateral filter was later extended to surface meshes [9] and subsequently to pointclouds [10]. The formulation presented below follows the pointcloud adaptation introduced in [10].

Instead of simply taking the mean of neighboring points, it assigns a weight to each neighbor based on spatial proximity (closer points in space contribute more) and geometric similarity (points with similar normals contribute more). Thanks to this, points located across sharp features (i.e., having very different normals) contribute little to the smoothing process. As a result, the filter effectively denoises flat or smoothly curved regions while preserving sharp edges.

The algorithm is summarized as follows:

For each point p_i with normal n_i :

1. Retrieve all neighbors within radius r and form the set N_i :

$$N_i = \{q_j, ||q_j - p_i|| < r\}$$

2. Compute the Euclidean distances between p_i and its neighbors:

$$d_d^j = \|q_j - p_i\|$$

3. Compute the projected onto the normal n_i distances between p_i and its neighbors:

$$d_n^j = (q_j - p_i) \cdot n_i$$

4. Compute the bilateral weights associated with each neighbor:

$$w_j = e^{-\frac{d_d^{j2}}{2\sigma_d^2}} e^{-\frac{d_n^{j2}}{2\sigma_n^2}}$$

5. Compute the displacement scalar δ_i and move the point to its new position p'_i :

$$p'_i = \delta_i n_i \quad \text{with} \quad \delta_i = \frac{\sum_{q_j \in N_i} w_j d_n^j}{\sum_{q_j \in N_i} w_j}$$

This process can be repeated several times, resulting in N iterations. The other and main parameters of the bilateral algorithms are then the radius r used for the ball search and σ_d and σ_n the Gaussian weights for the Euclidean and projected distances. The article cited above proposes a coarse heuristic for setting their values, based on the assumption that the contribution of neighbors farther than $3\sigma_d$ is negligible. This leads to $\sigma_d = \sigma_n = r/3$ and $r = L\sqrt{20/n}$ with n the size of the pointcloud and L the length of its cubic bounding box.

In practice this heuristic does not give the best result on our bunny pointcloud and smaller values are used in the example below.

```
[40]: def denoise_bilateral_filter(points, normals, neighbor_finder, sigma_d, sigma_n):
    """Bilateral filter for pointcloud"""

    points_denoised = np.empty_like(points)

    # Neighborhood search
    inds = neighbor_finder(points)

    for i, (point, normal) in enumerate(zip(points, normals)):
        # Neighbors
        neighbors = points[inds[i]]
        # Distances
        d_dist = np.linalg.norm(neighbors - point, axis=1)
        # Projected distances
        d_normal = np.dot(neighbors - point, normal)
        # Weights
        weights = (
            np.exp(-(d_dist**2) / (2 * sigma_d**2)) # spatial weights
            * np.exp(-(d_normal**2) / (2 * sigma_n**2)) # normal weights
        )
```

```

    # Displacement along the normal
    delta_p = np.sum(weights * d_normal) / np.sum(weights)
    # New point position
    points_denoised[i] = point + delta_p * normal

return points_denoised

```

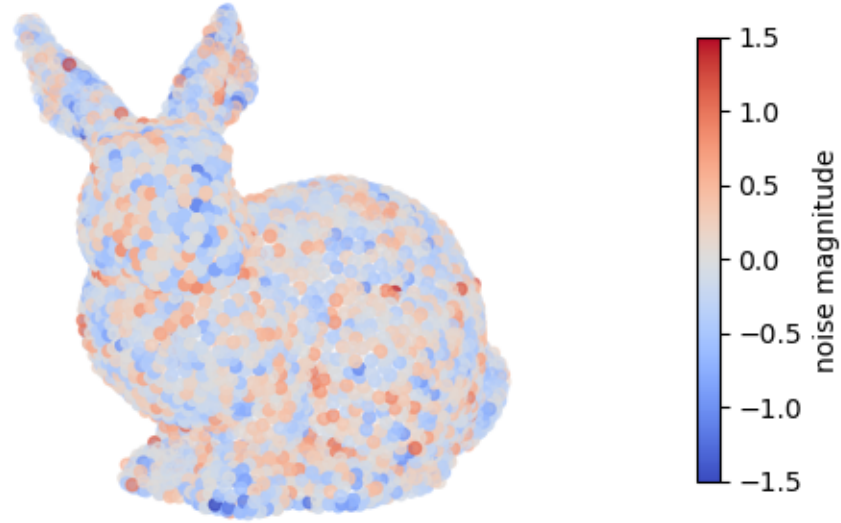
```

[41]: # Apply bilateral filter
neighbor_finder = lambda points: kdtree.query_ball_point(points, r=10.)
points_denoised_bilateral = denoise_bilateral_filter(
    points_noise,
    normals,
    neighbor_finder,
    sigma_d=2.,
    sigma_n=2.
)
noise_bilateral = np.sum(normals*(points_denoised_bilateral - points), axis=1)

# Visualize the denoised pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
norm = matplotlib.colors.TwoSlopeNorm(
    vcenter=0,
    vmin=-3*sigma_noise,
    vmax=3*sigma_noise
)
im = ax.scatter(points_denoised_bilateral[:,0], points_denoised_bilateral[:,1],
                points_denoised_bilateral[:,2],
                c=noise_bilateral, cmap="coolwarm", norm=norm)
cbar = fig.colorbar(im, ax=ax, shrink=0.5, pad=0)
cbar.set_label("noise magnitude")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.set_title(f"pointcloud denoised with bilateral filter ($\\sigma_d={
    noise_bilateral.std():.2f}$\\sigma_n={
plt.show()

```

pointcloud denoised with bilateral filter ($\sigma = 0.38$)



5.2.2 Moving Least Squares

Moving Least Squares (MLS) is a technique for approximating a smooth surface from an unstructured and possibly noisy pointcloud by fitting local surface models in a neighborhood of each point. The method was originally proposed for point-set surface computation and rendering [11] and is now a foundational technique in point-based geometry processing.

The central idea of MLS is to define, for any point of interest, a local neighborhood and to estimate a surface by solving a weighted least-squares problem, typically fitting a low-degree polynomial. The weights are chosen as a decreasing function of distance, so that nearby points exert a stronger influence than distant ones. As the evaluation point moves across the pointcloud, both the neighborhood and the associated weights change continuously, causing the fitted surface to vary smoothly (hence the term moving least squares). A projection operator is then defined by mapping a point onto the locally approximated surface.

MLS has been widely used for surface reconstruction, resampling, and point-based rendering. It can also be employed as a denoising technique by projecting each input point onto the MLS surface defined by the original point set, thereby reducing high-frequency noise.

A simplified variant of MLS is considered below for denoising purposes. The overall procedure is outlined below:

For each point p_i with unit normal n_i :

1. Retrieve all neighbors within radius r and form the set N_i :

$$N_i = \{q_j, ||q_j - p_i|| < r\}$$

where the radius r is chosen proportional to a scale parameter h (typically $r = 2h$ to $3h$) used to assign each neighbor with a distance based weight:

$$\theta(d_j) = e^{-d_j^2/h^2} \quad \text{with} \quad d_j = ||q_j - p_i||$$

2. Define the local reference plane H_i as passing through p_i and having normal n_i (avoiding optimization over normal and offset as described in the original paper). Choose two orthonormal tangent vectors u_i and v_i such that (u_i, v_i, n_i) is an orthonormal basis.
3. Project neighbors onto the plane H_i :

$$q'_j = q_j - (n_i \cdot (q_j - p_i))n_i$$

4. Compute local coordinates signed heights to the plane H_i :

$$x_j = u_i \cdot (q'_j - p_i) \quad y_j = v_i \cdot (q'_j - p_i) \quad f_j = n_i \cdot (q_j - p_i)$$

yielding scattered height samples (x_j, y_j, f_j) .

5. Fit a local MLS height function, typically a bivariate polynomial $g(x, y)$ of degree 3:

$$\min_g \sum_{q_j \in N_i} (g(x_j, y_j) - f_j)^2 \theta(||q_j - p_i||)$$

6. Compute the denoised point p'_i by evaluating the fitted height at the origin of the local frame:

$$p'_i = p_i + g(0, 0)n_i$$

In particular, the optimization step required to estimate an optimal local reference plane is omitted. Instead, the reference plane is assumed to pass through the point being processed, with a known normal direction. As noted in the original paper [11], this approximation is reasonable when the input points are already close to the underlying surface and allows the computationally expensive nonlinear optimization to be avoided. For denoising applications, where only small displacements are expected, this simplification provides a favorable trade-off between accuracy and efficiency.

The main parameter of the algorithm is the weighting scale h , which, as stated in the original work, should reflect the anticipated spacing between neighboring points. This parameter controls the locality of the surface approximation, as smaller values of h lead to more localized fits that better preserve fine-scale details, whereas larger values of h result in smoother, more global approximations.

As h increases, sharp geometric features such as edges and corners tend to be smoothed out, highlighting a well-known limitation of the standard MLS technique. Since it relies on local averaging and smooth polynomial surface representations, the preservation of sharp features generally requires additional strategies.

```
[42]: def MLS_denoise(points, normals, neighbor_finder, h):
    """Denoise a pointcloud using a simplified MLS projection."""

    points_denoised = np.empty_like(points)

    # Neighborhood search
    inds = neighbor_finder(points, 2.0*h)

    for i, point in enumerate(points):

        # Neighbors
        neighbors = points[inds[i]]
        dists = np.linalg.norm(neighbors - point, axis=1)
        weights = np.exp(-dists**2/h**2)

        # Local reference frame
        u, v, n = orthonormal_basis_from_w(normals[i])

        # Project neighbors onto the local plane
        q_prime = neighbors - np.dot(neighbors - point, n)[:, None] * n

        # Compute local coordinates
        x = np.dot(q_prime - point, u)
        y = np.dot(q_prime - point, v)
        f = np.dot(neighbors - point, n)

        # Fit a local MLS height function (here bivariate polynomial of degree 3)
        A = np.column_stack((np.ones_like(x), x, y, x**2, x*y, y**2, x**3,
→x**2*y, x*y**2, y**3))
        A = weights[:, None] * A
        b = weights * f
        coeffs, *_ = np.linalg.lstsq(A, b, rcond=None)

        # Compute the denoised point by evaluating the fitted height at origin
        g_00 = coeffs[0] # constant term of the polynomial
        points_denoised[i] = point + g_00 * n

    return points_denoised
```

```
[43]: # Apply MLS denoising
neighbor_finder = lambda points, r: kdtree.query_ball_point(points, r)
points_denoised_MLS = MLS_denoise(
    points_noise,
    normals,
    neighbor_finder,
    h=5.
)
```

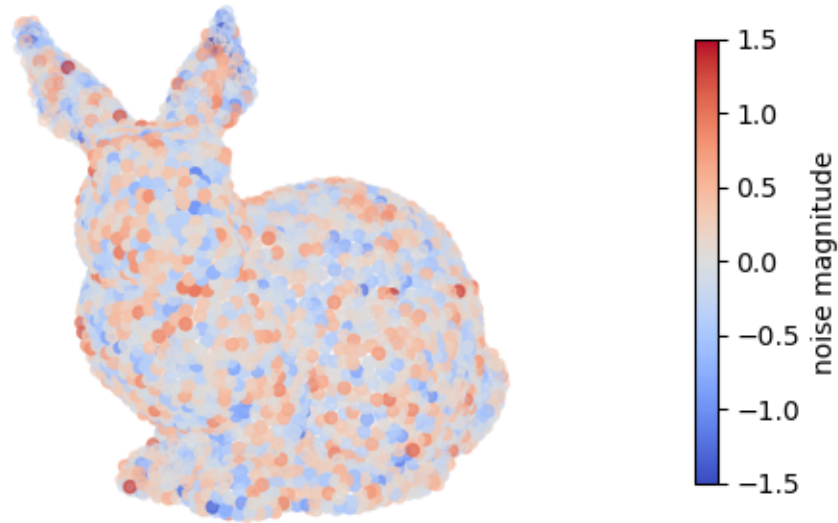
```

noise_MLS = np.sum(normals*(points_denoised_MLS - points), axis=1)

# Visualize the denoised pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
norm = matplotlib.colors.TwoSlopeNorm(
    vcenter=0,
    vmin=-3*sigma_noise,
    vmax=3*sigma_noise
)
im = ax.scatter(points_denoised_MLS[:, 0], points_denoised_MLS[:, 1],
                points_denoised_MLS[:, 2],
                c=noise_MLS, cmap="coolwarm", norm=norm)
cbar = fig.colorbar(im, ax=ax, shrink=0.5, pad=0)
cbar.set_label("noise magnitude")
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.set_title(f"pointcloud denoised with MLS filter ( $\sigma=\{$ 
    noise_MLS.std():.2f $\}$ )")
plt.show()

```

pointcloud denoised with MLS filter ($\sigma = 0.41$)



5.3 Going further

If the techniques discussed above constitute the foundation of many pipelines, numerous alternative methods have been proposed to address more complex noise and outlier characteristics.

Outlier removal

Efficient implementations of Statistical Outlier Removal (SOR) and Radius Outlier Removal (ROR) are available in most major pointcloud processing libraries, such as PCL (with `StatisticalOutlierRemoval` and `RadiusOutlierRemoval`), Open3D (with `remove_statistical_outlier` and `remove_radius_outlier`), and CGAL (with `remove_outliers`). While these methods primarily rely on Euclidean distances, alternative criteria such as normal consistency or color similarity are sometimes incorporated.

SOR and ROR can be viewed as simple statistical filters operating on local neighborhoods. More elaborate techniques from the broader field of statistics extend these ideas but are often formulated for data in arbitrary dimensional spaces, without being specifically tailored to the geometric properties of 3D pointclouds. Similarly, learning-based approaches identify outliers by learning patterns

and deviations directly from points [12].

A distinct approach, often called model-based, consists of defining outliers as points that do not conform to an assumed geometric model. A common example is RANSAC-based filtering, where surfaces such as planes or cylinders are robustly estimated and inconsistent points rejected (see notebook about segmentation). This approach is effective in scenes dominated by simple geometric primitives, but requires prior knowledge of the underlying surfaces and often manual parameter tuning.

In practice, outlier removal is rarely a one-size-fits-all operation. Simple filters such as SOR and ROR are typically applied early in a processing pipeline to remove obvious outliers, while model-based or learning-based techniques are often employed at later stages, once the data has been roughly cleaned and structured.

Denoising

Most classical denoising techniques for pointclouds rely on local surface estimation followed by point projection. The underlying surface is typically modeled as a plane or a low-degree polynomial, as illustrated by the bilateral filter and Moving Least Squares. Moving Least Squares is widely used in practice and is notably implemented in PCL as `MovingLeastSquares`. A related alternative is *jet fitting* [13], which estimates higher-order local surface approximations, with an implementation called `jet_smooth_point_set` found in CGAL.

More recent approaches replace explicit surface fitting with data-driven models [12]. These techniques can handle complex and structured noise patterns that are difficult to model analytically, but they require representative training datasets and introduce higher computational and implementation costs.

In practical pipelines, classical denoising methods remain widely used due to their simplicity, interpretability, and efficiency.

5.4 Wrapping up

You should now have a clearer understanding of 3D pointcloud cleaning, including outlier removal, which targets isolated or inconsistent points, and denoising, which aims to reduce acquisition noise. Together, these operations form an essential preprocessing stage for many downstream tasks.

Outlier-removal techniques such as SOR and ROR rely on neighborhood statistics and are well suited for eliminating gross artifacts early in a processing pipeline. In contrast, denoising techniques address subtler perturbations by estimating local surface structure and projecting points onto smoother representations, as illustrated by bilateral filtering and Moving Least Squares. The effectiveness of these techniques depends on assumptions about point density, noise and underlying geometry.

In practice, pointcloud cleaning is an iterative and data-dependent process. Simple, robust methods are typically applied first to stabilize the data, while more elaborate techniques may be used as refinement steps.

6 Normals and curvatures

This notebook focuses on the estimation of normals and curvatures in 3D pointclouds. These geometric properties **describe the local orientation and shape of the underlying surface** and provide more useful geometric information than point coordinates alone. They are widely used in 3D processing tasks such as visualization, segmentation, surface reconstruction or registration.

This notebook presents common methods for estimating normals and curvatures, primarily through local surface fitting techniques.

```
[1]: # Necessary imports
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from scipy.spatial import KDTree, Delaunay
from scipy.sparse import dok_matrix
from scipy.sparse.csgraph import minimum_spanning_tree, breadth_first_order

sys.path.append("./utils")
from manip_utils import orthonormal_basis_from_w

# Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
ref_normals = data[:, 3:6]
H_ref = data[:, -2]
K_ref = data[:, -1]
# Build KDTree for neighbor search
kdtree = KDTree(points)
```

6.1 Normals

A normal is a vector associated with each point that is perpendicular to the underlying surface it samples. By convention, **normals are oriented outward from the surface** and are typically represented as unit vectors. While some acquisition systems provide normals directly, they are most often estimated from the point coordinates.

6.1.1 Tangent plane estimation

The most common approach for estimating normals is based on **tangent plane estimation**, where the underlying surface is locally approximated by a plane. For each point, a plane is fitted in a least-squares sense to a neighborhood composed of the point and its nearest neighbors. The normal is then approximated by the normal vector of the fitted plane [14].

For each point p_i of the pointcloud:

1. Query the k -nearest neighbors of p_i and form the neighborhood N_i .
2. Compute the covariance matrix cov_i of this neighborhood.

3. Compute the eigenvalues $\lambda_1^i \geq \lambda_2^i \geq \lambda_3^i$ and eigenvectors $(\nu_1^i, \nu_2^i, \nu_3^i)$ of cov_i .
4. Estimate **the normal as the eigenvector associated with the smallest eigenvalue**:

$$n_i \pm \nu_3^i$$

Indeed, the eigenvector ν_3^i corresponds to the direction of minimum variation in the neighborhood, which is assumed to be orthogonal to the local surface. Note that tangent plane estimation does not determine the orientation of the normals, it only provides a direction (i.e., $\pm n$). A separate orientation step is required, as discussed later.

The covariance matrix considered here is computed without dividing by the number of points in the neighborhood:

$$cov_i = \sum_{p_j \in N_i} (p_j - c_i) \cdot (p_j - c_i)^T$$

where c_i is the centroid of the neighborhood:

$$c_i = \frac{1}{|N_i|} \sum_{p_j \in N_i} p_j$$

The plane is fitted in a least-squares sense, and the corresponding fitting error, defined as the sum of squared orthogonal distances, is given by λ_3^i .

The implementation below is fully vectorized, which significantly reduces execution time compared to a point-by-point implementation, at the cost of increased memory usage since all neighborhoods and covariance matrices must be stored in memory at once.

```
[2]: def compute_normals(points, neighbor_finder):
    """Normals by tangent plane estimation using k-nearest neighbors"""

    # Neighborhoods
    inds = neighbor_finder(points) # (N, k)
    neighbors = points[inds] # (N, k, 3)
    # Eigenvalues from unnormalized covariance matrix
    neighbors -= neighbors.mean(axis=1, keepdims=True) # center in-place
    cov = (neighbors.transpose(0, 2, 1) @ neighbors) # (N, 3, 3)
    eigenvalues, eigenvectors = np.linalg.eigh(cov) # (N, 3, 3)
    # Normals and residuals
    normals = eigenvectors[:, :, 0] # already normalized
    residuals = eigenvalues[:, 0]

    return normals, residuals
```

```
[3]: # Estimate normals using k-neighborhoods
k_normals = 10
neighbor_finder = lambda points: kdtree.query(points, k_normals)[1]
normals, residuals = compute_normals(points, neighbor_finder)

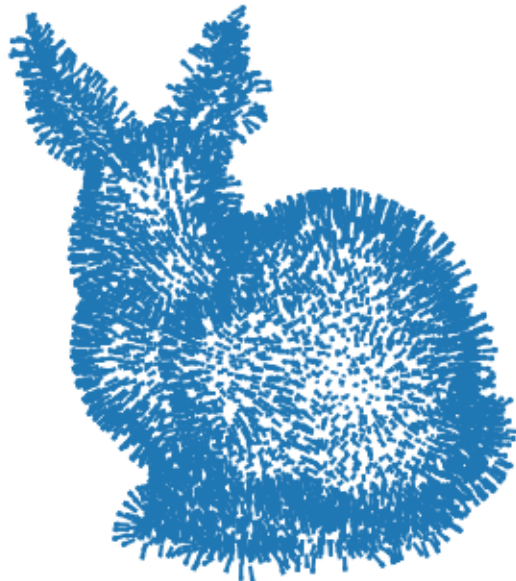
# Visualize the pointcloud and normals
```

```

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.quiver(points[:, 0], points[:, 1], points[:, 2],
          normals[:, 0], normals[:, 1], normals[:, 2],
          length=5.) # make normals appear a little bit larger
ax.set_box_aspect([1, 1, 1])
ax.view_init(10, 60)
ax.set_axis_off()
ax.set_title(f"Estimated normals using tangent plane (k={k_normals})")
plt.show()

```

Estimated normals using tangent plane (k=10)



The accuracy of tangent plane estimation strongly depends on the neighborhood used to fit the plane. Small neighborhoods better capture local surface geometry but are more sensitive to noise. Larger neighborhoods are more robust to noise, but may smooth out geometric details and violate the local planarity assumption. Choosing the neighborhood size therefore involves a trade-off between locality and robustness.

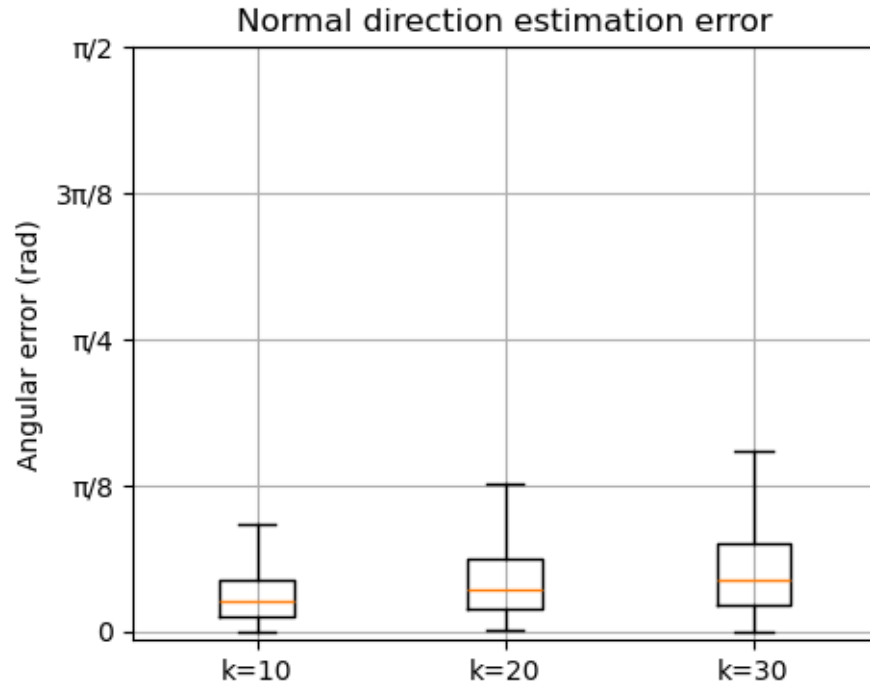
In practice, the neighborhood is usually defined either by the k -nearest neighbors or by all points

within a radius r . A common starting point is $k = 10$, which can then be adjusted according to point density, noise level, and surface complexity.

To illustrate the influence of k , normals are estimated for several neighborhood sizes and compared with the reference normals (derived from the original bunny mesh). At this stage, only the normal directions are evaluated, regardless of their orientation. The resulting angular error distributions are summarized in the boxplot below.

```
[4]: # Compute error between estimated and reference normals for different k
k_values = [10, 20, 30]
normal_errors_k = []
for k in k_values:
    neighbor_finder = lambda points: kdtree.query(points, k)[1]
    normals_k, _ = compute_normals(points, neighbor_finder)
    error_k = np.abs(np.sum(normals_k * ref_normals, axis=1))
    error_k = np.clip(error_k, -1.0, 1.0)
    error_k = np.arccos(error_k)
    normal_errors_k.append(error_k)

# Plot angular error
fig, ax = plt.subplots(figsize=(5, 4))
ax.boxplot(
    normal_errors_k,
    tick_labels=[f"k={k}" for k in k_values],
    showfliers=False
)
ax.set_ylabel("Angular error (rad)")
ax.set_yticks(
    [0., np.pi/8, np.pi/4, 3*np.pi/8, np.pi/2],
    ["0", " $\pi/8$ ", " $\pi/4$ ", " $3\pi/8$ ", " $\pi/2$ "]
)
ax.set_title("Normal direction estimation error")
ax.grid(True)
plt.show()
```

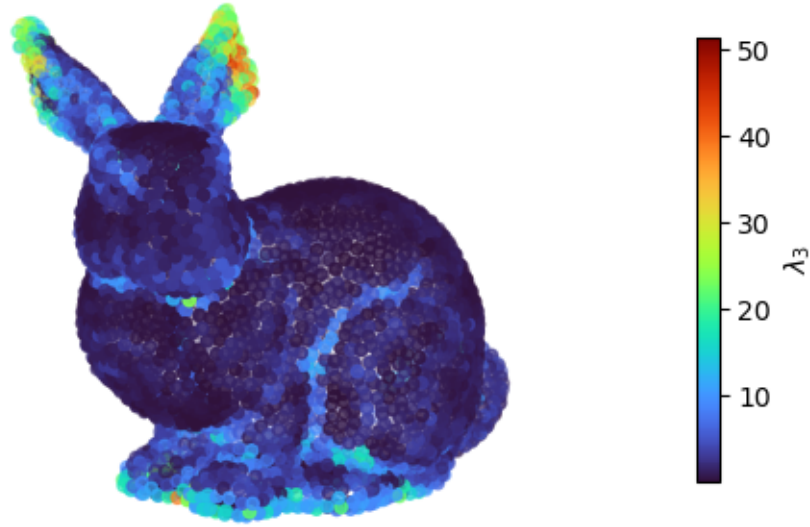


The quality of the local plane fit can be assessed from the smallest eigenvalue λ_3 . A small value indicates that the neighborhood is close to planar, while larger values suggest either noise or significant surface curvature. However, λ_3 is scale-dependant. A scale-independent measure is obtained by normalizing λ_3 by the sum of the eigenvalues, a quantity known as *surface variation* (see next section on curvatures)

The residuals highlight regions where the tangent plane approximation is less valid. As expected, larger residuals are observed near highly curved areas such as the ears of the bunny below.

```
[5]: # Visualize residuals
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
im = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                c=residuals, cmap="turbo")
cbar = fig.colorbar(im, ax=ax, shrink=0.5, pad=0)
cbar.set_label("$\\lambda_3$")
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=10, azim=60)
ax.set_axis_off()
ax.set_title(f"Residuals for k={k_normals}")
plt.show()
```

Residuals for k=10



6.1.2 Orientation with Riemannian Graph

The tangent plane estimation procedure only determines the normal direction up to a sign (i.e., $\pm n$). As a result, neighboring points may have normals pointing in opposite directions, producing inconsistent orientations across the surface.

To resolve this ambiguity, it is commonly assumed that the pointcloud is sufficiently dense and that the underlying surface is locally smooth. Under these conditions, tangent planes estimated at two nearby points p_i and p_j are expected to be nearly parallel. Consequently, their associated normals should satisfy:

$$n_i \cdot n_j \approx \pm 1$$

Two normals are considered consistently oriented when:

$$n_i \cdot n_j > 0$$

Otherwise, one of them must be flipped.

The main difficulty is determining which pairs of neighboring points should be compared. This is usually addressed by **finding a path through neighboring points that minimizes variations in normal directions**, using a *Riemannian Graph*. In this graph, vertices correspond to points, edges connect nearby points, and edge weights are derived from the angle between neighboring normals. **The desired path is obtained by extracting the minimum spanning tree of the graph**, which connects all points while minimizing the total variation in normal directions. Starting from a chosen seed point, the tree is then traversed and normals are flipped when necessary to enforce a consistent orientation over the entire pointcloud.

The procedure can be summarized as follows:

1. Compute the Euclidean Minimum Spanning Tree (EMST), which connects all points while minimizing the total edge length.
2. Add edges to the EMST to connect each point to its k -nearest neighbors to form the *Riemannian Graph*.
3. Define edge weights as:

$$e_{ij} = 1 - |n_i \cdot n_j|$$

4. Extract the minimum spanning tree of the *Riemannian Graph*.
5. Select an initial point and orient its normal (for example make the point with the highest z coordinate pointing towards z^+).
6. Traverse the tree and flip normal n_j whenever $n_i \cdot n_j < 0$.

Note that the EMST is a subgraph of the Delaunay triangulation. Computing the Delaunay triangulation is therefore an efficient way of obtaining the EMST.

The implementation proposed below mostly relies on `scipy` routines for nearest-neighbor search, Delaunay triangulation, minimum spanning tree extraction, and graph traversal. Graphs are represented as sparse matrices, where the value at position (i, j) corresponds to the weight of the edge connecting vertices v_i and v_j .

```
[6]: def orient_normals(points, normals, neighbor_finder):
    """Orient normals using a Riemannian Minimum Spanning Tree (EMST) approach.
    ↪ """

    normals = normals.copy()
    # Create graph (adjacency matrix)
    n_points = len(points)
    riemannian_graph = dok_matrix((n_points, n_points), dtype=np.float32)

    # Compute the Euclidean graph using Delaunay triangulation
    tri = Delaunay(points)
    indptr, indices = tri.vertex_neighbor_vertices
    for i in range(n_points) :
        neighbors = indices[indptr[i]:indptr[i+1]]
        dists = np.linalg.norm(points[i] - points[neighbors], axis=1)
        riemannian_graph[i, neighbors] = dists
```



```

# Compute the minimum spanning tree of the Euclidean graph (EMST)
riemannian_graph = minimum_spanning_tree(riemannian_graph, overwrite=True)
riemannian_graph = riemannian_graph.todok() # faster manipulation with DOK

# Add k-nearest neighbors to the Riemannian graph
neighbors = neighbor_finder(points) # point itself is included
neighbors = neighbors[:, 1:] # Remove the point itself
for i, nbrs in enumerate(neighbors):
    riemannian_graph[i, nbrs] = 1.

# Weight the edges of the Riemannian graph according to normal similarity
for (i, j) in riemannian_graph.keys():
    riemannian_graph[i, j] = 1 - np.abs(normals[i] @ normals[j]) + 1e-6

# Compute the minimum spanning tree of the Reimannian graph
riemannian_graph = minimum_spanning_tree(riemannian_graph, overwrite=True)

# Choose the point with the highest z-coordinate as the root
root_index = np.argmax(points[:, 2])
# and orient its normal upwards
if normals[root_index, 2] < 0:
    normals[root_index] *= -1

# Propagate normals through the MST
node_array, predecessors = breadth_first_order(
    riemannian_graph, root_index, directed=False, return_predecessors=True
)
node_array = node_array[1:] # Exclude the root node from processing
for current in node_array:
    previous = predecessors[current]
    # Ensure normal consistency
    if normals[current] @ normals[previous] < 0:
        normals[current] *= -1

return normals

```

The orientation procedure is now applied to the estimated normals. Since the sign ambiguity has been resolved, the angular error is computed without taking the absolute value of the dot product.

```

[7]: # Orient normals (reuse k=10)
neighbor_finder = lambda points: kdtree.query(points, k_normals)[1]
oriented_normals = orient_normals(points, normals, neighbor_finder)

# Estimate angular error between estimated, oriented and reference normals
normal_orientation_errors = []
for n in [normals, oriented_normals]:
    error_orient = np.sum(n * ref_normals, axis=1) # no np.abs here

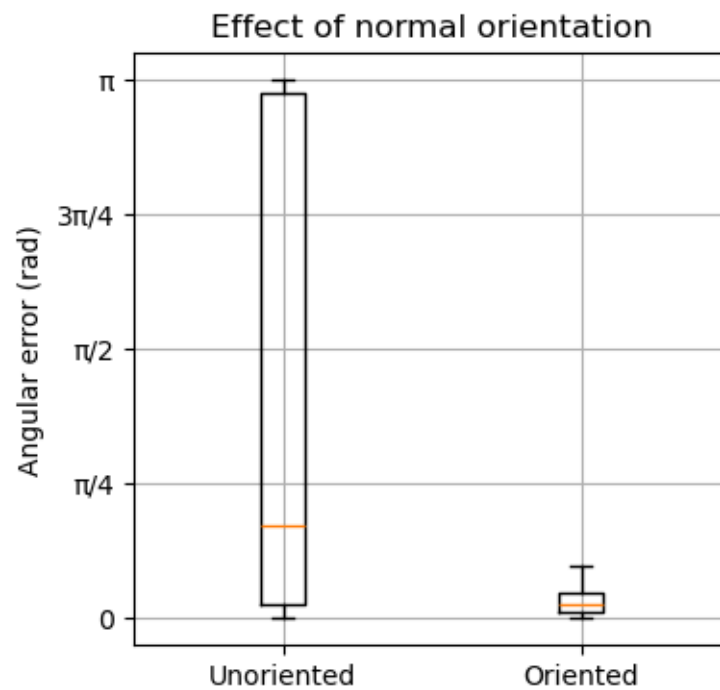
```

```

error_orient = np.clip(error_orient, -1.0, 1.0)
error_orient = np.arccos(error_orient)
normal_orientation_errors.append(error_orient)

# Plot angular error between estimated normals and reference normals
fig, ax = plt.subplots(figsize=(4, 4))
ax.boxplot(
    normal_orientation_errors,
    tick_labels=["Unoriented", "Oriented"],
    showfliers=False
)
ax.set_ylabel("Angular error (rad)")
ax.set_yticks(
    [0., np.pi/4, np.pi/2, 3*np.pi/4, np.pi],
    ["0", " $\pi/4$ ", " $\pi/2$ ", " $3\pi/4$ ", " $\pi$ "]
)
ax.set_title("Effect of normal orientation")
ax.grid(True)
plt.show()

```



6.2 Curvatures

Curvature measures how the underlying surface bends around a point. Unlike normals, which only describe surface orientation, curvatures characterize local shape and distinguish between flat, convex, concave, and saddle-like regions.

Most curvature measures come from differential geometry and the study of the local behavior of smooth surfaces. From this viewpoint, any sufficiently smooth surface admits at each point a tangent plane and an associated normal vector. Curvature expresses how the surface deviates from this tangent plane when moving in different tangent directions. **Principal curvatures** (κ_1, κ_2) **correspond to the maximum and minimum bending of the surface and are associated with two orthogonal directions** (t_1, t_2) , **called principal directions.**

Several commonly used curvature measures are derived from the principal curvatures. Among them, the most important are the **Gaussian curvature** K and the **mean curvature** H , defined by:

$$K = \kappa_1 \cdot \kappa_2 \quad H = \frac{\kappa_1 + \kappa_2}{2}$$

Gaussian curvature characterizes the local shape of the surface (saddle, planar, or dome-like), while mean curvature measures its average bending and distinguishes convex from concave regions. Together, they allow a local classification of surface shape, described below. Note that the distinction between a ridge and a valley, or between a peak and a pit, is entirely determined by the orientation of the surface.

	$K < 0$	$K = 0$	$K > 0$
$H < 0$	Saddle ridge	Ridge	Peak
$H = 0$	Minimal	Plane	(impossible)
$H > 0$	Saddle valley	Valley	Pit

The principal curvatures may also be recovered from K and H :

$$\kappa_1, \kappa_2 = H \pm \sqrt{H^2 - K}$$

Many other descriptors are also derived from the principal curvatures. Among these descriptors, the **shape index** S and **curvedness** C form a particularly influential pair [15]. The first provides a continuous description of the local shape type (e.g., dome, ridge, saddle, valley), while the second measures the overall magnitude of bending. Together, they provide an intuitive and scale-independent description of local surface geometry, making them useful for tasks such as segmentation, classification, and geometric feature detection.

$$S = \frac{2}{\pi} \arctan\left(\frac{\kappa_2 + \kappa_1}{\kappa_2 - \kappa_1}\right) \quad C = \sqrt{\frac{\kappa_1^2 + \kappa_2^2}{2}}$$

In differential geometry theory, principal curvatures κ_1 and κ_2 are obtained from the *Shape operator* (also called the *Weingarten map*), which describes how the surface normal varies along tangent directions. This requires constructing a local tangent frame, computing the first and second fundamental forms, assembling the shape operator, and diagonalizing it to obtain the principal curvatures and their directions.

In practice, these **differential geometry tools are not directly applicable to pointclouds, which are noisy discrete samples of an underlying surface**. As a consequence, curvatures cannot be computed exactly and must instead be estimated from local neighborhoods.

To bridge this gap, a variety of **estimation techniques** have been proposed, each offering a different compromise between robustness, computational cost and adherence to the principles of differential geometry. The following sections present some of the most common approaches.

6.2.1 PCA-based surface variation

As previously discussed, curvature is related to how a surface bends around a point. In a pointcloud, one way to estimate this bending is to measure how much the neighboring points deviate from the local tangent plane.

The previous section showed that a tangent plane can be estimated from the covariance matrix of a local neighborhood. The eigenvalues of this matrix describe the variation of the points along the principal directions of the neighborhood. In particular, the smallest eigenvalue λ_3 measures the variation along the normal direction and corresponds to the residual of the tangent plane fit.

This leads to a commonly used curvature indicator, called *surface variation* or *curvature change*, defined as the **ratio between this smallest eigenvalue and the total variance of the neighborhood** [16]:

$$C_\lambda = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$$

Higher surface variation indicates a less planar neighborhood. Its value ranges from 0 to 1/3, where 0 corresponds to a perfectly planar neighborhood, and 1/3 corresponds to an isotropic distribution.

It should be noted that surface variation does not provide the principal curvatures κ_1 and κ_2 , nor any direct estimate of Gaussian or mean curvature. Instead, it acts as **a simple proxy for the overall curvature magnitude**. Because it only requires the covariance eigenvalues already computed for normal estimation, it is inexpensive to evaluate and widely used in large-scale pointcloud processing. As with normal estimation, its accuracy depends strongly on the neighborhood size and the level of noise.

```
[8]: def compute_curvature_pca(points, neighbor_finder):
    """Compute curvature change indicator using PCA."""

    # Neighborhoods
    inds = neighbor_finder(points) # (N, k)
    neighbors = points[inds] # (N, k, 3)
    # Eigenvalues from unnormalized covariance matrix
    neighbors -= neighbors.mean(axis=1, keepdims=True) # center in-place
    cov = (neighbors.transpose(0, 2, 1) @ neighbors) # (N, 3, 3)
    eigenvalues = np.linalg.eigvalsh(cov) # (N, 3, 3)

    return eigenvalues[:, 0] / np.sum(eigenvalues, axis=1)

[9]: # Compute curvature change indicator
neighbor_finder = lambda point: kdtree.query(point, k=10)[1]
```

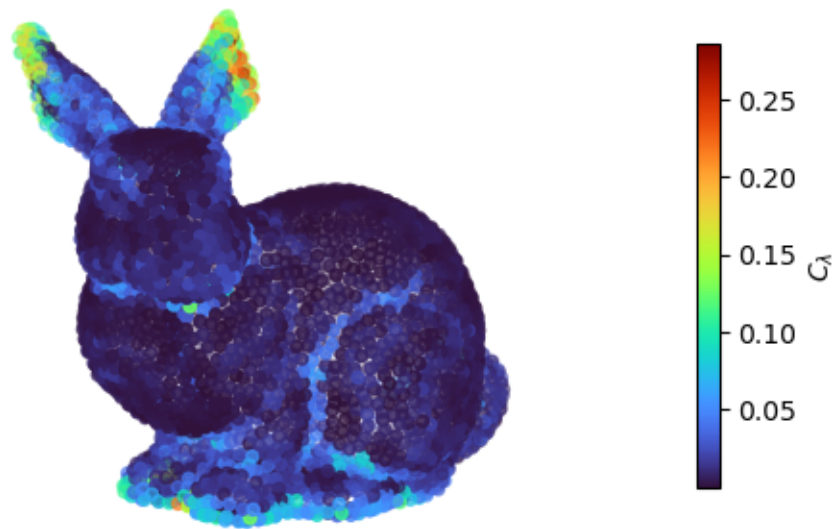
```

curvature_change = compute_curvature_pca(points, neighbor_finder)

# Visualize curvature change indicator
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
im = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                c=curvature_change, cmap="turbo")
cbar = fig.colorbar(im, ax=ax, shrink=0.5, pad=0)
cbar.set_label("$C_{\lambda}$")
ax.view_init(10, 60)
ax.set_box_aspect([1, 1, 1])
ax.set_axis_off()
ax.set_title("PCA-based surface variation")
plt.show()

```

PCA-based surface variation



6.2.2 Local surface fitting

Another common strategy for estimating curvature on pointclouds is to **locally approximate the underlying smooth surface by fitting a simple surface patch**, and then **apply standard differential-geometry formulas directly** to the fitted surface. This approach is appealing because it provides direct estimates of quantities such as principal, Gaussian, and mean curvatures. It is widely used in practice and performs particularly well on dense and smooth pointclouds. However, its accuracy depends strongly on the choice of neighborhood and it can be sensitive to noise.

Several types of surface models can be used. The choice generally involves a trade-off between simplicity and the ability to represent complex local geometry. The next subsections present two common approaches based on parametric and implicit surface fitting.

Parametric surfaces In 3D space, parametric surfaces express the coordinates of surface points as functions of two parameters:

$$r(u, v) = (x(u, v), y(u, v), z(u, v))$$

The generic process for estimating curvature from a parametric surface consists of computing its first- and second-order derivatives, deriving the associated fundamental forms, and extracting the principal curvatures from the corresponding shape operator.

The required derivatives are:

$$r_u = \frac{\partial r}{\partial u} \quad r_v = \frac{\partial r}{\partial v} \quad r_{uv} = \frac{\partial^2 r}{\partial u \partial v} \quad r_{uu} = \frac{\partial^2 r}{\partial^2 u} \quad r_{vv} = \frac{\partial^2 r}{\partial^2 v}$$

The surface normal is then given by:

$$n(u, v) = \frac{r_u \times r_v}{\|r_u \times r_v\|}$$

The first and second fundamental forms are constructed from these quantities:

$$I = Edu^2 + 2Fdudv + Gdv^2 \quad II = Ldu^2 + 2Mdudv + Ndv^2$$

where

$$E = r_u \cdot r_u = \|r_u\|^2 \quad F = r_u \cdot r_v \quad G = r_v \cdot r_v = \|r_v\|^2$$

and

$$L = r_{uu} \cdot n \quad M = r_{uv} \cdot n \quad N = r_{vv} \cdot n$$

The corresponding shape operator is:

$$P = \begin{pmatrix} E & F \\ F & G \end{pmatrix}^{-1} \begin{pmatrix} L & M \\ M & N \end{pmatrix}$$

Its eigenvalues correspond to the principal curvatures (κ_1, κ_2) , while the associated eigenvectors (ν_1, ν_2) define the principal directions in the tangent basis (r_u, r_v) . The principal directions may then be expressed in the local tangent frame as:

$$t_1 = \nu_{11}r_u + \nu_{12}r_v \quad t_2 = \nu_{21}r_u + \nu_{22}r_v$$

Gaussian and mean curvatures may also be computed directly from the coefficients of the first and second fundamental forms:

$$K = \frac{LN - M^2}{EG - F^2} \quad H = \frac{EN - 2FM + GL}{2(EG - F^2)}$$

In practice, many types of parametric surfaces can be used. A popular choice is the *quadric surface*, which provides a simple second-order approximation of the local geometry while remaining inexpensive to fit:

$$r(u, v) = (u, v, g(u, v))$$

with

$$w = g(u, v) = au^2 + bv^2 + cuv + du + ev + f$$

Because g is a second-degree polynomial, its derivatives are straightforward to compute. For example:

$$r_u = (1, 0, g_u) \quad g_u = 2au + cv + d$$

and

$$r_v = (0, 1, g_v) \quad g_v = 2bv + cu + e$$

After simplification, the normal, Gaussian curvature, and mean curvature at the origin $(0, 0)$ are given by:

$$n_0 = \frac{1}{\sqrt{d^2 + e^2 + 1}}(-d, -e, 1)$$

$$K_0 = \frac{4ab - c^2}{(d^2 + e^2 + 1)^2} \quad H_0 = \frac{a(1 + e^2) + b(1 + d^2) - cde}{(d^2 + e^2 + 1)^{3/2}}$$

A further simplification is obtained by placing the query point at the origin and aligning its normal with the vertical axis. This corresponds to setting $d = e = f = 0$, leading to:

$$K_0 = 4ab - c^2 \quad H_0 = a + b$$

The first challenge is to define a local coordinate system in which the query point lies at the origin and the normal is aligned with the vertical axis. For unorganized pointclouds, this local frame is typically constructed from the tangent plane estimated by PCA in the previous section.

The second challenge is to fit the quadric surface in this local coordinate system. Once the neighborhood has been projected into the local frame, the coefficients are obtained by least-squares fitting.

In summary, the curvature estimation process using a parametric surface follows these steps:

1. Compute the (consistently oriented) tangent plane for each point and construct the local coordinate system (see previous section about normals).
2. Express the neighborhood in the local frame and fit the quadric surface:

$$w = au^2 + bv^2 + cuv$$

using least-squares.

3. Compute the Gaussian and mean curvatures using the fitted surface coefficients:

$$K_0 = 4ab - c^2 \quad H_0 = a + b$$

The process is illustrated below using reference normals (derived from the bunny mesh) to isolate curvature estimation errors from errors introduced during tangent plane estimation and normal orientation. The implementation is partially vectorized: all neighborhoods are queried and extracted simultaneously, while local frame construction and quadric fitting are performed independently for each point. These latter steps could be further vectorized, but at the expense of readability.

```
[ ]: def compute_curvatures_parametric(points, normals, neighbor_finder):
    """Compute Gaussian and mean curvatures using parametric fitting."""

    N, _ = points.shape
    K_0 = np.empty(N)
    H_0 = np.empty(N)

    # Neighborhoods
    inds = neighbor_finder(points) # (N, k)
    neighbors = points[inds] # (N, k, 3)
    neighbors -= points[:, None, :] # center on the query point

    for i in range(N):
        # Project neighbors onto the tangent plane defined by the local normal
        basis = orthonormal_basis_from_w(normals[i])
        projected_neighbors = (basis @ neighbors[i].T).T
        # Fit a local quadratic patch to the projected neighbors
        # w = a.u^2 + b.v^2 + c.u.v
```



```

A = np.column_stack((
    projected_neighbors[:, 0]**2, #  $u^2$ 
    projected_neighbors[:, 1]**2, #  $v^2$ 
    projected_neighbors[:, 0] * projected_neighbors[:, 1], #  $u.v$ 
))
heights = projected_neighbors[:, 2] #  $w$ 
coeffs, *_ = np.linalg.lstsq(A, heights, rcond=None)
a, b, c = coeffs
# Compute Gaussian and Mean curvatures from the fitted coefficients
K_0[i] = 4*a*b - c**2
H_0[i] = a + b

return K_0, H_0

```

```

[11]: # Estimate curvatures using the parametric fitting approach
neighbor_finder = lambda points: kdtree.query(points, k=12)[1]
K_parametric, H_parametric = compute_curvatures_parametric(
    points,
    ref_normals, # instead of estimated normals
    neighbor_finder
)

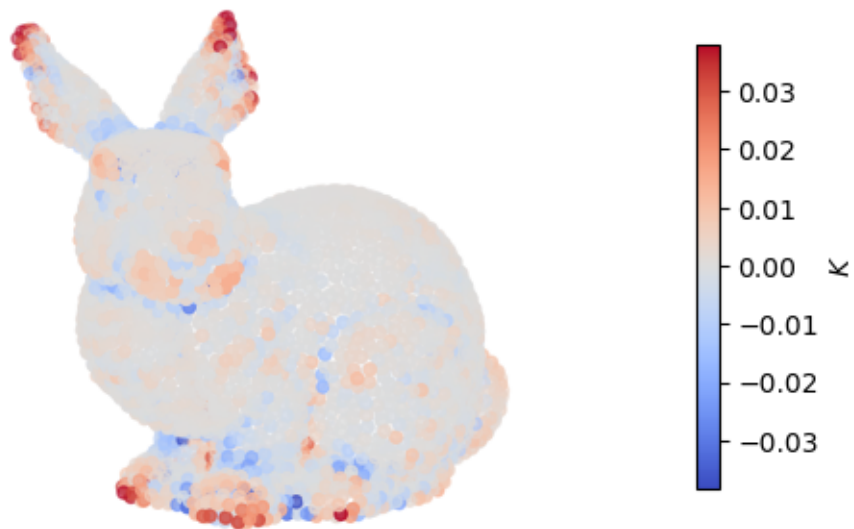
# Visualize curvatures
# Use reference values H_ref and K_ref for scales
ref_norms_k = {
    "Gaussian": matplotlib.colors.TwoSlopeNorm(
        vmin = -np.percentile(np.abs(K_ref), 99),
        vcenter = 0.,
        vmax = np.percentile(np.abs(K_ref), 99)
    ),
    "Mean": matplotlib.colors.TwoSlopeNorm(
        vmin = -np.percentile(np.abs(H_ref), 99),
        vcenter = 0.,
        vmax = np.percentile(np.abs(H_ref), 99)
    )
}

# Plot mean and Gaussian curvatures
for curvature_type, unit, values in [
    ("Gaussian", "$K$", K_parametric),
    ("Mean", "$H$", H_parametric)
]:
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection='3d')
    p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                   c=values, cmap="coolwarm", norm=ref_norms_k[curvature_type])
    cbar = fig.colorbar(p, shrink=0.5, pad=0)
    cbar.set_label(unit)

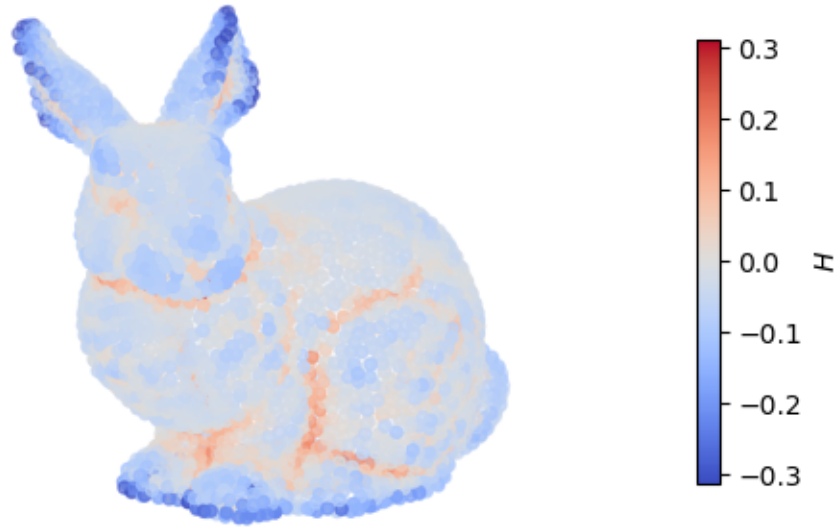
```

```
ax.set_title(f"{curvature_type} curvature from parametric quadric fitting")
ax.set_box_aspect([1, 1, 1])
ax.set_axis_off()
ax.view_init(10, 60)
plt.show()
```

Gaussian curvature from parametric quadric fitting



Mean curvature from parametric quadric fitting



Implicit surfaces 3D implicit surfaces represent the geometry as the zero-level set of a scalar field, such as:

$$F(x, y, z) = 0$$

Like parametric surfaces, implicit surfaces provide direct access to differential quantities such as normals and curvatures. Their main advantage is that no surface parameterization is required.

The generic process for computing curvatures from an implicit surface begins by computing the gradient, from which the unit normal is obtained:

$$n = \frac{\nabla F}{\|\nabla F\|} \quad \text{with} \quad \nabla F = (F_x, F_y, F_z)$$

Then computing the Hessian matrix:

$$H_F = \begin{pmatrix} F_{xx} & F_{xy} & F_{xz} \\ F_{yx} & F_{yy} & F_{yz} \\ F_{zx} & F_{zy} & F_{zz} \end{pmatrix}$$

Finally, projecting the Hessian onto the tangent plane to obtain the shape operator:

$$S = \frac{-1}{\|\nabla F\|} P^T H_F P \quad \text{with} \quad P = I - n \cdot n^T$$

The principal curvatures κ_1 and κ_2 are the two non-zero eigenvalues of this *shape operator* S , while the associated eigenvectors define the principal directions. Because the operator is represented in ambient three-dimensional space, it contains an additional eigenvalue associated with the surface normal, which is theoretically zero and must be discarded.

Many implicit surfaces can be used to estimate curvature. Choosing one is again a balance between capturing local shape variations and fitting the model efficiently. A common choice is the quadratic implicit surface, also called a *quadric*, which can be viewed as the implicit counterpart of the quadratic parametric surface introduced above:

$$ax^2 + by^2 + cz^2 + 2exy + 2fyz + 2gzx + 2lx + 2my + 2nz + d = 0$$

Because F is a second-degree polynomial, its partial derivatives are straightforward. For example, $F_x = 2ax + 2ey + 2gz + 2l$, and so on.

The first order derivatives are used to compute the gradient for our query point $p_0 = (x_0, y_0, z_0)$:

$$\nabla F_0 = \begin{pmatrix} 2ax_0 + 2ey_0 + 2gz_0 + 2l \\ 2by_0 + 2ex_0 + 2fz_0 + 2m \\ 2cz_0 + 2fx_0 + 2gy_0 + 2n \end{pmatrix}$$

The second order derivatives are used to compute the Hessian, which is constant for our quadric:

$$H_0 = \begin{pmatrix} 2a & 2e & 2g \\ 2e & 2b & 2f \\ 2g & 2f & 2c \end{pmatrix}$$

The normal and the shape operator are then computed as described above.

As in the parametric case, the query point is often translated to the origin before fitting. This eliminates the constant term of the quadric and generally improves stability during fitting.

Implicit functions can be **fit directly to unorganized 3D points using least squares techniques**, making them especially appealing for modern scanning data. Unlike parametric fitting, no local tangent frame needs to be constructed before fitting the surface.

In summary, the curvatures estimation process using an implicit surface follows these steps:

1. Extract a neighborhood around each point.

2. Fit a quadratic implicit surface using least squares.
3. Compute the gradient, Hessian, and shape operator of the fitted surface.
4. Extract the two principal curvatures from the shape operator (and derive Gaussian and mean curvatures from the principal curvatures).

The implicit fit naturally provides a normal estimate through the gradient of the fitted function. However, an implicit surface is defined only up to a multiplicative constant: $F(x, y, z) = 0$ and $-F(x, y, z) = 0$ describe the same geometry. So, the fitted normal and principal curvatures are only determined up to a sign. A subsequent normal-orientation step is then required to recover signed principal and mean curvatures. Gaussian curvature is unaffected because it is independent of surface orientation.

The implementation is partially vectorized: all neighborhoods are queried and extracted simultaneously, while quadric fitting and shape-operator computation are performed independently for each point. Because each neighborhood defines a separate least-squares problem, further vectorization would significantly increase implementation complexity for limited practical gains.

Implicit fitting occasionally generates unrealistically large curvature values because some fitted quadrics are nearly singular. Although these fits match the local neighborhood well in an algebraic sense, the small gradient norm can make curvature estimates numerically unstable.

```
[ ]: def compute_curvature_implicit(points, neighbor_finder):
    """Estimate Gaussian and mean curvatures using implicit fitting."""

    N, _ = points.shape
    normals = np.empty_like(points)
    kappa_1 = np.empty(N)
    kappa_2 = np.empty(N)

    # Neighborhoods
    inds = neighbor_finder(points) # (N, k)
    neighbors = points[inds] # (N, k, 3)
    neighbors -= points[:, None, :] # center on the query point

    for i in range(N):
        X = neighbors[i, :, 0]
        Y = neighbors[i, :, 1]
        Z = neighbors[i, :, 2]

        # Fit a local implicit quadratic surface to the neighbors
        # F(x, y, z) = ax^2 + by^2 + cz^2 + 2exy + 2fyz + 2gzx + 2lx + 2my + 2nz = 0
        # (d omitted because the query point is translated to origin)
        design_matrix = np.column_stack((
            X**2, Y**2, Z**2,
            2*X*Y, 2*Y*Z, 2*Z*X,
            2*X, 2*Y, 2*Z,
        ))
```

```

# Fit using SVD
_, S, Vt = np.linalg.svd(design_matrix, full_matrices=False)
coeffs = Vt[-1]
coeffs /= np.linalg.norm(coeffs) # improves stability
a, b, c, e, f, g, l, m, n = coeffs

# Gradient of F at the query point
gradient = np.array([2*l, 2*m, 2*n])
# Unit normal vector at the query point
norm_grad = np.linalg.norm(gradient)
normal = gradient / norm_grad
normals[i] = normal

# Projection matrix onto the tangent plane at the query point
projection = np.eye(3) - np.outer(normal, normal)
# Hessian of F (which is constant for a quadric)
hessian = np.array([
    [2*a, 2*e, 2*g],
    [2*e, 2*b, 2*f],
    [2*g, 2*f, 2*c],
])

# Shape operator projected onto the tangent plane
S = -(projection @ hessian @ projection) / norm_grad
S = (S + S.T) / 2 # Force symmetry to improve stability

# Principal curvatures
eigenvalues = np.linalg.eigvalsh(S)
# Remove the null eigenvalue associated with the normal
null_idx = np.argmin(np.abs(eigenvalues))
principal = np.delete(eigenvalues, null_idx)
# Order remaining eigenvalues
kappa_1[i] = np.max(principal)
kappa_2[i] = np.min(principal)

return kappa_1, kappa_2, normals

```

```

[13]: # Compute curvatures using the implicit surface fitting approach
neighbor_finder = lambda points: kdtree.query(points, k=12)[1]
kappa_1_implicit, kappa_2_implicit, normals_implicit = \
    compute_curvature_implicit(points, neighbor_finder)

# Orient with reference normals (alternatively with graph)
orientation = np.sign(
    np.sum(normals_implicit * ref_normals, axis=1)
)
normals_implicit *= orientation[:, None]

```

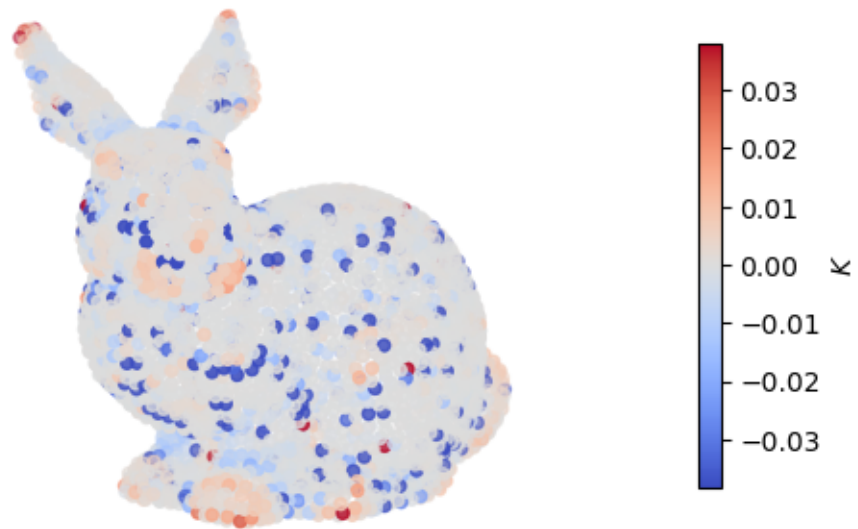
```

kappa_1_implicit *= orientation
kappa_2_implicit *= orientation
K_implicit = kappa_1_implicit * kappa_2_implicit
H_implicit = (kappa_1_implicit + kappa_2_implicit)/2

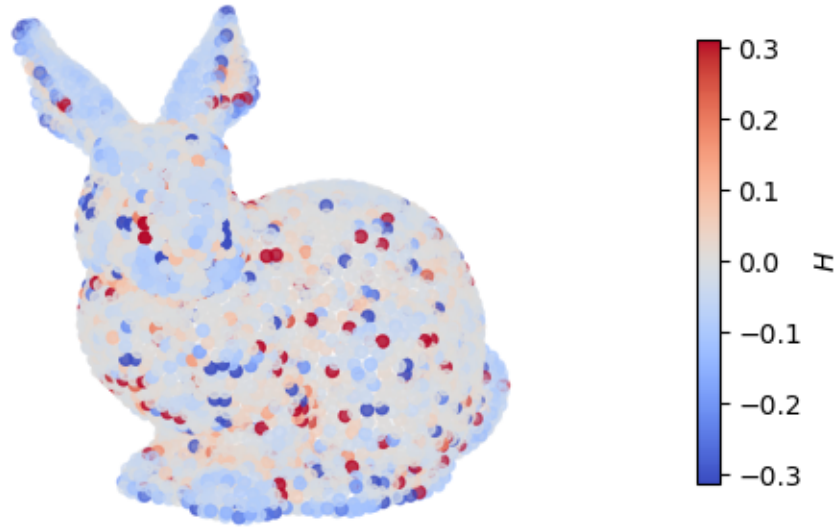
# Plot mean and Gaussian curvatures
for curvature_type, unit, values in [
    ("Gaussian", "$K$", K_implicit),
    ("Mean", "$H$", H_implicit)
]:
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection='3d')
    p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                   c=values, cmap='coolwarm', norm=ref_norms_k[curvature_type])
    cbar = fig.colorbar(p, shrink=0.5, pad=0)
    cbar.set_label(unit)
    ax.set_box_aspect([1, 1, 1])
    ax.set_axis_off()
    ax.view_init(10, 60)
    ax.set_title(f"{curvature_type} curvature from implicit surface fitting")
    plt.show()

```

Gaussian curvature from implicit surface fitting



Mean curvature from implicit surface fitting



6.2.3 Discrete differential operators

Discrete differential operator methods aim to **adapt smooth differential-geometry operators directly to pointclouds**, eliminating the need to reconstruct a local smooth surface. The core idea is to approximate operators, such as the shape operator using only local neighborhoods of points, producing curvature estimates without explicit surface fitting. Most discrete differential operators were originally developed for triangle meshes, where connectivity naturally supports discrete analogs of differential geometry. When adapted to pointclouds, mesh connectivity is typically replaced by local neighborhoods defined through k -nearest neighbors.

The explanation below focuses on an early influential technique, often referred to as **Taubin's method**, which **infers curvature from how vertex normals change over local neighborhoods** [17].

Taubin's method relies on the *tensor of curvature*, a quadratic form that associates a directional curvature $\kappa(t)$ with every tangent direction t . The directional curvature is approximated from neighboring points using the finite-difference expression:

$$\kappa(t) \approx \frac{2n \cdot (q - p)}{\|q - p\|^2}$$

where p is the query point, n its normal, and q its neighbor in direction t . This quantity measures how far the neighbor deviates from the tangent plane along the considered direction.

Curvature estimates collected from all neighboring directions are then combined to build a symmetric matrix approximating the local curvature tensor, from which the principal curvatures and directions may be recovered.

The quality of this approximation depends strongly on how uniformly the neighboring points sample the tangent plane. Anisotropic neighborhoods can introduce an artificial difference between the principal curvatures, even on surfaces whose curvature is theoretically identical in all directions (e.g., a sphere). Increasing the neighborhood size or using more uniformly distributed samples generally reduces this bias by improving the coverage of tangent directions.

The original algorithm is formulated for triangular meshes, where vertex normals are estimated from the weighted average of incident face normals and curvature tensors are built from edge-based neighborhoods. For pointclouds, the same procedure can be adapted by estimating normals using local tangent-plane fitting and defining neighborhoods using k -nearest neighbors instead of mesh connectivity. An adaptation of the method to pointclouds is summarized below.

For each point p_i of the pointcloud with normal n_i :

1. Query the k -nearest neighbors of p_i and form the set N_i .
2. Approximate the curvature tensor matrix as a weighted sum over neighbors:

$$M_i = \sum_{p_j \in N_i} \omega_{ij} \kappa_{ij} t_{ij} t_{ij}^T$$

with projected tangent directions:

$$t_{ij} = \frac{(I - n_i n_i^T) \cdot (p_i - p_j)}{\|(I - n_i n_i^T) \cdot (p_i - p_j)\|}$$

and directional normal curvature:

$$\kappa_{ij} = \frac{2n_i \cdot (p_j - p_i)}{\|p_j - p_i\|^2}$$

and distance-based weights (or optionally constant weights):

$$\omega_{ij} = \frac{\|p_j - p_i\|^{-2}}{\sum_{p_j \in N_i} \|p_j - p_i\|^{-2}}$$

3. Restrict the curvature tensor approximation to a basis aligned with the tangent plane. This is done using a Householder transformation:

$$Q_i M_i Q_i^T = \begin{pmatrix} 0 & 0 & 0 \\ 0 & m_{11}^i & m_{12}^i \\ 0 & m_{21}^i & m_{22}^i \end{pmatrix}$$

with

$$Q_i = I - 2w_i w_i^T \quad w_i = \frac{e_1 \pm n_i}{\|e_1 \pm n_i\|}$$

where the sign is chosen to maximize the denominator.

4. Diagonalize the resulting 2×2 tangent-plane block to obtain its eigenvalues and eigenvectors. This can be done using either a standard eigendecomposition or a Givens rotation (as in the original paper):

$$R(\theta)^T M_i^{2 \times 2} R(\theta) = \begin{pmatrix} m_{11'}^i & 0 \\ 0 & m_{22'}^i \end{pmatrix}$$

where

$$\tan(2\theta) = \frac{2m_{12}^i}{m_{11}^i - m_{22}^i}$$

5. Recover κ_1^i and κ_2^i principal curvatures associated with p_i as:

$$\kappa_1^i = 3m_{11'}^i - m_{22'}^i \quad \kappa_2^i = 3m_{22'}^i - m_{11'}^i$$

The curvature tensor approximation averages directional curvature in such a way that:

$$m_{11'}^i = \frac{3}{8}\kappa_1^i + \frac{1}{8}\kappa_2^i \quad m_{22'}^i = \frac{1}{8}\kappa_1^i + \frac{3}{8}\kappa_2^i$$

In practice, **the weighting strategy used to construct the curvature tensor can significantly affect the quality of the estimated principal curvatures**. Uniform weights are simple and faithful to the original formulation, while distance-based weights often reduce the bias caused by irregular point distributions by giving more importance to nearby neighbors. As a result, they generally produce more isotropic curvature estimates and can improve accuracy on non-uniformly sampled pointclouds.

This process is illustrated below. Reference normals (derived from the bunny mesh) are again used to isolate curvature estimation errors from errors introduced during normal estimation. The implementation is partially vectorized and all neighborhoods are queried and extracted simultaneously.

```
[14]: def compute_curvature_taubin(points, normals, neighbor_finder):
    """Compute principal curvatures and directions using Taubin's method."""

    N, _ = points.shape
    kappa_1 = np.empty(N)
    kappa_2 = np.empty(N)

    # Neighborhoods
    inds = neighbor_finder(points) # (N, k)
    neighbors = points[inds] # (N, k, 3)
    neighbors -= points[:, None, :] # center on the query point

    # Unit vector for Householder
    e_1 = np.array([1., 0., 0.])

    for i in range(N):
```

```

# Query normal
n_i = normals[i]

# Projection matrix onto the tangent plane at p_i ( $P = I - n.n^T$ )
proj_i = np.eye(3) - np.outer(n_i, n_i)

# Curvature tensor matrix
diffs = neighbors[i, 1:] # remove the point itself from neighbors
dists2 = np.sum(diffs**2, axis=1)

# Tangent vectors from neighbors projected onto the tangent plane
t_ij = diffs @ proj_i
t_ij /= np.linalg.norm(t_ij, axis=1, keepdims=True) # (check zero-div?)
# Curvature contribution from neighbors
kappa_ij = 2 * (diffs @ n_i) / dists2
# Accumulate curvature contributions
weights = 1.0 / dists2
weights /= weights.sum()
M_i = (((weights * kappa_ij)[: , None]) * t_ij).T @ t_ij

# Restrict M_i to the tangent plane (2D)
if n_i[0] < 0:
    w_i = (e_1 - n_i) / np.linalg.norm(e_1 - n_i)
else:
    w_i = (e_1 + n_i) / np.linalg.norm(e_1 + n_i)
# Householder projection matrix to restrict to the tangent plane
Q_i = np.eye(3) - 2 * np.outer(w_i, w_i)
# Restrict M_i to the tangent plane
Mt_i = Q_i @ M_i @ Q_i # Householder matrix is orthogonal and symmetric
m_11 = Mt_i[1, 1]
m_12 = Mt_i[1, 2]
m_22 = Mt_i[2, 2]

# Diagonalize the restricted curvature tensor
# (using linalg instead of Givens rotations)
eigenvalues = np.linalg.eigvalsh([[m_11, m_12], [m_12, m_22]])
m_11_p, m_22_p = eigenvalues[::-1]

# Equation 5 from Taubin's paper
kappas = np.array([3*m_11_p - m_22_p, 3*m_22_p - m_11_p])
# enforce kappa_1 > kappa_2 convention
kappa_1[i] = np.max(kappas)
kappa_2[i] = np.min(kappas)

return kappa_1, kappa_2

```

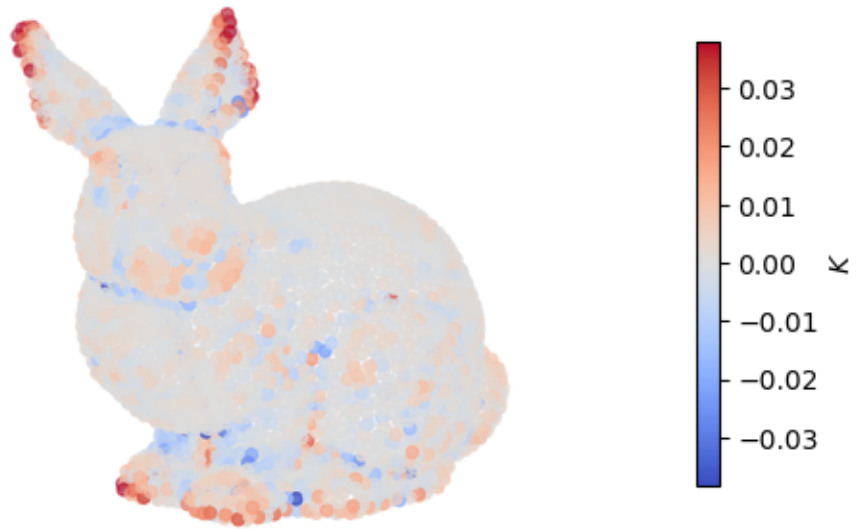
```

[15]: # Compute curvatures using Taubin's method
neighbor_finder = lambda points: kdtree.query(points, k=12)[1]
kappa_1_taubin, kappa_2_taubin = compute_curvature_taubin(
    points,
    ref_normals, # instead of estimated normals
    neighbor_finder
)
# Gaussian and mean curvatures for comparison with previous methods
K_taubin = kappa_1_taubin * kappa_2_taubin
H_taubin = 0.5 * (kappa_1_taubin + kappa_2_taubin)

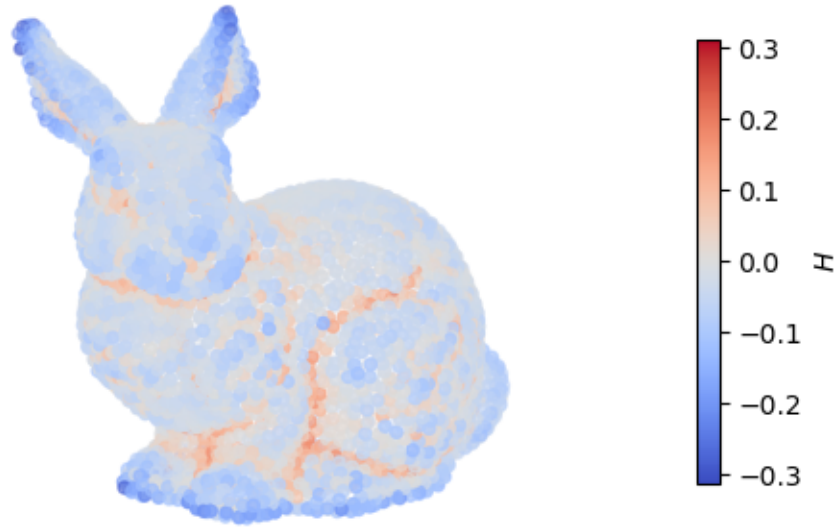
# Plot mean and Gaussian curvatures
for curvature_type, unit, values in [
    ("Gaussian", "$K$", K_taubin),
    ("Mean", "$H$", H_taubin)
]:
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection='3d')
    p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                    c=values, cmap='coolwarm', norm=ref_norms_k[curvature_type])
    cbar = fig.colorbar(p, shrink=0.5, pad=0)
    cbar.set_label(unit)
    ax.set_box_aspect([1, 1, 1])
    ax.set_axis_off()
    ax.view_init(10, 60)
    ax.set_title(f"{curvature_type} curvature from Taubin's method")
    plt.show()

```

Gaussian curvature from Taubin's method



Mean curvature from Taubin's method



6.2.4 Quick comparison

Below is a comparison of curvature estimates obtained with parametric fitting, implicit fitting, and Taubin's method, together with reference values computed on the original bunny mesh using a surface fitting approach. All implemented methods produce similar results on this relatively clean and evenly sampled pointcloud.

Parametric fitting formulation matches the differential geometry definition of curvature, is computationally efficient and relatively easy to implement. However, its accuracy depends on a reliable prior normal estimate and on the validity of the local parameterization. Performance can degrade near steep features, boundaries, or in the presence of significant noise.

Implicit fitting does not require local parameterization, so it can naturally handle arbitrary surface orientations and is generally less sensitive to errors in the estimated tangent frame. This additional flexibility often improves robustness on noisy pointclouds, although at the cost of solving a larger fitting problem.

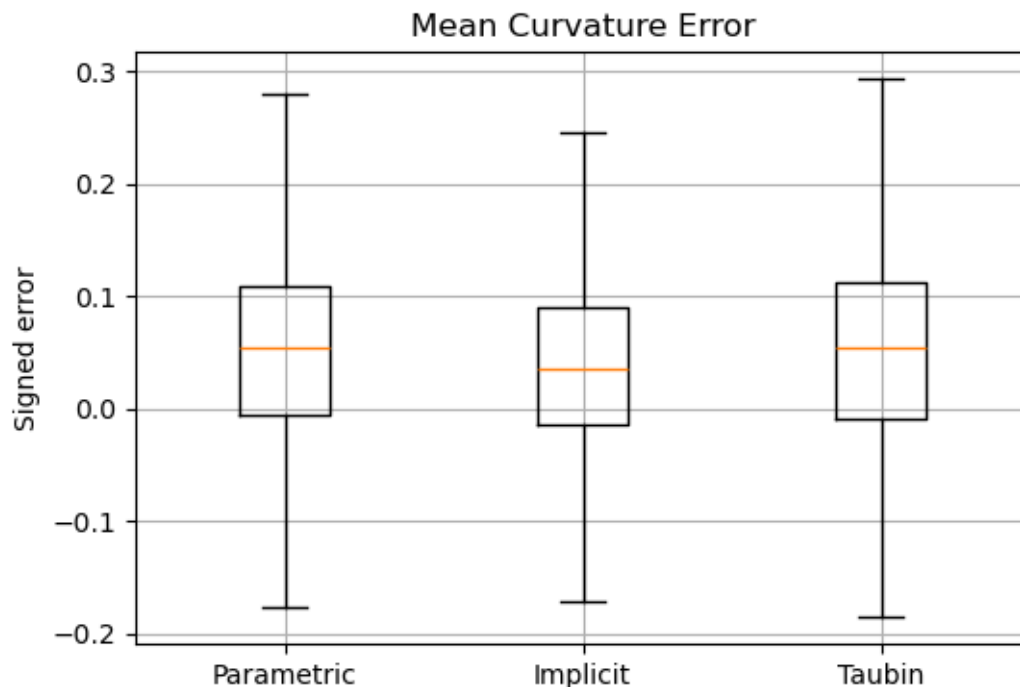
Rather than fitting a geometric surface, Taubin's method assumes that curvature can be recovered from the variation of normals within a neighborhood. This approach is elegant and computationally

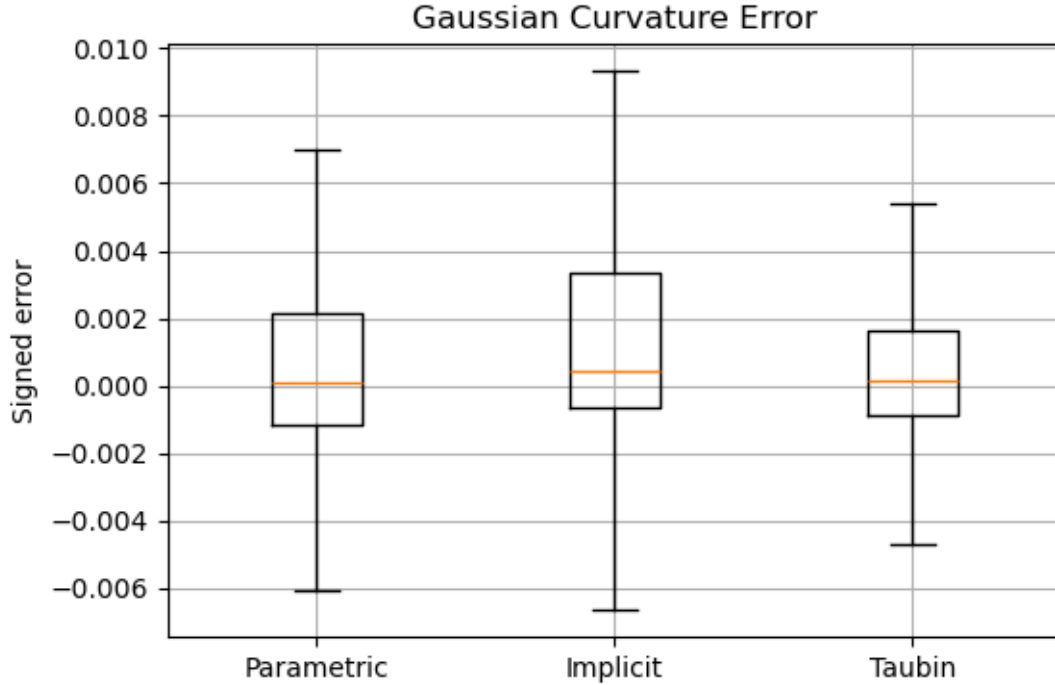
efficient once normals are available. However, its accuracy depends directly on the quality of the normal estimates, so noise, normal errors, and irregular sampling can have a stronger impact on the results.

In practice, the differences between the methods become more visible as pointcloud quality decreases. Parametric fitting generally works well on smooth and densely sampled surfaces, implicit fitting is often the most robust to noise and orientation changes, while Taubin's method is attractive for large datasets because of its low computational cost.

```
[16]: # Estimate angular error between estimated and reference curvatures
for title, ref, estimates in [
    ("Mean Curvature Error", H_ref, [H_parametric, H_implicit, H_taubin]),
    ("Gaussian Curvature Error", K_ref, [K_parametric, K_implicit, K_taubin]),
]:
    error_curvatures = [ref - est for est in estimates]

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.boxplot(
        error_curvatures,
        tick_labels=["Parametric", "Implicit", "Taubin"],
        showfliers=False
    )
    ax.set_ylabel("Signed error")
    ax.set_title(title)
    ax.grid(True)
    plt.show()
```





6.3 Going further

This section briefly discusses more advanced variants of the techniques detailed earlier for normal estimation, normal orientation, and curvature estimation.

Normal estimation

Despite being introduced several decades ago, PCA-based tangent-plane fitting remains one of the most widely used techniques for estimating normals in pointclouds. As discussed previously, it assumes that the underlying surface is locally smooth, an assumption that is often violated near sharp features, boundaries, or regions of high curvature.

One important source of error comes from the choice of neighborhood. Standard k -nearest-neighbor or spherical neighborhoods are not always well suited to approximate a local tangent plane, particularly in regions with high curvature or close surfaces. The `orient_normals_consistent_tangent_plane` implementation of `open3D` addresses this issue by using a pseudo-spherical neighborhood that penalizes the distance of neighboring points to the tangent plane and optionally rejects neighbors whose normals deviate excessively from the local surface orientation. However, this approach requires an initial normal estimate of sufficient quality. Other strategies include adapting the neighborhood size to the local surface curvature and noise level [18], or weighting neighbors according to their distance from the query point.

Another way to improve normal estimation is to fit more expressive local surface models, such as spheres, quadrics, or higher-order polynomial patches. One notable example is *jet fitting* [13],

implemented for example in **CGAL** through `jet_estimate_normals`. Another influential family of techniques relies on *Moving Least Squares* (MLS) surface reconstruction [11]. These approaches generally capture surface variations more accurately than a planar model, but they also introduce additional parameters and may become more sensitive to noise.

Surface fitting is not the only approach to normal estimation. Alternatives rely on RANSAC, Voronoi-based constructions, or Hough-transform techniques. A notable example combines a Hough-transform accumulator with convolutional neural networks to improve robustness to sharp edges, density variations, noise, and outliers [19]. The implementation is open-source and available in CloudCompare through the **HoughNormals** plug-in. Such alternatives often improve robustness at the expense of computational efficiency. More recently, deep-learning approaches have emerged, driven by the availability of large-scale datasets and benchmarks.

Normal orientation

Obtaining a consistent orientation is often more challenging than estimating the normal directions themselves. The most widely used orientation algorithms propagate orientation across local neighborhoods under the assumption that neighboring tangent planes are nearly parallel, which is equivalent to assuming low local curvature.

To reduce the impact of this assumption, several variants modify the edge-weighting functions and flipping criteria used during propagation. One such approach assumes locally constant curvature, effectively treating neighboring points as lying on a sphere rather than on parallel planes, and replaces the incident normal n_i by its reflection n'_i across the bisector plane of the segment $e_{ij} = p_i - p_j$ [20]. This modification preserves most of the original algorithm while improving robustness in regions of high curvature. This is not the case with some other proposed techniques, which rely on more complex edge weighting functions and flip criteria, whose often modest improvements come with higher implementation and computational costs.

Another family of approaches formulates orientation as a propagation problem on a spatial grid. For example, CloudCompare implements a Fast Marching strategy through the **Orient Normals With Fast Marching** tool. These methods are extremely fast and scale well to large pointclouds, but their accuracy depends strongly on the chosen grid resolution.

Alternative strategies rely on additional information, such as scanner positions, visibility constraints, or surface reconstruction techniques. More recently, deep-learning approaches have emerged, and now provide state-of-the-art results on several normal-orientation tasks.

Curvatures

Curvature estimation from 3D pointclouds is particularly challenging because differential-geometric quantities are defined on smooth surfaces, whereas pointclouds provide only a discrete sampling of the geometry. As a result, many workflows rely on simple curvature proxies, such as *PCA-based surface variation*, rather than attempting to estimate full principal curvatures.

Local surface fitting remains one of the most widely adopted strategies for estimating point-wise curvatures. The simplest models approximate the local geometry using parametric or implicit quadrics, while higher-order approaches such as *jet fitting* [13] extend this idea with richer polynomial models. A reference implementation is for example available in **CGAL** through `Monge_via_jet_fitting`. Robustness can also be improved by incorporating normal information directly into the fitting process, reducing sensitivity to noise and irregular sampling [21].

Another influential family of approaches is based on *Moving Least Squares* (MLS) surfaces [11]. Rather than fitting an independent surface to each neighborhood, MLS constructs a smooth surface approximation from weighted local polynomial regressions. This often improves robustness to noise and has become a standard tool in pointcloud processing pipelines. Implementations are available in libraries such as PCL and CGAL.

Discrete differential operators are less commonly applied directly to raw pointclouds, as they were originally developed for triangle meshes where connectivity supports discrete analogues of smooth operators. Some techniques can be adapted to pointclouds using only nearest-neighbor information. For example, `PrincipalCurvaturesEstimation` found in PCL computes curvatures by projecting neighbor normals into the tangent plane and applying PCA. In the general case, many discrete curvature estimators still require constructing a mesh as an intermediate representation. Once a mesh is available, a broad range of tools from discrete differential geometry become accessible, including angle-defect Gaussian curvature, cotangent-Laplacian mean curvature, normal-cycles estimators, and numerous other operators developed for mesh processing.

6.4 Wrapping up

You should now have a clearer understanding of how to extract meaningful local geometric information from pointclouds through the estimation of normals and curvatures. These quantities form the foundation of many geometry-processing algorithms. Normals describe the local surface orientation, while curvatures quantify how the surface bends, providing a richer description of the underlying shape.

Normals are most commonly estimated through local plane fitting using Principal Component Analysis (PCA), which simultaneously provides a tangent plane and its least-squares normal direction. Because these normals are initially unoriented, a second step is required to ensure consistency across the pointcloud, typically through graph-based orientation propagation. Curvature estimation often builds upon these local reference frames. Common approaches range from PCA-based descriptors such as *surface variation*, which measures the deviation of points from the local tangent plane, to **local surface fitting techniques that recover principal, mean, and Gaussian curvatures by approximating the geometry with smooth second-order surfaces**.

The next notebook focuses on task-oriented descriptors, or *features*, that are not intrinsic surface properties, but higher level representations designed to capture patterns relevant to matching, retrieval, or learning-based tasks. These representations bridge the gap between low-level geometry and high-level reasoning, enabling more powerful analysis of 3D data.

7 Descriptors

This notebook focuses on descriptors or *features*, which aim to provide a rich and expressive characterization of local geometry. Instead of describing only orientation or bending, **descriptors capture higher-level geometric patterns**. Such features play a central role in many downstream tasks, including segmentation, registration, and classification. In practice, many geometric and learning-based algorithms rely on these descriptors to assess similarity or infer semantics within a pointcloud.

This notebook explores both simple geometric descriptors such as **eigenvalue-based features**, which extend normal and curvature analysis, and more advanced histogram-based descriptors such as **Point Feature Histograms (PFH)** and **Signature of Histograms of Orientation (SHOT)**, which capture richer spatial relationships.

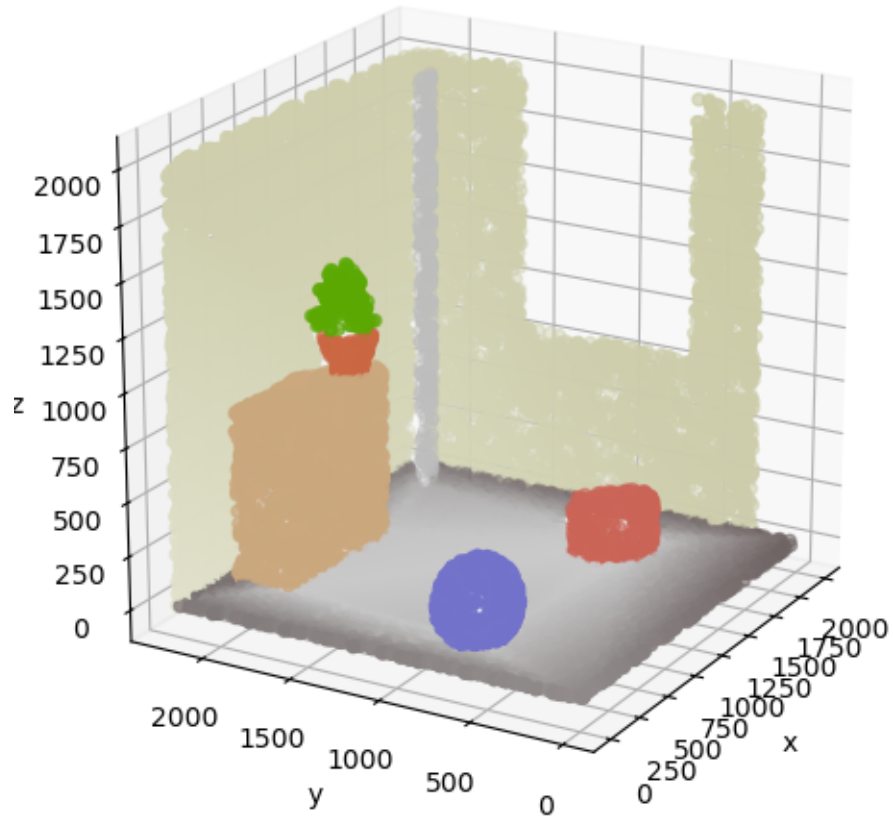
```
[1]: # Necessary imports
from itertools import combinations
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import KDTree

# Load second example pointcloud
data = np.loadtxt("./data/indoor_scene.xyz")
points = data[:, :3]
normals = data[:, 3:6]
colors = data[:, 6:]
# Build KDTree for neighbor search
kdtree = KDTree(points)
```

For the sake of illustration, let's consider a synthetic pointcloud representing an indoor scene, represented below.

```
[2]: # Plot original pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2], c=colors)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_box_aspect([1, 1, 1]) # equal aspect ratio
ax.view_init(elev=20, azim=210) # adjust the view angle
ax.set_title("Synthetic indoor scene")
plt.show()
```

Synthetic indoor scene



7.1 Eigenvalue-based Features

As introduced in the previous notebook, the decomposition of the covariance matrix computed over a local neighborhood is a fundamental tool for estimating the local tangent plane. From this analysis, geometric quantities such as surface normals and curvature proxies can be derived [14, 16].

Given a point p_i and its neighborhood N_i , the local covariance matrix is defined as:

$$cov_i = \frac{1}{|N_i|} \sum_{p_j \in N_i} (p_j - c_i) \cdot (p_j - c_i)^T$$

where c_i denotes the centroid of the neighborhood:

$$c_i = \frac{1}{|N_i|} \sum_{p_j \in N_i} p_j$$

The eigen-decomposition of this matrix yields three eigenvalues $\lambda_1^i \geq \lambda_2^i \geq \lambda_3^i \geq 0$, along with their associated eigenvectors.

The eigenvector ν_3^i associated with the smallest eigenvalue λ_3^i provides an estimate of the surface normal n_i and the ratio between this smallest eigenvalue and the sum of eigenvalues, also called *curvature change*:

$$C_\lambda^i = \frac{\lambda_3^i}{\lambda_1^i + \lambda_2^i + \lambda_3^i}$$

is commonly used as a measure of local surface variation, often interpreted as a curvature proxy.

Eigenvalue-based features extend this analysis by exploiting the relative magnitudes of the eigenvalues to characterize the local geometric structure. Instead of focusing solely on normal estimation, they describe how the point distribution is organized in the neighborhood of p_i .

A useful way to interpret these quantities is through the ellipsoid defined by the covariance matrix. The principal axes of this ellipsoid correspond to the eigenvectors, while their lengths are proportional to the square roots of the eigenvalues. This representation provides an intuitive geometric interpretation: for example, if two eigenvalues dominate, the neighborhood lies close to a plane, indicating a planar structure. However, note that it is not strictly equivalent to fitting the ellipsoid to the points and its size is arbitrary.

```
[3]: def compute_ellipsoid(points):
    """Compute the ellipsoid from the points."""

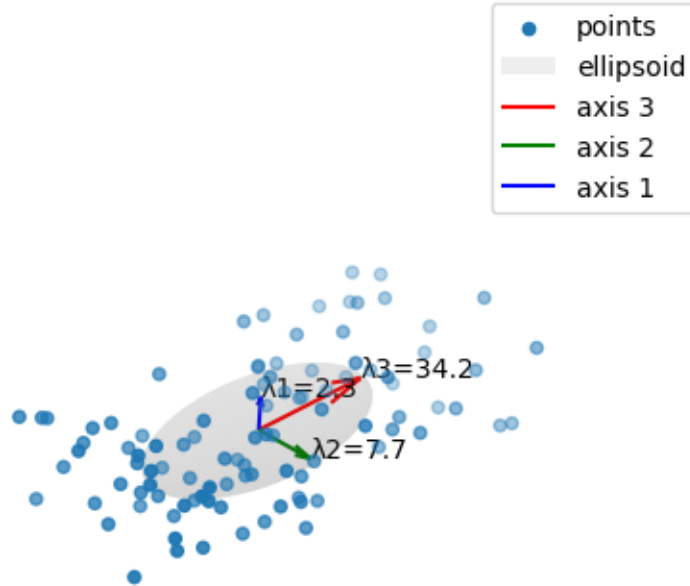
    # Center pointcloud
    centroid = points.mean(axis=0)
    centered_points = points - centroid
    # Covariance matrix
    cov = (centered_points.T @ centered_points) / len(points)
    # Eigen decomposition (eigenvalues are in ascending order here!)
    eigenvalues, eigenvectors = np.linalg.eigh(cov)
    # Create ellipsoid mesh
    u = np.linspace(0, 2 * np.pi, 50)
    v = np.linspace(0, np.pi, 50)
    x = np.outer(np.cos(u), np.sin(v))
    y = np.outer(np.sin(u), np.sin(v))
    z = np.outer(np.ones_like(u), np.cos(v))
    ellipsoid = np.array([x, y, z])
    # Scale by eigenvalues
    for i in range(3):
        ellipsoid[i] *= np.sqrt(eigenvalues[i])
    # Rotate by eigenvectors
    ellipsoid_rotated = np.tensordot(eigenvectors, ellipsoid.reshape(3, -1),
                                     axes=(1, 0))
    ellipsoid_rotated = ellipsoid_rotated.reshape(3, 50, 50)
    # Translate by centroid
    ellipsoid_rotated += centroid[:, np.newaxis, np.newaxis]
```

```
return centroid, eigenvalues, eigenvectors, ellipsoid_rotated
```

```
[4]: # Generate random 3D points
random_points = np.random.random((100, 3))
random_points *= np.array([20, 10, 5]) # adjust the scale
# Compute ellipsoid and principal axes
mean, eigvalues, eigvectors, ellipsoid = compute_ellipsoid(random_points)

# Plot points and ellipsoid
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
           label="points")
ax.plot_surface(ellipsoid[0], ellipsoid[1], ellipsoid[2],
               color="grey", alpha=0.1, label="ellipsoid")
for i, c in enumerate(["red", "green", "blue"]):
    j = 2 - i # to be consistent with eigenvalue ordering (smallest to largest)
    axis_vector = np.sqrt(eigvalues[j]) * eigvectors[:, j]
    ax.quiver(mean[0], mean[1], mean[2],
              axis_vector[0], axis_vector[1], axis_vector[2],
              color=c, length=1., label=f"axis {j+1}")
    ax.text(mean[0] + axis_vector[0],
            mean[1] + axis_vector[1],
            mean[2] + axis_vector[2],
            f" $\lambda_{j+1}={eigvalues[j]:.1f}$ ", color="black")
ax.set_title("Ellipsoid analogy for understanding eigenvalues-based features")
ax.set_axis_off()
ax.view_init(30, 45)
plt.legend()
plt.axis("equal")
plt.show()
```

Ellipsoid analogy for understanding eigenvalues-based features



This ellipsoid analogy provides an intuitive understanding of the local geometry around each point and motivates the definition of the features described below.

Feature	Formula	Intuitive meaning
Linearity	$L_\lambda = \frac{\lambda_1 - \lambda_2}{\lambda_1}$	Degree of elongation (1D structure)
Planarity	$P_\lambda = \frac{\lambda_2 - \lambda_3}{\lambda_1}$	Degree of flatness (2D structure)
Scattering	$S_\lambda = \frac{\lambda_3}{\lambda_1}$	Degree of volumetric dispersion (3D structure)
Omnivariance	$O_\lambda = (\lambda_1 \lambda_2 \lambda_3)^{\frac{1}{3}}$	Overall spread in all directions
Anisotropy	$A_\lambda = \frac{\lambda_1 - \lambda_3}{\lambda_1}$	Difference between strongest and weakest variation
Eigenentropy	$E_\lambda = \sum_{i=1}^3 -\lambda_i \ln(\lambda_i)$	Degree of disorder in the distribution
Sum of eigenvalues	$\Sigma_\lambda = \lambda_1 + \lambda_2 + \lambda_3$	Total variance (scale-dependent)

Feature	Formula	Intuitive meaning
Change of curvature	$C_\lambda = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$	Degree of bending (curvature proxy)

Scattering is also commonly referred to as *sphericity*, as it reflects the isotropy of the neighborhood.

In practice, a small subset of these features is sufficient for many tasks. In particular, linearity, planarity, scattering (sphericity), and change of curvature are the most widely used, as they provide a compact and discriminative description of local shape. These are represented below.

Eigenvalue-based features are simple, interpretable, and computationally efficient, making them well suited for large-scale pointclouds. However, they are sensitive to the choice of neighborhood scale and can be affected by noise and variations in point density.

These implementations below compute the k nearest neighbors for all points at once and use vectorized operations whenever possible instead of explicit loops. While improving execution speed, it is also memory-intensive and switching to batch processing is advised for large pointclouds.

```
[5]: def compute_3D_eigen_features(points, neighbor_finder, norm_eigenvalues=False):
    """Compute local eigenvalue-based features."""

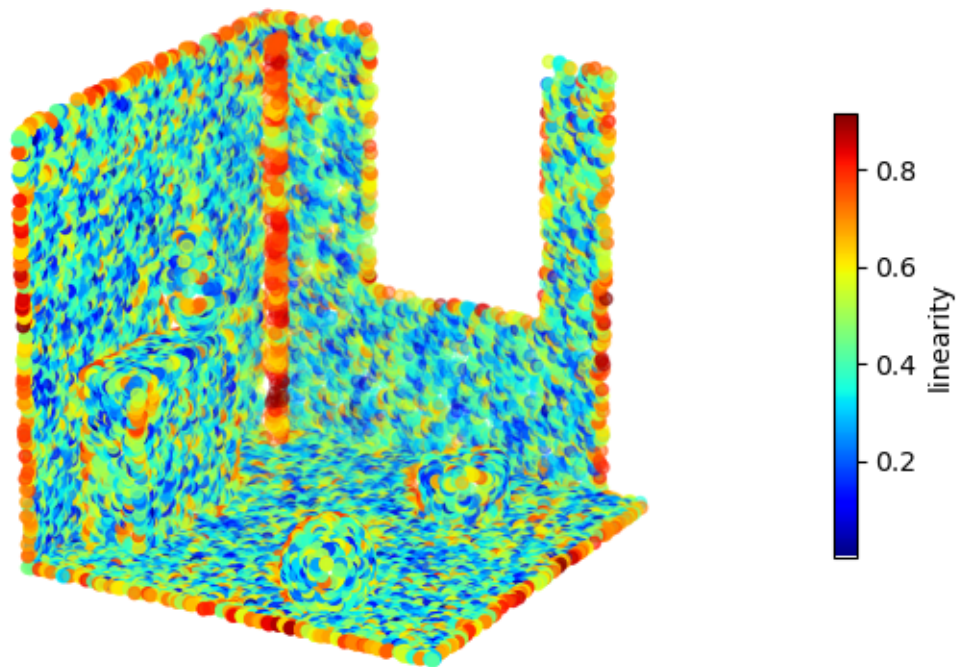
    # Neighbors for all query points
    inds = neighbor_finder(points) # (N, k)
    k = inds.shape[-1]
    neighbors = points[inds]
    # Eigenvalues from covariance matrix
    neighbors -= neighbors.mean(axis=1, keepdims=True) # center in-place
    cov = (neighbors.transpose(0, 2, 1) @ neighbors) / k # (N, 3, 3)
    eigenvalues = np.linalg.eigvalsh(cov) # (N, 3)
    eigenvalues = np.clip(eigenvalues, 1e-10, None) # fix numerical negatives
    # Compute sum before normalization (equal to 1. if normalized)
    eigensum = eigenvalues.sum(axis=1)
    # Normalize eigenvalues if required
    if norm_eigenvalues:
        eigenvalues /= eigensum.sum(axis=1, keepdims=True)
    # Compute features based on eigenvalues
    lambda_1 = eigenvalues[:, 2]
    lambda_2 = eigenvalues[:, 1]
    lambda_3 = eigenvalues[:, 0]
    linearity = (lambda_1 - lambda_2) / lambda_1
    planarity = (lambda_2 - lambda_3) / lambda_1
    scattering = lambda_3 / lambda_1
    omnivariance = (lambda_1 * lambda_2 * lambda_3) ** (1/3)
    anisotropy = (lambda_1 - lambda_3) / lambda_1
    eigentropy = -np.sum(eigenvalues * np.log(eigenvalues), axis=1)
    curvature = lambda_3 / (lambda_1 + lambda_2 + lambda_3)

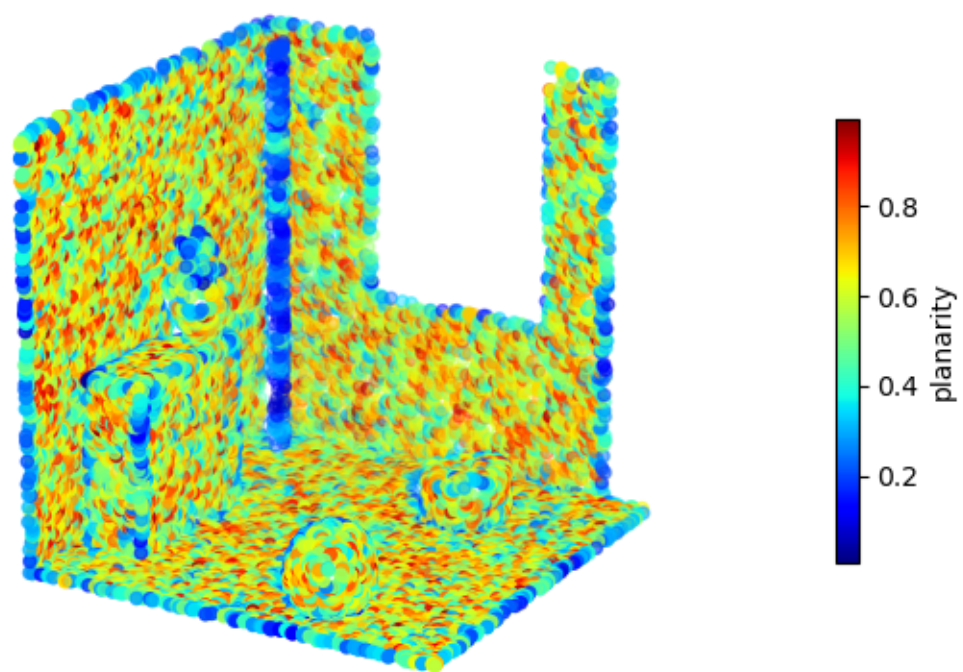
    return (linearity, planarity, scattering, omnivariance, anisotropy,
```

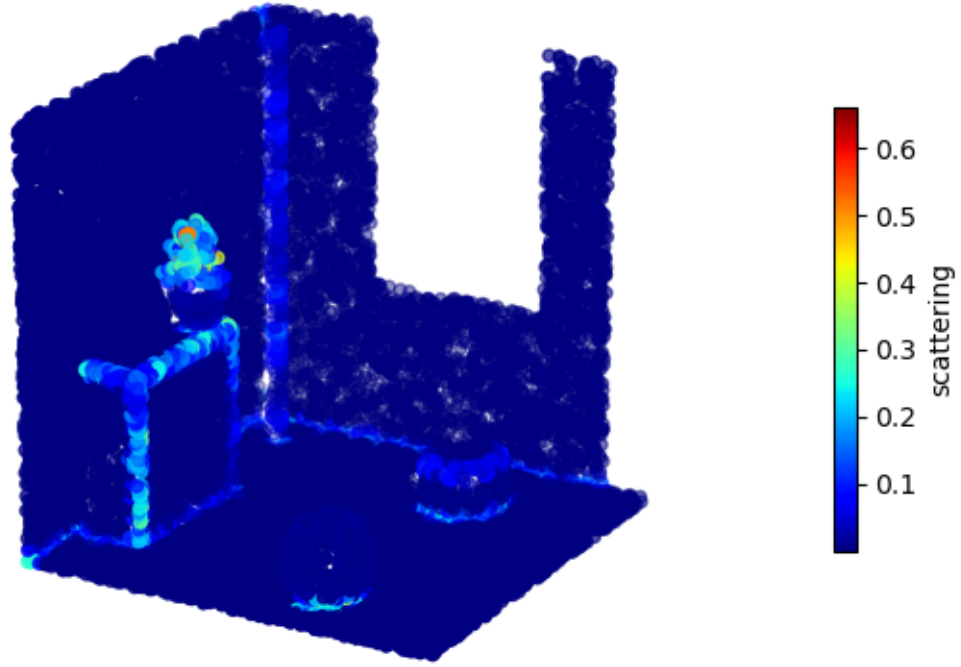
```
eigentropy, eigensum, curvature)
```

```
[ ]: # Compute local 3D features
neighbor_finder = lambda points: kdtree.query(points, 20)[1]
eigenfeatures = compute_3D_eigen_features(points, neighbor_finder, True)

# Visualize linearity, planarity and scattering
for feature_name, feature in zip(
    ["linearity", "planarity", "scattering"],
    eigenfeatures[:3]
):
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection="3d")
    im = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                    c=feature, cmap='jet')
    cbar = fig.colorbar(im, ax=ax, shrink=0.5)
    cbar.set_label(feature_name)
    ax.set_box_aspect([1, 1, 1])
    ax.view_init(20, 210)
    ax.set_axis_off()
plt.show()
```







7.2 Point Feature Histograms (PFH) and Fast Point Feature Histograms (FPFH)

Local descriptors such as eigenvalue-based features characterize the geometry around a point, but they do not explicitly encode relationships between neighboring points. Point Feature Histograms (PFH) were introduced by Radu Bogdan Rusu (one of the main contributors behind the Point Cloud Library PCL) to address this limitation [22]. **The main idea behind PFH is to capture *pairwise geometric relationships* within a local neighborhood using both point coordinates and normals.**

More specifically, **PFH aggregates geometric interactions between neighboring points into a multi-dimensional histogram representation.** Unlike simpler descriptors, which summarize point-wise properties independently, PFH encode how points relate to one another, leading to a richer description of the underlying surface structure.

The PFH computation algorithm can be summarized in a few steps:

For each point p_i with normal n_i of the pointcloud:

1. Query its neighbors p_j within a radius r (or alternatively its k -nearest neighbors).

2. Form all possible pairs (p_j, p_k) within this neighborhood.
3. Define a local reference frame for each pair, consisting of a source point and target point (p_s, p_t) , using the normal of the source point n_s , such as:

$$u = n_s \quad v = \frac{(p_t - p_s \times u)}{\|(p_t - p_s \times u)\|} \quad w = u \times v$$

4. Compute angular features (α, ϕ, θ) and distance d for each pair (p_s, p_t) , such as:

$$\alpha = v \cdot n_t \quad \phi = u \cdot \frac{p_t - p_s}{\|p_t - p_s\|} \quad \theta = \arctan(w \cdot n_t, u \cdot n_t)$$

$$d = \|p_t - p_s\|$$

5. Discretize these features $(\alpha, \phi, \theta, d)$ and accumulate them into a 4D-histogram, which is normalized so it is independent of the number of neighbors.

The resulting normalized histogram represents the PFH descriptor of p_i .

A crucial aspect of PFH is that **it captures the joint distribution of all features**. This means that PFH does not build four independent histograms, but a single multi-dimensional histogram (a bin represents statements like: *“the pair has a small α , medium ϕ , large θ , and medium distance”*). This joint encoding is what allows PFH to capture rich geometric relationships between points.

In practice, the distance term d is often omitted or treated separately. Most implementations (including the one found in PCL) focus primarily on the angular features (α, ϕ, θ) . These angular features are naturally bounded within fixed intervals ($[-1, 1]$ for α and ϕ and $[-\pi, \pi]$ for θ), which makes them well-suited for consistent discretization (whereas distance can vary significantly depending on pointcloud scale and neighborhood selection).

The main drawback of PFH is its high computational complexity. Indeed, it requires evaluating all pairwise interactions within the neighborhood, resulting in $O(k^2)$ operations per point, where k is the number of neighbors. While being rotation-invariant, PFH is still sensitive to errors in normal estimation, noise and outliers. It is also dependent on the neighborhood type and size.

Below is an example of a PFH for a given point of our pointcloud.

```
[7]: def compute_PFH(points, normals, neighbor_finder, num_bins=5,
    normalize_histogram=True):
    """Compute Point Feature Histograms (PFH) for each point."""

    N, _ = points.shape
    # 3D histogram of angular features (alpha, phi, theta) put together
    pfh = np.zeros((N, num_bins ** 3), dtype=float)

    for i in range(N):
        neighbors = neighbor_finder(points[i])
        features = []
        # Pairwise combinations of neighbors (without repetition)
```

```

        for (ind_s, ind_t) in combinations(neighbors, 2): # from itertools
            p_s, n_s = points[ind_s], normals[ind_s]
            p_t, n_t = points[ind_t], normals[ind_t]
            alpha, phi, theta, _ = compute_PFH_features(p_s, n_s, p_t, n_t)
            features.append([alpha, phi, theta])
        # Convert to numpy array and handle case with no valid pairs
        if features:
            features = np.array(features)
        else:
            continue # if no valid pairs, skip this point
        # 3D histogram of angular features (alpha, phi, theta)
        histogram, _ = np.histogramdd(
            features,
            bins=num_bins,
            range=[[-1., 1.], [-1., 1.], [-np.pi, np.pi]],
        )
        histogram = histogram.flatten() # get a 1D descriptor vector
        # Normalize histogram
        if normalize_histogram:
            histogram /= histogram.sum()

        pfh[i] = histogram

    return pfh

def compute_PFH_features(p_s, n_s, p_t, n_t):
    """Compute features (alpha, phi, theta, dist) for a pair of points."""

    diff = p_t - p_s
    dist = np.linalg.norm(diff)
    #if dist < 1e-10:
    diff_unit = diff / dist
    u = n_s
    v = np.cross(diff_unit, u)
    v_norm = np.linalg.norm(v)
    #if v_norm < 1e-10:
    v /= v_norm
    w = np.cross(u, v)
    alpha = v @ n_t
    phi = u @ diff_unit
    theta = np.arctan2((w @ n_t), (u @ n_t))

    return alpha, phi, theta, dist

```

```

[8]: # Compute PFH descriptors
neighbor_finder = lambda point: kdtree.query_ball_point(point, 25.)

```

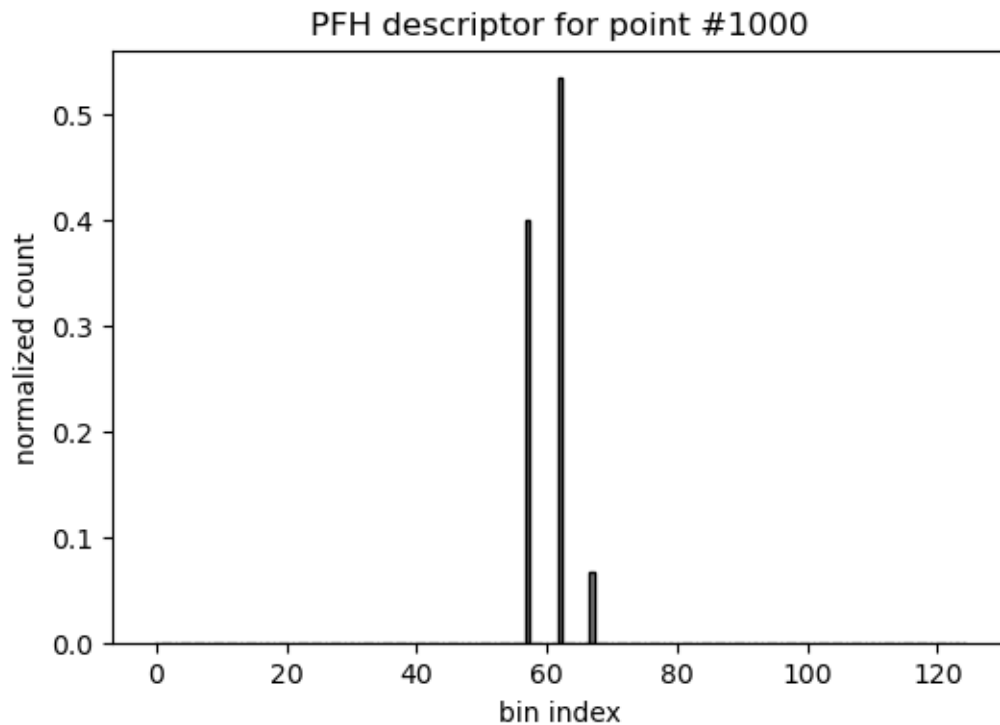
```

pfh_descriptors = compute_PFH(points, normals, neighbor_finder, num_bins=5)

# PFH angular descriptors associated with a point
i = 1000 # index of the point to visualize
descriptor = pfh_descriptors[i]

# Plot as a bar graph
plt.figure(figsize=(6, 4))
plt.bar(range(len(descriptor)), descriptor, color="gray", edgecolor="black")
plt.xlabel("bin index")
plt.ylabel("normalized count")
plt.title(f"PFH descriptor for point #{i}")
plt.show()

```



The high computational cost of Point Feature Histograms often make them difficult to apply to very large pointclouds or real-time applications. To overcome this computational limitations of PFH, Fast Point Feature Histograms (FPFH) were later introduced as an efficient approximation of the original descriptor [22].

FPFH retain the core idea of encoding local geometry through angular relationships between points and normals, but does not consider all pairwise interactions within a neighborhood. Instead, FPFH decompose the process into two stages:

1. Compute *Simplified Point Feature Histograms* (SPFH), containing angular features and dis-

tance $(\alpha, \phi, \theta, d)$, between each point p_i and its k immediate neighbors p_j .

2. Aggregate these local signatures using a distance-based weighting scheme:

$$FPFH(p_i) = SPFH(p_i) + \frac{1}{k} \sum_{j=0}^{k-1} \frac{1}{\omega_j} SPFH(p_j)$$

where the weight ω_j is usually the distance between p_i and p_j .

This approximation reduces the complexity from $O(k^2)$ to $O(k)$ per point while preserving most of the discriminative power of PFH.

One major difference with PFH is that FPFH builds independent histograms for each angular feature rather than a fully joint multi-dimensional distribution. This also results in a more compact descriptor, commonly a 33-dimensional vector. The weighting scheme allows FPFH to implicitly capture a broader neighborhood influence, partially compensating for the loss of direct pairwise combinations.

As a result, FPFH provides a good trade-off between descriptiveness and efficiency, making it widely used in practical applications such as real-time registration.

Below is the FPFH for the same point of our pointcloud.

```
[9]: def compute_FPFH(points, normals, neighbor_finder, num_bins=11,
        normalize_histogram=True):
    """Compute Fast Point Feature Histograms (FPFH) for each point."""

    N, _ = points.shape
    # 1D histogram of angular features (alpha, phi, theta) put together
    fpfh = np.zeros((N, num_bins * 3), dtype=float)
    # SPFH for each point (to be averaged for neighbors)
    spfh = np.zeros((N, num_bins * 3), dtype=float)

    for i in range(N):
        neighbors = neighbor_finder(points[i]) # point itself included
        neighbors = [ind for ind in neighbors if ind != i] # exclude point
        features = []
        for ind in neighbors:
            p_s, n_s = points[i], normals[i]
            p_t, n_t = points[ind], normals[ind]
            alpha, phi, theta, _ = compute_PFH_features(p_s, n_s, p_t, n_t)
            features.append([alpha, phi, theta])
        # Convert to numpy array and handle case with no valid neighbors
        if features:
            features = np.array(features)
        else:
            continue # if no valid neighbors, skip this point
        # 1D histograms of angular features (alpha, phi, theta) put together
        hist_alpha, _ = np.histogram(features[:, 0], bins=num_bins,
                                     range=(-1., 1.))
        hist_phi, _ = np.histogram(features[:, 1], bins=num_bins,
```



```

                                range=(-1., 1.))
hist_theta, _ = np.histogram(features[:, 2], bins=num_bins,
                                range=(-np.pi, np.pi))
histogram = np.concatenate([hist_alpha, hist_phi, hist_theta],
                             dtype=float) # for normalization
if normalize_histogram:
    histogram /= histogram.sum()
spf_h[i] = histogram

# Average SPFH of neighbors to get FPFH
for i in range(N):
    fpf_h[i] = spf_h[i]
    # Contribution of neighbors
    neighbors = neighbor_finder(points[i]) # point itself included
    neighbors = [ind for ind in neighbors if ind != i] # exclude point
    if neighbors:
        # Weights based on distances (but alternatives are possible)
        dists = np.linalg.norm(points[neighbors] - points[i], axis=1)
        weights = 1. / (dists + 1e-10) # inverse distance weights
        contribs = 1./len(neighbors) * weights[:, None] * spf_h[neighbors]
        fpf_h[i] += np.sum(contribs, axis=0)
    else:
        continue # if no neighbors, FPFH is just SPFH

return fpf_h

```

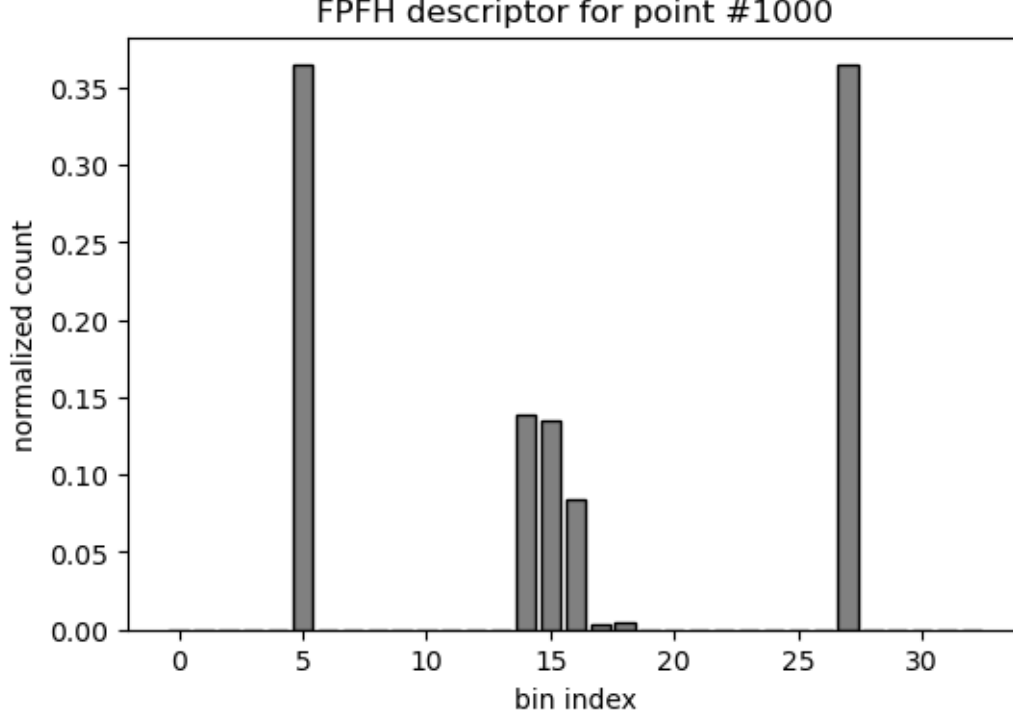
```

[10]: # Compute FPFH descriptors
neighbor_finder = lambda point: kdtree.query_ball_point(point, 25.)
fpf_h_descriptors = compute_FPFH(points, normals, neighbor_finder, num_bins=11)

# FPFH angular descriptors associated with a point
i = 1000 # index of the point to visualize
descriptor = fpf_h_descriptors[i]

# Plot as a bar graph
plt.figure(figsize=(6, 4))
plt.bar(range(len(descriptor)), descriptor, color="gray", edgecolor="black")
plt.xlabel("bin index")
plt.ylabel("normalized count")
plt.title(f"FPFH descriptor for point #{i}")
plt.show()

```



7.3 Signature of Histograms of Orientations (SHOT)

SHOT was motivated by the observation that geometric signatures (such as eigenvalue-based features) are often discriminative but sensitive to noise and unstable reference frames, whereas histogram-based methods (such as PFH) are more robust but less distinctive. SHOT addresses this trade-off by combining both ideas by introducing a repeatable local reference frame (LRF) [23].

The core idea of SHOT is to describe the local geometry around a point by expressing its neighborhood in a stable local coordinate system and capturing the spatial distribution of normals. The neighborhood is partitioned into spatial regions, and in each is built a **histogram of normal orientations**. By concatenating these histograms, SHOT aims to capture both the spatial arrangement and the geometric properties of the sampled surface.

The construction of the SHOT descriptor for a given point p_i follows these principal steps:

1. Query the neighbors of p_i within radius r and form the set N_i .
2. Compute a weighted covariance-like matrix:

$$M_i = \frac{1}{\sum_{p_j \in N_i} (r - d_j)} \sum_{p_j \in N_i} (r - d_j) (p_j - p_i) \cdot (p_j - p_i)^T$$

with $d_j = \|p_j - p_i\|$. Its eigen-decomposition yields eigenvectors (ν_1, ν_2, ν_3) and eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$.

3. Disambiguate the directions of eigenvectors to obtain the axes (u, v, w) of the LRF:

$$u = \text{sign}(S_x)\nu_1 \quad \text{with} \quad S_x = \sum_{p_j \in N_i} (p_j - p_i) \cdot \nu_1$$

and

$$w = \text{sign}(S_z)\nu_3 \quad \text{with} \quad S_z = \sum_{p_j \in N_i} (p_j - p_i) \cdot \nu_3$$

and

$$v = w \times u$$

4. Project each neighbor p_j into the LRF:

$$q_j = \begin{pmatrix} (p_j - p_i) \cdot u \\ (p_j - p_i) \cdot v \\ (p_j - p_i) \cdot w \end{pmatrix}$$

5. Partition the support region into 3D spatial bins based on radial distance ρ , azimuth ϕ and elevation angle ψ (with respect to w). Each point p_j is associated with a bin indexed by (ρ_j, ϕ_j, ψ_j) such as:

$$\rho_j = \|q_j\|$$

and

$$\phi_j = \arctan 2((p_j - p_i) \cdot v, (p_j - p_i) \cdot u)$$

and

$$\psi_j = \arccos((p_j - p_i) \cdot w / \|q_j\|)$$

6. Compute the angle each neighbor normal n_j and the w axis of the LRF:

$$\theta_j = n_j \cdot w$$

7. Accumulate these values into histograms within each spatial bin using quadrilinear interpolation in the 4D space $(\rho, \phi, \psi, \theta)$. The contribution of p_j is split among adjacent bins with weights ω_ρ , ω_ϕ , ω_ψ and ω_θ proportional to its distance from the bin centers.
8. Concatenate the histograms of angles θ (one per spatial bin) into a single feature vector and normalize it.

The first three steps can be grouped into the **computation of the Local Reference Frame (LRF)**, which is one of the key contributions of SHOT.

This LRF is derived from the eigen-analysis of the local neighborhood, previously introduced for normal estimation and eigenvalue-based features computations. SHOT proposes to improve the classic formulation in two ways: it assigns lower weights to distant neighbors and replaces the neighborhood centroid with the point of interest. It is important to note that, unlike in classical normal estimation, the eigenvector ν_3 associated with the smallest eigenvalue λ_3 does not necessarily correspond to the surface normal.

A key issue is that eigenvectors are defined up to a sign, which leads to ambiguity in the LRF. SHOT resolves this by enforcing a consistent orientation based on the spatial distribution of neighboring

points. For each axis, the sign is chosen such that the majority of neighbors lie in corresponding the positive half-space. This is supposed to yield a unique and repeatable LRF, which is essential for robust descriptor computation.

```
[11]: def local_reference_frame(point, neighbors, radius):
    """Compute local reference frame for each point."""

    # Center the neighbors and compute distances
    diffs = neighbors - point
    dists = np.linalg.norm(diffs, axis=1)
    # Compute and normalize weights (based on distance)
    weights = radius - dists
    weights /= (weights.sum() + 1e-12)
    # Compute weighted covariance matrix
    cov = (diffs * weights[:, None]).T @ diffs
    # Decompose covariance matrix
    _, eigenvectors = np.linalg.eigh(cov) # eigenvalues are in ascending order
    x_axis = eigenvectors[:, 2] # largest eigenvalue
    z_axis = eigenvectors[:, 0] # smallest eigenvalue
    # Sign disambiguation for consistent orientation
    projs = neighbors @ x_axis
    if np.sum(projs > 0) < np.sum(projs < 0):
        x_axis = -x_axis
    projs = neighbors @ z_axis
    if np.sum(projs > 0) < np.sum(projs < 0):
        z_axis = -z_axis
    # y-axis as cross product to ensure right-handed system
    y_axis = np.cross(z_axis, x_axis)
    y_axis /= (np.linalg.norm(y_axis) + 1e-12)

    return np.stack([x_axis, y_axis, z_axis], axis=1)
```

The last steps of SHOT focus on the **partitioning of the support region and the accumulation of local geometric information**. The neighborhood, expressed in the LRF, is subdivided into 3D spatial bins. Within each bin, a histogram of normal orientations is constructed, capturing the local surface geometry in a spatially structured manner.

In practice, the quality of the descriptor depends on several choices such as the number of spatial bins and the number of histogram bins. SHOT typically uses 32 spatial bins (2 radial, 8 azimuth and 2 elevation), each containing an 11-bin histogram, resulting in the widely used 352-dimensional descriptor.

```
[12]: def compute_SHOT(point, neighbors, normals, radius,
    n_azimuth=8, n_elevation=2, n_radial=2, n_bins=11):

    # Compute LRF and project neighbors into it
    R = local_reference_frame(point, neighbors, radius)
    neighbors_projected = (neighbors - point) @ R
```

```

dists = np.linalg.norm(neighbors_projected, axis=1)

# Compute Gaussian spatial weighs
sigma = radius / 2.0
w_spatial = np.exp(-(dists**2) / (2 * sigma**2))

# Compute continuous bin coordinates
# Radial
r = dists / radius * n_radial
r0 = np.floor(r).astype(int)
r1 = np.minimum(r0 + 1, n_radial - 1)
wr = r - r0
# Elevation (z direction)
z = neighbors_projected[:, 2] / (dists + 1e-12)
e = (z + 1) * 0.5 * n_elevation
e0 = np.floor(e).astype(int)
e1 = np.minimum(e0 + 1, n_elevation - 1)
we = e - e0
# Azimuth (angle in xy plane)
angles = np.arctan2(neighbors_projected[:, 1], neighbors_projected[:, 0])
a = (angles + np.pi) / (2*np.pi) * n_azimuth
a0 = np.floor(a).astype(int) % n_azimuth
a1 = (a0 + 1) % n_azimuth
wa = a - np.floor(a)
# Orientation (normal vs LRF z-axis)
cos_theta = normals @ R[:, 2]
h = (cos_theta + 1) * 0.5 * n_bins
h0 = np.floor(h).astype(int)
h1 = np.minimum(h0 + 1, n_bins - 1)
wh = h - h0

# Stack interpolation components
r_idx = np.stack([r0, r1], axis=1)
e_idx = np.stack([e0, e1], axis=1)
a_idx = np.stack([a0, a1], axis=1)
h_idx = np.stack([h0, h1], axis=1)

w_r = np.stack([1-wr, wr], axis=1)
w_e = np.stack([1-we, we], axis=1)
w_a = np.stack([1-wa, wa], axis=1)
w_h = np.stack([1-wh, wh], axis=1)

# Broadcast to all 16 combinations
weights = (w_r[:, :, None, None, None] *
           w_e[:, None, :, None, None] *
           w_a[:, None, None, :, None] *
           w_h[:, None, None, None, :])

```

```

# Apply spatial weighting
weights *= w_spatial[:, None, None, None, None]
rr = r_idx[:, :, None, None, None]
ee = e_idx[:, None, :, None, None]
aa = a_idx[:, None, None, :, None]
hh = h_idx[:, None, None, None, :]

# Flatten
# Force full broadcasting to same shape
rr, ee, aa, hh, weights = np.broadcast_arrays(rr, ee, aa, hh, weights)

# Now flatten safely
rr = rr.ravel()
ee = ee.ravel()
aa = aa.ravel()
hh = hh.ravel()
weights = weights.ravel()

# Convert to sector index
sector = rr + n_radial * (ee + n_elevation * aa)
n_sectors = n_radial * n_elevation * n_azimuth
descriptor = np.zeros((n_sectors, n_bins))

# Accumulate contributions
np.add.at(descriptor, (sector, hh), weights)

# Normalize descriptor
descriptor = descriptor.ravel()
descriptor /= (np.linalg.norm(descriptor) + 1e-12)

return descriptor

```

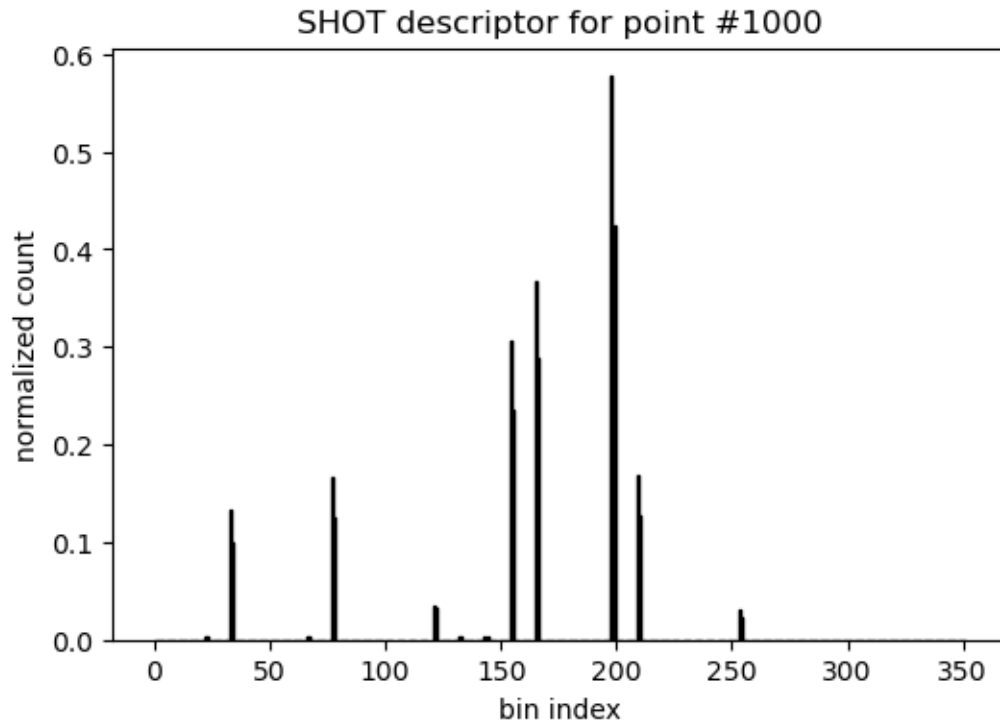
```

[13]: # Compute SHOT descriptor associated with a point
i = 1000 # index of the point to visualize
neighbor_finder = lambda point: kdtree.query_ball_point(point, 25.)
neighbor_inds = neighbor_finder(points[i])
descriptor = compute_SHOT(
    points[i], points[neighbor_inds], normals[neighbor_inds], radius=25.
)

# Plot as a bar graph
plt.figure(figsize=(6, 4))
plt.bar(range(len(descriptor)), descriptor, color="gray", edgecolor="black")
plt.xlabel("bin index")
plt.ylabel("normalized count")
plt.title(f"SHOT descriptor for point #{i}")

```

```
plt.show()
```



7.4 Going further

Selecting an appropriate descriptor typically involves balancing expressiveness, robustness, and computational cost, and strongly depends on the target application. This notebook explored three representative families of descriptors widely used in practice: eigenvalue-based features, PFH, and SHOT.

Eigenvalue-based features are simple while still capturing meaningful geometric information, and are relatively inexpensive to compute. They can be obtained directly from local covariance analysis, either explicitly as in `Open3D`, or implicitly as a by-product of normal estimation routines in libraries such as `PCL` and `CGAL`. Their compact description of the local structure makes them particularly well suited for classification tasks such as semantic segmentation, where each point is assigned a semantic label. Typical applications include urban and natural scene understanding. In such context, eigenvalue-based features are often combined with additional geometric features derived directly from point coordinates, such as local point density and the deviation between the estimated normal and the vertical axis (often the z -axis) [24]. This combination helps capture both intrinsic geometric properties and contextual information.

Histogram-based descriptors such as PFH and its faster variant FPFH provide a richer representation by encoding relationships within local neighborhoods. This makes them particularly suitable for matching and registration tasks, where reliable correspondences between pointclouds are required. Reference implementations are available in `PCL`, through `PFHEstimation` and `FPFHEstimation`.

Open3D focuses on the more efficient FPFH only with `compute_fpfh_feature`.

SHOT descriptors follow a similar objective, producing structured and highly discriminative signatures that are also widely used for matching and registration. Extensions of the original formulation incorporate color information, further improving distinctiveness in scenarios where geometric information alone is insufficient [25]. PCL provides a reference implementation with `SHOTEstimation` and `SHOTColorEstimation`.

Beyond the descriptors presented here, the landscape of 3D feature extraction remains active. Again, PCL includes both earlier approaches, such as Spin Images, and more recent descriptors like the *Viewpoint Feature Histogram* (VFH), which aggregates viewpoint-dependent information into global signatures [26, 27]. Such global descriptors are particularly useful for object-level recognition, rather than local matching.

More recent developments include learned descriptors, which have become a strong alternative to handcrafted features. The learning models are typically implemented in deep learning frameworks rather than classical geometry libraries. They learn task-specific representations directly from data and often outperform classical handcrafted descriptors in most applications. More broadly, the popularity of deep learning models illustrates a shift toward feature learning, where descriptors are no longer explicitly designed but instead learned directly from raw pointclouds. In practice, these techniques have not completely replaced handcrafted ones, as they typically require large training datasets and higher computational resources.

7.5 Wrapping up

You should now have a better grasp of how to compute descriptors, also called features, that enrich each point with higher-level geometric information. These descriptors go beyond normals and curvatures and provide a more expressive view of local geometry, enabling better analysis, classification, and interpretation complex 3D structures.

Eigenvalue-based features (such as *linearity*, *planarity* and *sphericity*) offer a compact and efficient way to characterize the local shape of a neighborhood. They are particularly well suited for tasks like segmentation and semantic labeling, where identifying structural patterns in the data is key. In contrast, histogram-based descriptors such as **FPFH** and **SHOT** capture richer spatial relationships by encoding the distribution of geometric interactions around a point. This makes them especially valuable for tasks such as registration and object recognition, where robustness and discriminative power are critical.

Together, these descriptors illustrate different trade-offs between simplicity, efficiency, and descriptive power, and form the basis of many practical pipelines in 3D pointcloud processing.

8 Segmentation

Segmentation is a fundamental task in pointcloud processing, where **the goal is to partition a pointcloud into meaningful regions or segments**. This notebook covers “traditional” or geometric segmentation techniques.

The first part focuses on **surface segmentation**, which involves isolating and potentially identifying different surfaces within a pointcloud, with three of the most popular surface segmentation algorithms: **Region Growing, the Hough Transform, and RANSAC**.

The second part is about **object segmentation** and isolating parts of the pointcloud consisting of distinct objects using groups of close points. This is often referred as **graph-based segmentation** and used as a baseline for more advanced techniques such as object recognition.

Note that other types of segmentation tasks are not described in this notebook. These include more advanced *instance segmentation* (separating individual instances of objects) and *semantic segmentation* (assigning semantic labels to each point). These tasks often rely on machine learning and deep learning models and are covered in the next notebooks.

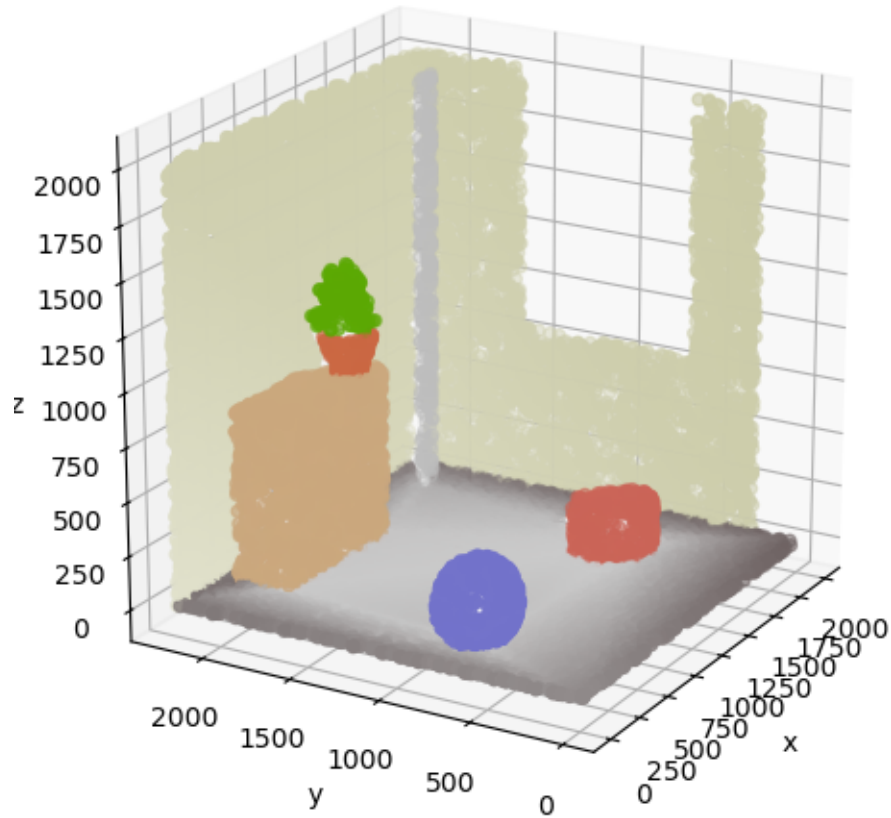
```
[1]: # Necessary imports
import numpy as np
from scipy import ndimage
from scipy.spatial import KDTree
from scipy.sparse.csgraph import connected_components
import matplotlib.pyplot as plt

# Load pointcloud
data = np.loadtxt("./data/indoor_scene.xyz")
points, colors = data[:, :3], data[:, 6:]
# Build the kd-tree (for neighbor search) once
kdtree = KDTree(points)
```

For the sake of illustration, let’s consider a synthetic pointcloud representing an indoor scene, represented below. This scene is composed of 8 “objects”: 1 floor (grey), 2 walls (beige), 1 drawer (brown), 1 pipe (grey), 1 ball (blue), 1 stool (red), and 1 plant in a pot (terracotta and green). Each of these object having one or more surfaces.

```
[2]: # Plot original pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=colors)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Synthetic indoor scene")
plt.show()
```

Synthetic indoor scene



8.1 Surface Segmentation

Surface segmentation divides a pointcloud into separate regions that represent individual surfaces by grouping points with similar geometric properties, such as location, orientation, or shape.

8.1.1 Region Growing

One of the most common approaches for surface segmentation is **Region Growing**. This algorithm (which is also widely used in image processing) works by **iteratively aggregating neighboring points with similar properties** in a bottom-up manner.

It consists of just two steps:

1. **Region initialization** by selecting a starting element (or *seed*).
2. **Region growth** by progressively incorporating neighboring elements that satisfy a predefined similarity criterion.

The growth process stops when no neighboring elements can be added to the current region. The procedure is then repeated with a new seed until all points in the pointcloud have been assigned to

a region, resulting in n segmented regions after n iterations.

Note that **Region Growing requires both a similarity criterion and a neighborhood structure**. Similarity can be evaluated using one or several attributes, such as normals, colors, or curvatures, while neighborhoods may be defined using k -nearest neighbors, a fixed-radius, or other connectivity relationships.

A popular choice for segmenting a pointcloud into smooth surface patches relies on a *smoothness constraint* based on the angular variation between local surface normals computed within a k -nearest neighborhood [28]. Normals are estimated by local plane fitting, and neighboring points are grouped together when the angle between their normals is sufficiently small. The fitting residual (or alternatively the *curvature change* C_λ) helps detect points located on edges (i.e., strongly curved areas) and is used both to select seed points and to stop region growth near surface boundaries.

The implementation below computes the normals and residuals using k nearest neighbors for all points at once and uses vectorized operations for execution speed. This may be memory intensive for large pointclouds, so consider switching to batch processing if memory becomes an issue.

```
[3]: def compute_normals_residuals(points, neighbor_finder):
      """Compute normals and residuals from tangent plane estimation."""

      # Local connectivity with knn
      inds = neighbor_finder(points) # (N, k)
      neighbors = points[inds]
      # Local plane fitting
      neighbors -= neighbors.mean(axis=1, keepdims=True) # center in-place
      cov = (neighbors.transpose(0, 2, 1) @ neighbors) # (N, 3, 3)
      eigenvalues, eigenvectors = np.linalg.eigh(cov) # (N, 3), (N, 3, 3)
      normals = eigenvectors[:, :, 0]
      residuals = eigenvalues[:, 0]

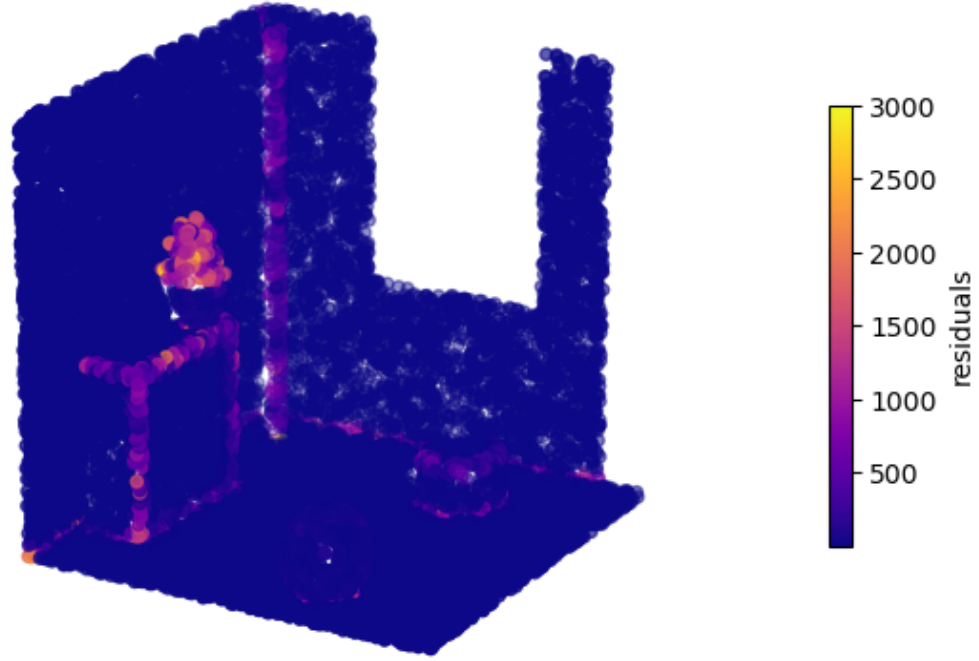
      return normals, residuals
```

```
[4]: # Compute and plot residuals
neighbor_finder = lambda points: kdtree.query(points, 10)[1]
# for spherical neighborhood: return kdtree.query_ball_point(x, r=10.)
normals, residuals = compute_normals_residuals(points, neighbor_finder)

# Plot pointcloud colored by residuals
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=residuals, cmap="plasma")
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('residuals')
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
ax.set_title("Pointcloud colored by residuals")
```

```
plt.show()
```

Pointcloud colored by residuals



The estimated normals and residuals are then used to drive the region-growing process. The algorithm starts from a seed point, chosen among the points with the lowest residuals. The region is then iteratively expanded according to the following rules:

1. A neighboring point p_j is added to the current region if the angle between its normal and that of p_i is smaller than a given threshold:

$$|n_i \cdot n_j| > \cos(\theta_{th})$$

2. The point p_j is selected for further growth only if its residual is below a given threshold:

$$r_j < r_{th}$$

In the first expression, the absolute value of the dot product accounts for the sign ambiguity of estimated normals, whose orientation is only defined up to 180° . Normals are also assumed to be unit vectors.

The angular threshold θ_{th} has a direct geometric interpretation, while the residual threshold r_{th} is somewhat less intuitive to tune. r_{th} is then typically defined as a relative threshold with respect to the distribution of residual values in the pointcloud. For this reason, some implementations use the *curvature change* C_λ instead, as it is normalized and generally easier to interpret and tune.

The neighborhood size and threshold values directly control the granularity of the segmentation. Smaller neighborhoods and stricter thresholds (i.e., lower values of θ_{th} and r_{th}) produce more conservative region growth and tend to increase over-segmentation. On the contrary, larger neighborhoods and more permissive thresholds favor the merging of surface patches, resulting in fewer and larger segments.

```
[5]: # Angle threshold (in degrees)
angle_threshold = 15.
# Residual threshold (as a percentile of the plane residuals)
residual_threshold = np.percentile(residuals, 95)
```

Once the neighborhood, thresholds, normals, and residuals have been computed, the region-growing phase can begin. In its simplest form, the algorithm starts from the non-segmented point with the lowest residual and iteratively explores its neighbors, adding points that satisfy the angular criterion and selecting new seeds according to the residual criterion. The process is repeated until all points have been assigned to a region.

In practice, the implementation can be made more efficient by computing the neighborhood relationships and evaluating the geometric criterion only once for the whole pointcloud. This produces an *adjacency graph* encoding the local smoothness constraint, after which Region Growing becomes a simple graph traversal problem. This approach significantly reduces the computational cost at the expense of storing the neighborhood graph in memory.

```
[6]: def region_growing(points, normals, residuals, neighbor_finder, angle_threshold,
                        residual_threshold):
    """Region growing for smooth surfaces."""

    N, _ = points.shape
    # Local connectivity with knn
    inds = neighbor_finder(points) # (N, k)
    # Smoothness criterion
    cos_threshold = np.cos(np.deg2rad(angle_threshold))
    smooth_neighbors = ( # (N, k)
        np.abs(np.sum(normals[:, None, :] * normals[inds], axis=2))
        > cos_threshold
    )
    # Adjacency graph
    adjacency = [inds[i, smooth_neighbors[i]] for i in range(N)]
    # Candidate seeds (allowed to propagate the region)
    valid_seed = residuals < residual_threshold
    # All points are available at start
    available = np.ones(N, dtype=bool)
    regions = []
```

```

while available.any():
    # Point (index) with minimal residual is chosen as seed
    remaining_points = np.flatnonzero(available)
    seed = remaining_points[np.argmin(residuals[remaining_points])]
    # Initiate region
    queue = [seed]
    current_region = [seed]
    available[seed] = False

    while queue:
        # Find neighbors of current point
        current = queue.pop()
        neighbors = adjacency[current]
        # Add valid neighbors
        neighbors = neighbors[available[neighbors]]
        available[neighbors] = False
        current_region.extend(neighbors.tolist())
        # Only points with small residual are used for queue
        queue.extend(neighbors[valid_seed[neighbors]])

    regions.append(np.asarray(current_region))

return sorted(regions, key=len, reverse=True) # sort according to region size

```

As shown below, smooth surfaces are well segmented, while edges and irregular surfaces (e.g., plant leaves) are generally assigned to the *others* category, corresponding to regions with few points.

```

[7]: # Segment pointcloud using region growing
regions = region_growing(points, normals, residuals, neighbor_finder,
                          angle_threshold, residual_threshold)

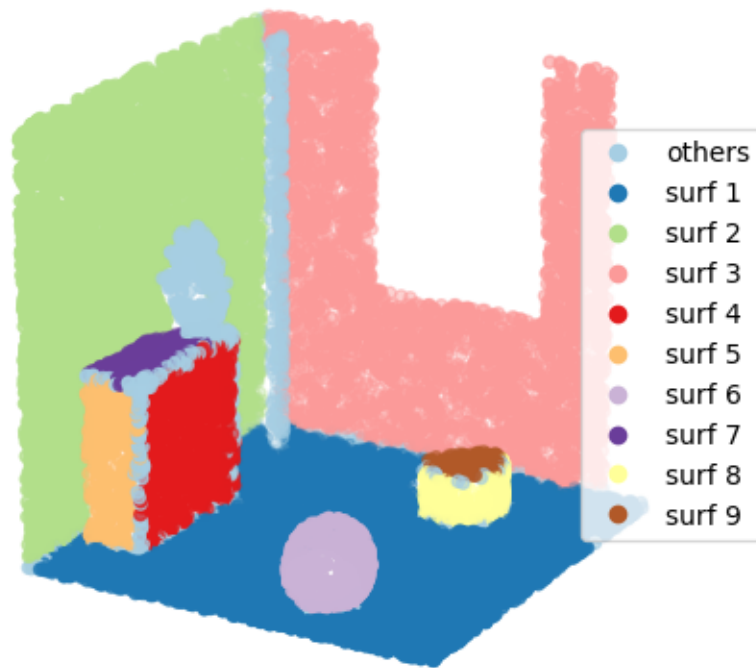
# Attribute a label to each point (number of attributed region)
labels = np.zeros(len(points))
for i, r in enumerate(regions):
    if len(r) < 100: break # disregard small regions for vizualisation purpose
    labels[r] = i + 1
classes = ["surf {}".format(i) for i in range(len(np.unique(labels)))]
classes[0] = "others"

# Plot segmented pointcloud using region growing
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=labels, cmap="Paired")
ax.legend(loc="right", handles=s.legend_elements()[0], labels=classes)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])

```

```
ax.view_init(20, 210)
ax.set_title("Segmented pointcloud using region growing")
plt.show()
```

Segmented pointcloud using region growing



8.1.2 The Hough Transform

The Hough Transform is a technique for detecting parametric shapes. Originally introduced for line detection in images [29], it was later extended to other 2D shapes, such as circles, and eventually generalized to simple 3D shapes such as planes. Indeed, planar surfaces are ubiquitous in many scenes and detecting more complex shapes quickly becomes computationally expensive, as discussed later.

Rather than searching for shapes directly in the pointcloud, the Hough Transform searches for their parameters. **Each point votes for all compatible parameter values**, and shapes supported by many points appear as peaks in the parameter space.

Since plane detection is the most common application of the Hough Transform in pointcloud processing, let us consider the case of a plane in three-dimensional space. A plane is often described

by its Cartesian equation:

$$a.x + b.y + c.z + d = 0$$

The first three parameters define the plane orientation through its normal vector $n = [a, b, c]$, while the last d parameter represents its signed distance from the origin.

Using spherical coordinates to express a unit normal vector ($\|n\| = 1$) leads to the alternative equation:

$$x.\cos(\theta).\sin(\phi) + y.\sin(\theta).\sin(\phi) + z.\cos(\phi) = \rho$$

Although less common, this representation reduces the number of independent parameters from four (a, b, c, d) to three (θ, ϕ, ρ) . Moreover, all three parameters lie within finite ranges, including ρ , making them well suited for discretization in the Hough Transform. Indeed, if a plane passes through a point p_i , then $\rho = n \cdot p_i$, and by the Cauchy-Schwarz inequality:

$$|\rho| = |n \cdot p_i| \leq \|n\| \cdot \|p_i\| = \|p_i\|$$

So, ρ cannot exceed the distance from the origin to the furthest point in the pointcloud. Moreover, the pairs (ρ, n) and $(-\rho, -n)$ describe the same plane. To avoid this redundancy, it is common to restrict ρ to non-negative values.

As a result, planes are usually represented in a three-dimensional Hough space. A point $q_j = [\rho_j, \theta_j, \phi_j]$ in the Hough space represents the plane $x.\cos(\theta_j).\sin(\phi_j) + y.\sin(\theta_j).\sin(\phi_j) + z.\cos(\phi_j) = \rho_j$ in the original 3D space.

Conversely, a point $p_i = [x_i, y_i, z_i]$ in the pointcloud belongs to infinitely many planes. In the Hough space, all planes passing through p_i satisfy $x_{\{i\}}.\cos(\theta).\sin(\phi) + y_{\{i\}}.\sin(\theta).\sin(\phi) + z_{\{i\}}.\cos(\phi) = \rho$, which forms a sinusoidal surface.

The Hough Transform maps every point of the pointcloud into this parameter space, producing one surface per point. Intersections between surfaces correspond to planes shared by the corresponding points. The more surfaces intersect at a given location, the more points belong to the same plane in the original 3D space.

```
[8]: def sample_hough_sinusoid_curve(point, n_theta=50, n_phi=50):
    """Build the unique sinusoid curve in the Hough space for a given point"""

    # Grid with theta and phi
    thetas = np.linspace(0., 2*np.pi, n_theta)
    phis = np.linspace(0., np.pi, n_phi)
    X, Y = np.meshgrid(thetas, phis)
    # Corresponding rho values
    ax = point[0] * np.cos(X) * np.sin(Y)
    by = point[1] * np.sin(X) * np.sin(Y)
    cz = point[2] * np.cos(Y)
    Z = ax + by + cz
```



```

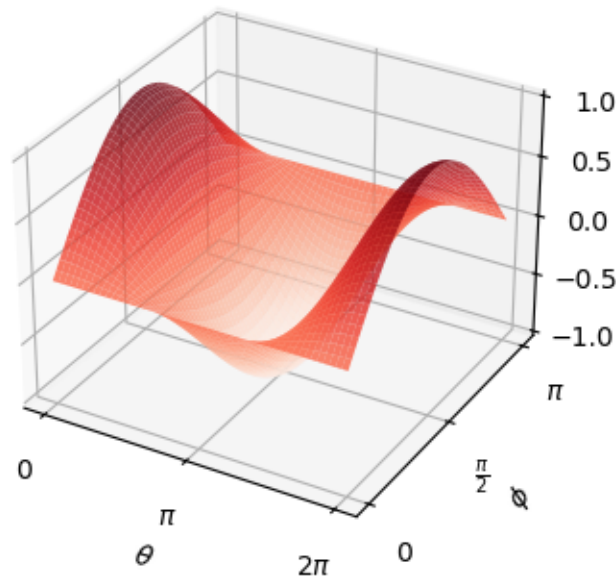
return X, Y, Z

# Example with one point
point_hough = np.array([1., 0., 0.])
X, Y, Z = sample_hough_sinusoid_curve(point_hough)

fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, Z, cmap="Reds", alpha=0.8)
ax.set_xlabel(r'$\theta$')
ax.set_ylabel(r'$\phi$')
ax.set_zlabel(r'$\rho$')
ax.set_xticks([0, np.pi, 2*np.pi])
ax.set_xticklabels([0, r'$\pi$', r'$2\pi$'])
ax.set_yticks([0, np.pi/2, np.pi])
ax.set_yticklabels([0, r'$\frac{\pi}{2}$', r'$\pi$'])
ax.set_title(f"Hough surface associated with $p={point\_hough}$")
plt.show()

```

Hough surface associated with $p = [1.0.0.]$



Consequently, plane detection with the Hough Transform consists of finding locations in the Hough space where many sinusoidal surfaces meet. The more surfaces intersect, the more points belong to the corresponding plane. In practice, however, directly computing these intersections is expensive and sensitive to noise.

To overcome this issue, the Hough space is divided into $n_\rho \times n_\theta \times n_\phi$ cells, called bins. Each bin

stores the number of surfaces passing through it, or equivalently, the number of points compatible with the corresponding plane parameters. This discretized space is known as the *accumulator*.

Plane detection can therefore be seen as a **voting process: each point votes for all planes that could pass through it**. Once all votes have been collected, the bins with the highest scores correspond to the most prominent planes in the pointcloud.

The resolution of the accumulator directly affects the results. A finer discretization allows similar planes to be distinguished more easily, but it also increases memory usage and computation time. For example, choosing $n_\theta = 360$ and $n_\phi = 180$ gives $64\,800 \times n_\rho$ bins. A compromise must be found between detection accuracy and computational cost. When memory becomes an issue, smaller integer types or sparse data structures can be used to reduce the size of the accumulator.

```
[9]: def hough_transform_plane(points, n_theta=120, n_phi=61, n_rho=None):
    """Hough Transform for plane detection."""

    # Angles at which to compute the transform
    theta_values = np.linspace(0., 2*np.pi, n_theta, endpoint=False) # [0, 2*pi[
    phi_values = np.linspace(0., np.pi, n_phi, endpoint=True) # [0, pi]
    # Compute the bins
    rho_max = np.max(np.linalg.norm(points, axis=1))
    # Unit size by default
    if n_rho is None:
        rho_max = np.ceil(rho_max).astype(int)
        n_rho = rho_max + 1
    rho_values, rho_step = np.linspace(0., rho_max, n_rho, # [0, rho_max]
                                     endpoint=True, retstep=True)

    # Allocate the accumulator array
    accumulator = np.zeros(
        (n_theta, n_phi, n_rho),
        dtype=np.uint32 # use insigned int
    )
    # Precompute normals to speed-up computation
    i = np.tile(np.arange(n_theta), n_phi) # grid indices for theta
    j = np.repeat(np.arange(n_phi), n_theta) # grid indices for phi
    normals = np.vstack([
        np.cos(theta_values[i]) * np.sin(phi_values[j]),
        np.sin(theta_values[i]) * np.sin(phi_values[j]),
        np.cos(phi_values[j])
    ])
    # Fill accumulator by computing rho values associated with (theta, phi) for
    ↪ each point
    for point in points:
        # Vectorized form of rho = a.x + b.y + c.z
        rho_calc = point @ normals
        # Compute the grid index associated with rho_calc
        k = (rho_calc//rho_step).astype(int)
```

```

        # Keep only indices associated with positive rho values (avoid
        ↳ redundancy)
        valid = (k >= 0)
        # Increment bins
        accumulator[i[valid], j[valid], k[valid]] += 1

    return accumulator, theta_values, phi_values, rho_values

```

```

[10]: def bin_to_plane_parameters(acc_bin, theta_values, phi_values, rho_values):
        """Convert Hough bin indices to plane parameters (a, b, c, d)"""

        theta_idx, phi_idx, rho_idx = acc_bin

        theta = theta_values[theta_idx]
        phi = phi_values[phi_idx]
        rho = rho_values[rho_idx]

        a = np.cos(theta) * np.sin(phi)
        b = np.sin(theta) * np.sin(phi)
        c = np.cos(phi)
        d = rho

        return np.array([a, b, c, d])

# Transform our pointcloud
accumulator, theta_values, phi_values, rho_values = hough_transform_plane(points)
print(f"The accumulator consists of {accumulator.size} cells,",
      f"of which {np.count_nonzero(accumulator)} are not empty")

# Find the bins with the highest scores
n_best = 5
best_bins = [np.unravel_index(ind, accumulator.shape)
              for ind in np.argpartition(accumulator.flatten(), -n_best)[-n_best:
↳]]
print(f"The best {n_best} bins are: {[tuple(map(int, bin)) for bin in
↳ best_bins]}")

# Retrieve the parameters of the the detected planes
best_planes = [
    bin_to_plane_parameters(b, theta_values, phi_values, rho_values)
    for b in best_bins
]
with np.printoptions(precision=4, suppress=True):
    print("Detected (cartesian) planes are:", best_planes)

```

The accumulator consists of 26352000 cells, of which 9532530 are not empty

The best 5 bins are: [(8, 60, 0), (30, 60, 0), (9, 60, 0), (10, 60, 0), (0, 30, 2000)]

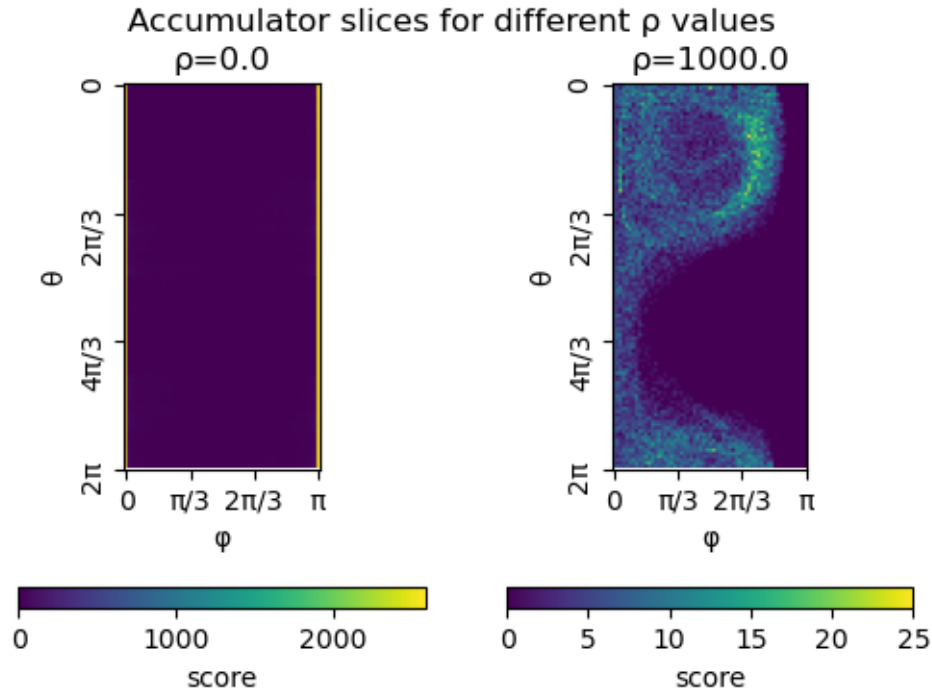
Detected (cartesian) planes are: [array([0., 0., -1., 0.]), array([0., 0., -1., 0.]), array([0., 0., -1., 0.]), array([0., 0., -1., 0.]), array([1., 0., 0., 2000.])]

As shown above, the result is somewhat disappointing, as the detection process returns several nearly identical planes.

These planes correspond to neighboring cells in the accumulator. This can be observed in the figure below, where the detected peaks are grouped into two vertical lines. In this example, the cells with the highest scores correspond to $\rho = 0$ and $\phi = 0$ or $\phi = \pi$. These parameters describe the plane $z = 0$. Indeed, when $\phi = 0$ or $\phi = \pi$, the normal vector is aligned with the z -axis and the parameter θ has no influence on the plane equation.

```
[11]: # Show accumulator values for layer rho_index
rho_indices = [0, 1000]

fig, axes = plt.subplots(1, len(rho_indices), figsize=(6, 4))
for ax, rho_index in zip(axes, rho_indices):
    im = ax.imshow(accumulator[:, :, rho_index], cmap='viridis')
    cbar = fig.colorbar(im, ax=ax,
                        location="bottom", orientation="horizontal", pad=0.2)
    cbar.set_label('score')
    ax.set_xticks(np.linspace(0, len(phi_values)-1, 4))
    ax.set_xticklabels(["0", " $\pi/3$ ", " $2\pi/3$ ", " $\pi$ "], rotation=0)
    ax.set_xlabel('φ')
    ax.set_yticks(np.linspace(0, len(theta_values), 4))
    ax.set_yticklabels(["0", " $2\pi/3$ ", " $4\pi/3$ ", " $2\pi$ "], rotation=90)
    ax.set_ylabel('θ')
    ax.set_title(f"ρ={rho_values[rho_index]}")
fig.suptitle("Accumulator slices for different ρ values")
plt.show()
```



The issue described above arises from the parameterization of planes and the presence of a perfectly horizontal plane in the data. In practice, such cases are rare. A simple solution is to retain the accumulator bins with the highest scores and remove duplicate or nearly identical planes.

```
[12]: # Find all bins/planes with a score higher than a given threshold
significant_bins = np.argwhere(accumulator >= 500)
significant_planes = [
    bin_to_plane_parameters(b, theta_values, phi_values, rho_values)
    for b in significant_bins
]
print(f"Number of significant planes (score >= 500): {len(significant_planes)}")

# Remove duplicate planes by rounding parameters and keeping unique rows
significant_planes = np.round(significant_planes, decimals=4)
unique_significant_planes = np.unique(significant_planes, axis=0)
print(f"Number of unique significant planes: {len(unique_significant_planes)}")

# Label each point according to the detected planes
labels = np.zeros(len(points))
for i, plane_equation in enumerate(unique_significant_planes):
    dists = np.abs(points @ plane_equation[:3] - plane_equation[3])
    inliers = dists < 1. # Adjust in function of rho_step
    labels[inliers] = i + 1
classes = ["plane {}".format(i) for i in range(len(np.unique(labels)))]
```

```

classes[0] = "others"

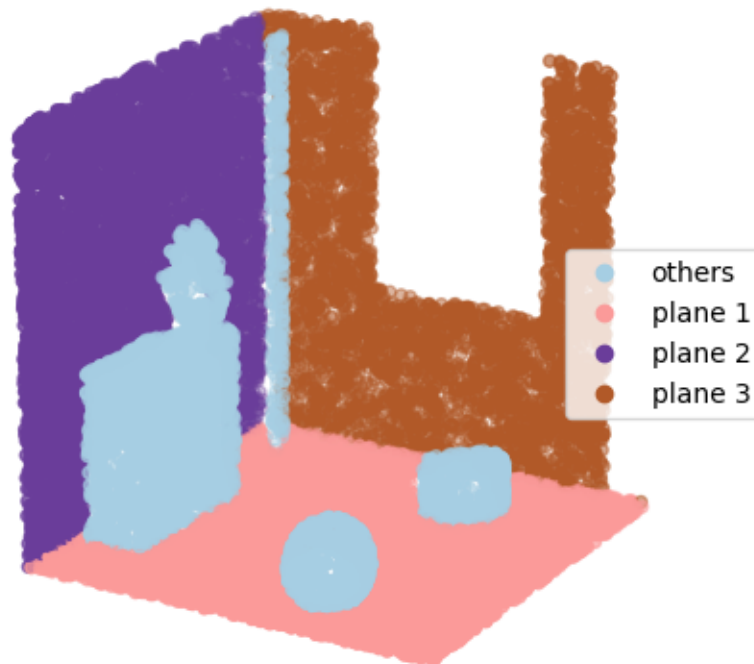
# Plot segmented pointcloud using Hough Transform
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=labels, cmap="Paired")
ax.legend(loc="right", handles=s.legend_elements()[0], labels=classes)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Segmented pointcloud using Hough Transform")
plt.show()

```

Number of significant planes (score ≥ 500): 243

Number of unique significant planes: 5

Segmented pointcloud using Hough Transform



The previous example shows that a single plane often produces several high-score bins in

the accumulator. This can be caused by the parameterization, as illustrated above, but also by discretization and noise. In practice, votes that should fall into the same bin are often spread across neighboring bins.

For this reason, it is generally better to look for groups of neighboring high-score bins rather than simply selecting the bins with the highest scores. A common solution is to **apply Non-Maximum Suppression (NMS) to the accumulator**. The idea is to keep only local maxima and remove lower-scoring bins in their neighborhood.

The simple implementation below first identifies local maxima using a sliding window, then removes peaks whose score is below a given threshold.

```
[13]: def peak_window(accumulator, size_theta=5, size_phi=5, size_rho=3,
    ↪ threshold=None):
    """Use a sliding window technique to filter out the maxima in the accumulator
    that are close to each other."""

    acc = accumulator.copy()
    # Threshold is set to half the max value of the accumulator by default
    if threshold is None:
        threshold = 0.5 * np.max(accumulator)
    # Slide the window and only keep the maximum value
    acc_max = ndimage.maximum_filter(acc, size=(size_theta, size_phi, size_rho),
                                     mode="constant")

    # Create a mask to keep only the peaks
    mask = (acc == acc_max)
    # Apply the mask
    acc *= mask
    # Only keep peaks above the threshold
    acc[acc < threshold] = 0

    return acc
```

As shown below, small peaks and low-scoring bins are filtered out. However, this only partially addresses the issue of the vertical lines discussed above, since they are formed by several adjacent bins with similarly high scores and therefore remain local maxima.

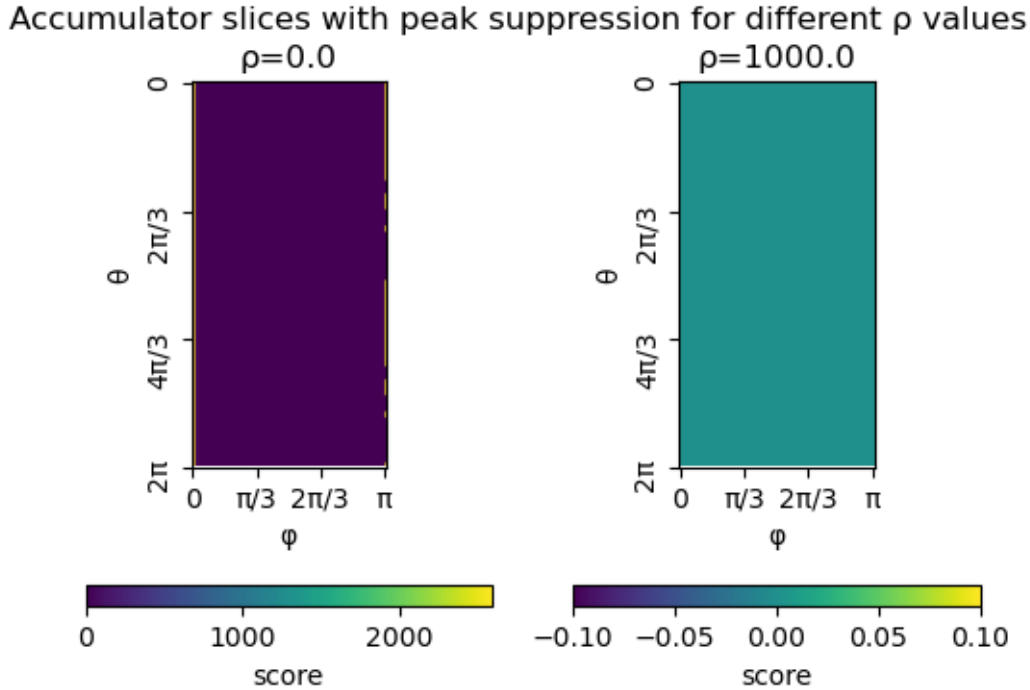
```
[14]: # Apply our peak detection
new_accumulator = peak_window(accumulator)

# Show new accumulator values (with peak suppression) for layer rho_index
# (rho_indices defined above)
fig, axes = plt.subplots(1, len(rho_indices), figsize=(6, 4))
for ax, rho_index in zip(axes, rho_indices):
    im = ax.imshow(new_accumulator[:, :, rho_index], cmap='viridis')
    cbar = fig.colorbar(im, ax=ax,
                        location="bottom", orientation="horizontal", pad=0.2)
    cbar.set_label('score')
    ax.set_xticks(np.linspace(0, len(phi_values)-1, 4))
```

```

ax.set_xticklabels(["0", " $\pi/3$ ", " $2\pi/3$ ", " $\pi$ "], rotation=0)
ax.set_xlabel('φ')
ax.set_yticks(np.linspace(0, len(theta_values), 4))
ax.set_yticklabels(["0", " $2\pi/3$ ", " $4\pi/3$ ", " $2\pi$ "], rotation=90)
ax.set_ylabel('θ')
ax.set_title(f"ρ={rho_values[rho_index]}")
fig.suptitle("Accumulator slices with peak suppression for different ρ values")
plt.show()

```



8.1.3 RANSAC

The Random Sample Consensus (RANSAC) algorithm is a robust model-fitting technique designed to estimate the parameters of a model from data containing noise and outliers. It was originally introduced as a general framework for robust parameter estimation [30].

The algorithm repeatedly performs two simple steps:

1. **Model generation** by randomly selecting the minimum number of points required to estimate the parameters of a candidate model.
2. **Model evaluation** by assessing the consistency of all points with the candidate model (e.g., through a distance measure) and counting the number of inliers.

After a given number of iterations, the model with the highest score is retained. The underlying idea is that both model generation and model evaluation are computationally inexpensive, allowing the process to be repeated many times.

In principle, RANSAC can be applied to any model for which these two operations can be defined. In pointcloud processing, it is commonly used to detect geometric primitives such as planes, cylinders, spheres, cones, and tori.

As an example, let us consider the detection of an infinite plane described by the Cartesian equation:

$$ax + by + cz + d = 0$$

A plane can be uniquely defined by three non-collinear points p_0 , p_1 , and p_2 , from which its parameters are obtained as:

$$n = [a, b, c] = (p_1 - p_0) \times (p_2 - p_0) \quad \text{and} \quad d = -p_0 \cdot n$$

Assuming that n is normalized ($\|n\| = 1$), the distance between a point p and the plane is:

$$d(p) = |n \cdot p + d|$$

A simple implementation of a plane model suitable for RANSAC is given below.

```
[15]: class PlaneModel:
    """Plane model for RANSAC plane fitting."""

    min_sample_size = 3

    def __init__(self, parameters=None):
        self.parameters = (
            np.array([1., 0., 0., 0.])
            if parameters is None
            else np.asarray(parameters, dtype=float)
        )

    @classmethod
    def from_points(cls, points):
        """Construct a plane from three non-collinear points."""

        n = np.cross(points[1] - points[0], points[2] - points[0])
        n /= np.linalg.norm(n) # May add check for degenerate case
        d = -np.dot(n, points[0])

        return cls([n[0], n[1], n[2], d])

    def compute_distance(self, points):
        """Compute point-to-plane distances."""

        n = self.parameters[:3]
        d = self.parameters[3]

        return np.abs(points @ n + d)
```

A large number of candidate shapes (here planes) are generated and evaluated before the best one is selected. Points located within a given distance threshold of a candidate shape are considered inliers and contribute to its score. The quality of the segmentation depends on this threshold: strict values may reject valid points, whereas permissive values may include false inliers.

The number of iterations can either be fixed beforehand or determined from a desired confidence level, the latter being preferred in practice because it provides a clear probability of detection rather than relying on an arbitrary number of iterations.

Assuming a pointcloud of N points containing a shape supported by n points, the probability that a random sample of k points (e.g., $k = 3$ for a plane) belongs entirely to the shape is approximately $(n/N)^k$ when $N \gg k$. After m iterations, the probability of having generated at least one valid candidate is:

$$P = 1 - (1 - (\frac{n}{N})^k)^m$$

Solving for m gives the number of candidate models required to detect shapes of size n with a probability $P \geq P_{th}$:

$$m \geq \frac{\log(1 - P_{th})}{\log(1 - (\frac{n}{N})^k)}$$

In practice, the size of the target shape is unknown and the parameter n corresponds to a minimum support n_{min} , ensuring that any shape containing at least n_{min} points is detected with a probability of at least P_{th} .

This highlights one of the main drawbacks of RANSAC: a large number of candidates is often required to achieve a high probability of detection, especially when the shapes of interest represent only a small fraction of the pointcloud. For example, detecting a plane covering only 10% of the points with a confidence level of 99% requires several thousand candidate models. To improve efficiency, advanced variants typically reduce the number of samples required to estimate a shape, restrict the set of points used for sampling, and reduce the cost of evaluating each candidate.

For clarity, the RANSAC implementation below follows the basic algorithm and does not include any of the advanced optimizations discussed above.

```
[16]: def RANSAC(points, model, threshold, min_support=1000, confidence=0.99):
    """RANSAC algorithm with confidence-based stopping criterion."""

    N, _ = points.shape
    k = model.min_sample_size

    # Number of iterations required
    n_draws = np.log(1.0 - confidence) / np.log(1.0 - (min_support / N)**k)
    n_draws = int(np.ceil(n_draws))

    best_score = 0
    best_model = None
    best_inliers = None
    for _ in range(n_draws):
```

```

    # Random sampling
    sampled_inds = np.random.choice(N, size=k, replace=False)
    sampled_points = points[sampled_inds]
    # Candidate model
    candidate = model.from_points(sampled_points)
    # Inliers
    dists = candidate.compute_distance(points)
    inliers = np.flatnonzero(dists < threshold)
    # Score
    score = len(inliers)
    # Keep best candidate
    if score > best_score:
        best_score = score
        best_model = candidate
        best_inliers = inliers

    return best_model, best_score, best_inliers

```

The RANSAC algorithm is typically applied iteratively to detect multiple shapes: once a shape has been detected, its inliers are removed and the process is repeated on the remaining points.

As illustrated below, the algorithm successfully detects the most prominent planar surfaces in the pointcloud.

```

[21]: # Run RANSAC for planar surfaces
    number_of_shapes = 5

    ransac_planes = []
    ransac_scores = []
    ransac_regions = []
    remaining_inds = np.arange(len(points), dtype=int)
    for _ in range(number_of_shapes):
        model, score, inliers = RANSAC(
            points[remaining_inds],
            PlaneModel,
            threshold=1.0,
            min_support=1000,
            confidence=0.99
        )
        ransac_planes.append(model)
        ransac_scores.append(score)
        ransac_regions.append(remaining_inds[inliers])
        remaining_inds = np.setdiff1d(remaining_inds, remaining_inds[inliers])

    # Attribute a label to each point (number of attributed region)
    labels = np.zeros(len(points))
    for i, r in enumerate(ransac_regions):
        labels[r] = i+1

```

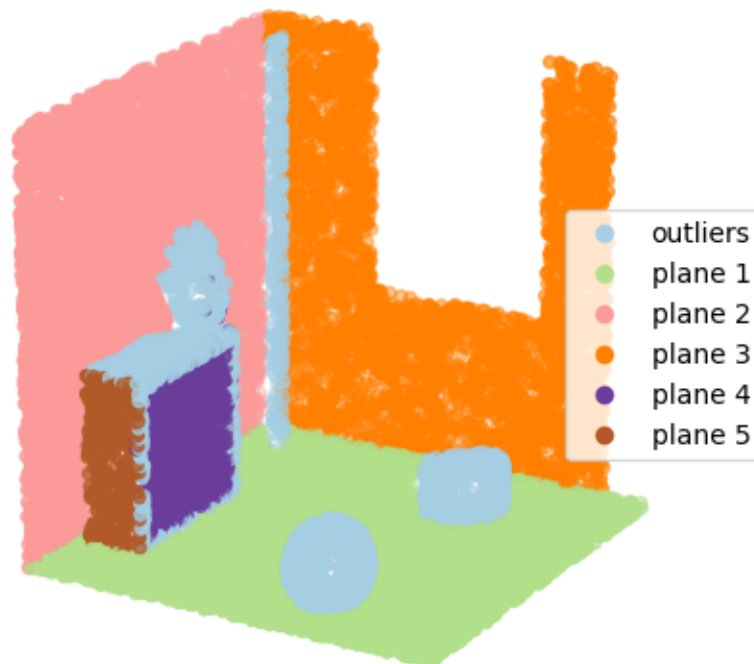
```

classes = ["plane {}".format(i) for i in range(len(np.unique(labels)))]
classes[0] = "outliers"

# Plot segmented pointcloud using RANSAC
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=labels, cmap="Paired")
ax.legend(loc="right", handles=s.legend_elements()[0], labels=classes)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Segmented pointcloud using RANSAC")
plt.show()

```

Segmented pointcloud using RANSAC



8.2 Object Segmentation

Object segmentation consists of partitioning a pointcloud into distinct regions corresponding to individual objects. The segmentation is typically based on spatial proximity, although additional features may also be used.

The approach presented here relies only on spatial proximity to group points belonging to the same object. In simpler terms, it assumes that points belonging to distinct objects are separated by a sufficient distance. This assumption is usually valid in many environments, including indoor, natural, and urban scenes, where physical entities such as furniture, vegetation, buildings, and vehicles generally exhibit clear spatial boundaries.

One common implementation iteratively groups neighboring points into clusters and can be viewed as a simplified version of the region-growing algorithm described above, using only a proximity criterion and ignoring normal compatibility. Alternatively, **a more direct approach represents the pointcloud as a graph that captures local spatial relationships**. It typically involves two steps:

1. **Graph Construction:** A *nearest neighbors graph* is built, connecting each point to its neighboring points within a given distance or among its k nearest neighbors.
2. **Connected Component Extraction:** Groups of points that are mutually reachable through the graph edges are identified and assumed to represent distinct objects.

The graph construction step is typically accelerated using a data structure such as an octree or a kd-tree (see notebook on *spatial indexing*). The `scipy` library provides an efficient kd-tree implementation, including a `sparse_distance_matrix` method, which can be used to compute a distance matrix between two kd-trees (here the pointcloud kd-tree and itself) while enforcing a maximum neighbor distance. Similarly, `scipy` provides a `connected_components` function for extracting connected components from a sparse graph, eliminating the need for a custom implementation.

Note that **this segmentation method generally requires a preprocessing stage**. In many real-world scenes, objects are connected through a *background* or *support surfaces*. These typically correspond to large horizontal planes such as the ground, floors, or tabletops, but may also include large vertical surfaces such as walls or building facades. If left in the pointcloud, these planes can create connections between otherwise distinct objects, leading to incorrect segmentation results. Consequently, points belonging to support surfaces are often removed beforehand using plane-detection techniques (typically RANSAC-based) or simple height-based filtering methods.

```
[18]: # Remove points belonging to the ground plane (z = 0)
      ground_mask = points[:, 2] < 0.05
      points_no_ground = points[~ground_mask]
      colors_no_ground = colors[~ground_mask]
      normals_no_ground = normals[~ground_mask]

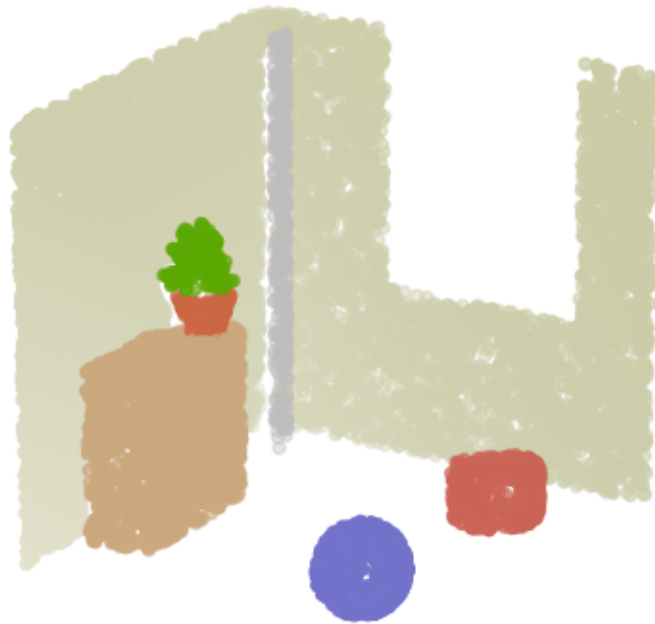
      # Plot remaining points
      fig = plt.figure(figsize=(8, 6))
      ax = fig.add_subplot(projection="3d")
      ax.scatter(
          points_no_ground[:, 0], points_no_ground[:, 1], points_no_ground[:, 2],
          c=colors_no_ground
```

```

)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Pointcloud after removing the gound plane")
plt.show()

```

Pointcloud after removing the gound plane



Despite its apparent simplicity, this technique is capable of segmenting objects in relatively complex scenes. It is computationally efficient and produces easily interpretable results, making it a strong baseline for object segmentation and a widely used approach in practical pointcloud processing pipelines.

```

[19]: # Find connected components based on euclidean distance
kdtree = KDTree(points_no_ground)
nn_graph = kdtree.sparse_distance_matrix(kdtree, max_distance=100.)
n_components, labels = connected_components(nn_graph, directed=False)

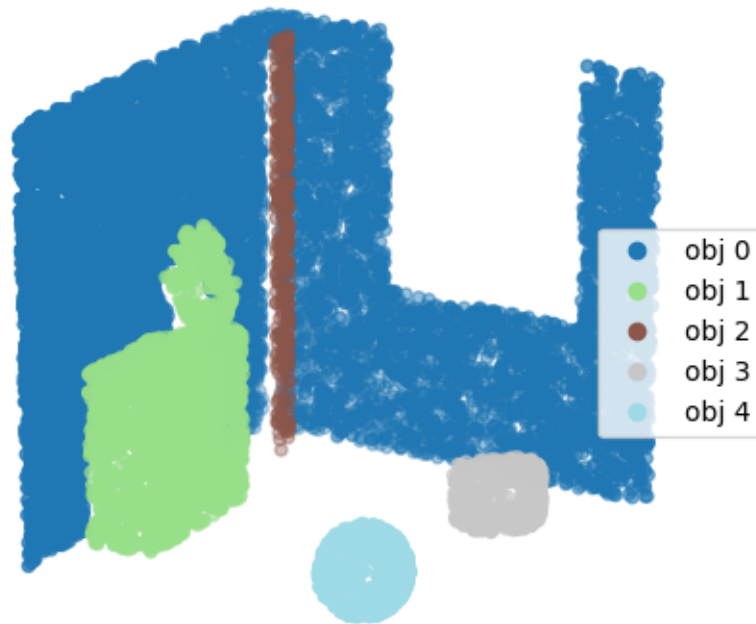
```

```

# Plot connected components
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(
    points_no_ground[:, 0], points_no_ground[:, 1], points_no_ground[:, 2],
    c=labels, cmap="tab20"
)
ax.legend(loc="right", handles=s.legend_elements()[0],
        labels=["obj {}".format(i) for i in range(n_components)])
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Segmented objects based on connected components")
plt.show()

```

Segmented objects based on connected components



8.3 Going further

This section briefly reviews more advanced variants of the segmentation algorithms introduced previously.

Surface Segmentation

Region Growing is simple, deterministic, and particularly effective for extracting smooth surfaces and planar regions. However, its performance depends heavily on local features such as normals, making it sensitive to noise and normal estimation errors.

More advanced variants extend the basic approach with additional constraints, including curvature and color consistency, or region-merging strategies. Another practical extension is *Voxel Cloud Connectivity Segmentation* (VCCS), which first over-segments the pointcloud into spatially coherent *supervoxels* and then performs segmentation on these larger entities rather than on individual points, improving robustness to noise and scalability [31].

For instance PCL provides a **RegionGrowing** implementation similar to the one presented in this notebook. It also offers **RegionGrowingRGB**, which uses color similarity instead of normal orientation and includes a post-processing stage where small regions (below a user-defined size threshold) are merged with the closest neighbouring regions. PCL also implements VCCS through **SupervoxelClustering**, allowing segmentation to be performed on supervoxels rather than individual points.

The **Hough Transform** segments pointclouds through the detection of parametric shapes, most commonly planes. Its main limitation is computational cost, as performance depends strongly on the discretization of the parameter space.

As a result, numerous variants have been proposed to improve efficiency and robustness for plane detection [32]. These include the *Probabilistic Hough Transform*, which reduces computational cost by randomly sampling a subset of points, or hierarchical accumulator structures that reduce memory requirements. Extending the Hough Transform to more complex 3D shapes is considerably more challenging because the associated parameter spaces become increasingly high-dimensional, resulting in a rapid growth of computational cost and memory requirements, a phenomenon known as the *curse of dimensionality*. In some cases, this limitation can be mitigated through multi-stage parameter estimation, as demonstrated for cylinder detection [33]. Nevertheless, such approaches remain relatively uncommon, and the Hough Transform is still used primarily for plane detection.

Despite its long history, the Hough Transform remains relatively uncommon in practical pointcloud processing software, largely because its computational and memory requirements limit its applicability. For instance, **CloudCompare** includes a Hough-based plugin for normal estimation rather than plane detection [19].

RANSAC detects and segments parametric shapes in pointclouds through robust model fitting. Its popularity stems from its robustness to outliers and its ability to handle a wide variety of geometric primitives, despite its computational cost and non-deterministic nature.

Many variants have been proposed to improve efficiency and robustness. One of the most influential is *Efficient RANSAC* [34], which improves efficiency through normal-based shape estimation, octree-guided sampling, a multi-criteria scoring function combining distance, normals, and connectivity, and accelerated evaluation based on local sampling and probabilistic early termination. Other notable variants include PROSAC, which prioritizes promising samples, and MSAC and MLESAC, which improve the evaluation of candidate models through alternative scoring functions.

The original implementation of the *Efficient RANSAC* described above is publicly available and has been integrated into `CGAL` and distributed in `CloudCompare` through a plugin. `Open3D` provides a simpler `segment_plane` implementation that is similar to the one implemented above.

Object Segmentation

The connected-component approach presented in this notebook is one of the simplest and most widely used methods for object segmentation. The approach can be viewed as a graph-based formulation of *Euclidean clustering*. One main difference is that Euclidean Cluster Extraction usually performs the clustering iteratively through region growing, avoiding the need to store the entire graph.

Another popular technique is *DBSCAN*, a density-based clustering algorithm that groups points according to local point density and can naturally classify isolated points as noise. It is often considered a data-clustering algorithm rather than a purely geometrical segmentation technique, and is covered in a next notebook along with other clustering algorithms such as K-Means.

Other approaches formulate segmentation as a graph-partitioning problem. A notable example is *Min-Cut segmentation*, which partitions a pointcloud into foreground and background regions by minimizing the cost of the removed connections. Unlike proximity-based clustering methods, graph-cut approaches can combine multiple geometric attributes (such as spatial proximity, normals, curvature, or color) and are better suited to separating objects with weak or ambiguous boundaries.

Another family of approaches also relies on *supervoxels*, which first group points into spatially coherent regions and then perform segmentation on the resulting adjacency graph [31]. Operating on supervoxels rather than individual points generally improves scalability and robustness while preserving object boundaries more effectively.

Implementations of Euclidean clustering, Min-Cut segmentation, and VCCS are available in `PCL` through `EuclideanClusterExtraction`, `MinCutSegmentation`, and `SupervoxelClustering`, respectively.

8.4 Wrapping up

You should now have a better grasp of some of the most popular surface segmentation algorithms: Region Growing, the Hough Transform, and RANSAC. These approaches rely only on geometrical reasoning and *a priori* assumptions about surfaces to detect: Region Growing groups points that belong to locally smooth surfaces, while the Hough Transform and RANSAC identify geometric primitives.

In practice, the choice of algorithm depends on the application. **Region Growing is well suited to extracting smooth surfaces** from dense and relatively clean pointclouds because it operates locally. The Hough Transform is useful for detecting planar shapes but is not often used in practice because it relies on discretizing the shape parameter space, which can lead to significant computational and memory costs. **RANSAC** is often preferred because it **can detect many types of parametric shapes** and is robust to noise and outliers. However, it is non-deterministic, can be computationally expensive due to its iterative nature, and may require careful parameter tuning.

Surface segmentation can also be used to remove less relevant parts of a pointcloud, such as *background* or *support surfaces*, as a first step toward object segmentation. A common approach is to group points based on proximity using nearest-neighbor graphs and connected-component analysis.

A key takeaway is that all these algorithms require an extensive phase of manual tuning and produce imperfect results on complex pointclouds. This is because they rely on relatively simple assumptions about the scene and on prior knowledge of pointcloud properties such as density and noise levels. Despite these limitations, the results are often good enough for later processing steps. In practice, some manual correction is usually needed.

These limitations partly explain the recent shift from traditional geometry-based segmentation methods to machine-learning and deep-learning approaches. Learning-based methods can segment and classify points using more global and abstract features. This topic will be covered in a future notebook. Before that, the next chapter introduces surface fitting, that is, estimating the parameters of segmented surfaces.

9 Primitive fitting

This notebook introduces techniques for fitting geometric primitives to pointclouds. **Instead of representing geometry using a large collection of sampled points, primitive fitting approximates it with a parametric shape described by a small number of parameters.** This process is often called *abstraction* as low-level point samples are replaced by higher-level geometric entities.

Common primitives include **planes, spheres, cylinders, cones, and tori**. The goal is to find the parameters of the primitive that best fits a set of points. The following sections present the parameterization, distance measures, and fitting algorithms used for each primitive.

9.1 About fitting

Primitive fitting is an optimization problem. Given a set of points and a primitive of known type, the goal is to find the parameters θ that best fit the points.

This is typically done by minimizing the sum of squared distances between the points and the primitive:

$$res(\theta) = \sum_{i=0}^{n-1} d(p_i, S(\theta))^2$$

This quantity is called the **residual**.

The above formulation assumes that all points are affected by the same level of measurement noise. If this is not the case, each point can be assigned a weight according to its expected uncertainty.

For simplicity, the remainder of this notebook uses the notation:

$$E(\theta) = \sum_{i=0}^{n-1} f_i(\theta)^2$$

where E is the error function and f_i is the residual associated with point p_i .

The choice of distance function strongly influences the optimization process. In general, it is preferable to keep the number of parameters as small as possible. For example, a unit normal vector can be represented using spherical coordinates instead of Cartesian coordinates. Since $\|n\| = 1$, only two parameters are needed instead of three.

```
[1]: # Necessary imports
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize, least_squares

sys.path.append("./utils")
from manip_utils import (rotation_f_to_t, cartesian_to_spherical,
                          spherical_to_cartesian)
from plot_utils import (generate_plane_mesh, generate_sphere_mesh,
```

```
generate_cylinder_mesh, generate_cone_mesh,
generate_torus_mesh)
```

9.2 Plane

Plane parameters

A plane can be represented in several ways. The most common is its Cartesian equation:

$$ax + by + cz + d = 0$$

where $[a, b, c]$ defines the plane normal direction and d is the plane offset. An equivalent representation uses a normal vector $n = [a, b, c]$ and a point p lying on the plane and satisfying $p \cdot n = -d$. Both representations describe an infinite plane.

A bounded planar surface can be represented by a rectangular plane defined by the parameters below. The width direction is implicitly given by $b = n \times t$, where n and t are orthogonal unit vectors.

Parameter	Description
c	center of the plane
n	unit normal vector
t	unit vector defining the length direction
l	length
w	width

```
[2]: def generate_plane_points(center, normal, tangent, length, width, n=1000):
    """Generate random points on a rectangular plane."""

    # Generate points on a unit plane
    x = np.random.uniform(-.5, .5, n)
    y = np.random.uniform(-.5, .5, n)
    z = np.zeros(n)
    points = np.stack((x, y, z), axis=1)
    # Scale to the desired dimensions
    points[:, 0] *= length
    points[:, 1] *= width
    # Rotate to the desired orientation
    R = np.stack((tangent, np.cross(normal, tangent), normal), axis=1)
    points = (R @ points.T).T
    # Translate to the desired position
    return points + center

[3]: # Define plane parameters
plane_parameters = (
    np.array([10., 20., 30.]), # center
    np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.]), # normal
```

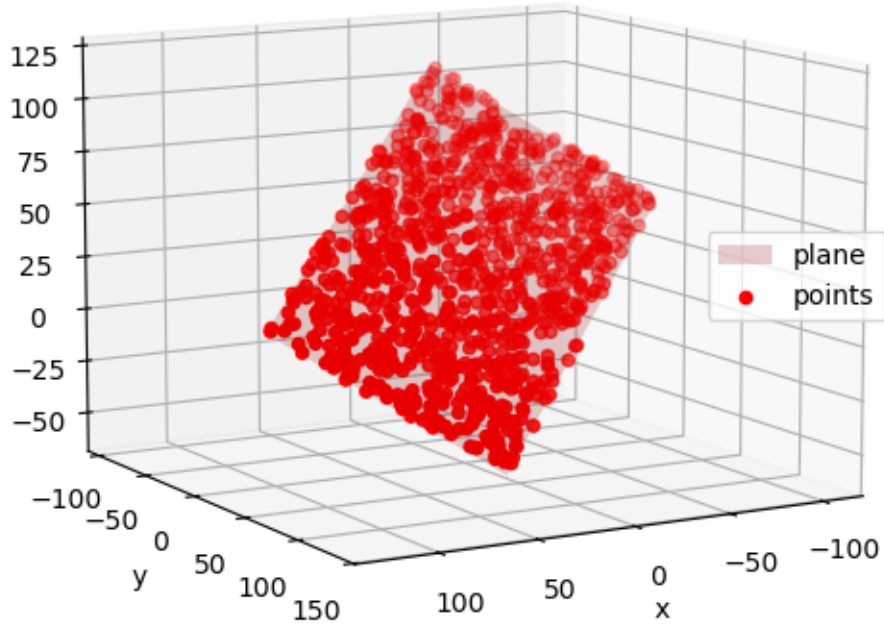
```

    np.array([np.sqrt(2)/2., 0.5, -0.5]), # tangent
    200., # length
    125. # width
)
# Generate points
points_plane = generate_plane_points(*plane_parameters)
# Generate a mesh for visualization
plane_vertices, plane_faces = generate_plane_mesh(*plane_parameters)

# Plot the plane and points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(plane_vertices[:, 0], plane_vertices[:,1], plane_vertices[:,2],
                triangles=plane_faces, color="red", alpha=0.2, label="plane")
ax.scatter(points_plane[:, 0], points_plane[:, 1], points_plane[:, 2],
           c="red", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
ax.axis("equal")
ax.set_title("Generated plane and points")
plt.show()

```

Generated plane and points



Point-to-plane distance

The distance between a point p and a rectangular plane P is given by:

$$d(p, P) = \sqrt{d_n^2 + \max(0, |d_l| - l/2)^2 + \max(0, |d_w| - w/2)^2}$$

where

$$d_n = n \cdot (p - c) \quad d_l = t \cdot (p - c) \quad d_w = (n \times t) \cdot (p - c)$$

For an infinite plane, the last two terms vanish and the distance reduces to $|d_n|$.

The above distance is unsigned. In some situations, it is useful to distinguish the side of the plane on which the point lies. A signed distance can then be defined as:

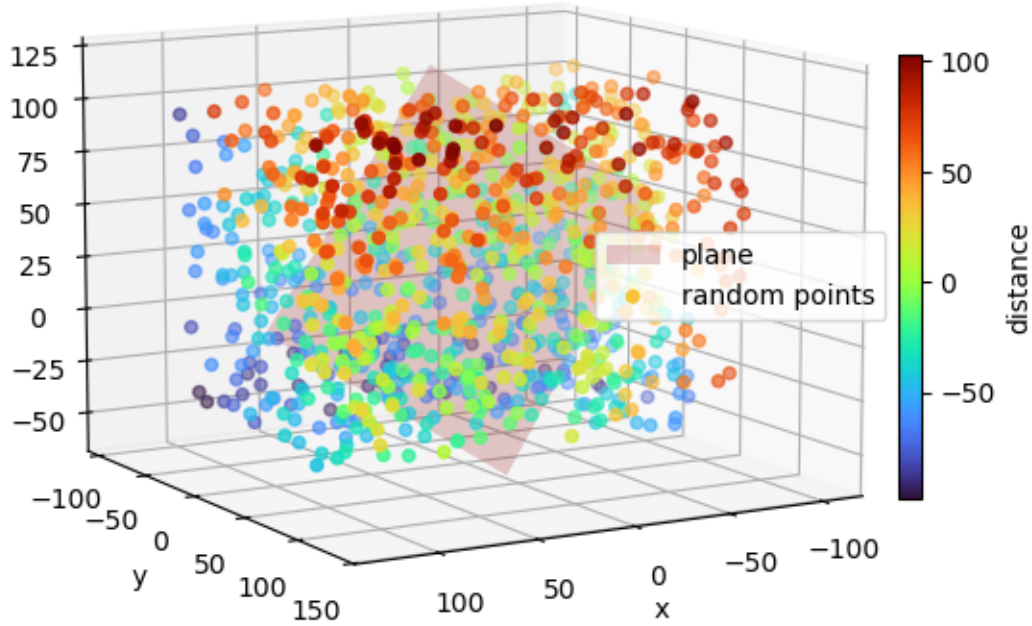
$$d(p, P) = \text{sign}(d_n) \cdot \sqrt{d_n^2 + \max(0, |d_l| - l/2)^2 + \max(0, |d_w| - w/2)^2}$$

Positive distances correspond to the side pointed to by the normal vector $\mathbf{n}$.

```
[4]: def dist_to_plane(points, center, normal, tangent=None, length=None, width=None):  
    """Signed distance to the plane."""  
  
    # Distance along the normal direction  
    vec_to_center = points - center  
    d_n = vec_to_center @ normal  
    if tangent is None:  
        return d_n  
    # Distance in the plane directions  
    d_l = vec_to_center @ tangent  
    d_w = vec_to_center @ np.cross(normal, tangent)  
    # Distance outside finite bounds  
    d_l_out = 0. if length is None else np.maximum(0., np.abs(d_l) - length/2)  
    d_w_out = 0. if width is None else np.maximum(0., np.abs(d_w) - width/2)  
    # Distance to a finite plane  
    d_total = np.sqrt(d_n**2 + d_l_out**2 + d_w_out**2)  
    # Signed distance  
    sign = np.where(d_n < 0, -1., 1.)  
    return sign * d_total
```

```
[5]: # Generate random points and compute distances to the plane  
random_points = np.random.uniform(  
    points_plane.min(axis=0), points_plane.max(axis=0), (1000, 3)  
)  
dists_plane = dist_to_plane(random_points, *plane_parameters)  
  
# Plot distances to the plane  
fig = plt.figure(figsize=(8, 6))  
ax = fig.add_subplot(111, projection="3d")  
ax.plot_trisurf(plane_vertices[:,0], plane_vertices[:,1], plane_vertices[:,2],  
    triangles=plane_faces, color="red", alpha=0.2, label="plane")  
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],  
    c=dists_plane, cmap="turbo", label="random points")  
ax.set_xlabel('x')  
ax.set_ylabel('y')  
ax.set_zlabel('z')  
ax.legend(loc="right")  
ax.view_init(10, 60)  
cbar = fig.colorbar(p, shrink=0.5, pad=0)  
cbar.set_label('distance')  
ax.axis("equal")  
ax.set_title("Distances to plane")  
plt.show()
```

Distances to plane



Least squares plane

The least-squares plane can be estimated either using a nonlinear optimizer or through a closed-form solution based on *Principal Component Analysis* (PCA). Since the PCA approach is simpler, faster, and does not require initialization, it is generally preferred.

First, the covariance matrix of the pointcloud is computed without dividing by the number of points in the neighborhood:

$$cov = \sum_{i=0}^{n-1} (p_i - \bar{p}) \cdot (p_i - \bar{p})^T \quad \bar{p} = \frac{1}{n} \sum_{i=0}^{n-1} p_i$$

Its decomposition gives the eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ and corresponding eigenvectors ν_1 , ν_2 and ν_3 .

The smallest eigenvalue λ_3 is equal to the residual, and the optimal plane directions are:

$$n = \nu_3 \quad t = \nu_1$$

Note that the signs of the eigenvectors are arbitrary.

For an infinite plane, the center is simply the centroid $\bar{p}$.

For a bounded rectangular plane, the center, length, and width are obtained by projecting the points onto the principal axes. Note that the area of the resulting rectangle is not minimal.

```
[6]: def fit_plane(points):
      """Best-fit plane to a pointcloud."""

      # Center pointcloud
      centroid = points.mean(axis=0)
      centered_points = points - centroid
      # Covariance matrix
      cov = centered_points.T @ centered_points
      # Eigenvalues are returned in ascending order
      eigenvalues, eigenvectors = np.linalg.eigh(cov)
      # Smallest eigenvalue = residual
      residual = eigenvalues[0]
      # Plane directions
      normal = eigenvectors[:, 0]
      tangent = eigenvectors[:, 2]
      # Project points onto the principal axes
      projected_points = (eigenvectors.T @ centered_points.T).T
      # Bounding rectangle dimensions
      _, width, length = np.ptp(projected_points, axis=0)
      bbox_center = (projected_points.max(axis=0)+projected_points.min(axis=0))/2
      # Center of the bounding rectangle
      shift = np.array([0., bbox_center[1], bbox_center[2]])
      center = centroid + eigenvectors @ shift

      return (center, normal, tangent, length, width), residual
```

```
[7]: # Fit plane to pointcloud
plane_fit, plane_residual = fit_plane(points_plane)
# Display estimated parameters
with np.printoptions(precision=3, suppress=True):
    for name, val in zip(
        ["center", "normal", "tangent", "length", "width"],
        plane_fit
    ):
        print(f"best-fit plane {name} is: {val}")
    print(f"residual is: {plane_residual}")
# Generate mesh for visualization purposes
plane_fit_vertices, plane_fit_faces = generate_plane_mesh(*plane_fit)
# Show distance to best-fit plane
dists_plane_fit = dist_to_plane(points_plane, *plane_fit)
```

```

# Plot best-fit plane and distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_plane[:, 0], points_plane[:, 1], points_plane[:, 2],
               c=dists_plane_fit, cmap="turbo", label="initial points")
ax.plot_trisurf(plane_fit_vertices[:, 0], plane_fit_vertices[:, 1],
               plane_fit_vertices[:, 2], triangles=plane_fit_faces,
               color="white", alpha=0.2, label="best-fit plane")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label("distance")
ax.axis("equal")
ax.set_title("Distances to fitted plane")
plt.show()

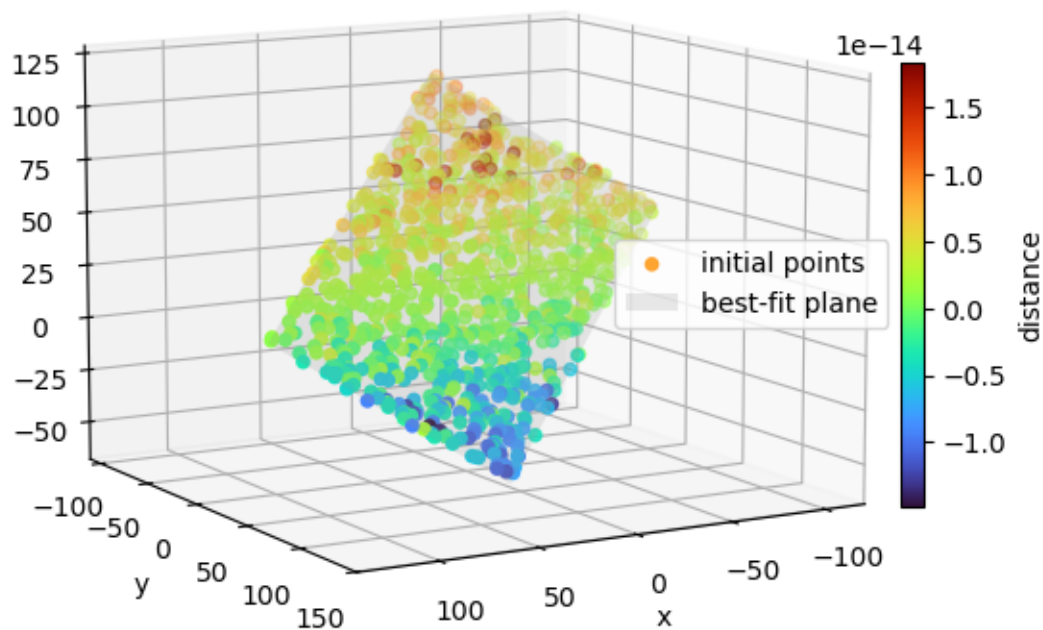
```

```

best-fit plane center is: [ 9.691 19.753 30.247]
best-fit plane normal is: [0.    0.707 0.707]
best-fit plane tangent is: [-0.703 -0.503  0.503]
best-fit plane length is: 199.41294934223237
best-fit plane width is: 124.6795003816994
residual is: 3.6195346199418295e-11

```

Distances to fitted plane



9.3 Sphere

Sphere parameters

A full sphere is defined by its center c and radius r . Its Cartesian equation is:

$$(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 = r^2$$

Parameter	Description
c	center
r	radius

```
[8]: def generate_sphere_points(center, radius, n=1000):
      """Random points on a sphere."""

      # Generate points on a unit sphere
```

```

phi = np.random.uniform(0., 2*np.pi, (n, 1))
z = np.random.uniform(-1., 1., (n, 1))
rho = np.sqrt(1 - z**2)
x = rho * np.cos(phi)
y = rho * np.sin(phi)
points = np.hstack((x, y, z))
# Scale to the desired radius
points *= radius
# Translate to the desired position
return points + center

```

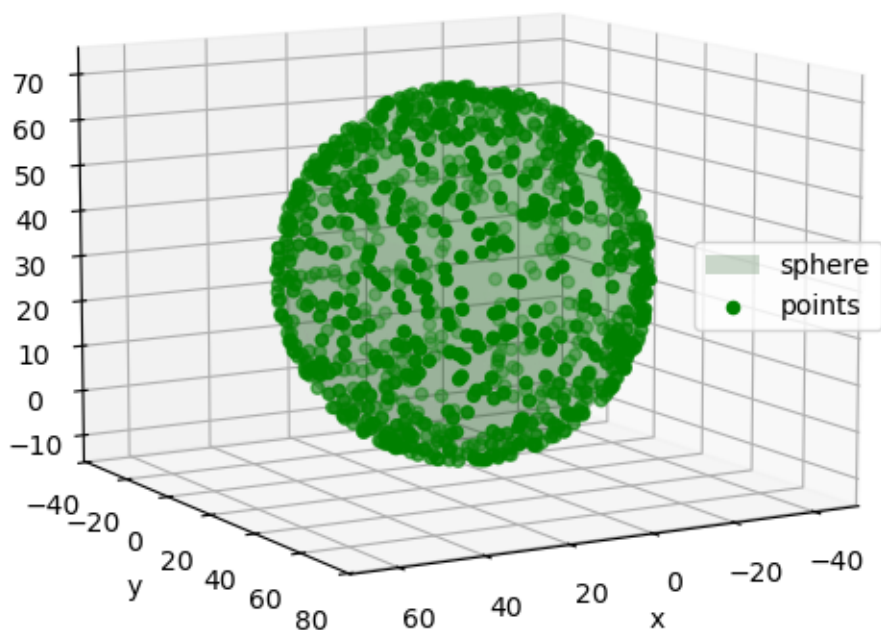
```

[9]: # Define sphere parameters
sphere_parameters = (
    np.array([10., 20., 30.]), # center
    40. # radius
)
# Generate points
points_sphere = generate_sphere_points(*sphere_parameters)
# Generate a mesh for visualization
sphere_vertices, sphere_faces = generate_sphere_mesh(*sphere_parameters)

# Plot points and sphere
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(sphere_vertices[:, 0], sphere_vertices[:, 1],
                sphere_vertices[:, 2], triangles=sphere_faces, color="green",
                alpha=.2, label="sphere")
ax.scatter(points_sphere[:, 0], points_sphere[:, 1], points_sphere[:, 2],
           c="green", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
ax.axis("equal")
ax.set_title("Generated sphere and points")
plt.show()

```

Generated sphere and points



Point-to-sphere distance

The signed distance between a point p and a sphere S is simply given by:

$$d(p, S) = \|p - c\| - r$$

Positive distances correspond to points outside the sphere, while negative distances correspond to points inside it.

```
[10]: def dist_to_sphere(points, center, radius):  
        """Signed distance to a sphere."""  
  
        return np.linalg.norm(points - center, axis=1) - radius  
  
[11]: # Generate random points and compute their distance to the sphere  
random_points = np.random.uniform(  
    points_sphere.min(axis=0), points_sphere.max(axis=0), (1000, 3)
```

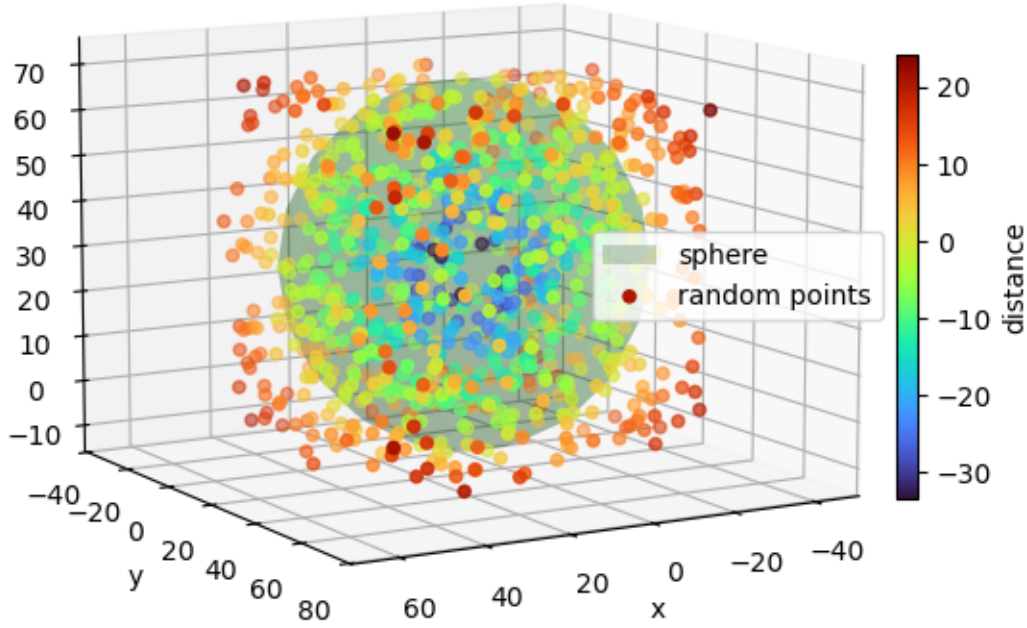
```

)
dists_sphere = dist_to_sphere(random_points, *sphere_parameters)

# Plot distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(sphere_vertices[:, 0], sphere_vertices[:, 1],
                sphere_vertices[:, 2], triangles=sphere_faces, color="green",
                alpha=0.2, label="sphere")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
               c=dists_sphere, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label('distance')
ax.axis("equal")
ax.set_title("Distances to sphere")
plt.show()

```

Distances to sphere



Least squares sphere

A least-squares sphere can be estimated either using a nonlinear optimizer or through a direct linear least-squares formulation. The latter is simpler and does not require initialization.

Starting from the Cartesian equation of the sphere, expanding and regrouping terms leads to:

$$2xc_x + 2yc_y + 2zc_z + r^2 - c_x^2 - c_y^2 - c_z^2 = x^2 + y^2 + z^2$$

Introducing:

$$t = r^2 - c_x^2 - c_y^2 - c_z^2$$

gives the linear system:

$$2xc_x + 2yc_y + 2zc_z + t = x^2 + y^2 + z^2$$

The fitting problem can therefore be written as a linear least-squares problem $Ax = b$ with unknowns $x = [c_x, c_y, c_z, t]$. Once solved, the sphere center is obtained directly from (c_x, c_y, c_z) , and the radius is recovered from t .

```
[12]: def fit_sphere(points):
    """Best-fit sphere to a pointcloud."""

    # Linear least-squares system
    A = np.column_stack((2*points, np.ones(len(points))))
    b = (points**2).sum(axis=1)
    # Solve  $Ax = b$ 
    x, residual, _, _ = np.linalg.lstsq(A, b, rcond=None)
    # Recover sphere parameters
    center = x[:3]
    radius = np.sqrt(x[0]**2 + x[1]**2 + x[2]**2 + x[3])

    return (center, radius), residual

[13]: # Fit sphere to pointcloud
sphere_fit, sphere_residual = fit_sphere(points_sphere)
# Display estimated parameters
with np.printoptions(precision=3, suppress=True):
    for name, val in zip(["center", "radius"], sphere_fit):
        print(f"best-fit sphere {name} is: {val}")
    print(f"residual is: {sphere_residual}")
# Generate a mesh for visualization
sphere_fit_vertices, sphere_fit_faces = generate_sphere_mesh(*sphere_fit)
# Show distance to fitted sphere
dists_sphere_fit = dist_to_sphere(points_sphere, *sphere_fit)

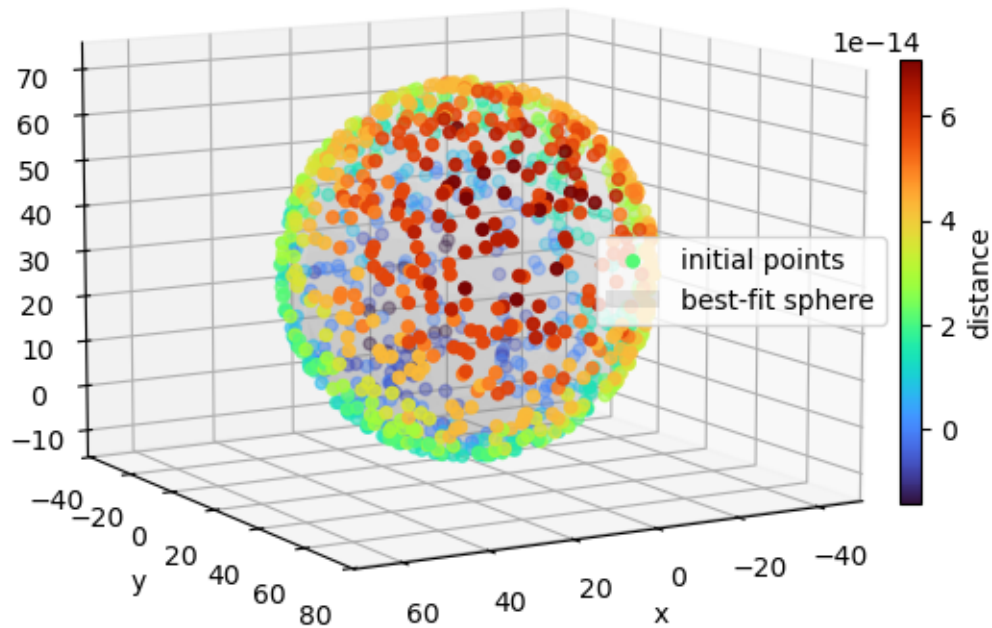
# Plot distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_sphere[:, 0], points_sphere[:, 1], points_sphere[:, 2],
               c=dists_sphere_fit, cmap="turbo", label="initial points")
ax.plot_trisurf(sphere_fit_vertices[:, 0], sphere_fit_vertices[:, 1],
               sphere_fit_vertices[:, 2], triangles=sphere_fit_faces,
               color="white", alpha=0.2, label="best-fit sphere")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label('distance')
ax.axis("equal")
ax.set_title("Distances to fitted sphere")
```



```
plt.show()
```

```
best-fit sphere center is: [10. 20. 30.]  
best-fit sphere radius is: 39.99999999999997  
residual is: [0.]
```

Distances to fitted sphere



9.4 Cylinder

Cylinder parameters

An infinite cylinder aligned with the z -axis can be represented by the Cartesian equation:

$$(x - c_x)^2 + (y - c_y)^2 = r^2$$

A finite cylinder of arbitrary orientation is considered here. It is defined by its center c , axis direction n , radius r and length l . The cylinder is bounded by two planes perpendicular to n and separated by a distance l . The center c lies halfway between these planes.

Parameter	Description
c	center
n	unit vector defining the axis direction
r	radius
l	length

```
[14]: def generate_cylinder_points(center, normal, radius, length, n=1000):
    """Generate random points on a cylinder."""

    # Generate points on a unit cylinder
    theta = np.random.uniform(0., 2*np.pi, (n, 1))
    x = np.cos(theta)
    y = np.sin(theta)
    z = np.random.uniform(-0.5, 0.5, (n, 1))
    points = np.hstack((x, y, z))
    # Scale to the desired dimensions
    points[:, :2] *= radius
    points[:, 2] *= length
    # Rotate to the desired orientation
    R = rotation_f_to_t(np.array([0., 0., 1.]), normal)
    points = (R @ points.T).T
    # Translate to the desired position
    return points + center

[15]: # Define cylinder parameters
cylinder_parameters = (
    np.array([10., 20., 30.]), # center
    np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.]), # axis direction
    40., # radius
    50. # length
)
# Generate mesh for visualization
cylinder_vertices, cylinder_faces = generate_cylinder_mesh(*cylinder_parameters)
# Generate points
points_cylinder = generate_cylinder_points(*cylinder_parameters)

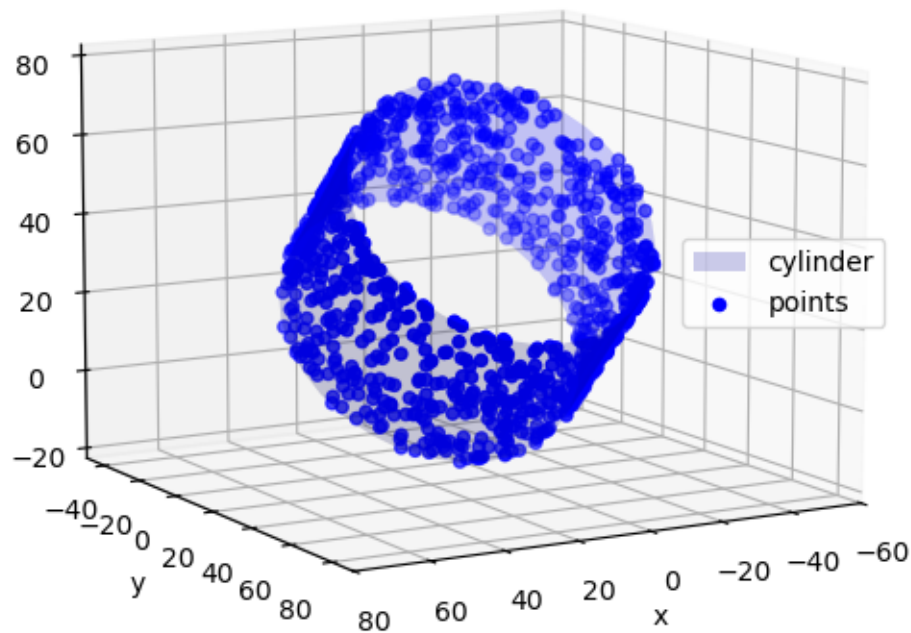
# Plot the cylinder and points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cylinder_vertices[:, 0], cylinder_vertices[:, 1],
                cylinder_vertices[:, 2], triangles=cylinder_faces,
                color="blue", alpha=0.2, label="cylinder")
ax.scatter(points_cylinder[:, 0], points_cylinder[:, 1], points_cylinder[:, 2],
            c="blue", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
```

```

ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
ax.axis("equal")
ax.set_title("Generated cylinder and points")
plt.show()

```

Generated cylinder and points



Points-to-cylinder distance

For an infinite cylinder, the signed distance between a point p and a cylinder C is:

$$d_r(p, C) = \|n \times (p - c)\| - r$$

where the first term is the distance between the point and the cylinder axis. Positive distances correspond to points outside the cylinder, while negative distances correspond to points inside it.

For a finite cylinder, the position of the point along the axis must also be considered:

$$d_a(p, C) = n \cdot (p - c)$$

Points satisfying:

$$|d_a(p, C)| > l/2$$

lie beyond one of the cylinder bounds. Their distance is then computed with respect to the corresponding bounding circle C_1 or C_2 :

$$d(p, C_i) = \sqrt{[n \cdot (p - c_i)]^2 + (\|n \times (p - c_i)\| - r)^2}$$

where

$$c_1 = c - \frac{l}{2}n \quad c_2 = c + \frac{l}{2}n.$$

For all other points, the cylinder distance is simply

$$d(p, C) = d_r(p, C)$$

```
[16]: def dist_to_cylinder(points, center, normal, radius, length=None):
    """Signed distance to the cylinder."""

    # Distance to the cylinder axis
    vec_to_center = points - center
    dist_to_axis = np.linalg.norm(np.cross(normal, vec_to_center), axis=1)
    # Infinite cylinder
    d_r = dist_to_axis - radius
    if length is None:
        return d_r
    d_t = np.zeros(len(d_r), dtype=float) # total distance
    # Position along the axis
    d_a = vec_to_center @ normal
    # Points between the two bounds
    inds_in = np.nonzero(np.abs(d_a) <= length / 2)[0]
    d_t[inds_in] = d_r[inds_in]
    # Points beyond the first bound
    inds_out_1 = np.nonzero(d_a < -length / 2)[0]
    c_1 = center - length / 2 * normal
    vec_to_c_1 = points[inds_out_1] - c_1
    dist_to_axis_1 = np.linalg.norm(np.cross(normal, vec_to_c_1), axis=1)
    d_t[inds_out_1] = np.sqrt(
        (vec_to_c_1 @ normal)**2 + (dist_to_axis_1 - radius)**2
    )
    # Points beyond the second bound
    inds_out_2 = np.nonzero(d_a > length / 2)[0]
```

```

c_2 = center + length / 2 * normal
vec_to_c_2 = points[inds_out_2] - c_2
dist_to_axis_2 = np.linalg.norm(np.cross(normal, vec_to_c_2), axis=1)
d_t[inds_out_2] = np.sqrt(
    (vec_to_c_2 @ normal)**2 + (dist_to_axis_2 - radius)**2
)

return d_t

```

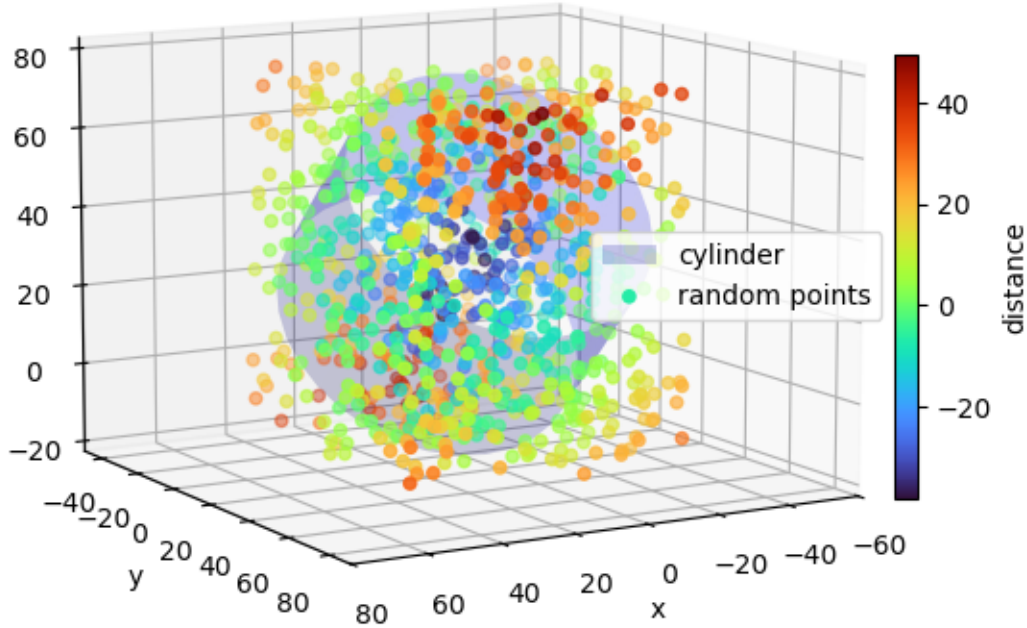
```

[17]: # Generate random points and compute distance to the cylinder
random_points = np.random.uniform(
    points_cylinder.min(axis=0), points_cylinder.max(axis=0), (1000, 3)
)
dists_cylinder = dist_to_cylinder(random_points, *cylinder_parameters)

# Plot the distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cylinder_vertices[:, 0], cylinder_vertices[:, 1],
                cylinder_vertices[:, 2], triangles=cylinder_faces,
                color="blue", alpha=0.2, label="cylinder")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
                c=dists_cylinder, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label("distance")
ax.axis("equal")
ax.set_title("Distances to cylinder")
plt.show()

```

Distances to cylinder



Least-squares cylinder

Unlike planes and spheres, cylinders do not admit a simple closed-form least-squares solution. The approach presented here follows [35].

Using the Cartesian equation of the cylinder, a point p belongs to the cylinder if its squared distance to the axis of unit direction n is equal to the squared radius, leading to the residual function:

$$f(c, n, r^2) = (p - c)^T (I - nn^T) (p - c) - r^2$$

The unknowns of the associated least-squares problem are the cylinder center c , axis direction n and squared radius r^2 .

The parameters may be estimated using a nonlinear least-squares optimizer such as Levenberg-Marquardt. However, convergence strongly depends on the quality of the initial guess.

An alternative approach, proposed by Eberly, minimizes an auxiliary function G derived from the stationary points of the problem. Although still iterative, this method avoids directly minimizing E and generally provides more robust estimates.

The implementation below follows this second approach. It first estimates the cylinder axis by minimizing the function G , then recovers the center, radius, and length analytically. Most computations are vectorized. The minimization is performed with Powell's method, which provides good results in practice and avoids the need to compute derivatives of G .

```
[18]: def fit_cylinder(points, normal_guess=[0., 0., 1.]):
    """Best-fit cylinder to a pointcloud."""

    # Initial direction estimate
    start_point = cartesian_to_spherical(normal_guess)
    # Center the pointcloud
    centroid = points.mean(axis=0)
    centered_points = points - centroid
    # Estimate the cylinder axis
    best_fit = minimize(
        lambda x : G(spherical_to_cartesian(x), centered_points),
        start_point,
        method="Powell",
    )
    normal = spherical_to_cartesian(best_fit.x)
    residual = best_fit.fun
    # Recover the remaining parameters
    # Projection matrix P
    P = np.eye(3) - normal.reshape(-1, 1) @ normal.reshape(1, -1)
    # Projected points
    projected_points = (P @ centered_points.T).T
    # Inertia matrix
    A = projected_points.T @ projected_points
    # Skew symmetric matrix
    S = np.array([
        [0, -normal[2], normal[1]],
        [normal[2], 0, -normal[0]],
        [-normal[1], normal[0], 0]
    ])
    # Adjugate matrix
    A_hat = S @ (A @ S.T)
    # Center
    center = (
        A_hat/np.trace(A_hat @ A)
    ) @ (
        (projected_points * projected_points).sum(axis=1).reshape(-1, 1)
        * projected_points
    ).sum(axis=0)
    center += centroid
    # Radius
    r2 = np.mean(np.sum(((points - center) @ P) * (points - center), axis=1))
    radius = np.sqrt(r2)
```

```

# Length
length = np.ptp((points - center) @ normal)

return (center, normal, radius, length), residual # residual of G function!

def G(normal, centered_points):
    """Cylinder objective function."""

    # Projection matrix
    P = np.eye(3) - normal.reshape(3, 1) @ normal.reshape(1, 3)
    # Projected points
    projected_points = (P @ centered_points.T).T
    # Inertia matrix
    A = projected_points.T @ projected_points
    # Skew symmetric matrix
    S = np.array([
        [0, -normal[2], normal[1]],
        [normal[2], 0, -normal[0]],
        [-normal[1], normal[0], 0]
    ])
    # Adjugate matrix
    A_hat = S @ (A @ S.T)
    # Intermediate terms for the G function
    s = (projected_points**2).sum(axis=1).reshape(-1, 1)
    u = s.sum()/len(centered_points)
    v = (A_hat / np.trace(A_hat @ A)) @ (s * projected_points).sum(axis=0)

    return ((s - u - 2*(projected_points @ v).reshape(-1, 1))*2).sum()

```

```

[19]: # Fit the cylinder
cylinder_fit, cylinder_residual = fit_cylinder(points_cylinder)
# Display estimated parameters
with np.printoptions(precision=3, suppress=True):
    for name, val in zip(
        ["center", "normal", "radius", "length"],
        cylinder_fit
    ):
        print(f"best-fit cylinder {name} is: {val}")
    print(f"residual of G is: {cylinder_residual}")

# Generate a mesh for visualization
cylinder_fit_vertices, cylinder_fit_faces = generate_cylinder_mesh(*cylinder_fit)
# Compute distances to the fitted infinite cylinder to avoid numerical errors
dists_cylinder_fit = dist_to_cylinder(points_cylinder, *cylinder_fit[:-1])

# Plot the fitted cylinder and distances

```



```

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_cylinder[:, 0], points_cylinder[:, 1],
               points_cylinder[:, 2], c=dists_cylinder_fit, cmap="turbo",
               label="initial points")
ax.plot_trisurf(cylinder_fit_vertices[:, 0], cylinder_fit_vertices[:, 1],
               cylinder_fit_vertices[:, 2], triangles=cylinder_fit_faces,
               color="white", alpha=0.2, label="best-fit cylinder")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label("distance")
ax.axis("equal")
ax.set_title("Distances to fitted cylinder")
plt.show()

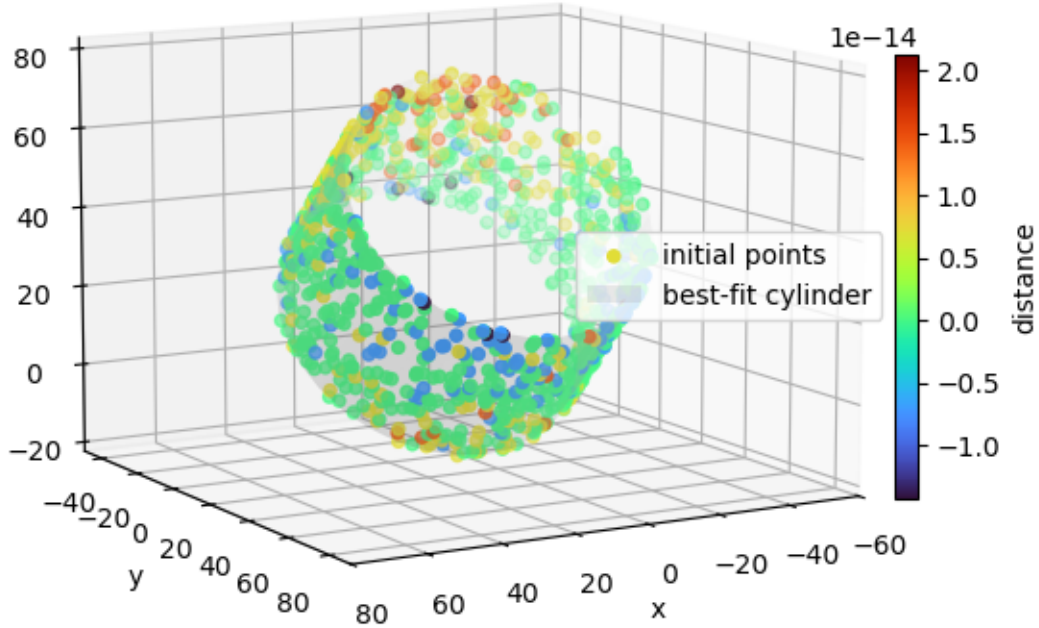
```

```

best-fit cylinder center is: [10.    20.158 30.158]
best-fit cylinder normal is: [0.    0.707 0.707]
best-fit cylinder radius is: 40.0
best-fit cylinder length is: 49.74512542381647
residual of G is: 2.0650588348950193e-22

```

Distances to fitted cylinder



9.5 Cone

Cone parameters

An infinite cone aligned with the z -axis can be represented by:

$$(x - c_x)^2 + (y - c_y)^2 = (z - c_z)^2 \cdot \tan^2 \alpha$$

This equation defines a double cone of half-angle α , with two symmetric branches meeting at the apex c .

A cone of arbitrary orientation with only one branch is considered here, resulting in a semi-infinite cone bounded by its apex and extending in the direction $-n$. A finite cone is obtained by truncating this semi-infinite cone with a plane perpendicular to the axis at a distance l from the apex. It is therefore defined by its apex c , axis direction n , half-angle α , and length l .

Parameter	Description
c	apex position
n	unit vector defining the axis direction
α	half-angle
l	length

```
[20]: def generate_cone_points(apex, normal, half_angle, length, n=1000):
        """Generate random points on a cone."""

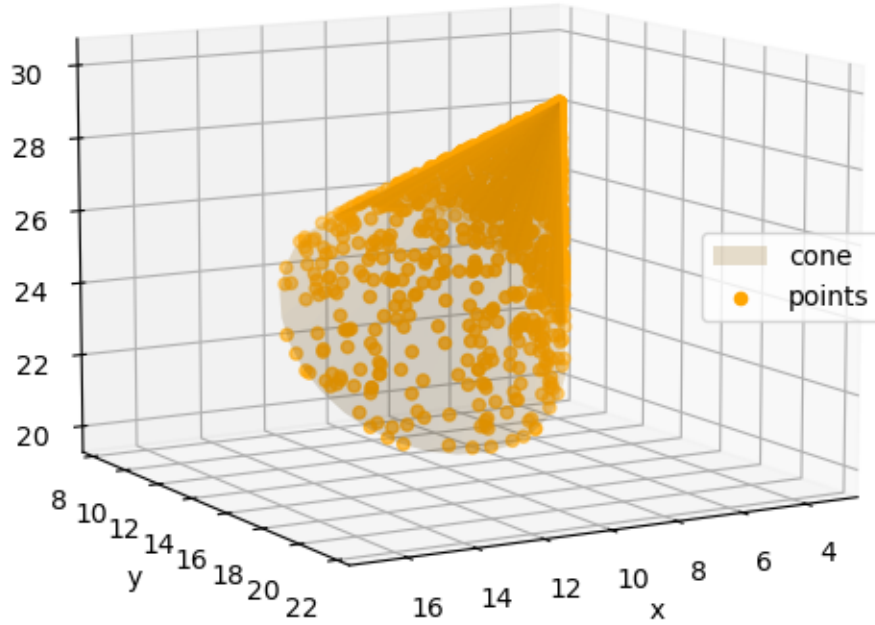
        # Generate points on a cone aligned with the z-axis
        z = np.random.uniform(-length, 0., (n, 1))
        theta = np.random.uniform(0., 2*np.pi, (n, 1))
        x = z * np.tan(half_angle) * np.cos(theta)
        y = z * np.tan(half_angle) * np.sin(theta)
        points = np.hstack((x, y, z))
        # Rotate to the desired orientation
        R = rotation_f_to_t(np.array([0., 0., 1.]), normal)
        points = (R @ points.T).T
        # Translate to the desired position
        return points + apex
```

```
[21]: # Define cone parameters
cone_parameters = (
    np.array([10., 20., 30.]), # apex
    np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.]), # axis direction
    np.pi/8, # half-angle
    10. # length
)
# Generate points
points_cone = generate_cone_points(*cone_parameters)
# Generate a mesh for visualization
cone_vertices, cone_faces = generate_cone_mesh(*cone_parameters)

# Plot the cone and points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cone_vertices[:, 0], cone_vertices[:, 1], cone_vertices[:, 2],
                triangles=cone_faces, color="orange", alpha=0.2, label="cone")
ax.scatter(points_cone[:, 0], points_cone[:, 1], points_cone[:, 2],
           c="orange", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
ax.axis("equal")
```

```
ax.set_title("Generated cone and points")
plt.show()
```

Generated cone and points



Point-to-cone distance

For an infinite cone, the signed distance between a point p and a cone C is:

$$d_r(p, C) = (||n \times (p - c)|| - |(p - c) \cdot n| \tan \alpha) \cos \alpha$$

The first term is the distance between the point and the cone axis, while the second is the radius of the cone at the same position along the axis. Positive values correspond to points outside the cone, while negative values correspond to points inside it.

For a finite cone, the position of the point along the axis must also be considered:

$$d_a(p, C) = n \cdot (p - c)$$

The cone is bounded by its apex c and by a circular base C_1 of center and radius:

$$c_1 = c - ln \quad r = l \tan \alpha$$

Points whose closest point on the cone is the apex satisfy:

$$d_a(p, C) > \|p - c\| \sin \alpha$$

Their distance to the cone is simply the distance to the apex:

$$d(p, c) = \|p - c\|$$

Similarly, points whose closest point on the cone lies on the base circle satisfy:

$$d_a(p, C) < -l - \|p - c_1\| \sin \alpha$$

Their distance is computed with respect to the bounding circle:

$$d(p, C_1) = \sqrt{[n \cdot (p - c_1)]^2 + (\|n \times (p - c_1)\| - r)^2}$$

For all other points, the cone distance is simply:

$$d(p, C) = d_r(p, C)$$

```
[22]: def dist_to_cone(points, apex, normal, half_angle, length=None):
    """Signed distance to the cone."""

    # Distance to the axis
    vec_to_apex = points - apex
    dist_to_axis = np.linalg.norm(np.cross(normal, vec_to_apex), axis=1)
    # Position along the axis
    d_a = vec_to_apex @ normal
    # Distance to the infinite cone lateral surface
    d_t = (dist_to_axis - np.abs(d_a) * np.tan(half_angle)) * np.cos(half_angle)
    # Apex
    inds_beyond_apex = np.flatnonzero(d_a > 0) # subset of interest
    if len(inds_beyond_apex):
        dist_to_apex = np.linalg.norm(vec_to_apex[inds_beyond_apex], axis=1)
        mask_apex = d_a[inds_beyond_apex] > dist_to_apex * np.sin(half_angle)
        if np.any(mask_apex):
            d_t[inds_beyond_apex[mask_apex]] = dist_to_apex[mask_apex]
    # Base circle
    if length is not None:
        inds_beyond_base = np.flatnonzero(d_a < -length) # subset of interest
        if len(inds_beyond_base):
```

```

c_base = apex - length * normal
r_base = length * np.tan(half_angle)
vec_to_base = points[inds_beyond_base] - c_base
dist_to_c_base = np.linalg.norm(vec_to_base, axis=1)
mask_base = (d_a[inds_beyond_base]
              < -length - dist_to_c_base * np.sin(half_angle))
if np.any(mask_base):
    vec_to_circle = vec_to_base[mask_base]
    dist_to_axis_base = np.linalg.norm(
        np.cross(normal, vec_to_circle),
        axis=1
    )
    dist_to_circle = np.sqrt(
        (vec_to_circle @ normal) ** 2
        + (dist_to_axis_base - r_base) ** 2
    )
    d_t[inds_beyond_base[mask_base]] = dist_to_circle

return d_t

```

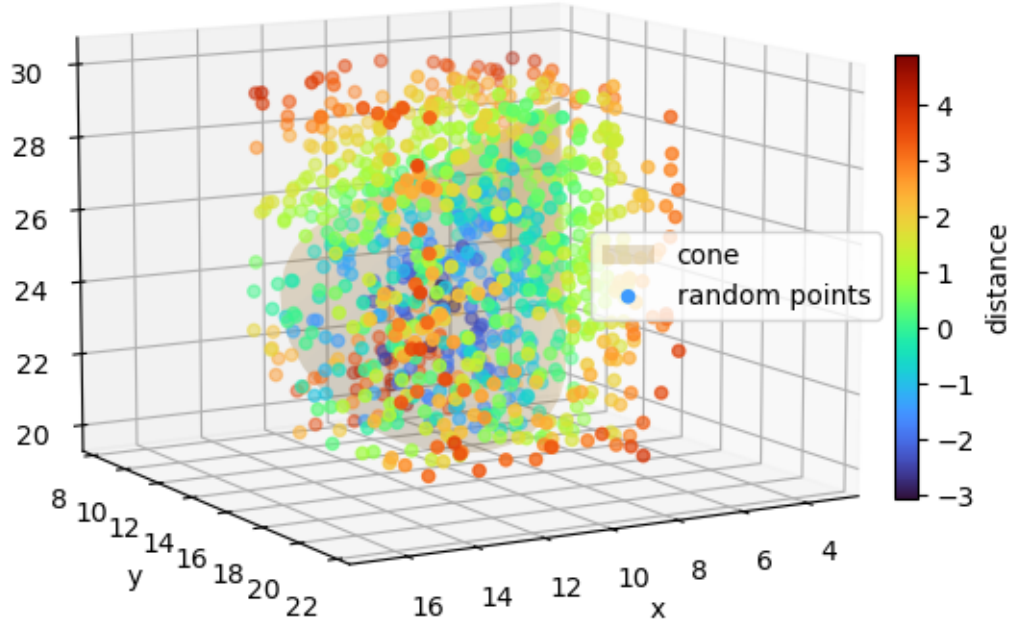
```

[23]: # Generate random points and compute their distance to the cone
random_points = np.random.uniform(
    points_cone.min(axis=0), points_cone.max(axis=0), (1000, 3)
)
dists_cone = dist_to_cone(random_points, *cone_parameters)

# Plot the distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cone_vertices[:, 0], cone_vertices[:, 1], cone_vertices[:, 2],
                triangles=cone_faces, color="orange", alpha=0.2, label="cone")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
               c=dists_cone, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label("distance")
ax.axis("equal")
ax.set_title("Distances to cone")
plt.show()

```

Distances to cone



Least squares cone

Unlike planes and spheres, cones do not admit a simple closed-form least-squares solution. The approach presented here is inspired by [35].

Using the Cartesian equation of the cone, a point p belongs to the cone if its squared distance to the apex c equals $(1 + \tan^2 \alpha)$ times the squared projection onto the cone axis of unit direction n , leading to the residual function:

$$f(c, n, \alpha) = \|p - c\|^2 - (1 + \tan^2 \alpha)[n \cdot (p - c)]^2$$

The unknowns of the associated least-squares problem are the cone apex c , axis direction n , and half-angle α .

These parameters can be estimated using a nonlinear least-squares optimizer such as Levenberg-Marquardt. In practice, the `least_squares` function from `scipy.optimize` makes this straightforward: only the distance function and an initial parameter estimate need to be provided.

The implementation below follows this approach. The cone axis is represented using spherical coordinates, reducing the number of optimized parameters from three to two. The point cloud is also centered on its centroid before optimization to improve numerical stability. The Levenberg-Marquardt algorithm (`method="lm"`) is used because it is efficient for small nonlinear least-squares problems. Since it does not support parameter bounds, non-physical parameter values may occasionally be produced and require post-processing.

```
[24]: def fit_cone(points, apex_guess, normal_guess, half_angle_guess):
    """Least squares cone."""

    # Center the pointcloud
    centroid = points.mean(axis=0)
    centered_points = points - centroid
    # Initial parameter estimate
    initial_parameters = np.empty(6)
    initial_parameters[:3] = apex_guess - centroid # points are already centered
    initial_parameters[3:5] = cartesian_to_spherical(normal_guess)
    initial_parameters[5] = half_angle_guess
    # Optimal cone parameters
    result = least_squares(
        lambda x : cone_residuals(x, centered_points),
        initial_parameters,
        method="lm", # Levenberg-Marquardt method
    )
    # Recover parameters
    parameters = result.x
    apex = parameters[:3] + centroid
    normal = spherical_to_cartesian(parameters[3:5])
    half_angle = parameters[-1]
    length = -np.min((points - apex) @ normal)

    return (apex, normal, half_angle, length), result.cost

def cone_residuals(parameters, points):
    """Function which computes the vector of residuals."""

    # Recover parameters
    apex = parameters[:3]
    normal = spherical_to_cartesian(parameters[3:5])
    half_angle = parameters[-1]
    # Squared distances to apex
    squared_distance = np.sum((points - apex)**2, axis=1)
    # Projection on axis
    proj_axis = (points - apex) @ normal

    return squared_distance - (1 + np.tan(half_angle)**2) * proj_axis**2
```


The quality of the initial guesses has a strong influence on convergence and the ability of the optimizer to avoid local minima.

A simple, application-independent approach consists in running the optimization from multiple starting points and keeping the solution with the lowest residual.

Alternatively, geometric information can be exploited. For example, *Principal Component Analysis* (PCA) may be used to estimate the cone axis. For sufficiently elongated cones, the first principal component often provides a good approximation of the axis direction. The apex can then be approximated from the extremal points along this axis, while the half-angle can be estimated from the relationship between the radial distance to the axis and the distance to the apex.

```
[25]: def cone_initial_guess(points):
    """PCA-based cone initialization."""

    # PCA
    centroid = points.mean(axis=0)
    centered_points = points - centroid
    cov = centered_points.T @ centered_points
    _, eigenvectors = np.linalg.eigh(cov)
    # Largest variance direction
    normal_guess = eigenvectors[:, -1]
    # Apex candidate
    projections = centered_points @ normal_guess
    apex_guess = points[np.argmax(projections)]
    # Ensure the axis points from base to apex
    projections = (points - apex_guess) @ normal_guess
    if projections.mean() > 0:
        normal_guess *= -1
        projections *= -1
    # Estimate half-angle
    radial_distances = np.linalg.norm(
        np.cross(normal_guess, points - apex_guess),
        axis=1,
    )
    mask = projections < -1e-6
    tangents = radial_distances[mask] / (-projections[mask])
    half_angle_guess = np.arctan(np.median(tangents))

    return apex_guess, normal_guess, half_angle_guess
```

```
[26]: # Initial parameter estimation with PCA
initial_parameters_cone = cone_initial_guess(points_cone)
# Fit cone
cone_fit, cone_residual = fit_cone(points_cone, *initial_parameters_cone)
# Display estimated parameters
with np.printoptions(precision=3, suppress=True):
    for name, val in zip(
```

```

        ["apex", "normal", "half-angle", "length"],
        cone_fit
    ):
        print(f"best-fit cone {name} is: {val}")
    print(f"residual is: {cone_residual}")

# Generate mesh for visualization
cone_fit_vertices, cone_fit_faces = generate_cone_mesh(*cone_fit)
# Show distance to best-fit
dists_cone_fit = dist_to_cone(points_cone, *cone_fit)

# Plot the fitted cone and distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_cone[:, 0], points_cone[:, 1], points_cone[:, 2],
               c=dists_cone_fit, cmap="turbo", label="initial points")
ax.plot_trisurf(cone_fit_vertices[:, 0], cone_fit_vertices[:, 1],
               cone_fit_vertices[:, 2], triangles=cone_fit_faces,
               color="white", alpha=0.2, label="best-fit cone")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label("distance")
ax.axis("equal")
ax.set_title("Distances to fitted cone")
plt.show()

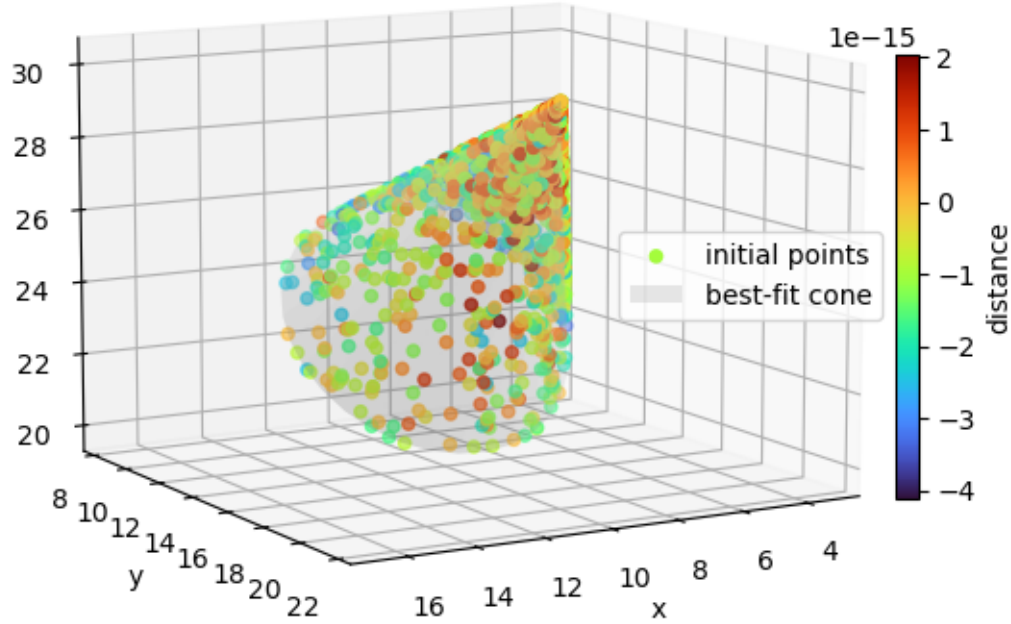
```

```

best-fit cone apex is: [10. 20. 30.]
best-fit cone normal is: [0.    0.707 0.707]
best-fit cone half-angle is: 0.3926990816987243
best-fit cone length is: 9.996239807250092
residual is: 7.996989906257051e-26

```

Distances to fitted cone



9.6 Torus

Torus parameters

A torus aligned with the z -axis can be represented by:

$$(\sqrt{(x - c_x)^2 + (y - c_y)^2} - R)^2 + (z - c_z)^2 = r^2$$

A torus of arbitrary orientation is considered here. It is defined by its center c , axis direction n , major radius R , and minor radius r . The major radius R is the distance between the torus center and the center of its tube, while the minor radius r is the radius of the tube itself.

Parameter	Description
c	center
n	unit vector defining the axis direction
R	major radius
r	minor radius

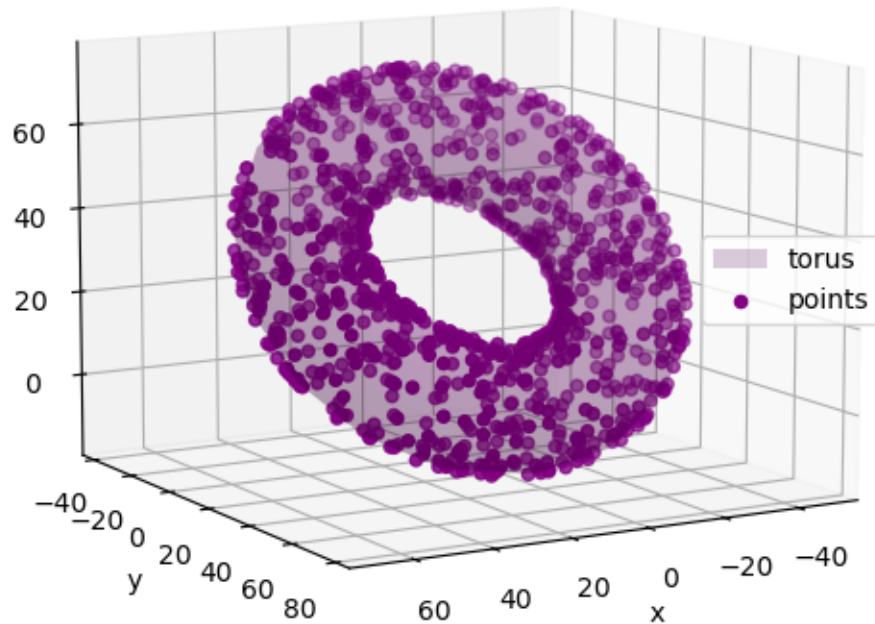
```
[27]: def generate_torus_points(center, normal, major_radius, minor_radius, n=1000):
        """Generate random points on a torus."""

        # Generate points on a torus aligned with the z-axis
        theta = np.random.uniform(0., 2*np.pi, (n, 1))
        phi = np.random.uniform(0., 2*np.pi, (n, 1))
        x = (major_radius+ minor_radius * np.cos(theta)) * np.cos(phi)
        y = (major_radius + minor_radius * np.cos(theta)) * np.sin(phi)
        z = minor_radius * np.sin(theta)
        points = np.hstack((x, y, z))
        # Rotate to the desired orientation
        R = rotation_f_to_t(np.array([0., 0., 1.]), normal)
        points = (R @ points.T).T
        # Translate to the desired position
        return points + center
```

```
[28]: # Define torus parameters
torus_parameters = (
    np.array([10., 20., 30.]), # center
    np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.]), # axis direction
    40., # major radius
    15. # minor radius
)
# Generate points
points_torus = generate_torus_points(*torus_parameters)
# Generate a mesh for visualization
torus_vertices, torus_faces = generate_torus_mesh(*torus_parameters)

# Plot the torus and points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(torus_vertices[:, 0], torus_vertices[:,1], torus_vertices[:,2],
                triangles=torus_faces, color="purple", alpha=0.2,label="torus")
ax.scatter(points_torus[:, 0], points_torus[:, 1], points_torus[:, 2],
           c="purple", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
ax.axis("equal")
ax.set_title("Generated torus and points")
plt.show()
```

Generated torus and points



Point-to-torus distance

The signed distance between a point p and a torus T is:

$$d(p, T) = \sqrt{[n \cdot (p - c)]^2 + (\|n \times (p - c)\| - R)^2} - r$$

The first term is the distance between the point and the circle of center c , radius R , and axis direction n .

Positive distances correspond to points outside the torus, while negative distances correspond to points inside it.

```
[29]: def dist_to_torus(points, center, normal, major_radius, minor_radius):
        """Signed distance to the torus."""

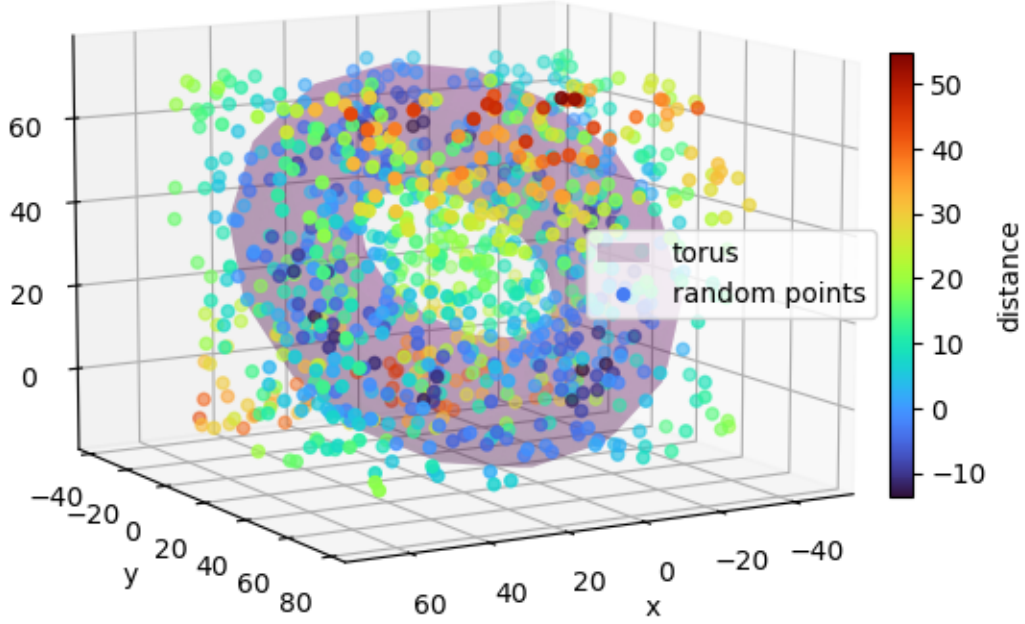
        vec_to_center = points - center
        d_a = vec_to_center @ normal # axial distance
        d_r = np.linalg.norm(np.cross(normal, vec_to_center), axis=1) # radial
```

```
return np.sqrt(d_a**2 + (d_r - major_radius)**2) - minor_radius
```

```
[30]: # Generate random points and compute their distance to the torus
random_points = np.random.uniform(
    points_torus.min(axis=0), points_torus.max(axis=0), (1000, 3)
)
dists_torus = dist_to_torus(random_points, *torus_parameters)

# Plot the distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(torus_vertices[:, 0], torus_vertices[:, 1], torus_vertices[:, 2],
                triangles=torus_faces, color="purple", alpha=0.2, label="torus")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
               c=dists_torus, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label("distance")
ax.axis("equal")
ax.set_title("Distances to torus")
plt.show()
```

Distances to torus



Least squares torus

Unlike planes and spheres, tori do not admit a simple closed-form least-squares solution. The approach presented here follows [36].

Using the squared and expanded Cartesian equation of the torus, the residual function is defined as:

$$f(c, n, R^2, r^2) = (||p - c||^2 + R^2 - r^2)^2 - 4R^2(||p - c||^2 - [n \cdot (p - c)]^2)$$

It is also possible to use the unsquared implicit equation, which leads to the alternative residual function used here:

$$f(c, n, R, r) = ||p - c||^2 + R^2 - r^2 - 2R\sqrt{||p - c||^2 - [n \cdot (p - c)]^2}$$

Either way, the unknowns are the torus center c , axis direction n , and the two torus radii.

These parameters can be estimated using a nonlinear least-squares optimizer such as Levenberg-Marquardt. In practice, the `least_squares` function from `scipy.optimize` only requires the residual function and an initial parameter estimate.

The torus axis is represented using spherical coordinates, reducing the number of optimized parameters from three to two. The pointcloud is also centered on its centroid before optimization to improve numerical stability. The Levenberg-Marquardt algorithm (`method="lm"`) is used because it is efficient for small nonlinear least-squares problems. Since it does not support parameter bounds, the fitted radii may occasionally take non-physical values and are therefore post-processed after optimization.

```
[31]: def fit_torus(points, center_guess, normal_guess, major_radius_guess,
                minor_radius_guess):
    """Least squares torus."""

    # Center the pointcloud
    centroid = points.mean(axis=0)
    centered_points = points - centroid
    # Initial parameter estimate
    initial_parameters = np.empty(7)
    initial_parameters[:3] = center_guess - centroid
    initial_parameters[3:5] = cartesian_to_spherical(normal_guess)
    initial_parameters[5] = major_radius_guess
    initial_parameters[6] = minor_radius_guess
    # Optimal parameters
    result = least_squares(
        lambda x : torus_residuals(x, centered_points),
        initial_parameters,
        method="lm", # Levenberg-Marquardt method
    )
    # Retrieve parameters
    parameters = result.x
    center = parameters[:3] + centroid
    normal = spherical_to_cartesian(parameters[3:5])
    major_radius = np.abs(parameters[5])
    minor_radius = np.abs(parameters[6])

    return (center, normal, major_radius, minor_radius), result.cost

def torus_residuals(parameters, points):
    """Function which computes the vector of residuals."""

    center = parameters[:3]
    normal = spherical_to_cartesian(parameters[3:5])
    major_radius = parameters[-2]
    minor_radius = parameters[-1]
    # squared distances to center
```



```

squared_distance = np.sum((points - center)**2, axis=1)
# projection on axis
proj_axis = (points - center) @ normal

return (squared_distance + major_radius**2 - minor_radius**2
        - 2 * major_radius * np.sqrt(squared_distance - proj_axis**2))

```

The quality of the initial guesses has a strong influence on convergence and robustness.

A generic strategy consists in testing multiple starting points and selecting the solution with the lowest residual.

Alternatively, geometric information can be exploited. For example, *Principal Component Analysis* (PCA) may be used to estimate the torus axis from the direction of smallest variance (under the hypothesis that the torus is relatively thin along its normal direction). The torus center can then be approximated by the centroid, while the major and minor radii may be estimated from the distribution of distances between the points and the estimated axis.

```

[32]: def torus_initial_guess(points):
        """PCA-based torus initialization."""

        # Center estimate
        center = points.mean(axis=0)
        # Axis estimate
        centered_points = points - center
        cov = centered_points.T @ centered_points
        _, eigenvectors = np.linalg.eigh(cov)
        # Smallest variance direction
        normal = eigenvectors[:, 0]
        # Distance to the estimated axis
        d_r = np.linalg.norm(
            np.cross(normal, centered_points),
            axis=1
        )
        # Radii estimates
        major_radius = d_r.mean()
        minor_radius = np.sqrt(np.mean((d_r - major_radius) ** 2))

        return center, normal, major_radius, minor_radius

```

```

[33]: # Initial parameter estimate
initial_parameters_torus = torus_initial_guess(points_torus)
# Fit the torus
torus_fit, torus_residual = fit_torus(points_torus, *initial_parameters_torus)
# Display estimated parameters
with np.printoptions(precision=3, suppress=True):
    for name, val in zip(
        ["center", "normal", "major_radius", "minor_radius"],

```

```

        torus_fit
    ):
        print(f"best-fit torus {name} is: {val}")
    print(f"residual is: {torus_residual}")

# Generate a mesh for visualization
torus_fit_vertices, torus_fit_faces = generate_torus_mesh(*torus_fit)
# Compute distances to the fitted torus
dists_torus_fit = dist_to_torus(points_torus, *torus_fit)

# Plot the fitted torus and distances
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_torus[:, 0], points_torus[:, 1], points_torus[:, 2],
               c=dists_torus_fit, cmap="turbo", label="initial points")
ax.plot_trisurf(torus_fit_vertices[:, 0], torus_fit_vertices[:, 1],
               torus_fit_vertices[:, 2], triangles=torus_fit_faces,
               color="white", alpha=0.2, label="best-fit torus")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5, pad=0)
cbar.set_label("distance")
ax.axis("equal")
ax.set_title("Distances to fitted torus")
plt.show()

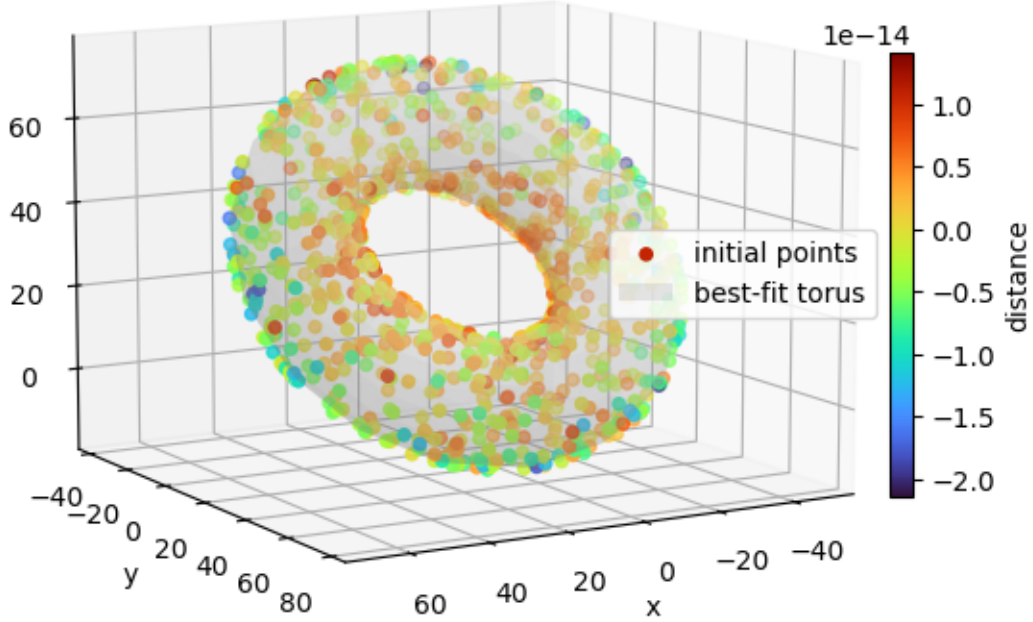
```

```

best-fit torus center is: [10. 20. 30.]
best-fit torus normal is: [0.    0.707 0.707]
best-fit torus major_radius is: 40.000000000000001
best-fit torus minor_radius is: 15.000000000000002
residual is: 7.894404971052958e-23

```

Distances to fitted torus



9.7 Validity of fitted primitives

Least-squares fitting always produces a solution, even when the chosen primitive is not a good representation of the points. It is therefore important to assess the validity of the fitted primitive.

A common approach consists in analyzing the **distribution of residuals**, which quantify the deviation between the points and the fitted primitive. Under the assumption that the points are independently affected by zero-mean Gaussian noise with standard deviation σ , the residual follows a χ^2 distribution with k degrees of freedom, where k is approximately equal to the number of points n for large pointclouds.

Based on this observation, a simple residual-based validity test [37] considers that a point is compatible with the fitted primitive if its squared residual satisfies:

$$d(p_i, S(\theta))^2 < 3\sigma^2$$

Assuming that a large majority of points should satisfy this criterion, it may be extended to a primitive-level test by considering the total residual. A fitted primitive can then be regarded as

valid if:

$$res(\theta) < 3n\sigma^2$$

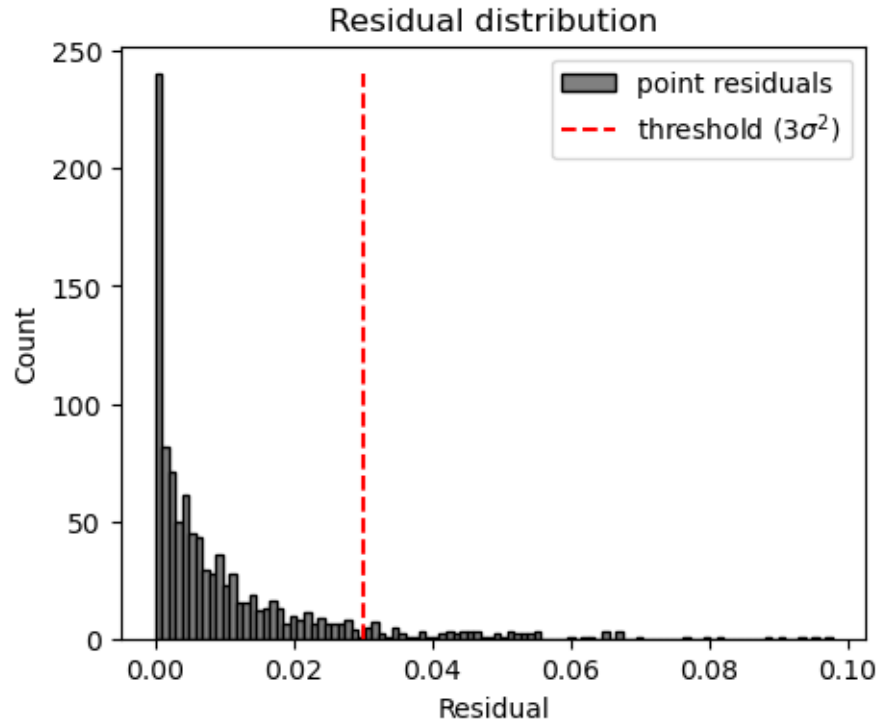
This provides a simple way to reject primitives whose residuals are inconsistent with the expected measurement noise.

Other validation strategies exist. When the Gaussian-noise assumption is questionable, residuals may be compared against a theoretical or empirical distribution using goodness-of-fit tests such as the Kolmogorov-Smirnov test. Another option is cross-validation, in which part of the points is withheld during fitting and later used for evaluation. Such methods can provide a more robust assessment of model quality and parameter stability, but generally require additional assumptions or incur a higher computational cost.

```
[34]: # Generate points on a plane
points_plane_noise = generate_plane_points(
    [0., 0., 0.], [0., 0., 1.], [0., 1., 0.], 50, 80, n=1000
)
# Add some Gaussian noise
sigma = 0.1
points_plane_noise += np.random.normal(0., sigma, points_plane_noise.shape)
# Fit plane to noisy pointcloud
plane_fit, plane_residual = fit_plane(points_plane_noise)
# Compute residual for each point
dists_best_fit_plane = dist_to_plane(points_plane_noise, *plane_fit)
residuals = dists_best_fit_plane**2
# Test that residuals are compatible with the noise level
threshold = 3 * len(points_plane_noise) * sigma**2
print(f"Total residual is {plane_residual:.1f} and validity threshold is {
    threshold:.1f}")

# Plot residuals distribution
fig, ax = plt.subplots(figsize=(5, 4))
n, *_ = ax.hist(residuals, bins=100, color="gray", edgecolor="black",
    label="point residuals")
ax.vlines(3*(sigma**2), 0, np.max(n), colors="red", linestyle="dashed",
    label="threshold ($3\\sigma^2$)")
ax.legend()
ax.set_xlabel("Residual")
ax.set_ylabel("Count")
ax.set_title("Residual distribution")
plt.show()
```

Total residual is 10.1 and validity threshold is 30.0



9.8 Wrapping up

This notebook introduced some of the most common geometric primitives used in 3D processing, namely planes, spheres, cylinders, cones, and tori. For each primitive, the associated parameters, distance function, and fitting algorithms were presented.

The least-squares approach was used throughout the notebook because of its simplicity and the availability of efficient numerical solvers. However, its performance depends on the quality of the data and may be affected by noise, outliers, or poor initialization.

Primitive fitting is a common way to simplify and organize pointclouds. By replacing many points with a small number of parametric surfaces, it produces a representation that is easier to store, analyze, and manipulate. In practice, primitive fitting is often only one step of a larger workflow, where the fitted primitives are used for tasks such as object modeling, reverse engineering, or scene understanding.

10 Registration

This notebook discusses the notion of registration, that is **the process of finding a spatial transformation that aligns two overlapping pointclouds**. The result may then be used for 3D reconstruction (i.e., merging pointclouds obtained by scanning the same object/scene from different points of view) or for 3D pose estimation (i.e., finding the relative transformation between objects).

The scope is here limited to rigid registration, yielding a rigid transformation (i.e., a rotation and a translation), but note that non-rigid registration is also a thing. Also, note that this problem differs from a mere change of reference frames (where the reference frames of both pointclouds are known).

```
[15]: # Necessary imports
import sys
import time
import numpy as np
from scipy.spatial import KDTree
import matplotlib.pyplot as plt

sys.path.append("./utils")
from manip_utils import rotation_zyx

# Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
normals = data[:, 3:6]
```

Just as in the case of fitting a model to data, the registration process comes down to minimizing a certain quantity materializing the distance between the two considered pointclouds. As often found in literature, **the pointcloud to be aligned is called the source** (“moving” pointcloud) and **the reference pointcloud is called the target** (“fixed” pointcloud). In this notebook, the source pointcloud is noted $P = \{p_0, p_1, \dots, p_{n-1}\}$ and the target pointcloud is noted $Q = \{q_0, q_1, \dots, q_{m-1}\}$.

This notion of distance to be minimized may take multiple forms, as we are about to see below.

10.1 Registration between corresponding sets

In this section, we assume the correspondence of points between pointclouds P and Q . This means that P and Q have the same number of points ($n = m$) and that (p_i, q_i) is a pair of corresponding points $\forall i \in [0, n[$.

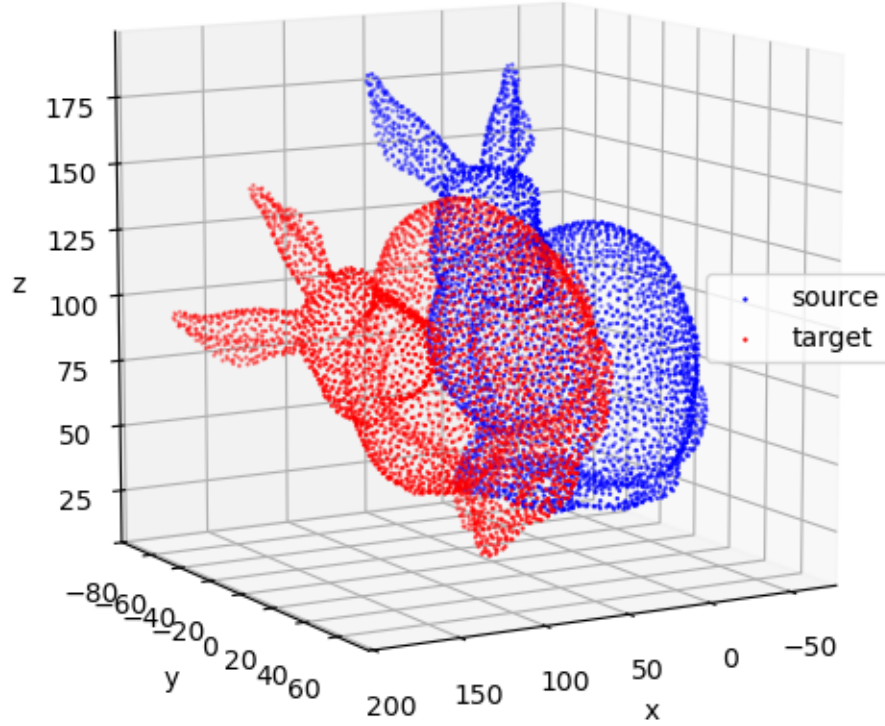
This configuration is for example found when artificial targets (e.g., adhesive reflective dots, spheres, or other artefacts) are used during the 3D acquisition process, providing a set of corresponding points between views. Correspondences may also be established using remarkable points present in both pointclouds.

For the sake of the example, the same pointcloud is used for the source and the target (it is assumed that `R_true` and `t_true` below are not known).

```
[16]: # Apply a known transformation to create a target pointcloud
R_true = rotation_zyx(theta_x=np.pi/5, theta_y=np.pi/6, theta_z=np.pi/7)
T_true = np.array([10., 20., 30.])
points_transformed = (R_true @ points.T).T + T_true
normals_transformed = (R_true @ normals.T).T

# Visualize source and target pointclouds
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', s=.5, label="source")
ax.scatter(points_transformed[:, 0], points_transformed[:, 1],
           points_transformed[:, 2], c='red', s=.5, label="target")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_box_aspect([1, 1, 1])
ax.set_title("Source and target pointclouds")
plt.show()
```

Source and target pointclouds



10.1.1 Point-to-point

The most straightforward approach consists in minimizing the sum of squared distances between pairs (i.e., each source point and its corresponding target point). This so called **point-to-point error metric**, may be written as follow:

$$(R_{opt}, t_{opt}) = \arg \min_{R, t} \sum_{i=0}^{n-1} \omega_i \|R \cdot p_i + t - q_i\|^2$$

Here ω_i represents the weight associated with each pair (which is equal to $1/n$ in case that all pairs have the same weight).

Fortunately, there is a direct method for calculating optimal parameters without resorting to traditional algorithms used to solve non-linear least squares problems.

The steps to computing the optimal rigid transformation are summarized as follow: 1. Compute the weighted centroids of both pointclouds

$$\bar{p} = \frac{\sum_{i=0}^{n-1} \omega_i p_i}{\sum_{i=0}^{n-1} \omega_i} \quad \bar{q} = \frac{\sum_{i=0}^{n-1} \omega_i q_i}{\sum_{i=0}^{n-1} \omega_i}$$

2. Center both pointclouds around their centroids

$$X = \{x_0, x_1, \dots, x_{n-1}\}, \quad x_i = p_i - \bar{p}$$

$$Y = \{y_0, y_1, \dots, y_{n-1}\}, \quad y_i = q_i - \bar{q}$$

3. Compute the cross-covariance matrix of the centered pointclouds

$$S = XY^T \quad \text{with} \quad W = \text{diag}(\omega_0, \omega_1, \dots, \omega_{n-1})$$

4. Compute the singular value decomposition and deduce the optimal rotation

$$R = VU^T \quad \text{with} \quad S = U\Sigma V^T$$

5. Deduce the optimal translation

$$t = \bar{q} - R\bar{p}$$

More details can be found in [38].

```
[17]: def point_to_point_minimization(p, q, w):
    """Compute the best fitting rigid transformation (p to q) that minimizes
    the sum of squared distances between two sets of corresponding points"""

    # Compute the weighted centroids of both pointclouds
    p_bar = (w @ p)/w.sum()
    q_bar = (w @ q)/w.sum()
    # Compute the centered pointclouds
    x = p - p_bar
    y = q - q_bar
    # Cross covariance matrix
    S = x.T @ (w[:, None] * y)
    # Singular Value Decomposition
    U, _, V = np.linalg.svd(S)
    # Optimal rotation
    R = (U @ V).T
    # Symmetry test
    if np.linalg.det(R) < 0.0:
        V[2, :] *= -1
        R = (U @ V).T
    # Optimal translation
    T = q_bar - (R @ p_bar)
```

```

    return R, T

def point_to_point_error(p, q, w):
    """Compute the Root Mean Squared Error (RMS) for point-to-point distance"""

    se = ((w[:, None] * (p - q))**2).sum(axis=1)
    mse = se.mean()

    return np.sqrt(mse)

```

```

[18]: # Initial situation
weights = np.ones(len(points)) # use constant weights
initial_error = point_to_point_error(points, points_transformed, weights)
print("Initial error of", initial_error)

# Compute and apply the optimal transformation
R_point, T_point = point_to_point_minimization(points, points_transformed,
                                                weights)
points_transformed_calc = (R_point @ points.T).T + T_point
final_error = point_to_point_error(points_transformed_calc, points_transformed,
                                    weights)
print("Final error of", final_error)

# Check if similar to ground truth
print("Check if rotations R_true and R_point are the same:",
      np.allclose(R_true, R_point))
print("Check if translations T_true and T_point are the same:",
      np.allclose(T_true, T_point))

```

Initial error of 76.39164942230356

Final error of 3.9909595861801535e-14

Check if rotations R_true and R_point are the same: True

Check if translations T_true and T_point are the same: True

10.1.2 Point to plane

Another popular approach consists in minimizing the sum of squared distances between each source point and the tangent plane at its corresponding target point. This so called **point-to-plane error metric** may be written as follow:

$$(R_{opt}, t_{opt}) = \arg \min_{R, t} \sum_{i=0}^{n-1} \omega_i [(R \cdot p_i + t - q_i) \cdot n_{q,i}]^2$$

ω_i is, again, the weight associated with each pair and $n_{q,i}$ is the unit normal vector associated to q_i .

Finding the optimal transformation (R_{opt}, t_{opt}) requires to solve a non-linear least squares optimization problem. While doable using minimization techniques, such as the Levenberg-Marquardt algorithm, it is in practice quite computationally costly (moreover if it must be repeated multiple

times, but we'll get to it in the next section). Instead, an often-found solution consists in using a linear approximation, which is more easily solved.

Starting with the expression of the rotation matrix (extrinsic rotation z-y-x)

$$R = \begin{pmatrix} \cos \theta_y \cos \theta_z & \sin \theta_x \sin \theta_y \cos \theta_z - \cos \theta_x \sin \theta_z & \cos \theta_x \sin \theta_y \cos \theta_z + \sin \theta_x \sin \theta_z \\ \cos \theta_y \sin \theta_z & \sin \theta_x \sin \theta_y \sin \theta_z + \cos \theta_x \cos \theta_z & \cos \theta_x \sin \theta_y \sin \theta_z - \sin \theta_x \cos \theta_z \\ -\sin \theta_y & \sin \theta_x \cos \theta_y & \cos \theta_x \cos \theta_y \end{pmatrix}$$

for small rotations ($\theta \approx 0$ so $\cos \theta \approx 1$ and $\sin \theta \approx \theta$) this matrix can be approximated by

$$R_{linear} = \begin{pmatrix} 1 & \theta_z & \theta_y \\ \theta_z & 1 & \theta_x \\ -\theta_y & \theta_x & 1 \end{pmatrix}$$

Thanks to this approximation, this problem can be formulated as a linear least squares problem $Ax = b$ with $x = [\theta_x, \theta_y, \theta_z, t_x, t_y, t_z]$, which requires far less computational resources.

Of course, the goodness of the result depends on the validity of the small-rotations approximation.

More details can be found in [39].

```
[19]: def point_to_plane_minimization(p, q, n_q, w):
    """Compute the best fitting rigid transformation (p to q) that minimizes
    the sum of squared distances between a data set of points and the tangent
    plane at its corresponding set of points in ref (linearized form)"""

    # Apply weight
    n_w = w[:, None] * n_q
    # Solving A.X = b (linearized form of the equation)
    A = np.hstack((np.cross(p, n_w), n_w))
    b = np.sum(n_w*(q-p), axis=1)
    x = np.linalg.pinv(A) @ b
    # Unpack result
    theta_x, theta_y, theta_z, t_x, t_y, t_z = x
    R = rotation_zyx(theta_x, theta_y, theta_z)
    T = np.array([t_x, t_y, t_z])

    return R, T

def point_to_plane_error(p, q, n_q, w):
    """Compute the Root Mean Squared Error (RMS) for point-to-plane distance"""

    se = (w[:, None] * (n_q * (p - q)**2).sum(axis=1)

    return se.mean()

[20]: # Initial situation (point-to-plane)
weights = np.ones(len(points))
```

```

initial_error = point_to_plane_error(points, points_transformed,
                                     normals_transformed, weights)
print("Initial error of", initial_error)

# Compute and apply the optimal transformation (point-to-plane)
R_plane, T_plane = point_to_plane_minimization(points, points_transformed,
                                              normals_transformed, weights)
points_transformed_calc = (R_point @ points.T).T + T_point
final_error = point_to_plane_error(points_transformed_calc, points_transformed,
                                   normals_transformed, weights)
print("Final error of", final_error)

# Check if similar to ground truth (comparing apples and oranges here)
print("Check if rotations R_true and R_plane are the same:",
      np.allclose(R_true, R_plane))
print("Check if translations T_true and T_plane are the same:",
      np.allclose(T_true, T_plane))

```

```

Initial error of 1814.0808148093574
Final error of 4.964768616561959e-28
Check if rotations R_true and R_plane are the same: False
Check if translations T_true and T_plane are the same: False

```

10.2 Registration between non-corresponding sets

In this section, we do not assume the correspondence between pointclouds P and Q .

This configuration is for example found when the position of the acquisition device is not tracked or that no artificial targets are placed on the object of interest.

For the sake of the example, our initial pointcloud is split into two halves, one for the source and one for the target (it is assumed that R_true and t_true below are not known).

10.2.1 The Iterative Closest Point (ICP) Algorithm

The Iterative Closest Point algorithm or ICP is unquestionably the most popular algorithm when it comes to pointcloud registration. The algorithm was first described (almost simultaneously, with a few variations) in [40] and [41].

The basic algorithm is conceptually simple. Starting from two pointclouds, it iteratively:

1. Generates pairs of corresponding points between the source and the target pointclouds
2. Calculates the optimal transformation between these pairs
3. Applies this optimal transformation to the source pointcloud

Having already seen the last two steps of the algorithm (see previous sections), let's focus on the generation of corresponding pairs.

As described in the name of the algorithm, the corresponding point of a given point of the source pointcloud is its closest point (i.e., nearest neighbor) in the target pointcloud. This search process

is repeated several times, meaning that pairs of corresponding points may change from one iteration to the next one.

In practice, the number of iterations is often a parameter of the algorithm, as seen below.

```
[21]: def register_simple_icp(source, target, max_iter):

    # Initialization
    kdtree = KDTree(target)
    Rt = [np.eye(3)]
    Tt = [np.zeros(3)]
    err = []

    for i in range(max_iter):
        # 1. Generate pairs
        _, match = kdtree.query(source)
        target_match = target[match]
        # 2. Compute the optimal transformation
        weights = np.ones(len(source))
        R, T = point_to_point_minimization(source, target_match, weights)
        # 3. Apply this transformation
        source = (R @ source.T).T + T
        rms = point_to_point_error(source, target_match, weights)

        # Store the result
        Rt.append(R @ Rt[i])
        Tt.append(R @ Tt[i] + T)
        err.append(rms)

    return Rt, Tt, err
```

```
[29]: # Perform ICP registration
Rt, Tt, err = register_simple_icp(points, points_transformed, 25)
points_sub_icp_1 = (Rt[1] @ points.T).T + Tt[1]
points_sub_icp = (Rt[-1] @ points.T).T + Tt[-1]

# Visualize source, intermediate and final aligned pointclouds
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='skyblue', label="source")
ax.scatter(points_sub_icp_1[:, 0], points_sub_icp_1[:, 1],
           points_sub_icp_1[:, 2], c='palegreen', label="icp iter #1")
ax.scatter(points_sub_icp[:, 0], points_sub_icp[:, 1], points_sub_icp[:, 2],
           c='gold', label="icp final")
ax.scatter(points_transformed[:, 0], points_transformed[:, 1],
           points_transformed[:, 2], c='coral', label="target")
ax.legend(loc="right")
```

```

ax.view_init(10, 60)
ax.set_box_aspect([1, 1, 1])
ax.set_axis_off()
ax.set_title("ICP registration progress")
plt.show()

```

ICP registration progress



It is then possible to track the error metric at each iteration to assess whether the algorithm is converging. Of course, the closest the final error value to zero the better but note that it would not likely fall down to zero if the source and target pointclouds are not strictly identical.

```

[23]: print("Check if rotations R_true and R_icp are the same:",
        np.allclose(R_true, Rt[-1]))
print("Check if translations T_true and T_icp are the same:",
        np.allclose(T_true, Tt[-1]))

fig = plt.figure(figsize=(5, 4))
ax = fig.add_subplot()

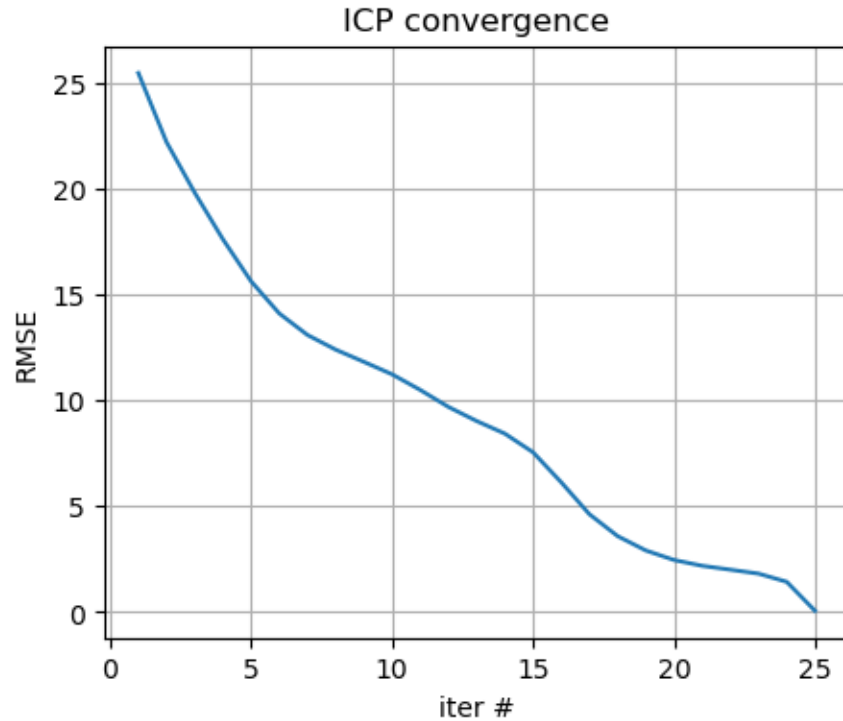
```

```

ax.plot(range(1, len(err)+1), err)
ax.set(xlabel='iter #', ylabel='RMSE',
       xticks=range(0, len(err)+1, 5))
ax.grid()
ax.set_title("ICP convergence")
plt.show()

```

Check if rotations R_{true} and R_{icp} are the same: False
 Check if translations T_{true} and T_{icp} are the same: False



While three main phases are generally distinguished in the basic ICP algorithm (generation of pairs, estimation of the optimal transformation, and its application to the source pointcloud), some more advanced variants have been introduced over time. Several tentative have been made in an effort to summarize them [inproceedings], introducing a few additional steps to categorize the different variants of the ICP algorithm. These steps are:

1. Selection of points
2. Matching points
3. Weighting of pairs
4. Rejecting pairs
5. Minimization of error metric

More details can be found in the paper, which also discusses the performances of the different variants depending on various considerations.

For the example, the following piece of code implement some of the most found strategies:

- Random selection of the source points

```
[24]: def register_icp(source, target, max_iter):

    # Initialization
    kdtree = KDTree(target)
    Rt = [np.eye(3)]
    Tt = [np.zeros(3)]
    err = []

    n_samples = int(len(source)*0.5)

    for i in range(max_iter):

        # 1. selection of points
        random_source_inds = np.random.choice(n_samples, replace=False)
        source_sampled = source[random_source_inds]
        # 2. matching of points
        _, match = kdtree.query(source_sampled)
        target_match = target[match]
        # 3. weighting of pairs
        weights = np.ones(n_samples)
        # 4. rejecting pairs
        # 5. minimization of error metric
        R, T = point_to_point_minimization(source_sampled, target_match, weights)
        # Apply transform
        source = (R @ source.T).T + T # apply to the whole initial pointcloud

        rms = point_to_point_error(source_sampled, target_match, weights)

        # Store the result
        Rt.append(R @ Rt[i])
        Tt.append(R @ Tt[i] + T)
        err.append(rms)

    return Rt, Tt, err
```

While some strategies may improve the behavior of the ICP algorithm, its capacity to return the best transformation (i.e., the rotation matrix and the translation vector that minimize the distance between the two pointclouds) is not always guaranteed in practice. Indeed, the results of the ICP algorithm and its variants are known for being very sensitive to the initial alignment of the source and target pointclouds.

Said differently, if the rotation required to align both pointclouds is too large, the ICP algorithm often fails to find it. It “gets stuck” to a local optimum instead of converging to the global optimum, as illustrated below.


```

[25]: # Flip source
Rf_z = np.array([
    [1., 0., 0.],
    [0., 1., 0.],
    [0., 0., -1.]
])
points_flipped = (Rf_z @ points.T).T
# Perform ICP registration
Rt, Tt, err = register_simple_icp(points_flipped, points_transformed, 20)
points_flipped_icp = (Rt[-1] @ points_flipped.T).T + Tt[-1]

# Visualize source (flipped), final aligned and target pointclouds
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points_flipped[:, 0], points_flipped[:, 1],
           points_flipped[:, 2], c='turquoise', label="source (flipped)")
ax.scatter(points_flipped_icp[:, 0], points_flipped_icp[:, 1],
           points_flipped_icp[:, 2], c='tan', label="icp final")
ax.scatter(points_transformed[:, 0], points_transformed[:, 1],
           points_transformed[:, 2], c='coral', label="target")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_box_aspect([1, 1, 1])
ax.set_axis_off()
ax.set_title("ICP registration with flipped source")
plt.show()

```

ICP registration with flipped source



This is why the ICP algorithm and its variants are often classified as local registration techniques. A rough estimate of the transformation between the views is often necessary for the algorithm to converge correctly.

This estimation can be carried out manually or by means of other algorithms.

10.2.2 Global registration and Principal Axis Alignment

Unlike local registration techniques, global registration techniques do not rely on any initial alignment or hypothesis on the pose of the source and the target pointclouds. Their main quality typically lies in their robustness to initial alignment conditions or even to some other parameters such as noise or the percentage of overlap between the source and the target pointclouds.

Plenty of global registration algorithms have been proposed in the last decades. One of the easiest to understand relies on Principal Component Analysis (or PCA). It simply aims to align the principal directions of both pointclouds. Although not very robust to noise and requiring a good amount of overlap between the two pointclouds, this is one of the fastest techniques available and often a good first go-to algorithm. Furthermore, its steps are also quite similar to those of the algorithms discussed in the previous sections of this notebook.

The steps of this PCA-based global registration technique are summarized as follow:

1. Compute the centroids of both pointclouds

$$\bar{p} = \frac{1}{n} \sum_{i=0}^{n-1} p_i \quad \bar{q} = \frac{1}{m} \sum_{i=0}^{m-1} q_i$$

2. Center both pointclouds around their centroids

$$M = \{m_0, m_1, \dots, m_{n-1}\}, \quad m_i = p_i - \bar{p}$$

$$N = \{n_0, n_1, \dots, n_{m-1}\}, \quad n_i = q_i - \bar{q}$$

3. Compute the singular value decomposition of the centered pointclouds and deduce the optimal rotation

$$R = V_N^T V_M \quad \text{with} \quad M = U_M \Sigma_M V_M^T \quad \text{and} \quad N = U_N \Sigma_N V_N^T$$

4. Deduce the optimal translation

$$t = \bar{q} - R\bar{p}$$

As you may see, this is quite similar to the point-to-point error minimization technique presented earlier.

```
[26]: def register_pca(p, q):
    """Compute the rigid transformation (p to q) that aligns the principal
    axis of the pointclouds"""

    # Compute centroids of both pointclouds
    p_bar = p.mean(axis=0)
    q_bar = q.mean(axis=0)
    # Compute the centered pointclouds
    M = p - p_bar
    N = q - q_bar
    # Singular Value Decomposition
    U_M, s_M, V_M = np.linalg.svd(M)
    U_N, s_N, V_N = np.linalg.svd(N)
    # Rotation matrix
    R = V_N.T @ V_M
    # Choice of the good symmetries
    candidates = []
    # x, y, and z
    for i in range(3):
        V_M1 = np.array(V_M)
        V_M1[i, :] *= -1
        candidates.append(V_N.T @ V_M1)
```

```

# xyz
candidates.append(-R)
# xy, xz, and yz
for i in range(3):
    V_M1 = np.array(V_M)
    V_M1[i, :] *= -1
    V_M1[i-1, :] *= -1
    candidates.append(V_N.T @ V_M1)
# and 0
candidates.append(R)
# The best candidate is the one that obtains the best bounding box matches
N_min = N.min(axis=0)
N_max = N.max(axis=0)
mismatch = np.inf
for R in candidates:
    RM = R @ M.T
    RM_min = RM.min(axis=1)
    RM_max = RM.max(axis=1)
    d = np.linalg.norm(RM_min - N_min) + np.linalg.norm(RM_max - N_max)
    if d < mismatch:
        R_opt = R
        mismatch = d
# Translation vector
T_opt = q_bar - R_opt @ p_bar

return R_opt, T_opt

```

In the example below, the PCA-based global registration technique aligns the two pointclouds almost perfectly.

```

[27]: # PCA-based registration
R_pca, T_pca = register_pca(points_flipped, points_transformed)
points_sub_pca = (R_pca @ points_flipped.T).T + T_pca

# Visualize source (flipped), pca aligned and target pointclouds
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points_flipped[:, 0], points_flipped[:, 1],
           points_flipped[:, 2], c='skyblue', label="source")
ax.scatter(points_sub_pca[:, 0], points_sub_pca[:, 1], points_sub_pca[:, 2],
           c='orchid', label="pca")
ax.scatter(points_transformed[:, 0], points_transformed[:, 1],
           points_transformed[:, 2], c='coral', label="target")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_box_aspect([1, 1, 1])
ax.set_axis_off()

```

```
ax.set_title("PCA-based registration with flipped source")  
plt.show()
```

PCA-based registration with flipped source



10.3 Wrapping up

You should now have a better grasp of 3D point cloud registration, which aims to estimate the transformation that aligns two overlapping point clouds. This problem is often found in computer vision and robotics in the context of scene/object reconstruction, localization/mapping, and object pose estimation.

Registration algorithms are commonly classified into local and global categories. The first one includes the well-known ICP algorithm and its variants. Such algorithms return the best transformation parameters when both point clouds are initially already quite-well aligned but are likely to fail to converge otherwise. On the other hand, global registration algorithms are robust to the initial alignment conditions but provide a less precise result. For this reason, global and local algorithms are often used sequentially during the registration process.

11 Machine learning

This notebook introduces some of the most commonly used machine learning techniques for processing 3D pointclouds. Unlike purely geometric approaches, these methods leverage both geometric structure and **statistical properties** of the data while remaining **interpretable and efficient**.

Machine learning algorithms often outperform classical methods for tasks such as **outlier detection** and **object segmentation**, and also extend to more advanced applications like **semantic segmentation**.

```
[1]: # Necessary imports
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

from sklearn.utils import resample
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
from sklearn.neighbors import KDTree, kneighbors_graph, LocalOutlierFactor
from sklearn.cluster import MiniBatchKMeans, DBSCAN, AgglomerativeClustering
from sklearn.ensemble import RandomForestClassifier
```

11.1 Cleaning

As a reminder, the goal of cleaning is to reduce both the number of outliers and the level of noise affecting a pointcloud, as these can impact subsequent processing steps (see the notebook on cleaning).

From a machine learning perspective, outlier detection can be framed as a subtask of **unsupervised learning**, often referred to as **anomaly detection**, where the goal is to find points that are “different” from the rest of the data, without using labeled examples.

Although there is a lot of work on *anomaly detection*, this section focuses on the **Local Outlier Factor (LOF)**, which extends the neighborhood-based principles of traditional techniques in a more adaptive way.

```
[2]: # Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
normals = data[:, 9:12]

# Add outliers to the pointcloud
n_outliers = 100
outliers = np.random.uniform(
    points.min(axis=0),
    points.max(axis=0),
    (n_outliers, 3)
)
points_outliers = np.vstack((points, outliers))
# Create a mask to identify the outliers in the combined pointcloud
```

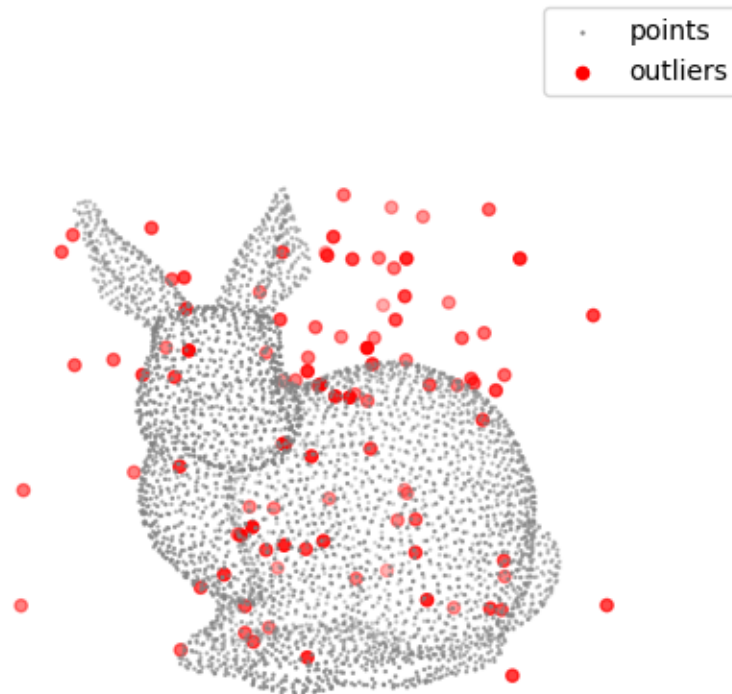
```

outliers_mask_true = np.hstack((np.zeros(points.shape[0], dtype=bool),
                                np.ones(outliers.shape[0], dtype=bool)))

# Visualize the pointcloud with outliers
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers[:, 0], outliers[:, 1], outliers[:, 2],
           c="red", label="outliers")
ax.set_box_aspect([1, 1, 1])
ax.view_init(10, 60)
ax.set_axis_off()
ax.legend()
ax.set_title("pointcloud with generated outliers")
plt.show()

```

pointcloud with generated outliers



The most commonly used traditional outlier detection techniques, namely *Statistical Outlier Re-*

moval (SOR) and *Radius Outlier Removal* (ROR), rely on the assumption of a globally consistent point density, using fixed thresholds based on distances or neighborhood sizes to identify outliers. As a result, they often struggle when the density varies across the pointcloud, since a single global criterion cannot adapt to heterogeneous regions. A natural improvement is therefore to account for local variations in the data distribution rather than applying a uniform rule.

This is precisely the idea behind the **Local Outlier Factor (LOF)**, which adopts an **unsupervised learning approach** by estimating the local density of each point and comparing it to that of its neighbors [42]. By relying on relative density instead of absolute thresholds, LOF is able to automatically adapt to spatial heterogeneity and detect more subtle anomalies in complex pointclouds.

LOF can be applied directly to point coordinates or extended to higher-dimensional feature spaces by incorporating additional attributes such as intensity, color, normals, or eigenvalue-based descriptors. This is an additional advantage over more traditional techniques that rely solely on geometric Euclidean distances. However, careful feature selection and proper data normalization are essential to ensure meaningful results.

LOF computation for given point p_i follows these steps:

1. Query the k -nearest neighbors of p_i and form the set N_i .
2. Compute the k -distance of each neighbor $p_j \in N_i$, which corresponds to the distance between p_j and its k -th nearest neighbor:

$$k\text{-dist}(p_j)$$

3. Define the reachability distance between p_i and each neighbor p_j :

$$\text{reach-dist}_k(p_i, p_j) = \max(k\text{-dist}(p_j), d(p_i, p_j))$$

4. Estimate the local reachability density (LRD) of p_i :

$$LRD_k(p_i) = \left(\frac{1}{|N_i|} \sum_{p_j \in N_i} \text{reach-dist}_k(p_i, p_j) \right)^{-1}$$

5. Compute the Local Outlier Factor (LOF) of p_i by comparing its density to that of its neighbors

$$LOF_k(p_i) = \frac{1}{|N_i|} \sum_{p_j \in N_i} \frac{LRD_k(p_j)}{LRD_k(p_i)}$$

As a result, LOF assigns a score to each point measuring how isolated it is with respect to its local neighborhood. Values close to 1 indicate inliers, while significantly larger values indicate potential outliers.

LOF is conceptually straightforward and can be directly implemented using Python main scientific libraries. The `scikit-learn` library also provides an out-of-the box implementation called `LocalOutlierFactor`, which is used in the example below.

```
[3]: # Detect outliers using Local Outlier Factor (LOF)
      clf_lof = LocalOutlierFactor(n_neighbors=10)
```



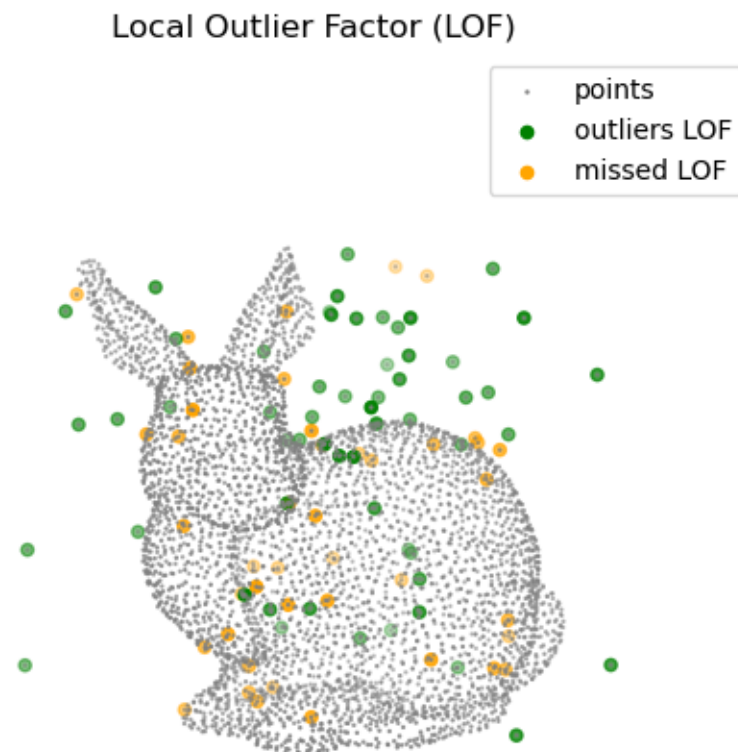
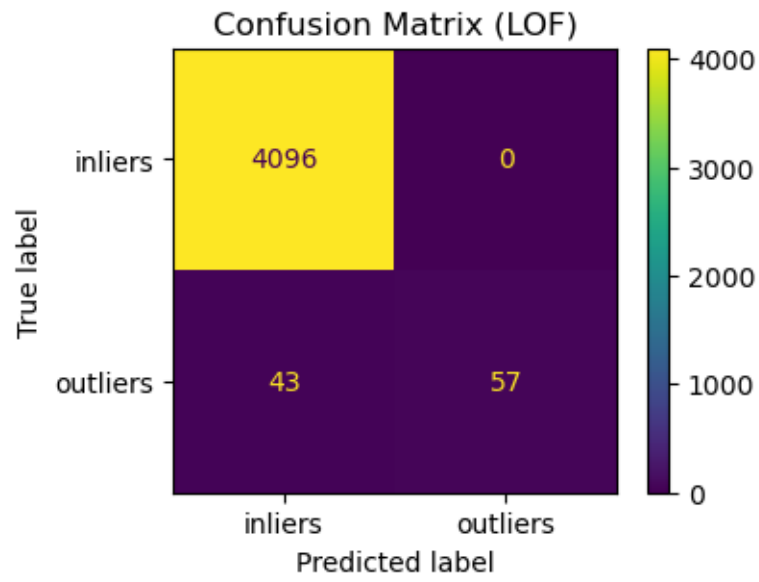
```

y_pred = clf_lof.fit_predict(points_outliers)
outlier_mask_lof = y_pred == -1
outliers_lof = points_outliers[outlier_mask_lof]
missed_mask_lof = np.logical_xor(outlier_mask_lof, outliers_mask_true)
missed_lof = points_outliers[missed_mask_lof]

# Evaluate the performance of LOF using a confusion matrix
fig, ax = plt.subplots(figsize=(4, 3))
cmd_lof = ConfusionMatrixDisplay.from_predictions(
    outliers_mask_true, outlier_mask_lof,
    display_labels=["inliers", "outliers"],
    ax=ax
)
ax.set_title("Confusion Matrix (LOF)")
plt.show()

# Visualize the inliers and outliers
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_outliers[:, 0], points_outliers[:, 1], points_outliers[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers_lof[:, 0], outliers_lof[:, 1], outliers_lof[:, 2],
           c="green", label="outliers LOF")
ax.scatter(missed_lof[:, 0], missed_lof[:, 1], missed_lof[:, 2],
           c="orange", label="missed LOF")
ax.set_box_aspect([1, 1, 1])
ax.view_init(10, 60)
ax.set_axis_off()
ax.legend()
ax.set_title("Local Outlier Factor (LOF)")
plt.show()

```



11.2 Object Segmentation

As a reminder, object segmentation in 3D pointclouds aims to group points that belong to the same physical object. In practice, each object corresponds to a set of nearby points and segmentation seeks to identify and separate these sets.

This goal is closely aligned with **clustering**, a core concept in **unsupervised learning**, which consists of partition data into groups based on similarity without relying on labeled examples. As a result, clustering algorithms provide a natural and flexible framework for object segmentation.

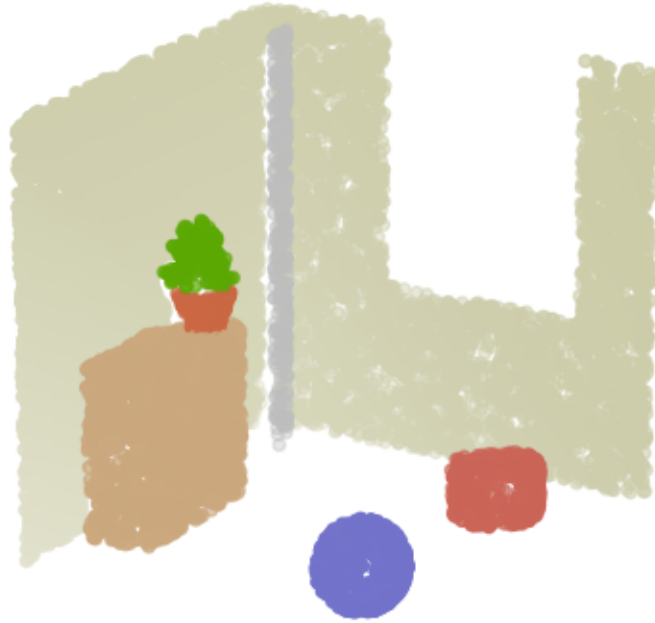
The following subsections explore how classical clustering methods such as KMeans, DBSCAN, and Agglomerative Clustering can be applied to segment 3D pointclouds.

```
[4]: # Load synthetic indoor scene pointcloud
data = np.loadtxt("./data/indoor_scene.xyz")
points, colors, normals = data[:, :3], data[:, 6:9], data[:, 9:12]

# Remove points belonging to the ground plane (z = 0)
ground_mask = points[:, 2] < 0.05
points_no_ground = points[~ground_mask]
colors_no_ground = colors[~ground_mask]
normals_no_ground = normals[~ground_mask]

# Visualize the original pointcloud and the points without the ground plane
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_no_ground[:, 0], points_no_ground[:, 1], points_no_ground[:, 2],
           c=colors_no_ground)
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("pointcloud without ground plane")
plt.show()
```

pointcloud without ground plane



11.2.1 K-means

KMeans is one of the most widely used clustering algorithms and is sometimes considered for object segmentation in 3D point clouds. It **partitions the data into a fixed number of clusters by minimizing the distance between each point and its assigned cluster centroid**.

Because KMeans tends to produce compact and roughly spherical clusters, it is rarely applied to raw spatial coordinates alone. Instead, it is typically used with enriched feature representations, combining position with attributes such as color or intensity. In practice, careful feature selection and proper normalization are essential to ensure stable and meaningful results.

However, the reliance on centroid distance introduces several important limitations. Nearby objects may be grouped into a single cluster, and the method implicitly **assumes clusters of similar size and convex shape**. It does not enforce spatial continuity and struggles with varying point densities, which are common in real-world data. In addition, the number of clusters must be specified *a priori*, although heuristic methods such as the *elbow curve* can help estimate a suitable value *a posteriori*.

These limitations make KMeans not much used in practice for complex scenes. The example below

even underlines its limitation on our rather simple synthetic indoor scene. Note that the implementation below uses `MiniBatchKMeans`, which is a variant of the `KMeans` algorithm which uses mini-batches to reduce the computation time. A `StandardScaler` is applied to normalize the features.

```
[5]: # Stack points and colors into a feature matrix X
X_kmeans = np.hstack((points_no_ground, colors_no_ground))
# Standardize features
X_kmeans = StandardScaler().fit_transform(X_kmeans)
# KMeans clustering with a chosen number of clusters
labels_kmeans = MiniBatchKMeans(n_clusters=7, n_init=10).fit_predict(X_kmeans)

# Visualize KMeans clustering results
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points_no_ground[:, 0], points_no_ground[:, 1],
                    ↪points_no_ground[:, 2],
                        c=labels_kmeans, cmap="tab10", s=0.5)
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("KMeans Clustering")
ax.legend(*scatter.legend_elements(), title="clusters", loc="center left")
plt.show()
```

```
c:\Users\frup00090363\AppData\Local\miniconda3\envs\sci\Lib\site-
packages\threadpoolctl.py:1226: RuntimeWarning:
```

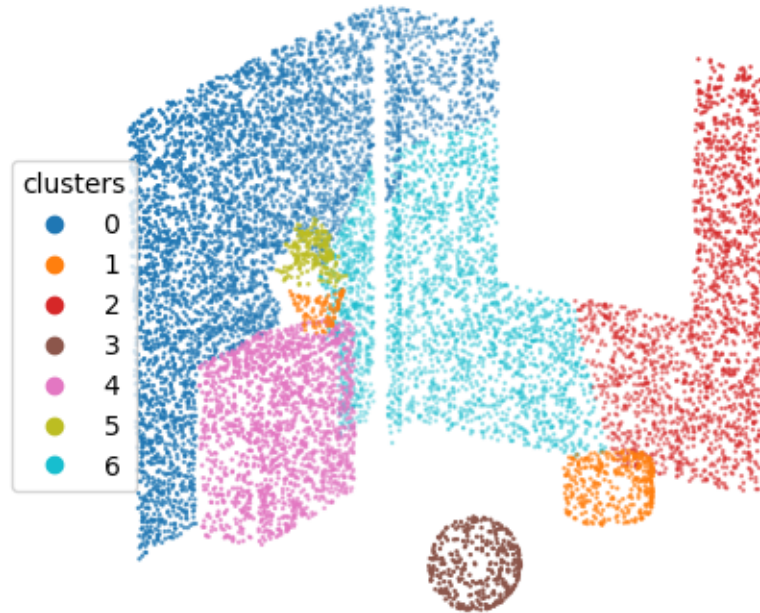
```
Found Intel OpenMP ('libiomp') and LLVM OpenMP ('libomp') loaded at
the same time. Both libraries are known to be incompatible and this
can cause random crashes or deadlocks on Linux when loaded in the
same Python program.
```

```
Using threadpoolctl may cause crashes or deadlocks. For more
information and possible workarounds, please see
```

```
https://github.com/joblib/threadpoolctl/blob/master/multiple\_openmp.md
```

```
warnings.warn(msg, RuntimeWarning)
```

KMeans Clustering



11.2.2 DBSCAN

DBSCAN is a density-based clustering algorithm that is particularly well-suited for 3D pointclouds. It groups points based on spatial proximity and local density, using a distance threshold to connect neighboring points. Unlike KMeans, DBSCAN does not require specifying the number of clusters in advance and can naturally detect outliers as noise, making it robust in cluttered scenes.

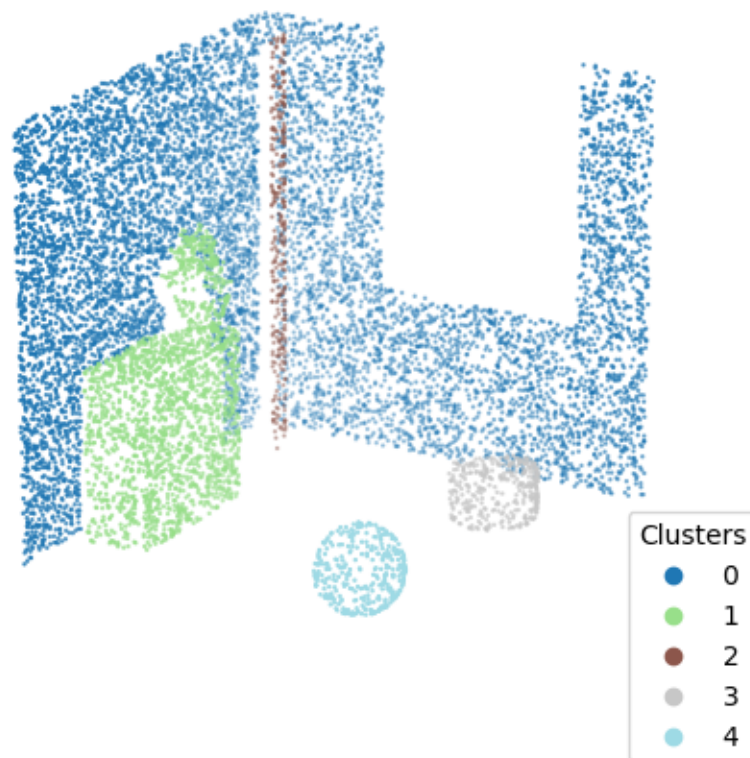
Additional features such as color or normals can help better separate objects, but they also increase the feature space dimension. In high dimensions, distances become less meaningful and neighborhoods more sparse, which can degrade performance. To address this, it is common to either weight features carefully or decouple the problem (for example, first clustering in XYZ space, then refining clusters using additional features).

The implementation below uses the DBSCAN algorithm from `scikit-learn`.

```
[6]: # DBSCAN clustering to segment the scene
labels_dbscan = DBSCAN(eps=100., min_samples=20).fit_predict(points_no_ground)
```

```
# Visualize the clustered pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points_no_ground[:, 0], points_no_ground[:, 1],
                    ↪points_no_ground[:, 2],
                        c=labels_dbscan, cmap="tab20", s=0.5)
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("DBSCAN clustering of the indoor scene")
ax.legend(*scatter.legend_elements(), title="Clusters")
plt.show()
```

DBSCAN clustering of the indoor scene



11.2.3 Agglomerative Clustering

Agglomerative Clustering is a hierarchical clustering method that progressively builds clusters by merging nearby points or groups of points. Starting from individual points, it iteratively combines

the closest clusters according to a chosen linkage criterion, producing a hierarchy that can be cut at different levels to obtain a segmentation.

A key advantage for pointcloud segmentation is the ability to incorporate a connectivity matrix, which restricts merges to spatially neighboring points and helps preserve local structure. This makes the method well-suited for refining structured regions such as walls, roofs, or indoor partitions. However, its computational cost is high, which makes it more appropriate as a refinement step rather than a first clustering approach. Agglomerative clustering is also sometimes used in surface segmentation and simplification [16], following the same strategy of merging groups of points based on similarity.

The implementation below uses the `AgglomerativeClustering` algorithm from `scikit-learn`. A `kneighbors_graph` is used to define connectivity and a `StandardScaler` is applied to normalize the features.

```
[ ]: # Select a cluster of interest from the DBSCAN results
object_mask = labels_dbscan == 1
points_object = points_no_ground[object_mask]
points_object -= points_object.mean(axis=0) # center the object
normals_object = normals_no_ground[object_mask]
colors_object = colors_no_ground[object_mask]

# Stack features for Agglomerative Clustering (only normals and colors)
X_agglo = np.hstack((normals_object, colors_object))
X_agglo = StandardScaler().fit_transform(X_agglo)
# Compute the connectivity matrix from the coordinates of the points
connectivity = kneighbors_graph(points_object, n_neighbors=10,
                                mode='connectivity', include_self=False)
# Perform Agglomerative Clustering with connectivity constraints
labels_agglo = AgglomerativeClustering(
    n_clusters=5,
    connectivity=connectivity,
    linkage="ward"
).fit_predict(X_agglo)

# Visualize Agglomerative Clustering results
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points_object[:, 0], points_object[:, 1], points_object[:, 2],
                    c=labels_agglo, cmap="tab10")
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("Agglomerative Clustering of selected object")
ax.legend(*scatter.legend_elements(), title="clusters", loc="center left")
plt.axis("equal")
plt.show()
```


Agglomerative Clustering of selected object



11.3 Semantic Segmentation

In machine learning, semantic segmentation is a task within **supervised learning**, where a model is trained on labeled data to assign a class to each individual element of an input. Applied to 3D point clouds, **semantic segmentation consists of predicting a semantic label for every point** (e.g., *car*, *tree*, *road*). This enables the interpretation of complex 3D scenes, including urban and natural environments as well as indoor spaces, using high-level semantic information.

In a “traditional” machine learning framework (excluding deep learning), semantic segmentation is typically structured into four key steps:

1. Definition of **neighborhoods** around each point to capture its local context
2. Computation of **descriptors** that describe the geometry of these neighborhoods
3. Selection and tuning of a **classifier** to assign a semantic label based on these descriptors
4. Contextual **regularization** to enforce spatial consistency of the predictions (Optional)

The design of efficient neighborhoods, discriminative descriptors, suitable classifiers, and spatial regularization strategies for semantic segmentation has been extensively explored in the literature

[43, 44, 24, 45]. The following paragraphs try to summarize key practical insights derived from existing works.

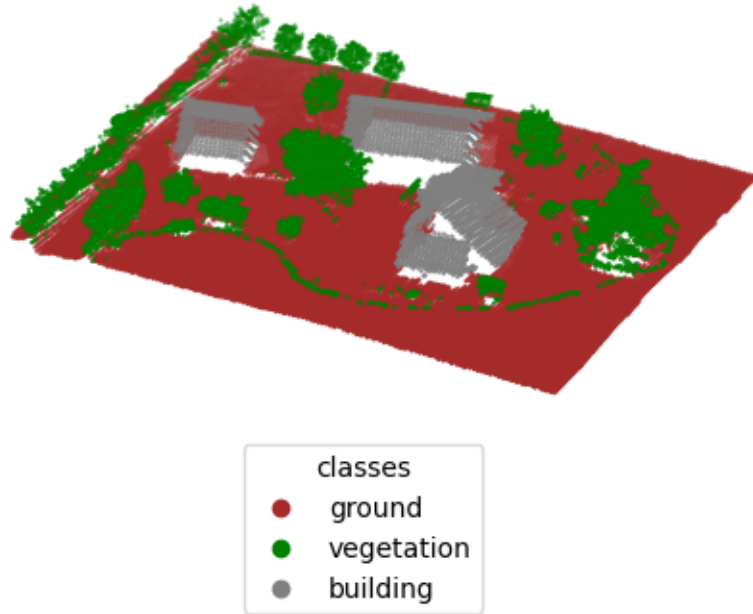
The example below comes from the IGN LiDAR HD dataset, a dense airborne LiDAR dataset providing a detailed 3D representation of landscapes in France, with an average density of about 10 points per square meter [46]. The original pointcloud is classified into 11 categories, including eight standard ASPRS classes and three custom classes. For clarity and to reduce complexity, the point cloud is simplified to three classes (*ground*, *vegetation*, and *building*) by merging vegetation subclasses (*low*, *medium*, and *high vegetation*) and excluding less relevant categories such as *water* and *not classified*.

```
[8]: # Load LHD training data
data = np.loadtxt("./data/LHD_train.xyz", dtype=np.float32)
points, intensity, y_train = data[:, :3], data[:, 3], data[:, 4]
class_names = {0: "ground", 1: "vegetation", 2: "building"}

# Spatial structure
kdtree = KDTree(points)

# Visualize the pointcloud colored by class labels
lidar_cmap = matplotlib.colors.ListedColormap(["brown", "green", "gray"])
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=y_train, cmap=lidar_cmap, s=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Train pointcloud")
# Create legend with class names
legend_handles = []
legend_labels = []
for cls in class_names.keys():
    color = lidar_cmap(cls)
    legend_handles.append(plt.Line2D([0], [0], marker='o', linestyle='',
                                     color=color, markersize=6))
    legend_labels.append(class_names[int(cls)])
ax.legend(legend_handles, legend_labels, title="classes", loc="lower center")
plt.show()
```

Train pointcloud



11.3.1 Neighborhoods

The first step of the semantic segmentation pipeline is the **definition of neighborhoods**, which directly impacts the scale of the analysis.

Neighborhoods are usually characterized by their geometric **nature** (spatial support) and their **scale** (spatial extent).

Common neighborhood types include:

- **k -neighborhoods**, defined by the k closest points. They naturally adapt to varying point densities, but their spatial extent varies across the scene.
- **Spherical neighborhoods**, consisting of all points within a 3D sphere of a given radius around a point. They provide a fixed spatial extent, but the number of contained points varies with local density.
- **Cylindrical neighborhoods**, consisting of points whose projection falls within a circle on a reference plane (e.g., the ground plane). They are well suited to scenes with a dominant vertical direction, but may connect points that are very far apart in elevation.

- **Voxel-based neighborhoods**, defined by grouping points within the same voxel. They enable efficient spatial indexing and processing, but remain sensitive to density variations and voxel size selection.

The neighborhood scale can be selected using different strategies:

- **Fixed-scale neighborhoods**, using a constant radius or a fixed number of neighbors. They are simple to implement, but may not adapt well to varying densities or object sizes.
- **Adaptive single-scale neighborhoods**, where the neighborhood size is selected independently for each point based on local geometric criteria. They better capture local geometry, but require additional computation.
- **Multi-scale neighborhoods**, where features are computed at several predefined neighborhood sizes. They capture geometric information at different levels of detail, but increase both computational cost and feature dimensionality.
- **Adaptive multi-scale neighborhoods**, where multiple scales are combined through data-driven selection or weighting. They offer greater flexibility and robustness, but are more complex to design and compute.

One popular adaptive single-scale strategy determines the “optimal” neighborhood size by evaluating the eigentropy E_λ (computed from the normalized eigenvalues of the local covariance matrix, see notebook about descriptors) over a range of scales $k \in [k_{\min}, k_{\max}]$. The selected size is the one minimizing E_λ , corresponding to the local geometry with the lowest uncertainty according to a Shannon entropy-based criterion.

Although an adaptive single-scale strategy leads to locally “optimal” neighborhoods, a single scale rarely captures all relevant geometric information present in complex scenes. In practice, **multi-scale neighborhoods remain the most effective and popular strategy**, because they capture geometry at different levels of detail. A common baseline is to **use 3 to 6 scales with a logarithmic progression**, small scales capturing local geometry, medium scales describing object parts, and larger scales providing full object context.

In the case of airborne LiDAR pointclouds such as the one presented above, k-neighborhoods are generally preferred since they better handle density variations and result in more stable features. For a density of approximately 10 pts/m² (average point spacing around 30 cm), neighborhood sizes of $k = [10, 20, 50]$ typically capture geometric information ranging from local surface details to object-level context. These values provide a reasonable initial configuration but may be refined later based on classification results.

```
[9]: # k values used for multiscale neighborhoods
k_values_multiscale = [10, 20, 50]
```

11.3.2 Descriptors

The second key step of the semantic segmentation pipeline is the **computation of descriptors** or features, which characterize the local geometry and attributes of each point based on its neighborhood.

Common types of features include:

- **Eigenvalue-based features** (e.g., linearity, planarity, scattering), capturing the local 3D shape.
- **Height-based features** (e.g., height above ground, height range), describing the vertical position of points.
- **Density-based features** (e.g., point density, spacing), characterizing the local point distribution.
- **Angle-based features** (e.g., verticality, normal variation), describing surface orientation and smoothness.
- **Histogram-based features** (e.g., FPFH, SHOT), encoding distributions of geometric relationships within the neighborhood.
- **Radiometric features** (e.g., intensity and intensity statistics), linked to the reflectance properties of the scanned materials (when available from sensor).
- **RGB features** (e.g., colors, color gradients and statistics), incorporating spectral information (when available from sensor).

Additionally, some of these descriptors can also be computed from a 2D projection (e.g. onto the ground plane), providing complementary information in structured environments where horizontal organization is meaningful.

In practice, **eigenvalue-based, height-based and density-based features remain among the most widely used descriptors** for semantic segmentation, as they provide a good compromise between discriminative power, robustness and computational efficiency. More sophisticated descriptors such as FPFH or SHOT may capture richer geometric patterns but are generally more computationally expensive.

These implementations below compute the k nearest neighbors for all points at once and use vectorized operations whenever possible instead of explicit loops. While improving execution speed, it is also memory-intensive and switching to batch processing is advised for large pointclouds. Height, density and angle-based features are grouped into the “geometric features” category. Separate functions are used for eigenvalue-based features and geometric features for clarity and readability, although grouping them would reduce redundant neighborhood computations. The `indices` parameter enables descriptor computation on a subset of points only (useful for adaptive neighborhoods for example).

```
[10]: def compute_3D_eigen_features(points, neighbor_finder, norm_eigenvalues=True,
                                     indices=None):
    """Compute normals and principal eigenvalue-based features."""

    # Neighbors for all query points
    query_points = points if indices is None else points[indices]
    inds = neighbor_finder(query_points) # (N, k)
    k = inds.shape[-1]
    neighbors = points[inds]
    # Eigenvalues from covariance matrix
    neighbors -= neighbors.mean(axis=1, keepdims=True) # center in-place
    cov = (neighbors.transpose(0, 2, 1) @ neighbors) / k # (N, 3, 3)
```

```

eigenvalues, eigenvectors = np.linalg.eigh(cov) # (N, 3), (N, 3, 3)
eigenvalues = np.clip(eigenvalues, 1e-10, None) # fix numerical negatives
# Normals
normals = eigenvectors[:, :, 0]
# Compute sum before normalization (equal to 1. if normalized)
eigensum = eigenvalues.sum(axis=1)
# Normalize eigenvalues if required
if norm_eigenvalues:
    eigenvalues /= eigenvalues.sum(axis=1, keepdims=True)
# Compute features based on eigenvalues
lambda_1 = eigenvalues[:, 2]
lambda_2 = eigenvalues[:, 1]
lambda_3 = eigenvalues[:, 0]
linearity = (lambda_1 - lambda_2) / lambda_1
planarity = (lambda_2 - lambda_3) / lambda_1
scattering = lambda_3 / lambda_1
omnivariance = (lambda_1 * lambda_2 * lambda_3) ** (1/3)
anisotropy = (lambda_1 - lambda_3) / lambda_1
eigentropy = -np.sum(eigenvalues * np.log(eigenvalues), axis=1)
curvature = lambda_3 / (lambda_1 + lambda_2 + lambda_3)

return (normals, linearity, planarity, scattering, omnivariance,
        anisotropy, eigentropy, eigensum, curvature)

```

The 3D eigenvalue-based features have already been introduced in the previous notebook on descriptors. One important thing to remember is that the eigensum should not be normalized. If so, its value becomes the same (equal to 1) for every point, which is not useful for training or prediction.

Normals are computed at the same time and at different scales. Another option is to compute normals separately as a preprocessing step, using a different (adaptive) scale, as shown in the notebook about normals and curvatures. For aerial data, normals can be easily oriented using the sensor position and direction, so they point upward (toward z^+).

```

[12]: # Compute normals and 3D eigenvalue-based features
normals_multiscale = []
eigenfeatures_3D_multiscale = []
eigenfeatures_3D_names = []

for k in k_values_multiscale:
    neighbor_finder = lambda points: kdtree.query(points,
                                                    k,
                                                    return_distance=False)
    eigenfeatures_3D_k = compute_3D_eigen_features(points, neighbor_finder)
    # Remove normals from eigenfeatures list (computed at same time)
    normals_k = eigenfeatures_3D_k[0]
    eigenfeatures_3D_k = eigenfeatures_3D_k[1:]
    # Orient normals consistently towards +z direction (acquisition direction)
    mask_n = normals_k[:, 2] < 0

```

```

normals_k[mask_n] = -normals_k[mask_n]
# Store results
normals_multiscale.append(normals_k)
eigenfeatures_3D_multiscale.append(eigenfeatures_3D_k)
eigenfeatures_3D_names.extend([
    f"linearity_{k}", f"planarity_{k}", f"scattering_{k}",
    f"omnivariance_{k}",
    f"anisotropy_{k}", f"eigentropy_{k}", f"eigensum_{k}", f"curvature_{k}"
])

# Concatenate all multiscale features in one array of shape (N, 8*k)
eigenfeatures_3D_multiscale = np.stack(
    [arr for tup in eigenfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

Contrary to eigenvalue-based features, geometric features capture the global or contextual position of each point in the scene, rather than the local shape of its neighborhood. The tables below list some of the most used height-based, density-based and angle-based features.

Height-based features assume a preferred direction for height, often z . Features are computed withing each neighborhood but also relatively to the ground thanks to a *Digital Terrain Model* (DTM), which represents the bare-earth surface. A DTM is typically created as a preprocessing step by extracting ground points from the data and building a surface that represents the terrain height (not covered here). Height-based features typically help to capture elevated objects such as buildings or vegetation.

Feature	Formula	Intuitive meaning
Normalized height	$z - z_{ground}$	Height relative to ground (need DTM)
Height variation	$std(z)$	Height variation within neighborhood
Height range	$z_{max} - z_{min}$	Height range within neighborhood

Density-based features describe how points are distributed in space. They provide information about how crowded or sparse a local region is, which can help differentiate between solid structures and more irregular or scattered objects.

Feature	Formula	Intuitive meaning
Density	$k/\frac{4}{3}\pi r^3$	Local density
Radius (knn distance)	$r = d_{knn}$	Size of the local neighborhood
Mean neighbor distance	$1/k \sum d_i$	Average spacing between nearby points

Angle-based features rely on the orientation of the local surface, typically derived from normal vectors. They capture how much local surface varies in orientation, making it easier to identify horizontal/vertical or highly irregular structures.

Feature	Formula	Intuitive meaning
Verticality	$1 - \ n_z\ $	How “upright” the local surface is
Angular variation	$std(n)$	Variation of surface orientation

```
[13]: def compute_3D_geometric_features(points, normals, neighbor_finder,
                                         indices=None):
    """Compute 3D features"""

    # Neighbors for all query points
    query_points = points if indices is None else points[indices]
    query_normals = normals if indices is None else normals[indices]
    dists, inds = neighbor_finder(query_points) # (N, k)
    k = inds.shape[-1]
    # Density
    radius = dists[:, -1]
    density = k / ((4/3) * np.pi * radius**3)
    mean_dists = np.mean(dists, axis=1)
    # Height
    knn_z_coords = points[inds, 2] # (N, k)
    d_z = knn_z_coords.max(axis=1) - knn_z_coords.min(axis=1)
    std_z = knn_z_coords.std(axis=1)
    # Angle (normals are supposed to be oriented, otherwise use abs)
    verticality = 1 - np.abs(query_normals[:, 2])
    normals_dot = np.sum(query_normals[:, None, :] * normals[inds], axis=2)
    normals_dot = np.clip(normals_dot, -1.0, 1.0) # (N, k)
    angular_variation = np.mean(1 - normals_dot, axis=1)

    return radius, density, mean_dists, d_z, std_z, verticality, ↵
    ↪angular_variation
```

```
[14]: # 3D geometric features
geomfeatures_3D_multiscale = []
geomfeatures_3D_names = []

for i, k in enumerate(k_values_multiscale):
    neighbor_finder = lambda points: kdtree.query(points,
                                                    k,
                                                    return_distance=True,
                                                    sort_results=True)

    geomfeatures_3D_k = compute_3D_geometric_features(points,
                                                         normals_multiscale[i],
                                                         neighbor_finder)

    # Store results
    geomfeatures_3D_multiscale.append(geomfeatures_3D_k)
    geomfeatures_3D_names.extend([
```



```

        f"radius_{k}", f"density_{k}", f"mean_dists_{k}", f"d_z_{k}",
        f"std_z_{k}", f"verticality_{k}", f"angular_variation_{k}"
    ])

# Concatenate all multiscale features in one array of shape (N, 7*k)
geomfeatures_3D_multiscale = np.stack(
    [arr for tup in geomfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

Finally, eigenvalue-based and geometric **features computed at multiple scales are concatenated into a single feature vector** (note that intensity is also included here). This feature vector is typically high-dimensional, with up to 150 features being common in practice.

However, not all descriptors are equally informative, and **feature selection is often used to identify the most relevant features** and remove redundant ones. This is a standard machine learning step and is not specific to point cloud semantic segmentation. Feature selection methods can be broadly divided into **filter-based** and **embedded** approaches. Filter methods assess feature relevance independently of any classifier, either by evaluating each feature individually (*univariate* methods) or by accounting for dependencies between features (*multivariate* methods). Embedded methods estimate feature importance directly during classifier training.

In practice, feature selection is often left to the classifier, and explicit selection is mainly used to improve performance, reduce dimensionality, or increase interpretability.

```

[15]: # All available features
X_train = np.concatenate(
    [eigenfeatures_3D_multiscale, geomfeatures_3D_multiscale, intensity[:,
    ↪None]],
    axis=1
)
X_names = np.array(eigenfeatures_3D_names + geomfeatures_3D_names +
    ↪["intensity"])

print("Shape of X_train:", X_train.shape)
print("Feature names:", X_names)

```

Shape of X_train: (49056, 46)

```

Feature names: ['linearity_10' 'planarity_10' 'scattering_10' 'omnivariance_10'
'anisotropy_10' 'eigentropy_10' 'eigensum_10' 'curvature_10'
'linearity_20' 'planarity_20' 'scattering_20' 'omnivariance_20'
'anisotropy_20' 'eigentropy_20' 'eigensum_20' 'curvature_20'
'linearity_50' 'planarity_50' 'scattering_50' 'omnivariance_50'
'anisotropy_50' 'eigentropy_50' 'eigensum_50' 'curvature_50' 'radius_10'
'density_10' 'mean_dists_10' 'd_z_10' 'std_z_10' 'verticality_10'
'angular_variation_10' 'radius_20' 'density_20' 'mean_dists_20' 'd_z_20'
'std_z_20' 'verticality_20' 'angular_variation_20' 'radius_50'
'density_50' 'mean_dists_50' 'd_z_50' 'std_z_50' 'verticality_50'
'angular_variation_50' 'intensity']

```

Before training, it is good practice to **check for class imbalance**. In airborne LiDAR datasets, the ground class often contains many more samples than classes corresponding to smaller objects. As a rule of thumb, if the largest class contains more than 5 to 10 times as many samples as the smallest one, class rebalancing is usually recommended. Otherwise, the classifier may become biased toward the dominant classes.

A simple and widely used solution is **random undersampling**, which consists of keeping the same number of samples for each class in the training set. This **creates a balanced dataset and is often a good first choice when the training set is large**.

However, undersampling removes many samples from dominant classes and may therefore discard useful information. Alternatives include assigning larger weights to minority classes or increasing their number of samples through data augmentation. These approaches are generally preferred when preserving all available samples is important.

The example below illustrates how random undersampling can be applied in practice. Since the considered dataset is only moderately imbalanced and not particularly large, undersampling is shown for demonstration purposes only, and all available samples are ultimately retained for training.

```
[16]: def random_undersample_balance(X_train, y_train, n_samples_per_class=None,
                                     random_state=42):
    """Perform random undersampling to balance classes."""

    classes = np.unique(y_train)
    # Determine target number of samples per class
    if n_samples_per_class is None:
        class_counts = [np.sum(y_train==c) for c in classes]
        n_samples_per_class = min(class_counts)
    # Sample each class to the target number of samples
    X_list = []
    y_list = []
    for c in classes:
        X_c = X_train[y_train==c]
        y_c = y_train[y_train==c]
        # Random undersampling
        X_resampled, y_resampled = resample(X_c, y_c, replace=False,
                                             n_samples=n_samples_per_class, random_state=random_state)
        X_list.append(X_resampled)
        y_list.append(y_resampled)
    # Concatenate all classes
    X_balanced = np.vstack(X_list)
    y_balanced = np.concatenate(y_list)
    # Mix classes
    perm = np.random.RandomState(random_state).permutation(len(y_balanced))
    X_balanced = X_balanced[perm]
    y_balanced = y_balanced[perm]

    return X_balanced, y_balanced
```

```
[17]: # Balance dataset
X_balanced, y_balanced = random_undersample_balance(X_train, y_train)

print("Original shape:", X_train.shape)
print("Balanced shape:", X_balanced.shape)
print("Class distribution:", {class_names[int(c)]: int(np.sum(y_balanced==c))
                             for c in np.unique(y_balanced)})
```

Original shape: (49056, 46)

Balanced shape: (18726, 46)

Class distribution: {'ground': 6242, 'vegetation': 6242, 'building': 6242}

11.3.3 Classifiers

The third key step of the semantic segmentation pipeline is the **selection and tuning of a classifier**

The list of possible classifiers is large, ranging from simple *k-Nearest Neighbors* (kNN) to more sophisticated *Support Vector Machines* (SVM). In practice, **Random Forest (RF)** remains one of the most widely used baseline classifiers, offering a good balance between accuracy, robustness, computational efficiency, and ease of use. RF requires little parameter tuning, does not require feature normalization, naturally provides feature importance measures, and can be efficiently parallelized.

More importantly, **the impact of the classifier is often secondary compared to the quality of the input features**. A common mistake is to spend too much effort tuning the classifier while neglecting feature engineering.

The implementation below uses `scikit-learn` implementation of `RandomForestClassifier` with default mostly settings. A forest of 100 trees already provides good performance in most cases, although larger forests (typically 200–500 trees) may improve prediction stability. Class imbalance is handled through `class_weight="balanced"`, and training is parallelized using `n_jobs=-1`.

```
[18]: # Train a Random Forest classifier on all samples and features
rf_clf = RandomForestClassifier(n_estimators=100, n_jobs=-1, random_state=42,
                               class_weight="balanced")
rf_clf.fit(X_train, y_train)

# Evaluate the classifier on the training set
y_pred_train_all = rf_clf.predict(X_train)
print("Classification report on training set (all samples all features):")
print(classification_report(y_train, y_pred_train_all,
                            target_names=class_names.values()))
```

Classification report on training set (all samples all features):

	precision	recall	f1-score	support
ground	1.00	1.00	1.00	33428
vegetation	0.99	1.00	0.99	9386
building	1.00	1.00	1.00	6242

accuracy			1.00	49056
macro avg	1.00	1.00	1.00	49056
weighted avg	1.00	1.00	1.00	49056

The model achieves near-perfect performance on the training set when using all available features. This likely indicates some degree of overfitting and reflects the simplicity of the classification problem (three well-separated classes) rather than true generalization performance.

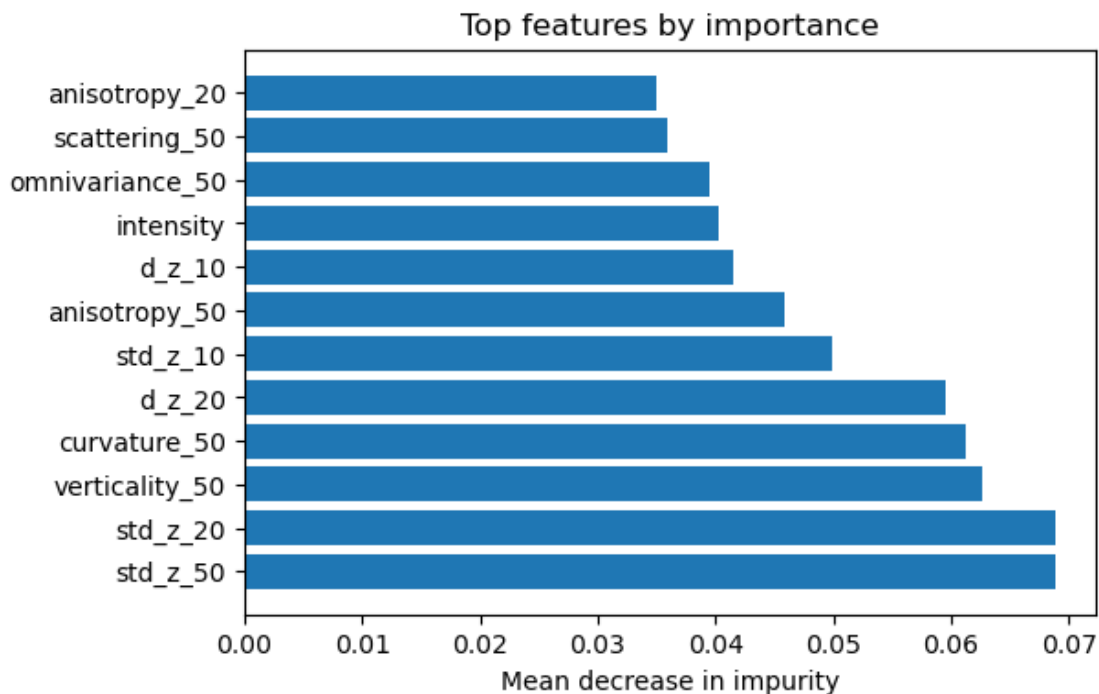
Once the classifier has been trained, embedded feature selection is performed by analysing feature importance. For Random Forests, the standard measure is the *Mean Decrease in Impurity* (MDI, available through `model.feature_importances_`). This metric quantifies the contribution of each feature to the splits performed by the trees. It is computationally inexpensive, but may be biased when features are highly correlated.

An alternative is permutation importance (`sklearn.inspection.permutation_importance`), which estimates feature relevance by measuring the decrease in model performance when the values of a feature are randomly shuffled. This approach is generally more reliable and model-agnostic, but comes at a significantly higher computational cost.

For simplicity and computational efficiency, the examples below rely on the MDI importance provided directly by the Random Forest implementation. The feature selection process confirms that 3D eigenvalue-based features alone are not sufficient to optimally characterize semantic classes, highlighting the importance of complementary features such as height or intensity.

```
[19]: # Compute feature importances
feature_importances = rf_clf.feature_importances_
# Keep only the most important features
top_features = np.flatnonzero(
    feature_importances > np.percentile(feature_importances, 75)
)
# Sort top features by importance
top_features = top_features[np.argsort(feature_importances[top_features])]

# Plot feature importances
fig, ax = plt.subplots(figsize=(6, 4))
ax.barh(
    X_names[top_features],
    feature_importances[top_features]
)
ax.invert_yaxis() # most important feature at the top
ax.set_xlabel("Mean decrease in impurity")
ax.set_title("Top features by importance")
plt.show()
```



The classifier is then retrained using only the dozen most important features identified by the Random Forest. No significant drop in training performance is observed, indicating that most of the discriminative information is captured by a small subset of descriptors. In practice, reducing the number of features also helps decrease memory usage, speed up training and inference, and improve model interpretability.

```
[20]: # Retrain with only the top features
X_train = X_train[:, top_features]

# Retrain the Random Forest classifier with the top features
rf_clf.fit(X_train, y_train)
y_pred_train_top = rf_clf.predict(X_train)
print("Classification report on training set (all samples top features):")
print(classification_report(y_train, y_pred_train_top,
                             target_names=class_names.values()))
```

Classification report on training set (all samples top features):

	precision	recall	f1-score	support
ground	1.00	0.99	1.00	33428
vegetation	0.98	1.00	0.99	9386
building	0.99	1.00	1.00	6242
accuracy			1.00	49056
macro avg	0.99	1.00	0.99	49056

weighted avg	1.00	1.00	1.00	49056
--------------	------	------	------	-------

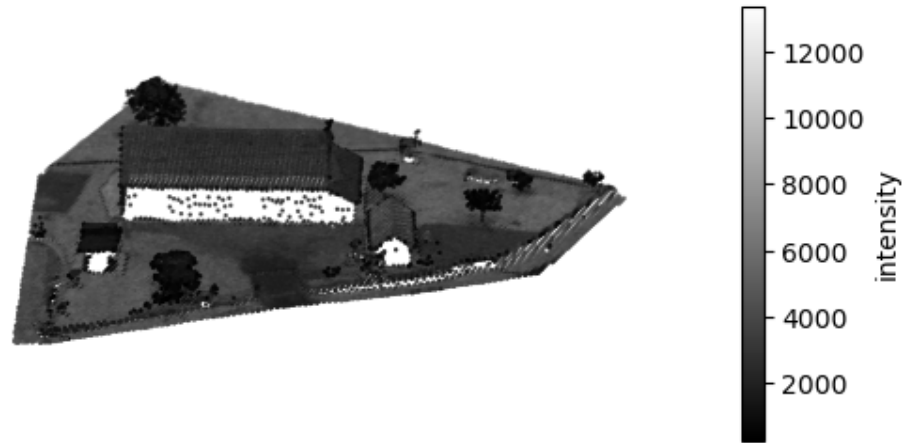
The classifier is then evaluated on the test set below. It consists of a different tile from the IGN LiDAR HD dataset (which was not used during training). This provides a more realistic estimate of the model's ability to generalize to unseen data.

```
[21]: # Load test pointcloud and compute features (similar to training)
data_test = np.loadtxt("./data/LHD_test.xyz")
points, intensity, y_test = data_test[:, :3], data_test[:, 3], data_test[:, 4]

# Spatial structure
kdtree = KDTree(points)

# Visualize the test pointcloud colored by intensity
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                    c=intensity, cmap="gray", s=0.5)
fig.colorbar(scatter, ax=ax, label="intensity", shrink=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Test pointcloud")
plt.show()
```

Test pointcloud



As for the training set, eigenvalue-based and geometric features are computed for the test set using the same neighborhood definitions.

```
[22]: normals_multiscale = []
      eigenfeatures_3D_multiscale = []

      for k in k_values_multiscale:
          neighbor_finder = lambda points: kdtree.query(points,
                                                         k,
                                                         return_distance=False)

          eigenfeatures_3D_k = compute_3D_eigen_features(points, neighbor_finder)
          # Remove normals from eigenfeatures list (computed at same time)
          normals_k = eigenfeatures_3D_k[0]
          eigenfeatures_3D_k = eigenfeatures_3D_k[1:]
          # Orient normals consistently towards +z direction (acquisition direction)
          mask_n = normals_k[:, 2] < 0
          normals_k[mask_n] = -normals_k[mask_n]
          # Store results
```

```

normals_multiscale.append(normals_k)
eigenfeatures_3D_multiscale.append(eigenfeatures_3D_k)

# Concatenate all multiscale features in one array of shape (N, 8*k)
eigenfeatures_3D_multiscale = np.stack(
    [arr for tup in eigenfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

```

[23]: geomfeatures_3D_multiscale = []

for i, k in enumerate(k_values_multiscale):
    neighbor_finder = lambda points: kdtree.query(points,
                                                    k,
                                                    return_distance=True,
                                                    sort_results=True)

    geomfeatures_3D_k = compute_3D_geometric_features(points,
                                                        normals_multiscale[i],
                                                        neighbor_finder)

    # Store results
    geomfeatures_3D_multiscale.append(geomfeatures_3D_k)

# Concatenate all multiscale features in one array of shape (N, 7*k)
geomfeatures_3D_multiscale = np.stack(
    [arr for tup in geomfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

Eigenvalue-based and geometric features are first concatenated into a single feature vector. Only the “top features” identified during the feature selection stage are then retained, ensuring that the test data is the same as the training data.

```

[24]: X_test = np.concatenate(
    [eigenfeatures_3D_multiscale, geomfeatures_3D_multiscale, intensity[:,
    ↪None]],
    axis=1
)
X_test = X_test[:, top_features]

print(X_test.shape)

```

```
(32901, 12)
```

The resulting feature vectors are then passed to the trained classifier. The predicted labels are finally compared with the ground-truth labels available for the test set in order to evaluate the classification performance.

The classifier generalizes well to the test set. Ground and building classes are detected with high

precision and recall, while vegetation is more challenging.

```
[25]: # Predict class labels for the test set
y_pred = rf_clf.predict(X_test)

# Classification report
print(
    classification_report(y_test, y_pred, target_names=class_names.values())
)
```

	precision	recall	f1-score	support
ground	0.98	0.97	0.97	25236
vegetation	0.74	0.84	0.79	2585
building	0.91	0.88	0.89	5080
accuracy			0.95	32901
macro avg	0.88	0.90	0.89	32901
weighted avg	0.95	0.95	0.95	32901

The plot below shows a globally coherent classification, with most points assigned to the correct class. Misclassified points are concentrated in a few localized areas, particularly on building roofs and near the lower parts of walls.

```
[26]: # Plot predicted labels on the test pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=y_pred, cmap=lidar_cmap, s=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Test pointcloud colored by predicted class labels")
# Create legend with class names
ax.legend(legend_handles, legend_labels, title="classes", loc="lower center")
plt.show()
```

Test pointcloud colored by predicted class labels



11.3.4 Regularization

The fourth step of the semantic segmentation pipeline is the **regularization** of predicted labels to enforce local consistency and remove isolated misclassifications. This step is optional but commonly applied. It relies on the assumption that neighboring points often belong to the same object and should therefore receive similar labels.

Common regularization techniques include:

- **Majority voting**, where each point is assigned the most frequent label among its neighbors.
- **Probability smoothing**, where class probabilities are averaged within a neighborhood before assigning the final label.
- **Markov Random Fields (MRF)** and **Conditional Random Fields (CRF)**, which combine classifier predictions with spatial smoothness constraints.

In practice, simple probability smoothing already provides a significant improvement at a very low computational cost. However, CRFs are generally the go-to regularization method in production pipelines, often significantly improving spatial consistency and boundary accuracy.

Below, regularization is performed by probability smoothing. The final label is obtained by averaging the classifier probabilities over the k -neighborhood of each point and selecting the most probable class. This approach is generally good at correcting isolated classification errors, while remaining simple to implement.

```
[27]: # Probabilities from the RF classifier
proba = rf_clf.predict_proba(X_test)

# Smooth probabilities over neighbors
inds = kdtree.query(points, k=20, return_distance=False)
smoothed_proba = np.zeros_like(proba)
for i in range(len(proba)):
    smoothed_proba[i] = proba[inds[i]].mean(axis=0)

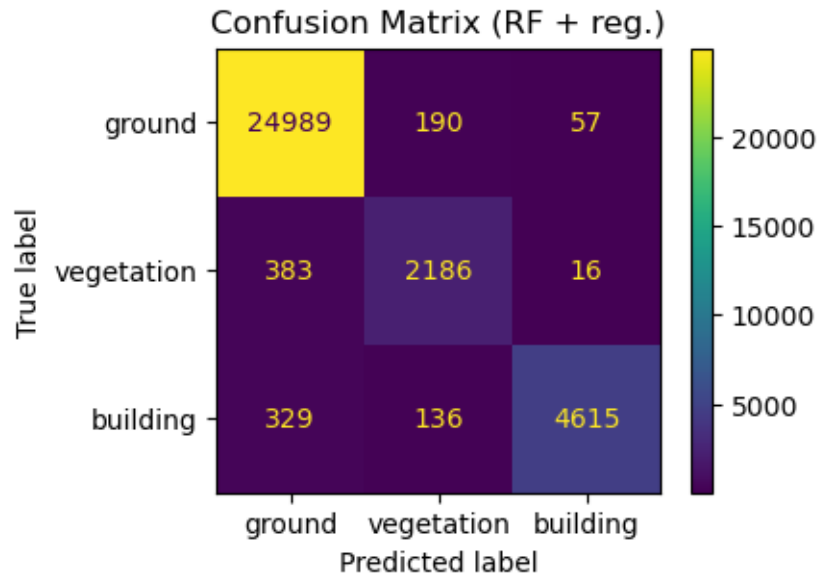
y_pred_smooth = smoothed_proba.argmax(axis=1)
```

Regularization improves the overall accuracy and significantly increases the classification quality of the vegetation class. This indicates that probability smoothing successfully corrects many isolated errors and enforces local consistency.

```
[28]: # Classification report
print(
    classification_report(y_test, y_pred_smooth, target_names=class_names,
        ↪values())
)

# Evaluate the classifier using a confusion matrix
fig, ax = plt.subplots(figsize=(4, 3))
cmd_clf = ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred_smooth,
    display_labels=class_names.values(),
    ax=ax
)
ax.set_title("Confusion Matrix (RF + reg.)")
plt.show()
```

	precision	recall	f1-score	support
ground	0.97	0.99	0.98	25236
vegetation	0.87	0.85	0.86	2585
building	0.98	0.91	0.94	5080
accuracy			0.97	32901
macro avg	0.94	0.91	0.93	32901
weighted avg	0.97	0.97	0.97	32901



The regularized classification is also more spatially coherent visually. This improvement is particularly noticeable on building roofs.

```
[29]: # Plot predicted labels on the test pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=y_pred_smooth, cmap=lidar_cmap, s=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Test pointcloud after regularization by probability smoothing")
# Create legend with class names
ax.legend(legend_handles, legend_labels, title="classes", loc="lower center")
plt.show()
```

Test pointcloud after regularization by probability smoothing



11.4 Wrapping up

You should now have a better grasp on how machine learning can be effectively applied to 3D point clouds. This notebook covered three areas where machine learning particularly shines: **cleaning**, **object segmentation**, and **semantic segmentation**. By leveraging both the geometric structure and statistical properties of the data, these methods provide solutions that are often more robust than purely geometric approaches while remaining efficient and interpretable.

For cleaning and object segmentation, machine learning models can often be used as **drop-in replacements** for classical geometric algorithms. **Local Outlier Factor (LOF)** improves **outlier detection** by identifying points that are inconsistent with their local neighborhood, making it a powerful alternative to threshold-based geometric filters. Similarly, clustering methods such as K-Means, DBSCAN, and Agglomerative Clustering can segment objects directly from the point cloud. In particular, **DBSCAN** often replaces traditional geometric **object segmentation** techniques while offering better handling of noise and objects with irregular shapes. Semantic segmentation, however, goes beyond geometry and assigns a semantic meaning to each point (such as ground, building, or vegetation). This type of understanding cannot be achieved with purely geometric algorithms alone, as it requires learning the relationship between geometric features and semantic

classes from labeled examples.

While this notebook focused on cleaning, object segmentation, and semantic segmentation, machine learning also plays an important role in other pointcloud processing tasks, including classification, registration, feature matching, and change detection. However, the performance of classical machine learning models depends heavily on **feature engineering** or the design geometric descriptors that capture the relevant characteristics of the data. As tasks become more complex, identifying the most informative features becomes increasingly challenging.

This limitation naturally motivates the use of deep learning models, which learn hierarchical representations directly from raw point clouds or derived representations rather than relying on hand-crafted features. As a result, they can capture more complex patterns and achieve state-of-the-art performance on tasks such as semantic segmentation. However, this additional power comes at the cost of increased complexity, lower interpretability, and a need for larger training datasets and computational resources. Classical machine learning therefore remains an excellent choice when data are limited, interpretability is important, or a simple and efficient solution is sufficient.

12 Going further

The lists of software and resources below are not exhaustive, but may be a good starting point for those who wish to dive deeper in the topic of pointcloud processing with Python.

12.1 Software

Libraries (in alphabetical order):

- **CGAL**, an open-source library for efficient and reliable geometric algorithms (in C++, with Python bindings)
- **CloudComPy**, a Python wrapper for CloudCompare (see below)
- **Open3D**, an open-source library for 3D data processing (in C++ and Python, with a 3D viewer app)
- **PCL**, a standalone, large scale, open project for 2D/3D image and pointcloud processing (in C++, with Python bindings)
- **PDAL**, an open-source library for translating and manipulating pointcloud data (in C++, with Python support)
- **PyMeshLab**, a Python library that interfaces to MeshLab (see below)
- **PyntCloud**, a Python library for working with 3D point clouds leveraging the power of the Python scientific stack
- **PyVista**, a library providing a pythonic interface to VTK (see below)
- **VTK**, an open-source software for manipulating and displaying scientific data (in C++, with wrappers in Python, Java and Tcl)

Applications (in alphabetical order):

- **Blender**, an open-source 3D computer graphics software that may be used to visualize and process pointclouds (with Python scripting capabilities)
- **CloudCompare**, an open-source 3D pointcloud (and triangular mesh) processing software (with Python scripting capabilities through **CloudComPy**)
- **MeshLab**, an open-source 3D triangular meshes (and pointclouds) processing and editing software (with Python scripting capabilities through **PyMeshLab**)
- **ParaView**, an open-source visualization application (with Python scripting capabilities)

12.2 Resources

Books (in reverse chronological order):

- Poux, F. (2025). *3D Data Science with Python*. O'Reilly Media.
- Liu, S., Zhang, M., Kadam, P., & Kuo, C. C. J. (2021). *3D Point Cloud Analysis: Traditional, Deep Learning, and Explainable Machine Learning Methods*. Springer.
- Vosselman, G., & Maas, H. G. (2010). *Airborne and terrestrial laser scanning*. Whittles Publishing.

- Samet, H. (2006). *Foundations of multidimensional and metric data structures*. Morgan Kaufmann.
- Schneider, P., & Eberly, D. H. (2002). *Geometric tools for computer graphics*. Elsevier.
- Goulette, F. (1999). *Modélisation 3D automatique : outils de géométrie différentielle*. Presses des Mines.

Videos (in alphabetical order):

- CVPR 2024 Tutorial: *All You Need To Know About Point Cloud Understanding*. <https://cvpr.thecvf.com/virtual/2024/tutorial/23725> (last accessed in August 2026)
- CVPR 2025 Tutorial: *The 2nd Point Cloud Tutorial: All You Need To Know About 3D Point Cloud*. <https://cvpr.thecvf.com/virtual/2025/tutorial/35903> (last accessed in August 2026)
- CloudCompare playlist on Daniel Girardeau-Montaut YouTube channel <https://www.youtube.com/@danielgirardeau-montaut9044> (last accessed in August 2026)
- Florent Poux YouTube channel: <https://www.youtube.com/@FlorentPoux> (last accessed in August 2026)

References

- [1] Donald Meagher. “Geometric modeling using octree encoding”. In: *Computer graphics and image processing* 19.2 (1982), pp. 129–147.
- [2] Jon Louis Bentley. “Multidimensional binary search trees used for associative searching”. In: *Communications of the ACM* 18.9 (1975), pp. 509–517.
- [3] Irene Gargantini. “An effective way to represent quadtrees”. In: *Communications of the ACM* 25.12 (1982), pp. 905–910.
- [4] Daniel Girardeau-Montaut et al. “Change detection on points cloud data acquired with a ground laser scanner”. In: *International archives of photogrammetry, remote sensing and spatial information sciences* 36.3 (2005), W19.
- [5] Songrit Maneewongvatana and David M Mount. “It’s okay to be skinny, if your friends are fat”. In: *Center for geometric computing 4th annual workshop on computational geometry*. Vol. 2. 1999, pp. 1–8.
- [6] Jeffrey S Vitter. “Random sampling with a reservoir”. In: *ACM Transactions on Mathematical Software (TOMS)* 11.1 (1985), pp. 37–57.
- [7] Cem Yuksel. “Sample elimination for generating poisson disk sample sets”. In: *Computer Graphics Forum*. Vol. 34. 2. Wiley Online Library. 2015, pp. 25–32.
- [8] Carlo Tomasi and Roberto Manduchi. “Bilateral filtering for gray and color images”. In: *Sixth international conference on computer vision (IEEE Cat. No. 98CH36271)*. IEEE. 1998, pp. 839–846.
- [9] Shachar Fleishman, Iddo Drori, and Daniel Cohen-Or. “Bilateral mesh denoising”. In: *ACM SIGGRAPH 2003 Papers*. 2003, pp. 950–953.
- [10] Julie Digne and Carlo De Franchis. “The bilateral filter for point clouds”. In: *Image Processing On Line* 7 (2017), pp. 278–287.
- [11] Marc Alexa et al. “Computing and rendering point set surfaces”. In: *IEEE Transactions on visualization and computer graphics* 9.1 (2003), pp. 3–15.
- [12] Marie-Julie Rakotosaona et al. “Pointcleannet: Learning to denoise and remove outliers from dense point clouds”. In: *Computer graphics forum*. Vol. 39. 1. Wiley Online Library. 2020, pp. 185–203.
- [13] Frédéric Cazals and Marc Pouget. “Estimating differential quantities using polynomial fitting of osculating jets”. In: *Computer aided geometric design* 22.2 (2005), pp. 121–146.
- [14] Hugues Hoppe et al. “Surface reconstruction from unorganized points”. In: *Proceedings of the 19th annual conference on computer graphics and interactive techniques*. 1992, pp. 71–78.
- [15] Jan J Koenderink and Andrea J Van Doorn. “Surface shape and curvature scales”. In: *Image and vision computing* 10.8 (1992), pp. 557–564.
- [16] Mark Pauly, Markus Gross, and Leif P Kobbelt. “Efficient simplification of point-sampled surfaces”. In: *IEEE Visualization, 2002. VIS 2002*. IEEE. 2002, pp. 163–170.
- [17] Gabriel Taubin. “Estimating the tensor of curvature of a surface from a polyhedral approximation”. In: *Proceedings of IEEE International Conference on Computer Vision*. IEEE. 1995, pp. 902–907.
- [18] Niloy J Mitra and An Nguyen. “Estimating surface normals in noisy point cloud data”. In: *Proceedings of the nineteenth annual symposium on Computational geometry*. 2003, pp. 322–328.
- [19] Alexandre Boulch and Renaud Marlet. “Fast and robust normal estimation for point clouds with sharp features”. In: *Computer graphics forum*. Vol. 31. 5. Wiley Online Library. 2012, pp. 1765–1774.

- [20] Hui Xie et al. “Piecewise $C/\sup 1$ /continuous surface reconstruction of noisy point clouds via local implicit quadric regression”. In: *IEEE Visualization, 2003. VIS 2003*. IEEE. 2003, pp. 91–98.
- [21] Jack Goldfeather and Victoria Interrante. “A novel cubic-order algorithm for approximating principal direction vectors”. In: *ACM Transactions on Graphics (TOG)* 23.1 (2004), pp. 45–63.
- [22] Radu Bogdan Rusu. “Semantic 3D object maps for everyday manipulation in human living environments”. In: *KI-Künstliche Intelligenz* 24.4 (2010), pp. 345–348.
- [23] Federico Tombari, Samuele Salti, and Luigi Di Stefano. “Unique signatures of histograms for local surface description”. In: *European conference on computer vision*. Springer. 2010, pp. 356–369.
- [24] Martin Weinmann et al. “Semantic point cloud interpretation based on optimal neighborhoods, relevant features and efficient classifiers”. In: *ISPRS Journal of Photogrammetry and Remote Sensing* 105 (2015), pp. 286–304.
- [25] Federico Tombari, Samuele Salti, Luigi Di Stefano, et al. “A combined texture-shape descriptor for enhanced 3D feature matching.” In: *ICIP*. 2011, pp. 809–812.
- [26] Andrew E Johnson and Martial Hebert. “Surface registration by matching oriented points”. In: *Proceedings. International Conference on Recent Advances in 3-D Digital Imaging and Modeling*. IEEE. 1997, pp. 121–128.
- [27] Radu Bogdan Rusu et al. “Fast 3d recognition and pose using the viewpoint feature histogram”. In: *2010 IEEE/RSJ international conference on intelligent robots and systems*. IEEE. 2010, pp. 2155–2162.
- [28] Tahir Rabbani, Frank Van Den Heuvel, and George Vosselmann. “Segmentation of point clouds using smoothness constraint”. In: *International archives of photogrammetry, remote sensing and spatial information sciences* 36.5 (2006), pp. 248–253.
- [29] Paul VC Hough. *Method and means for recognizing complex patterns*. US Patent 3,069,654. Dec. 1962.
- [30] Martin A Fischler and Robert C Bolles. “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography”. In: *Communications of the ACM* 24.6 (1981), pp. 381–395.
- [31] Jeremie Papon et al. “Voxel cloud connectivity segmentation-supervoxels for point clouds”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2013, pp. 2027–2034.
- [32] Dorit Borrmann et al. “The 3d hough transform for plane detection in point clouds: A review and a new accumulator design”. In: *3D Research* 2.2 (2011), p. 3.
- [33] Tahir Rabbani and Frank Van Den Heuvel. “Efficient hough transform for automatic detection of cylinders in point clouds”. In: *Isprs Wg Iii/3, Iii/4* 3 (2005), pp. 60–65.
- [34] Ruwen Schnabel, Roland Wahl, and Reinhard Klein. “Efficient RANSAC for point-cloud shape detection”. In: *Computer graphics forum*. Vol. 26. 2. Wiley Online Library. 2007, pp. 214–226.
- [35] David Eberly. *Least squares fitting of data by linear or quadratic structures*. Tech. rep. Technical report, Geometric Tools. <https://www.geometrictools.com>, 1999.
- [36] David Eberly. *Fitting 3d data with a torus*. Tech. rep. Technical report, Geometric Tools. <https://www.geometrictools.com>, 2018.
- [37] François Goulette. “Automatic CAD modeling of industrial pipes from range images”. In: *Proceedings. International Conference on Recent Advances in 3-D Digital Imaging and Modeling (Cat. No. 97TB100134)*. IEEE. 1997, pp. 229–233.
- [38] Olga Sorkine-Hornung and Michael Rabinovich. “Least-squares rigid motion using svd”. In: *Computing* 1.1 (2017), pp. 1–5.

- [39] Kok-Lim Low. “Linear least-squares optimization for point-to-plane icp surface registration”. In: *Chapel Hill, University of North Carolina* 4.10 (2004), pp. 1–3.
- [40] Yang Chen and Gérard Medioni. “Object modelling by registration of multiple range images”. In: *Image and vision computing* 10.3 (1992), pp. 145–155.
- [41] Paul J Besl and Neil D McKay. “Method for registration of 3-D shapes”. In: *Sensor fusion IV: control paradigms and data structures*. Vol. 1611. Spie. 1992, pp. 586–606.
- [42] Markus M Breunig et al. “LOF: identifying density-based local outliers”. In: *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 2000, pp. 93–104.
- [43] Jérôme Demantké et al. “Dimensionality based scale selection in 3D lidar point clouds”. In: *Laserscanning*. 2011.
- [44] Joachim Niemeyer, Franz Rottensteiner, and Uwe Soergel. “Contextual classification of lidar data and building object detection in urban areas”. In: *ISPRS journal of photogrammetry and remote sensing* 87 (2014), pp. 152–165.
- [45] Timo Hackel, Jan D Wegner, and Konrad Schindler. “Fast semantic segmentation of 3D point clouds with strongly varying density”. In: *ISPRS annals of the photogrammetry, remote sensing and spatial information sciences* 3 (2016), pp. 177–184.
- [46] Institut national de l’information géographique et forestière (IGN). *Nuages de points LiDAR HD*. Open Licence 2.0. 2026. URL: <https://www.data.gouv.fr/datasets/nuages-de-points-lidar-hd>.

